زبان اسمبلی رایانه‌های شخصی

پل ای. کارتر

ترجمهٔ

دکتر یاسین علی‌پور

۱۶ نوامبر ۲۰۱۹

این اثر تحت این پروانه منتشر شده است:
Creative Commons Attribution-NonCommercial-ShareAlike
4.0 International License
برای دیدن متن این پروانه به این نشانی مراجعه کنید:
http://creativecommons.org/licenses/by-nc-sa/4.0/

ترجمهٔ فارسی کتاب *PC Assembly Language* نوشتهٔ Paul A. Carter به قلم دکتر یاسین علی‌پور است و با همان پروانه منتشر می‌شود.

فهرست مطالب

پیشگفتار **ز**

۱ مقدمه **۱**

۱.۱ دستگاه‌های عددنویسی ۱

۱.۱.۱ دستگاه دهدهی ۱

۲.۱.۱ دستگاه دودویی ۱

۳.۱.۱ دستگاه شانزده‌شانزدهی ۳

۲.۱ سازمان رایانه ۵

۱.۲.۱ حافظه ۵

۲.۲.۱ پردازنده ۷

۳.۲.۱ خانوادهٔ پردازنده‌های 80x86 . . . . . ۸

۴.۲.۱ ثبات‌های ۱۶ بیتی 8086 . . . . . ۹

۵.۲.۱ ثبات‌های ۳۲ بیتی 80386 . . . . . ۱۰

۶.۲.۱ حالت حقیقی ۱۱

۷.۲.۱ حالت حفاظت‌شدهٔ ۱۶ بیتی ۱۲

۸.۲.۱ حالت حفاظت‌شدهٔ ۳۲ بیتی ۱۳

۹.۲.۱ وقفه‌ها ۱۴

۳.۱ زبان اسمبلی ۱۵

۱.۳.۱ زبان ماشین ۱۵

۲.۳.۱ زبان اسمبلی ۱۵

۳.۳.۱ عملگرهای دستورالعمل ۱۶

۴.۳.۱ دستورالعمل‌های پایه ۱۷

۵.۳.۱ رهنمودها ۱۸

۶.۳.۱ ورودی و خروجی ۲۲
۷.۳.۱ اشکال‌زدایی ۲۳
۴.۱ ساختن یک برنامه ۲۵
۱.۴.۱ نخستین برنامه ۲۵
۲.۴.۱ وابستگی به کامپایلر ۲۹
۳.۴.۱ اسمبل کردن کد ۳۰
۴.۴.۱ کامپایل کردن کد C ۳۰
۵.۴.۱ پیوند دادن فایل‌های شیء ۳۱
۶.۴.۱ درک فایل فهرست اسمبلی ۳۱
۵.۱ فایل اسکلت ۳۴

۲ مبانی زبان اسمبلی ۳۵
۱.۲ کار با اعداد صحیح ۳۵
۱.۱.۲ نمایش اعداد صحیح ۳۵
۲.۱.۲ گسترش علامت ۳۹
۳.۱.۲ محاسبات حسابی مکمل دو ۴۳
۴.۱.۲ برنامهٔ نمونه ۴۵
۵.۱.۲ حساب با دقت گسترش‌یافته ۴۷
۲.۲ ساختارهای کنترلی ۴۸
۱.۲.۲ مقایسه‌ها ۴۸
۲.۲.۲ دستورالعمل‌های انشعاب ۴۹
۳.۲.۲ دستورالعمل‌های حلقه ۵۳
۳.۲ ترجمهٔ ساختارهای کنترلی استاندارد ۵۴
۱.۳.۲ دستور **if** ۵۴
۲.۳.۲ حلقه‌های **while** ۵۵
۳.۳.۲ حلقه‌های **do while** ۵۵
۴.۲ مثال: یافتن اعداد اول ۵۶

۳ عملیات بیتی ۵۹
۱.۳ عملیات جابه‌جایی ۵۹
۱.۱.۳ جابه‌جایی‌های منطقی ۵۹
۲.۱.۳ کاربرد جابه‌جایی‌ها ۶۰
۳.۱.۳ جابه‌جایی‌های حسابی ۶۱

۴.۱.۳ جابه‌جایی‌های چرخشی ۶۱
۵.۱.۳ یک کاربرد ساده ۶۲
۲.۳ عملیات بیتی بولی ۶۳
۱.۲.۳ عمل AND ۶۳
۲.۲.۳ عمل OR ۶۴
۳.۲.۳ عمل XOR ۶۴
۴.۲.۳ عمل NOT ۶۵
۵.۲.۳ دستورالعمل TEST ۶۵
۶.۲.۳ کاربردهای عملیات بیتی ۶۵
۳.۳ پرهیز از انشعاب‌های شرطی ۶۷
۴.۳ دست‌کاری بیت‌ها در C ۷۰
۱.۴.۳ اپراتورهای بیتی C ۷۰
۲.۴.۳ به کارگیری اپراتورهای بیتی در C ۷۱
۵.۳ نمایش‌های بزرگ‌انتها و کوچک‌انتها ۷۲
۱.۵.۳ چه زمانی ترتیب کوچک‌انتها و بزرگ‌انتها اهمیت دارد؟ ۷۴
۶.۳ شمارش بیت‌ها ۷۵
۱.۶.۳ روش اول ۷۵
۲.۶.۳ روش دوم ۷۶
۳.۶.۳ روش سوم ۷۸

۴ زیربرنامه‌ها ۸۱
۱.۴ آدرس‌دهی غیرمستقیم ۸۱
۲.۴ نمونه‌ای ساده از زیربرنامه ۸۲
۳.۴ پشته ۸۵
۴.۴ دستورالعمل‌های CALL و RET ۸۶
۵.۴ قراردادهای فراخوانی ۸۷
۱.۵.۴ ارسال پارامترها از طریق پشته ۸۸
۲.۵.۴ متغیرهای محلی در پشته ۹۴
۶.۴ برنامه‌های چندپیمانه‌ای ۹۵
۷.۴ پیوند اسمبلی با C ۹۹
۱.۷.۴ ذخیره‌سازی ثبات‌ها ۱۰۰
۲.۷.۴ برچسب‌های توابع ۱۰۱
۳.۷.۴ ارسال پارامترها ۱۰۲

۴.۷.۴ محاسبهٔ آدرس متغیرهای محلی ۱۰۲
۵.۷.۴ بازگرداندن مقادیر ۱۰۳
۶.۷.۴ قراردادهای فراخوانی دیگر ۱۰۳
۷.۷.۴ مثال‌ها ۱۰۵
۸.۷.۴ فراخوانی توابع C از اسمبلی ۱۰۸
۸.۴ زیربرنامه‌های بازورودپذیر و بازگشتی ۱۰۹
۱.۸.۴ زیربرنامه‌های بازگشتی ۱۱۰
۲.۸.۴ مروری بر انواع ذخیره‌سازی متغیرها در C ۱۱۲

۵ آرایه‌ها ۱۱۷
۱.۵ مقدمه ۱۱۷
۱.۱.۵ تعریف آرایه‌ها ۱۱۷
۲.۱.۵ دسترسی به عناصر آرایه ۱۲۰
۳.۱.۵ آدرس‌دهی غیرمستقیم پیشرفته‌تر ۱۲۱
۴.۱.۵ مثال ۱۲۲
۵.۱.۵ آرایه‌های چندبعدی ۱۲۶
۲.۵ دستورالعمل‌های آرایه و رشته ۱۳۰
۱.۲.۵ خواندن از حافظه و نوشتن در آن ۱۳۱
۲.۲.۵ پیشوند دستورالعمل REP ۱۳۳
۳.۲.۵ دستورالعمل‌های رشته‌ای مقایسه ۱۳۴
۴.۲.۵ پیشوندهای دستورالعمل REPx ۱۳۵
۵.۲.۵ مثال ۱۳۶

۶ ممیز شناور ۱۴۳
۱.۶ نمایش ممیز شناور ۱۴۳
۱.۱.۶ اعداد دودویی غیرصحیح ۱۴۳
۲.۱.۶ نمایش ممیز شناور IEEE ۱۴۶
۲.۶ حساب ممیز شناور ۱۵۰
۱.۲.۶ جمع ۱۵۰
۲.۲.۶ تفریق ۱۵۱
۳.۲.۶ ضرب و تقسیم ۱۵۲
۴.۲.۶ پیامدها برای برنامه‌نویسی ۱۵۲
۳.۶ کمک‌پردازندهٔ ریاضی ۱۵۳

۱.۳.۶ سخت‌افزار ۱۵۳
۲.۳.۶ دستورالعمل‌ها ۱۵۴
۳.۳.۶ مثال‌ها ۱۶۴
۴.۳.۶ فرمول معادلهٔ درجهٔ دوم ۱۶۴
۵.۳.۶ خواندن آرایه از فایل ۱۶۷
۶.۳.۶ یافتن اعداد اول ۱۷۰

۷ ساختارها و C++ ۱۷۵
۱.۷ ساختارها ۱۷۵
۱.۱.۷ مقدمه ۱۷۵
۲.۱.۷ هم‌ترازی حافظه ۱۷۷
۳.۱.۷ میدان‌های بیتی ۱۷۹
۴.۱.۷ استفاده از ساختارها در اسمبلی ۱۸۳
۲.۷ اسمبلی و C++ ۱۸۴
۱.۲.۷ سربارگذاری و درهم‌سازی نام ۱۸۴
۲.۲.۷ ارجاع‌ها ۱۸۹
۳.۲.۷ توابع درون‌خطی ۱۹۰
۴.۲.۷ کلاس‌ها ۱۹۳
۵.۲.۷ وراثت و چندریختی ۲۰۲
۶.۲.۷ دیگر ویژگی‌های C++ ۲۱۰

الف دستورالعمل‌های 80x86 ۲۱۱
الف.۱ دستورالعمل‌های غیر ممیز شناور ۲۱۱
الف.۲ دستورالعمل‌های ممیز شناور ۲۱۸

پیشگفتار

هدف

هدف این کتاب این است که خواننده دریابد رایانه‌ها در سطحی پایین‌تر از زبان‌های برنامه‌نویسی‌ای مانند پاسکال واقعاً چگونه کار می‌کنند. خواننده با درک عمیق‌تر سازوکار رایانه‌ها اغلب می‌تواند در توسعهٔ نرم‌افزار با زبان‌های سطح بالاتری مانند C و C++ بسیار کارآمدتر باشد. یادگیری برنامه‌نویسی به زبان اسمبلی راه بسیار خوبی برای رسیدن به این هدف است. کتاب‌های دیگر زبان اسمبلی رایانه‌های شخصی هنوز برنامه‌نویسی پردازندهٔ 8086 را آموزش می‌دهند؛ پردازنده‌ای که نخستین رایانهٔ شخصی در سال ۱۹۸۱ از آن استفاده می‌کرد! پردازندهٔ 8086 فقط از حالت **حقیقی** پشتیبانی می‌کرد. در این حالت هر برنامه‌ای می‌تواند به هر بخشی از حافظه یا هر دستگاهی در رایانه دسترسی داشته باشد. این حالت برای یک سیستم‌عامل امن و چندوظیفه‌ای مناسب نیست. این کتاب در عوض برنامه‌نویسی پردازندهٔ 80386 و پردازنده‌های بعدی را در حالت **حفاظت‌شده** شرح می‌دهد (حالتی که ویندوز و لینوکس در آن اجرا می‌شوند). این حالت از امکاناتی که سیستم‌عامل‌های امروزی انتظار دارند، مانند حافظهٔ مجازی و حفاظت از حافظه، پشتیبانی می‌کند. دلایل گوناگونی برای استفاده از حالت حفاظت‌شده وجود دارد:

۱. برنامه‌نویسی در حالت حفاظت‌شده از برنامه‌نویسی در حالت حقیقی 8086، که کتاب‌های دیگر به کار می‌برند، آسان‌تر است.

۲. همهٔ سیستم‌عامل‌های امروزی رایانه‌های شخصی در حالت حفاظت‌شده اجرا می‌شوند.

۳. نرم‌افزارهای رایگانی وجود دارند که در این حالت اجرا می‌شوند.

نبود کتاب درسی برای برنامه‌نویسی اسمبلی رایانه‌های شخصی در حالت حفاظت‌شده دلیل اصلی نگارش این کتاب بوده است.

همان‌طور که اشاره شد، این متن از نرم‌افزارهای آزاد و متن‌باز استفاده می‌کند؛ یعنی اسمبلر NASM و کامپایلر C/C++ به نام DJGPP. هر دوی این‌ها را می‌توان از اینترنت دریافت کرد. این متن همچنین شیوهٔ استفاده از کد اسمبلی NASM را در سیستم‌عامل لینوکس و همراه با کامپایلرهای C/C++ شرکت‌های بورلند و مایکروسافت در ویندوز شرح می‌دهد. نمونه‌کدهای همهٔ این سکوها در وبگاه من در دسترس است: http://pacman128.github.io/pcasm/. اگر می‌خواهید بسیاری از مثال‌های این آموزش را اسمبل و اجرا کنید، **باید** نمونه‌کدها را دریافت کنید.

توجه داشته باشید که این متن همهٔ جنبه‌های برنامه‌نویسی اسمبلی را پوشش نمی‌دهد. نویسنده کوشیده است مهم‌ترین موضوعاتی را که **همهٔ** برنامه‌نویسان باید با آن‌ها آشنا باشند پوشش دهد.

قدردانی

نویسنده از برنامه‌نویسان بسیاری در سراسر جهان که در جنبش نرم‌افزار آزاد و متن‌باز مشارکت داشته‌اند سپاسگزاری می‌کند. همهٔ برنامه‌ها و حتی خود این کتاب با نرم‌افزار آزاد تولید شده‌اند. نویسنده به‌ویژه از این افراد سپاسگزار است: John S. Fine، Simon Tatham، Julian Hall و دیگرانی که اسمبلر NASM را، که همهٔ مثال‌های این کتاب بر پایهٔ آن است، توسعه دادند؛ DJ Delorie برای توسعهٔ کامپایلر C/C++ به نام DJGPP که در این کتاب به کار رفته است؛ افراد بی‌شماری که در توسعهٔ کامپایلر GNU gcc، که DJGPP بر پایهٔ آن ساخته شده است، مشارکت داشته‌اند؛ Donald Knuth و دیگرانی که زبان‌های حروف‌چینی TEX و LATEX 2ε را، که برای تولید این کتاب به کار رفتند، توسعه دادند؛ Richard Stallman (بنیان‌گذار بنیاد نرم‌افزار آزاد)، Linus Torvalds (پدیدآورندهٔ هستهٔ لینوکس) و دیگرانی که نرم‌افزارهای پایه‌ای را تولید کردند که نویسنده برای پدیدآوردن این اثر از آن‌ها استفاده کرد.

سپاس از افراد زیر برای اصلاحاتی که پیشنهاد کردند:

- John S. Fine
- Marcelo Henrique Pinto de Almeida
- Sam Hopkins
- Nick D'Imperio
- Jeremiah Lawrence

- Ed Beroset
- Jerry Gembarowski
- Ziqiang Peng
- Eno Compton
- Josh I Cates
- Mik Mifflin
- Luke Wallis
- Gaku Ueda
- Brian Heward
- Chad Gorshing
- F. Gotti
- Bob Wilkinson
- Markus Koegel
- Louis Taber
- Dave Kiddell
- Eduardo Horowitz
- Sébastien Le Ray
- Nehal Mistry
- Jianyue Wang
- Jeremias Kleer
- Marc Janicki
- Trevor Hansen

- Giacomo Bruschi
- Leonardo Rodríguez Mújica
- Ulrich Bicheler
- Wu Xing
- Oleksandr Baranyuk

منابع اینترنتی

http://pacman128.github.io/	صفحهٔ نویسنده
http://www.nasm.us/	صفحهٔ NASM در SourceForge
http://www.delorie.com/djgpp	DJGPP
http://webster.cs.ucr.edu/	The Art of Assembly
comp.lang.asm.x86	USENET

بازخورد

نویسنده از هرگونه بازخورد دربارهٔ این اثر استقبال می‌کند.

رایانامه: pacman128@gmail.com
وبگاه: http://pacman128.github.io/pcasm/

فصل ۱

مقدمه

۱.۱ دستگاه‌های عددنویسی

آنچه در حافظهٔ رایانه نگهداری می‌شود چیزی جز عدد نیست، اما رایانه این عددها را در دستگاه دهدهی[۱] (مبنای ۱۰) ذخیره نمی‌کند. از آنجا که نمایش دودویی[۲] (مبنای ۲) سخت‌افزار را بسیار ساده‌تر می‌کند، رایانه‌ها همهٔ اطلاعات را به شکل دودویی نگه می‌دارند. با این حال، بهتر است نخست مروری کوتاه بر دستگاه دهدهی داشته باشیم.

۱.۱.۱ دستگاه دهدهی

عددهای مبنای ۱۰ از ۱۰ رقم[۳] ممکن (۰ تا ۹) ساخته می‌شوند و هر رقم، بسته به جایگاهی که در عدد دارد، با توانی از ۱۰ متناظر است. برای نمونه:

$$234 = 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$$

۲.۱.۱ دستگاه دودویی

عددهای مبنای ۲ تنها از دو رقم ممکن (۰ و ۱) ساخته می‌شوند و در اینجا نیز هر رقم، بسته به جایگاهش در عدد، با توانی از ۲ متناظر است. (به هر رقم دودویی یک

[1] decimal
[2] binary
[3] digit

دودویی	دهدهی		دودویی	دهدهی
0000	0		1000	8
0001	1		1001	9
0010	2		1010	10
0011	3		1011	11
0100	4		1100	12
0101	5		1101	13
0110	6		1110	14
0111	7		1111	15

جدول ۱.۱: عددهای دهدهی ۰ تا ۱۵ در مبنای دودویی

بیت[۴] گفته می‌شود.) برای نمونه[۵]:

$$\begin{aligned} 11001_2 &= 1\times 2^4 + 1\times 2^3 + 0\times 2^2 + 0\times 2^1 + 1\times 2^0 \\ &= 16 + 8 + 1 \\ &= 25 \end{aligned}$$

این مثال نشان می‌دهد که چگونه می‌توان یک عدد دودویی را به دهدهی تبدیل کرد. جدول ۱.۱ نیز نمایش دودویی چند عدد نخست را نشان می‌دهد.

با رقم نقلی قبلی				بدون رقم نقلی قبلی			
1	1	0	1	1	1	0	0
+1	+0	+1	+0	+1	+0	+1	+0
1	0	0	1	0	1	1	0
c	c	c		c			

شکل ۱.۱: جمع دودویی (حرف c نشانهٔ *carry*، یعنی رقم نقلی، است)

شکل ۱.۱ نشان می‌دهد که رقم‌های دودویی تکی (یعنی بیت‌ها) چگونه با هم جمع می‌شوند. به این مثال توجه کنید:

[۴] bit

[۵] عدد ۲ که پایین و در سمت راست عدد نوشته شده است نشان می‌دهد که عدد در مبنای دودویی نمایش داده شده است، نه در مبنای دهدهی.

$$\begin{array}{r} 11011_2 \\ +10001_2 \\ \hline 101100_2 \end{array}$$

اکنون تقسیم دهدهی زیر را در نظر بگیرید (در اینجا r باقی‌ماندهٔ تقسیم را نشان می‌دهد):

$$1234 \div 10 = 123\ r\ 4$$

همان‌طور که می‌بینید، این تقسیم سمت‌راست‌ترین رقم دهدهی عدد را جدا می‌کند و بقیهٔ رقم‌ها را یک جایگاه به راست می‌برد. تقسیم بر دو نیز همین کار را با رقم‌های دودویی عدد انجام می‌دهد. برای نمونه، به تقسیم دودویی زیر نگاه کنید:

$$1101_2 \div 10_2 = 110_2\ r\ 1$$

از این ویژگی می‌توان برای تبدیل یک عدد دهدهی به نمایش دودویی معادل آن بهره گرفت، چنان‌که شکل ۲.۱ نشان می‌دهد. این روش نخست سمت‌راست‌ترین رقم را به دست می‌آورد که به آن **کم‌ارزش‌ترین بیت**[۶] گفته می‌شود. سمت‌چپ‌ترین رقم نیز **پرارزش‌ترین بیت**[۷] نام دارد. واحد پایهٔ حافظه از ۸ بیت تشکیل شده است و **بایت**[۸] خوانده می‌شود.

۳.۱.۱ دستگاه شانزده‌شانزدهی

عددهای شانزده‌شانزدهی[۹] در مبنای ۱۶ نوشته می‌شوند. از این دستگاه، که به اختصار **هگز**[۱۰] هم نامیده می‌شود، می‌توان به‌عنوان شیوه‌ای کوتاه‌نویسی برای عددهای دودویی بهره گرفت. دستگاه شانزده‌شانزدهی ۱۶ رقم ممکن دارد و همین نکته مشکلی پیش می‌آورد، زیرا برای رقم‌های اضافی پس از ۹ نمادی وجود ندارد. طبق قرارداد، این رقم‌های اضافی را با حرف نشان می‌دهند؛ بنابراین ۱۶ رقم این دستگاه، رقم‌های ۰ تا ۹ و سپس حرف‌های A، B، C، D، E و F هستند. رقم A با ۱۰ در دستگاه دهدهی برابر است، B با ۱۱ و به همین ترتیب تا آخر. در اینجا نیز هر رقم از یک عدد

[6] least significant bit (lsb)
[7] most significant bit (msb)
[8] byte
[9] hexadecimal
[10] hex

دودویی	دهدهی
$11001 \div 10 = 1100\ r\ 1$	$25 \div 2 = 12\ r\ 1$
$1100 \div 10 = 110\ r\ 0$	$12 \div 2 = 6\ r\ 0$
$110 \div 10 = 11\ r\ 0$	$6 \div 2 = 3\ r\ 0$
$11 \div 10 = 1\ r\ 1$	$3 \div 2 = 1\ r\ 1$
$1 \div 10 = 0\ r\ 1$	$1 \div 2 = 0\ r\ 1$

بنابراین $25_{10} = 11001_2$

شکل ۲.۱: تبدیل عدد دهدهی به دودویی

$$
\begin{aligned}
589 \div 16 &= 36\ r\ 13 \\
36 \div 16 &= 2\ r\ 4 \\
2 \div 16 &= 0\ r\ 2
\end{aligned}
$$

بنابراین $589 = 24\mathrm{D}_{16}$

شکل ۳.۱:

شانزده‌شانزدهی با توانی از ۱۶ متناظر است. برای نمونه:

$$
\begin{aligned}
2\mathrm{BD}_{16} &= 2 \times 16^2 + 11 \times 16^1 + 13 \times 16^0 \\
&= 512 + 176 + 13 \\
&= 701
\end{aligned}
$$

برای تبدیل از دهدهی به شانزده‌شانزدهی، همان ایدهٔ تبدیل به دودویی را به کار ببرید، با این تفاوت که این بار باید بر ۱۶ تقسیم کنید. نمونه‌ای از این کار را در شکل ۳.۱ می‌بینید.

سودمندی دستگاه شانزده‌شانزدهی در این است که تبدیل میان آن و دستگاه

دودویی راه بسیار ساده‌ای دارد. عددهای دودویی خیلی زود بزرگ و دست‌وپاگیر می‌شوند، در حالی که نمایش شانزده‌شانزدهی راهی بسیار فشرده‌تر برای نوشتن آن‌ها فراهم می‌کند.

برای تبدیل یک عدد شانزده‌شانزدهی به دودویی کافی است هر رقم آن را به یک عدد دودویی ۴ بیتی تبدیل کنید. برای نمونه، $24\mathrm{D}_{16}$ به $0010\ 0100\ 1101_2$ تبدیل می‌شود. توجه داشته باشید که صفرهای ابتدایی هر گروه ۴ بیتی اهمیت دارند! اگر صفر ابتدایی رقم میانی $24\mathrm{D}_{16}$ نوشته نشود، نتیجه نادرست خواهد بود. تبدیل از دودویی به شانزده‌شانزدهی نیز به همین سادگی است و کافی است همان فرایند را وارونه انجام دهید، یعنی هر بخش ۴ بیتی از عدد دودویی را به یک رقم شانزده‌شانزدهی تبدیل کنید. فقط باید کار را از انتهای سمت راست عدد دودویی آغاز کنید، نه از سمت چپ آن، تا فرایند با بخش‌های ۴ بیتی درست پیش برود[۱۱]. برای نمونه:

110	0000	0101	1010	0111	1110_2
6	0	5	A	7	E_{16}

به یک عدد ۴ بیتی **نیم‌بایت**[۱۲] گفته می‌شود. بنابراین هر رقم شانزده‌شانزدهی با یک نیم‌بایت متناظر است، و چون هر بایت از دو نیم‌بایت تشکیل می‌شود، می‌توان هر بایت را با یک عدد شانزده‌شانزدهی دورقمی نمایش داد. مقدار یک بایت در دودویی از 0 تا 11111111، در شانزده‌شانزدهی از 0 تا FF و در دهدهی از ۰ تا ۲۵۵ است.

۲.۱ سازمان رایانه

۱.۲.۱ حافظه

واحد پایهٔ حافظه[۱۳] بایت است. برای نمونه، رایانه‌ای با ۳۲ مگابایت حافظه می‌تواند تقریباً ۳۲ میلیون بایت اطلاعات را در خود نگه دارد. همان‌طور که شکل ۴.۱ نشان می‌دهد، هر بایت در حافظه با عددی یکتا مشخص می‌شود که آن را **آدرس**[۱۴] آن بایت می‌نامند.

حافظه را با واحدهای کیلوبایت ($2^{10} = 1,024$ بایت)، مگابایت ($2^{20} = 1,048,576$ بایت) و گیگابایت ($2^{30} = 1,073,741,824$ بایت) اندازه می‌گیرند.

حافظه اغلب در تکه‌هایی بزرگ‌تر از یک بایت به کار می‌رود. در معماری رایانه‌های شخصی (PC)، به این بخش‌های بزرگ‌تر حافظه نام‌هایی مانند کلمه[۱۵]،

[۱۱] اگر روشن نیست که چرا نقطهٔ شروع اهمیت دارد، یک بار مثال را از سمت چپ تبدیل کنید.
[12] nibble
[13] memory
[14] address
[15] word

آدرس	0	1	2	3	4	5	6	7
حافظه	2A	45	B8	20	8F	CD	12	2E

شکل ۴.۱: آدرس‌های حافظه

کلمه	۲ بایت
کلمهٔ مضاعف	۴ بایت
کلمهٔ چهارگانه	۸ بایت
پاراگراف	۱۶ بایت

جدول ۲.۱: واحدهای حافظه

کلمهٔ مضاعف[۱۶]، کلمهٔ چهارگانه[۱۷] و پاراگراف[۱۸] داده شده است که اندازهٔ هر یک را در جدول ۲.۱ می‌بینید.

همهٔ داده‌های درون حافظه عددی هستند. نویسه‌ها نیز با کمک یک **کد نویسه**[۱۹] ذخیره می‌شوند که عددها را به نویسه‌ها نگاشت می‌کند. یکی از رایج‌ترین کدهای نویسه *ASCII* (American Standard Code for Information Interchange) نام دارد. کد جدیدتر و کامل‌تری که اکنون در حال جایگزین شدن به جای ASCII است، یونیکد (Unicode) است. یکی از تفاوت‌های اصلی این دو کد آن است که ASCII برای رمزگذاری هر نویسه یک بایت به کار می‌برد، اما یونیکد از چند بایت استفاده می‌کند. یونیکد چند شکل گوناگون دارد؛ در کامپایلرهای[۲۰] C/C++ برای x86، یونیکد در کد با نوع `wchar_t` و رمزگذاری UTF-16 نمایش داده می‌شود که برای هر نویسه ۱۶ بیت (یعنی یک **کلمه**) به کار می‌برد. برای نمونه، ASCII بایت 41_{16} (65_{10}) را به حرف بزرگ A نگاشت می‌کند، در حالی که UTF-16 آن را به کلمهٔ 0041_{16} نگاشت می‌کند. از آنجا که ASCII تنها یک بایت به کار می‌برد، به ۲۵۶ نویسهٔ متفاوت محدود است[۲۱]. یونیکد مقادیر ASCII را گسترش می‌دهد و امکان نمایش نویسه‌های بسیار بیشتری را فراهم می‌کند، و این ویژگی برای نمایش نویسه‌های همهٔ

[16] double word
[17] quad word
[18] paragraph
[19] character code
[20] compiler
[۲۱] در واقع ASCII تنها از ۷ بیت پایینی استفاده می‌کند و در نتیجه فقط ۱۲۸ مقدار متفاوت در اختیار دارد.

زبان‌های جهان اهمیت زیادی دارد.

۲.۲.۱ پردازنده

واحد پردازش مرکزی[۲۲]، یا به اختصار پردازنده، قطعه‌ای فیزیکی است که دستورالعمل‌ها[۲۳] را اجرا می‌کند. دستورالعمل‌هایی که پردازنده‌ها اجرا می‌کنند معمولاً بسیار ساده هستند. گاهی لازم است داده‌هایی که یک دستورالعمل روی آن‌ها کار می‌کند در مکان‌های ذخیره‌سازی ویژه‌ای درون خود پردازنده قرار داشته باشند که **ثبات**[۲۴] نامیده می‌شوند. پردازنده به داده‌های درون ثبات‌ها بسیار سریع‌تر از داده‌های درون حافظه دسترسی پیدا می‌کند، اما تعداد ثبات‌های هر پردازنده محدود است و از این رو برنامه‌نویس باید مراقب باشد که تنها داده‌هایی را در ثبات‌ها نگه دارد که در همان لحظه به آن‌ها نیاز دارد.

مجموعهٔ دستورالعمل‌هایی که هر نوع پردازنده اجرا می‌کند، **زبان ماشین**[۲۵] آن پردازنده را تشکیل می‌دهد. ساختار برنامه‌های ماشین بسیار ابتدایی‌تر از زبان‌های سطح بالاتر[۲۶] است. دستورالعمل‌های زبان ماشین به صورت عددهای خام رمزگذاری می‌شوند، نه در قالب متنی خوانا و آشنا. پردازنده برای آنکه کارآمد باشد، باید بتواند هدف هر دستورالعمل را بسیار سریع رمزگشایی کند، و زبان ماشین نیز با همین هدف طراحی شده است، نه برای آنکه انسان به‌آسانی از آن سر درآورد. برنامه‌هایی که به زبان‌های دیگر نوشته می‌شوند، برای اجرا روی رایانه باید به زبان ماشین بومی پردازنده تبدیل شوند. **کامپایلر** برنامه‌ای است که برنامه‌های نوشته‌شده به یک زبان برنامه‌نویسی را به زبان ماشین یک معماری رایانه‌ای خاص ترجمه می‌کند. به طور کلی، هر نوع پردازنده زبان ماشین منحصربه‌فرد خود را دارد، و این یکی از دلایلی است که برنامه‌های نوشته‌شده برای مک روی رایانه‌های شخصی از نوع IBM اجرا نمی‌شوند.

رایانه‌ها برای هماهنگ کردن اجرای دستورالعمل‌ها از یک **ساعت**[۲۷] بهره می‌گیرند. این ساعت با بسامدی ثابت تپ می‌زند که به آن **سرعت ساعت**[۲۸] گفته می‌شود. وقتی رایانه‌ای ۱/۵ گیگاهرتزی می‌خرید، ۱/۵ گیگاهرتز همان بسامد این ساعت است[۲۹].

GHz کوتاه‌شدهٔ گیگاهرتز، یعنی یک میلیارد چرخه در ثانیه، است. یک پردازندهٔ ۱/۵ گیگاهرتزی در هر ثانیه ۱/۵ میلیارد تپ ساعت دارد.

[22] Central Processing Unit (CPU)
[23] instruction
[24] register
[25] machine language
[26] high-level language
[27] clock
[28] clock speed
[۲۹] در واقع، تپ‌های ساعت در بسیاری از قطعه‌های گوناگون رایانه به کار می‌روند و قطعه‌های دیگر اغلب با سرعت ساعتی متفاوت با پردازنده کار می‌کنند.

این ساعت دقیقه و ثانیه را نگه نمی‌دارد، بلکه تنها با آهنگی ثابت می‌تپد. مدارهای الکترونیکی پردازنده از این تپ‌ها کمک می‌گیرند تا عملیات خود را درست انجام دهند، درست همان‌گونه که ضربه‌های مترونوم به نوازنده کمک می‌کند موسیقی را با ریتم درست بنوازد. تعداد تپ‌هایی (یا به اصطلاح رایج، **چرخه‌هایی**[۳۰]) که یک دستورالعمل نیاز دارد، به نسل و مدل پردازنده بستگی دارد. افزون بر این، تعداد چرخه‌ها به دستورالعمل‌های پیش از آن و عوامل دیگری نیز وابسته است.

۳.۲.۱ خانوادهٔ پردازنده‌های 80x86

رایانه‌های شخصی از نوع IBM پردازنده‌ای از خانوادهٔ 80x86 اینتل (یا پردازنده‌ای سازگار که از روی یکی از آن‌ها ساخته شده است) دارند. پردازنده‌های این خانواده ویژگی‌های مشترکی دارند که یک زبان ماشین پایه از جملهٔ آن‌هاست، اما اعضای جدیدتر این ویژگی‌ها را بسیار گسترش داده‌اند.

8088، 8086: این دو پردازنده از دیدگاه برنامه‌نویسی یکسان هستند و در نخستین رایانه‌های شخصی به کار می‌رفتند. آن‌ها چند ثبات ۱۶ بیتی در اختیار برنامه‌نویس می‌گذارند: AX، BX، CX، DX، SI، DI، BP، SP، CS، DS، SS، ES، IP و FLAGS. این پردازنده‌ها حداکثر از یک مگابایت حافظه پشتیبانی می‌کنند و تنها در **حالت حقیقی**[۳۱] کار می‌کنند. در این حالت، برنامه می‌تواند به هر آدرسی از حافظه، حتی حافظهٔ برنامه‌های دیگر، دسترسی داشته باشد! همین موضوع اشکال‌زدایی[۳۲] و تأمین امنیت را بسیار دشوار می‌کند! افزون بر این، حافظهٔ برنامه باید به **قطعه‌هایی**[۳۳] تقسیم شود که هیچ‌کدام نمی‌توانند بزرگ‌تر از 64K باشند.

80286: این پردازنده در رایانه‌های شخصی کلاس AT به کار رفت. 80286 چند دستورالعمل تازه به زبان ماشین پایهٔ 8088/86 می‌افزاید، اما ویژگی تازهٔ اصلی آن **حالت حفاظت‌شدهٔ ۱۶ بیتی**[۳۴] است. پردازنده در این حالت می‌تواند به حداکثر ۱۶ مگابایت حافظه دسترسی داشته باشد و از دسترسی برنامه‌ها به حافظهٔ یکدیگر جلوگیری کند. با این حال، برنامه‌ها همچنان به قطعه‌هایی تقسیم می‌شوند که نمی‌توانند بزرگ‌تر از 64K باشند.

---

[30] cycle
[31] real mode
[32] debugging
[33] segment
[34] 16-bit protected mode

80386: این پردازنده 80286 را بسیار بهبود داد. نخست آنکه بسیاری از ثبات‌ها را گسترش داد تا ۳۲ بیت را در خود جای دهند (EAX، EBX، ECX، EDX، ESI، EDI، EBP، ESP و EIP) و دو ثبات ۱۶ بیتی تازه به نام‌های FS و GS نیز افزود. همچنین یک **حالت حفاظت‌شدهٔ ۳۲ بیتی** تازه به آن اضافه شد که پردازنده در آن می‌تواند به حداکثر ۴ گیگابایت حافظه دسترسی داشته باشد. در این حالت نیز برنامه‌ها به قطعه‌هایی تقسیم می‌شوند، اما اکنون اندازهٔ هر قطعه نیز می‌تواند تا ۴ گیگابایت باشد!

80486، پنتیوم و پنتیوم پرو: این اعضای خانوادهٔ 80x86 ویژگی تازهٔ چندانی نمی‌افزایند و بیشتر، اجرای دستورالعمل‌ها را سریع‌تر می‌کنند.

پنتیوم MMX: این پردازنده دستورالعمل‌های MMX را به پنتیوم می‌افزاید. نام MMX از عبارت MultiMedia eXtensions (افزونه‌های چندرسانه‌ای) گرفته شده است و این دستورالعمل‌ها می‌توانند عملیات گرافیکی رایج را سریع‌تر کنند.

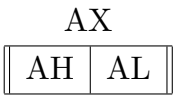
پنتیوم II: این پردازنده همان پنتیوم پرو است که دستورالعمل‌های MMX به آن افزوده شده است. (پنتیوم III نیز در اصل تنها نسخهٔ سریع‌تری از پنتیوم II است.)

۴.۲.۱ ثبات‌های ۱۶ بیتی 8086

پردازندهٔ اولیهٔ 8086 چهار ثبات ۱۶ بیتی همه‌منظوره[۳۵] در اختیار می‌گذاشت: AX، BX، CX و DX. هر یک از این ثبات‌ها را می‌شد به دو ثبات ۸ بیتی تجزیه کرد؛ برای نمونه، همان‌طور که شکل ۵.۱ نشان می‌دهد، ثبات AX به دو ثبات AH و AL تجزیه می‌شود. ثبات AH هشت بیت بالایی (یا پرارزش) AX را در خود دارد و AL هشت بیت پایینی آن را. AH و AL اغلب به‌عنوان ثبات‌هایی مستقل و یک‌بایتی به کار می‌روند، اما باید توجه داشت که این دو از AX مستقل نیستند؛ تغییر مقدار AX، مقدار AH و AL را نیز تغییر می‌دهد و **برعکس**. ثبات‌های همه‌منظوره در بسیاری از دستورالعمل‌های انتقال داده و دستورالعمل‌های حسابی به کار می‌روند.

دو ثبات اندیس[۳۶] ۱۶ بیتی نیز وجود دارد: SI و DI. این دو ثبات اغلب به‌عنوان اشاره‌گر[۳۷] به کار می‌روند، اما برای بسیاری از کاربردهای ثبات‌های همه‌منظوره نیز می‌توان از آن‌ها استفاده کرد. با این حال، نمی‌توان آن‌ها را به ثبات‌های ۸ بیتی تجزیه کرد.

[35] general purpose register
[36] index register
[37] pointer



شکل ۵.۱: ثبات AX

ثبات‌های ۱۶ بیتی BP و SP برای اشاره به داده‌های درون پشتهٔ[۳۸] زبان ماشین به کار می‌روند و به ترتیب اشاره‌گر پایه[۳۹] و اشاره‌گر پشته[۴۰] نام دارند. دربارهٔ این دو ثبات بعداً صحبت خواهیم کرد.

ثبات‌های ۱۶ بیتی CS، DS، SS و ES **ثبات‌های قطعه**[۴۱] هستند. این ثبات‌ها مشخص می‌کنند که هر یک از قسمت‌های برنامه از کدام بخش حافظه استفاده می‌کند. CS کوتاه‌شدهٔ Code Segment (قطعهٔ کد)، DS کوتاه‌شدهٔ Data Segment (قطعهٔ داده)، SS کوتاه‌شدهٔ Stack Segment (قطعهٔ پشته) و ES کوتاه‌شدهٔ Extra Segment (قطعهٔ اضافی) است. از ES به‌عنوان یک ثبات قطعهٔ موقت استفاده می‌شود. جزئیات این ثبات‌ها را در بخش‌های ۶.۲.۱ و ۷.۲.۱ خواهید دید.

ثبات اشاره‌گر دستورالعمل[۴۲] (IP) همراه با ثبات CS به کار می‌رود تا آدرس دستورالعمل بعدی که پردازنده باید اجرا کند، همواره مشخص باشد. معمولاً با اجرای هر دستورالعمل، مقدار IP جلو می‌رود تا به دستورالعمل بعدی در حافظه اشاره کند.

ثبات FLAGS اطلاعات مهمی دربارهٔ نتایج دستورالعمل قبلی نگه می‌دارد. این نتایج به صورت بیت‌هایی جداگانه در این ثبات ذخیره می‌شوند؛ برای نمونه، اگر نتیجهٔ دستورالعمل قبلی صفر باشد، بیت Z برابر ۱ و در غیر این صورت برابر ۰ است. همهٔ دستورالعمل‌ها بیت‌های FLAGS را تغییر نمی‌دهند؛ برای اینکه ببینید هر دستورالعمل چه اثری بر ثبات FLAGS دارد، به جدول پیوست مراجعه کنید.

۵.۲.۱ ثبات‌های ۳۲ بیتی 80386

پردازندهٔ 80386 و پردازنده‌های پس از آن ثبات‌های گسترش‌یافته دارند. برای نمونه، ثبات ۱۶ بیتی AX به ۳۲ بیت گسترش یافته است. برای حفظ سازگاری با نسل‌های پیشین، AX همچنان به ثبات ۱۶ بیتی اشاره دارد و برای اشاره به ثبات

[38] stack
[39] Base Pointer
[40] Stack Pointer
[41] segment registers
[42] Instruction Pointer

گسترش‌یافتهٔ ۳۲ بیتی از نام EAX استفاده می‌شود. AX همان ۱۶ بیت پایینی EAX است، درست همان‌طور که AL هشت بیت پایینی AX (و EAX) است. هیچ راهی برای دسترسی مستقیم به ۱۶ بیت بالایی EAX وجود ندارد. ثبات‌های گسترش‌یافتهٔ دیگر نیز EBX، ECX، EDX، ESI و EDI هستند.

بسیاری از ثبات‌های دیگر نیز گسترش یافته‌اند: BP به EBP، SP به ESP، FLAGS به EFLAGS و IP به EIP تبدیل شده است. با این حال، برخلاف ثبات‌های اندیس و همه‌منظوره، در حالت حفاظت‌شدهٔ ۳۲ بیتی (که در ادامه به آن می‌پردازیم) تنها نسخه‌های گسترش‌یافتهٔ این ثبات‌ها به کار می‌روند.

ثبات‌های قطعه در 80386 همچنان ۱۶ بیتی هستند. دو ثبات قطعهٔ تازه به نام‌های FS و GS نیز اضافه شده‌اند که نامشان کوتاه‌شدهٔ هیچ عبارتی نیست. این دو، مانند ES، ثبات‌های قطعهٔ موقت اضافی هستند.

یکی از تعریف‌های اصطلاح **کلمه** به اندازهٔ ثبات‌های دادهٔ پردازنده اشاره دارد. در خانوادهٔ 80x86، این اصطلاح اکنون کمی گیج‌کننده شده است. در جدول ۲.۱ دیدیم که **کلمه** برابر ۲ بایت (یا ۱۶ بیت) تعریف شده است. این معنا زمانی به آن داده شد که 8086 برای نخستین بار عرضه شد، و هنگام طراحی 80386 تصمیم گرفتند تعریف **کلمه** را بدون تغییر نگه دارند، هرچند اندازهٔ ثبات‌ها تغییر کرده بود.

۶.۲.۱ حالت حقیقی

پس محدودیت بدنام ۶۴۰ کیلوبایتی داس از کجا آمد؟ BIOS بخشی از این یک مگابایت را برای کد خود و برای دستگاه‌های سخت‌افزاری مانند صفحهٔ نمایش ویدئویی لازم داشت.

در حالت حقیقی، حافظه تنها به یک مگابایت (2^{20} بایت) محدود است و آدرس‌های معتبر (در مبنای شانزده) از 00000 تا FFFFF را در بر می‌گیرند. این آدرس‌ها به عددی ۲۰ بیتی نیاز دارند و روشن است که عدد ۲۰ بیتی در هیچ‌یک از ثبات‌های ۱۶ بیتی 8086 جا نمی‌گیرد. اینتل این مشکل را با به کار بردن دو مقدار ۱۶ بیتی برای تعیین هر آدرس حل کرد. مقدار ۱۶ بیتی نخست **انتخاب‌گر**[۴۳] نام دارد و باید در ثبات‌های قطعه ذخیره شود. مقدار ۱۶ بیتی دوم نیز **آفست**[۴۴] نامیده می‌شود. آدرس فیزیکی‌ای که یک زوج ۳۲ بیتی **انتخاب‌گر:آفست** به آن اشاره می‌کند، از این فرمول به دست می‌آید:

$$16 * \text{selector} + \text{offset}$$

ضرب در ۱۶ در مبنای شانزده کار ساده‌ای است و کافی است یک ۰ به سمت راست عدد اضافه کنید. برای نمونه، آدرس فیزیکی‌ای که 047C:0048 به آن اشاره می‌کند، این‌گونه به دست می‌آید:

[43] selector
[44] offset

$$\begin{array}{r} 047C0 \\ +0048 \\ \hline 04808 \end{array}$$

در واقع، مقدار انتخاب‌گر همان شمارهٔ یک پاراگراف است (جدول ۲.۱ را ببینید). آدرس‌های قطعه‌بندی‌شدهٔ حالت حقیقی معایبی دارند:

- هر مقدار انتخاب‌گر تنها می‌تواند به 64K از حافظه اشاره کند (که حد بالای آفست ۱۶ بیتی است). اگر کد یک برنامه بیش از 64K باشد، چه باید کرد؟ در این صورت یک مقدار ثابت در CS برای سراسر اجرای برنامه کافی نیست و برنامه باید به بخش‌هایی کوچک‌تر از 64K (به نام **قطعه**) تقسیم شود. هر بار که اجرا از یک قطعه به قطعهٔ دیگر می‌رود، مقدار CS باید تغییر کند. مشکلات مشابهی نیز هنگام کار با حجم زیادی از داده و ثبات DS پیش می‌آید. این وضعیت می‌تواند بسیار دست‌وپاگیر باشد!
- هر بایت در حافظه آدرس قطعه‌بندی‌شدهٔ یکتایی ندارد. برای نمونه، به آدرس فیزیکی 04808 می‌توان با 047C:0048، 047D:0038، 047E:0028 یا 047B:0058 اشاره کرد. این موضوع می‌تواند مقایسهٔ آدرس‌های قطعه‌بندی‌شده را پیچیده کند.

۷.۲.۱ حالت حفاظت‌شدهٔ ۱۶ بیتی

در حالت حفاظت‌شدهٔ ۱۶ بیتی 80286، مقادیر انتخاب‌گر به شکلی کاملاً متفاوت با حالت حقیقی تفسیر می‌شوند. در حالت حقیقی، مقدار انتخاب‌گر شمارهٔ یک پاراگراف از حافظهٔ فیزیکی است، اما در حالت حفاظت‌شده، مقدار انتخاب‌گر یک **اندیس**[۴۵] در یک **جدول توصیف‌گر**[۴۶] است. در هر دو حالت، برنامه‌ها به قطعه‌هایی تقسیم می‌شوند. در حالت حقیقی، این قطعه‌ها در جایگاه‌های ثابتی از حافظهٔ فیزیکی قرار دارند و مقدار انتخاب‌گر، شمارهٔ پاراگرافی را نشان می‌دهد که قطعه از آن آغاز می‌شود. اما در حالت حفاظت‌شده، قطعه‌ها جایگاه ثابتی در حافظهٔ فیزیکی ندارند و در واقع حتی لازم نیست اصلاً در حافظه باشند!

حالت حفاظت‌شده از روشی به نام **حافظهٔ مجازی**[۴۷] بهره می‌گیرد. ایدهٔ اصلی یک سیستم حافظهٔ مجازی این است که تنها داده‌ها و کدی در حافظه نگه داشته شوند

[45] index
[46] descriptor table
[47] virtual memory

که برنامه‌ها در همان زمان از آن‌ها استفاده می‌کنند، و داده‌ها و کدهای دیگر تا زمانی که دوباره به آن‌ها نیاز پیدا شود، موقتاً روی دیسک بمانند. در حالت حفاظت‌شدهٔ ۱۶ بیتی، قطعه‌ها بر حسب نیاز میان حافظه و دیسک جابه‌جا می‌شوند. هنگامی که یک قطعه از دیسک به حافظه بازگردانده می‌شود، به احتمال زیاد در ناحیه‌ای از حافظه قرار می‌گیرد که با جای پیشین آن، پیش از انتقال به دیسک، متفاوت است. همهٔ این کارها را سیستم‌عامل[۴۸] به‌طور شفاف، یعنی بی‌آنکه برنامه متوجه شود، انجام می‌دهد و برای کار کردن حافظهٔ مجازی لازم نیست برنامه به شکل دیگری نوشته شود.

در حالت حفاظت‌شده، برای هر قطعه یک مدخل در جدول توصیف‌گر در نظر گرفته می‌شود. این مدخل همهٔ اطلاعاتی را که سیستم دربارهٔ آن قطعه باید بداند در خود دارد؛ از جمله اینکه آیا قطعه در حال حاضر در حافظه است یا نه، اگر در حافظه است در کجای آن قرار دارد، و مجوزهای دسترسی به آن (برای نمونه، فقط‌خواندنی بودن) چیست. اندیس مدخل هر قطعه همان مقدار انتخاب‌گری است که در ثبات‌های قطعه ذخیره می‌شود.

ستون‌نویس سرشناسی در حوزهٔ رایانه‌های شخصی، پردازندهٔ 286 را «مرگ مغزی» خوانده بود.

یکی از معایب بزرگ حالت حفاظت‌شدهٔ ۱۶ بیتی این است که آفست‌ها همچنان مقادیری ۱۶ بیتی هستند. در نتیجه، اندازهٔ قطعه‌ها همچنان حداکثر به 64K محدود است و همین موضوع کار با آرایه‌های[۴۹] بزرگ را دشوار می‌کند!

۸.۲.۱ حالت حفاظت‌شدهٔ ۳۲ بیتی

پردازندهٔ 80386 حالت حفاظت‌شدهٔ ۳۲ بیتی را معرفی کرد. حالت حفاظت‌شدهٔ ۳۲ بیتی 386 دو تفاوت عمده با حالت حفاظت‌شدهٔ ۱۶ بیتی 286 دارد:

۱. آفست‌ها به ۳۲ بیت گسترش یافته‌اند. به این ترتیب، مقدار آفست می‌تواند تا ۴ میلیارد برسد و در نتیجه اندازهٔ قطعه‌ها می‌تواند تا ۴ گیگابایت باشد.

۲. قطعه‌ها را می‌توان به واحدهای کوچک‌تری به اندازهٔ 4K تقسیم کرد که **صفحه**[۵۰] نامیده می‌شوند. اکنون سیستم حافظهٔ مجازی به جای قطعه‌ها با صفحه‌ها کار می‌کند، و این یعنی در هر لحظه ممکن است تنها بخش‌هایی از یک قطعه در حافظه باشد. در حالت ۱۶ بیتی 286، یا کل قطعه در حافظه است یا هیچ بخشی از آن، و این روش برای قطعه‌های بزرگ‌تری که حالت ۳۲ بیتی امکان‌پذیر می‌کند، عملی نیست.

[48] operating system
[49] array
[50] page

در ویندوز 3.x، به حالت حفاظت‌شدهٔ ۱۶ بیتی 286 **حالت استاندارد**[۵۱] و به حالت ۳۲ بیتی **حالت پیشرفته**[۵۲] گفته می‌شد. ویندوز 9X، ویندوز NT/2000/XP، OS/2 و لینوکس همگی در حالت حفاظت‌شدهٔ ۳۲ بیتی صفحه‌بندی‌شده اجرا می‌شوند.

۹.۲.۱ وقفه‌ها

گاهی لازم است روند عادی اجرای یک برنامه قطع شود تا رویدادهایی که به واکنش سریع نیاز دارند، پردازش شوند. سخت‌افزار رایانه برای رسیدگی به این رویدادها سازوکاری به نام **وقفه**[۵۳] فراهم می‌کند. برای نمونه، هنگامی که ماوس حرکت می‌کند، سخت‌افزار ماوس برنامهٔ در حال اجرا را قطع می‌کند تا به حرکت ماوس رسیدگی شود (مثلاً مکان‌نمای ماوس جابه‌جا شود و کارهایی از این دست). وقفه‌ها باعث می‌شوند کنترل به یک **روال سرویس وقفه**[۵۴] سپرده شود که وظیفهٔ پردازش وقفه را بر عهده دارد. به هر نوع وقفه یک عدد صحیح نسبت داده می‌شود. در ابتدای حافظهٔ فیزیکی، جدولی از **بردارهای وقفه**[۵۵] قرار دارد که آدرس‌های قطعه‌بندی‌شدهٔ روال‌های سرویس وقفه را در خود نگه می‌دارد، و شمارهٔ هر وقفه در اصل یک اندیس در همین جدول است.

وقفه‌های خارجی از بیرون پردازنده ایجاد می‌شوند (وقفهٔ ماوس نمونه‌ای از این نوع است). بسیاری از دستگاه‌های ورودی و خروجی (I/O) وقفه ایجاد می‌کنند، مانند صفحه‌کلید، زمان‌سنج، دیسک‌گردان‌ها، CD-ROM و کارت‌های صوتی. وقفه‌های داخلی از درون خود پردازنده ایجاد می‌شوند و علت آن‌ها یا یک خطاست یا دستورالعمل وقفه. به وقفه‌های ناشی از خطا **تله**[۵۶] نیز گفته می‌شود، و وقفه‌هایی را که با دستورالعمل وقفه ایجاد می‌شوند **وقفه‌های نرم‌افزاری**[۵۷] می‌نامند. داس از این نوع وقفه‌ها برای پیاده‌سازی API (واسط برنامه‌نویسی کاربردی[۵۸]) خود استفاده می‌کند، اما سیستم‌عامل‌های جدیدتر (مانند ویندوز و یونیکس) از واسطی مبتنی بر زبان C بهره می‌گیرند[۵۹].

بسیاری از روال‌های سرویس وقفه پس از پایان کار، کنترل را به برنامهٔ قطع‌شده

---

[51] standard mode
[52] enhanced mode
[53] interrupt
[54] interrupt handler
[55] interrupt vectors
[56] trap
[57] software interrupts
[58] Application Programming Interface
[۵۹] البته ممکن است این سیستم‌عامل‌ها در سطح هسته از واسطی سطح پایین‌تر استفاده کنند.

بازمی‌گردانند. این روال‌ها همهٔ ثبات‌ها را به همان مقادیری که پیش از رخ دادن وقفه داشتند برمی‌گردانند، و به این ترتیب برنامهٔ قطع‌شده چنان به اجرای خود ادامه می‌دهد که گویی هیچ اتفاقی نیفتاده است (جز اینکه چند چرخهٔ پردازنده را از دست داده است). اما تله‌ها معمولاً بازنمی‌گردند و اغلب به اجرای برنامه پایان می‌دهند.

۳.۱ زبان اسمبلی

۱.۳.۱ زبان ماشین

هر نوع پردازنده زبان ماشین[۶۰] ویژهٔ خود را می‌فهمد. دستورالعمل‌های[۶۱] زبان ماشین در واقع عددهایی هستند که به شکل بایت در حافظه ذخیره می‌شوند و هر دستورالعمل کد عددی یکتای خود را دارد که **کد عملیاتی**[۶۲] نامیده می‌شود. اندازهٔ دستورالعمل‌های پردازنده‌های 80x86 یکسان نیست، اما کد عملیاتی همیشه در ابتدای دستورالعمل قرار می‌گیرد. بسیاری از دستورالعمل‌ها افزون بر کد عملیاتی، داده‌هایی (برای نمونه ثابت‌ها یا آدرس‌ها) را نیز در خود دارند که هنگام اجرا از آن‌ها استفاده می‌کنند.

برنامه‌نویسی مستقیم به زبان ماشین بسیار دشوار است، زیرا رمزگشایی از معنای دستورالعمل‌هایی که با عدد کدگذاری شده‌اند برای انسان کاری خسته‌کننده است. برای نمونه، دستورالعملی که محتوای ثبات‌های[۶۳] EAX و EBX را با هم جمع می‌کند و حاصل را دوباره در EAX می‌گذارد، با کدهای شانزده‌شانزدهی زیر نمایش داده می‌شود:

```
03 C3
```

روشن است که معنای این کد به هیچ وجه آشکار نیست. خوشبختانه برنامه‌ای به نام **اسمبلر**[۶۴] می‌تواند این کار طاقت‌فرسا را به جای برنامه‌نویس انجام دهد.

۲.۳.۱ زبان اسمبلی

برنامه‌ای که به زبان اسمبلی[۶۵] نوشته می‌شود، درست مانند برنامه‌های زبان‌های سطح بالاتر، به صورت متن ذخیره می‌شود و هر دستورالعمل اسمبلی دقیقاً معادل یک

[60] machine language
[61] instruction
[62] operation code (opcode)
[63] register
[64] assembler
[65] assembly language

دستورالعمل ماشین است. برای نمونه، دستورالعمل جمعی که در بالا توضیح داده شد، در زبان اسمبلی به این شکل نوشته می‌شود:

```
add eax, ebx
```

می‌بینید که معنای دستورالعمل در این حالت **بسیار** روشن‌تر از کد ماشین است. واژهٔ add **یادیار**[۶۶] دستورالعمل جمع است. شکل کلی یک دستورالعمل اسمبلی چنین است:

```
mnemonic operand(s)
```

اسمبلر برنامه‌ای است که فایلی متنی شامل دستورالعمل‌های اسمبلی را می‌خواند و آن را به کد ماشین تبدیل می‌کند. **کامپایلرها**[۶۷] نیز برنامه‌هایی هستند که تبدیل مشابهی را برای زبان‌های برنامه‌نویسی سطح بالا[۶۸] انجام می‌دهند، با این تفاوت که اسمبلر بسیار ساده‌تر از کامپایلر است. علت این است که هر عبارت زبان اسمبلی مستقیماً یک دستورالعمل ماشین را نمایش می‌دهد، در حالی که عبارت‌های زبان‌های سطح بالا **بسیار** پیچیده‌تر هستند و ممکن است به دستورالعمل‌های ماشین فراوانی نیاز داشته باشند.

چند سال طول کشید تا دانشمندان علوم رایانه حتی بفهمند که چگونه باید یک کامپایلر نوشت!

تفاوت مهم دیگر زبان اسمبلی با زبان‌های سطح بالا این است که چون هر نوع پردازنده زبان ماشین ویژهٔ خود را دارد، زبان اسمبلی ویژهٔ خود را نیز دارد. از این رو، انتقال برنامه‌های اسمبلی از یک معماری رایانه به معماری دیگر **بسیار** دشوارتر از انتقال برنامه‌های نوشته‌شده به زبان‌های سطح بالا است.

مثال‌های این کتاب از اسمبلر Netwide Assembler یا به اختصار NASM استفاده می‌کنند. این اسمبلر را می‌توان به رایگان از اینترنت دریافت کرد (نشانی آن در پیشگفتار آمده است). اسمبلرهای رایج‌تر، اسمبلر مایکروسافت (MASM) و اسمبلر بورلند (TASM) هستند. نحو اسمبلی در MASM/TASM با نحو NASM تفاوت‌هایی دارد.

۳.۳.۱ عملگرهای دستورالعمل

تعداد و نوع عملگرهای[۶۹] دستورالعمل‌های کد ماشین متفاوت است، اما به طور کلی هر دستورالعمل تعداد ثابتی عملگر (از صفر تا سه) دارد. عملگرها می‌توانند یکی از نوع‌های زیر باشند:

[66] mnemonic
[67] compiler
[68] high-level language
[69] operand

ثبات: این عملگرها مستقیماً به محتوای ثبات‌های پردازنده اشاره می‌کنند.

حافظه: این عملگرها به داده‌ای در حافظه[۷۰] اشاره می‌کنند. آدرس[۷۱] این داده ممکن است ثابتی باشد که مستقیماً در خود دستورالعمل نوشته شده است، یا با استفاده از مقادیر ثبات‌ها محاسبه شود. آدرس‌ها همیشه آفست‌هایی[۷۲] از ابتدای یک قطعه[۷۳] هستند.

بی‌واسطه: مقادیر این عملگرها[۷۴] ثابت است و در خود دستورالعمل فهرست می‌شوند. به بیان دیگر، این مقادیر در خود دستورالعمل (یعنی در قطعهٔ کد) ذخیره می‌شوند، نه در قطعهٔ داده.

ضمنی: این عملگرها به طور صریح نشان داده نمی‌شوند. برای نمونه، دستورالعمل افزایش، یک واحد به یک ثبات یا خانهٔ حافظه می‌افزاید و این عدد یک به صورت ضمنی در دستورالعمل وجود دارد.

۴.۳.۱ دستورالعمل‌های پایه

پایه‌ای‌ترین دستورالعمل، دستورالعمل `MOV` است که داده را از جایی به جای دیگر منتقل می‌کند (همانند اپراتور انتساب در زبان‌های سطح بالا). این دستورالعمل دو عملگر می‌گیرد:

```
mov dest, src
```

در اینجا داده‌ای که با src مشخص شده است در $dest$ کپی می‌شود. یکی از محدودیت‌های این دستورالعمل آن است که هر دو عملگر نمی‌توانند هم‌زمان عملگر حافظه باشند. همین نکته یکی دیگر از ویژگی‌های عجیب اسمبلی را نشان می‌دهد: اغلب قاعده‌هایی تا حدی دلبخواهی برای چگونگی استفاده از دستورالعمل‌های گوناگون وجود دارد. افزون بر این، دو عملگر باید هم‌اندازه باشند، بنابراین مقدار AX را نمی‌توان در BL ذخیره کرد.

در اینجا مثالی آورده شده است (هر توضیح[۷۵] با نقطه‌ویرگول آغاز می‌شود):

---

[70] memory
[71] address
[72] offset
[73] segment
[74] immediate operand
[75] comment

```
mov     eax, 3   ; store 3 into EAX register (3 is immediate operand)
mov     bx, ax   ; store the value of AX into the BX register
```

دستورالعمل ADD برای جمع کردن عددهای صحیح به کار می‌رود.

```
add     eax, 4   ; eax = eax + 4
add     al, ah   ; al = al + ah
```

دستورالعمل SUB نیز عددهای صحیح را از هم کم می‌کند.

```
sub     bx, 10   ; bx = bx - 10
sub     ebx, edi ; ebx = ebx - edi
```

دستورالعمل‌های INC و DEC مقدار را یک واحد افزایش یا کاهش می‌دهند. از آنجا که عدد یک در این دستورالعمل‌ها عملگری ضمنی است، کد ماشین INC و DEC کوچک‌تر از کد ماشین دستورالعمل‌های معادل آن‌ها، یعنی ADD و SUB، است.

```
inc     ecx      ; ecx++
dec     dl       ; dl--
```

۵.۳.۱ رهنمودها

رهنمود[۷۶] ساختهٔ اسمبلر است، نه پردازنده. رهنمودها معمولاً یا به اسمبلر دستور می‌دهند کاری را انجام دهد یا چیزی را به اطلاع آن می‌رسانند و به کد ماشین ترجمه نمی‌شوند. کاربردهای رایج رهنمودها از این قرار است:

- تعریف ثابت‌ها
- تعریف حافظه برای ذخیرهٔ داده
- گروه‌بندی حافظه در قطعه‌ها
- گنجاندن شرطی کد منبع
- گنجاندن فایل‌های دیگر

کد NASM، درست مانند زبان C، از یک پیش‌پردازنده[۷۷] عبور می‌کند و بسیاری از فرمان‌های پیش‌پردازندهٔ آن نیز همانند فرمان‌های C هستند. با این حال، رهنمودهای پیش‌پردازندهٔ NASM به جای # که در C به کار می‌رود، با % آغاز می‌شوند.

[76] directive
[77] preprocessor

واحد	حرف
بایت	B
کلمه	W
کلمهٔ مضاعف	D
کلمهٔ چهارگانه	Q
ده بایت	T

جدول ۳.۱: حرف‌های رهنمودهای RESX و DX

رهنمود equ

از رهنمود equ می‌توان برای تعریف یک **نماد**[۷۸] استفاده کرد. نمادها ثابت‌هایی نام‌دار هستند که در برنامهٔ اسمبلی به کار می‌روند و قالب تعریف آن‌ها چنین است:

symbol equ *value*

مقدار نمادها را **نمی‌توان** بعداً دوباره تعریف کرد.

رهنمود %define

این رهنمود همانند رهنمود #define در زبان C است و بیشتر برای تعریف ماکروهای[۷۹] ثابت به کار می‌رود، درست همان‌گونه که در C رایج است.

```
%define SIZE 100
      mov    eax, SIZE
```

کد بالا ماکرویی به نام SIZE تعریف می‌کند و چگونگی استفاده از آن را در یک دستورالعمل MOV نشان می‌دهد. ماکروها از دو جهت انعطاف‌پذیرتر از نمادها هستند: می‌توان آن‌ها را دوباره تعریف کرد و می‌توانند چیزی فراتر از عددهای ثابت ساده باشند.

رهنمودهای داده

رهنمودهای داده در قطعه‌های داده به کار می‌روند تا فضایی از حافظه را تعریف کنند. حافظه را به دو روش می‌توان رزرو کرد: در روش نخست تنها فضایی برای داده تعریف می‌شود، اما در روش دوم افزون بر فضا، مقدار اولیه‌ای نیز برای آن تعیین

[78] symbol
[79] macro

می‌شود. روش نخست از یکی از رهنمودهای RESX استفاده می‌کند که در آن به جای X حرفی قرار می‌گیرد که اندازهٔ شیء (یا اشیایی) را که قرار است ذخیره شود تعیین می‌کند. مقادیر ممکن برای این حرف در جدول ۳.۱ آمده است.

روش دوم، که مقدار اولیه را نیز تعیین می‌کند، از یکی از رهنمودهای DX استفاده می‌کند. حرف‌هایی که به جای X می‌نشینند همان حرف‌های رهنمودهای RESX هستند.

علامت‌گذاری خانه‌های حافظه با **برچسب‌ها**[۸۰] بسیار رایج است، زیرا برچسب‌ها به ما امکان می‌دهند در کد به‌آسانی به خانه‌های حافظه ارجاع دهیم. در زیر چند مثال آمده است:

```
L1    db     0         ; byte labeled L1 with initial value 0
L2    dw     1000      ; word labeled L2 with initial value 1000
L3    db     110101b   ; byte initialized to binary 110101 (53 in decimal)
L4    db     12h       ; byte initialized to hex 12 (18 in decimal)
L5    db     17o       ; byte initialized to octal 17 (15 in decimal)
L6    dd     1A92h     ; double word initialized to hex 1A92
L7    resb   1         ; 1 uninitialized byte
L8    db     "A"       ; byte initialized to ASCII code for A (65)
```

در این تعریف‌ها میان گیومهٔ دوتایی و گیومهٔ تکی تفاوتی وجود ندارد. تعریف‌های دادهٔ پشت سر هم نیز به همان ترتیب در حافظه ذخیره می‌شوند؛ یعنی کلمهٔ L2 بلافاصله پس از L1 در حافظه قرار می‌گیرد. افزون بر این، می‌توان دنباله‌ای از خانه‌های حافظه را نیز یک‌جا تعریف کرد.

```
L9    db     0, 1, 2, 3              ; defines 4 bytes
L10   db     "w", "o", "r", 'd', 0   ; defines a C string = "word"
L11   db     'word', 0               ; same as L10
```

با رهنمود DD می‌توان هم ثابت‌های صحیح و هم ثابت‌های ممیز شناور[۸۱] با دقت ساده[۸۲] را تعریف کرد، اما رهنمود DQ را تنها می‌توان برای تعریف ثابت‌های ممیز شناور با دقت مضاعف به کار برد.

برای دنباله‌های بلند، رهنمود TIMES در NASM اغلب سودمند است. این رهنمود عملگر خود را به تعداد دفعات مشخص تکرار می‌کند. برای نمونه:

```
L12   times 100 db 0                ; equivalent to 100 (db 0)'s
L13   resw   100                    ; reserves room for 100 words
```

[80] label

[81] floating point

[۸۲] عدد ممیز شناور با دقت ساده معادل متغیری از نوع `float` در زبان C است.

به یاد داشته باشید که برچسب‌ها را می‌توان برای ارجاع به داده‌ها در کد به کار برد، و این کار به دو شکل انجام می‌شود. اگر برچسب به تنهایی به کار رود، به عنوان آدرس (یا آفست) داده تفسیر می‌شود، اما اگر درون کروشه ([]) قرار گیرد، به عنوان دادهٔ موجود در آن آدرس تفسیر خواهد شد. به بیان دیگر، بهتر است برچسب را **اشاره‌گری**[۸۳] به داده در نظر بگیرید که کروشه‌ها آن را، درست مانند ستاره در زبان C، به داده‌ای که به آن اشاره می‌کند تبدیل می‌کنند. (MASM/TASM از قرارداد دیگری پیروی می‌کنند.) در حالت ۳۲ بیتی، آدرس‌ها ۳۲ بیتی هستند. چند مثال در زیر آمده است:

```
1   mov    al, [L1]      ; copy byte at L1 into AL
2   mov    eax, L1       ; EAX = address of byte at L1
3   mov    [L1], ah      ; copy AH into byte at L1
4   mov    eax, [L6]     ; copy double word at L6 into EAX
5   add    eax, [L6]     ; EAX = EAX + double word at L6
6   add    [L6], eax     ; double word at L6 += EAX
7   mov    al, [L6]      ; copy first byte of double word at L6 into AL
```

سطر ۷ این مثال‌ها ویژگی مهمی از NASM را نشان می‌دهد: اسمبلر نوع داده‌ای را که یک برچسب به آن اشاره می‌کند **دنبال نمی‌کند** و این برنامه‌نویس است که باید مطمئن شود برچسب را درست به کار می‌برد. در ادامه خواهیم دید که ذخیرهٔ آدرس داده‌ها در ثبات‌ها و استفاده از ثبات مانند یک متغیر اشاره‌گر در C بسیار رایج است. در این حالت نیز هیچ بررسی‌ای انجام نمی‌شود که اشاره‌گر درست به کار رفته باشد. از این رو، احتمال خطا در اسمبلی بسیار بیشتر از حتی زبان C است.

دستورالعمل زیر را در نظر بگیرید:

```
mov    [L6], 1             ; store a 1 at L6
```

اسمبل کردن این عبارت خطای `operation size not specified` را در پی دارد. چرا؟ زیرا اسمبلر نمی‌داند که عدد ۱ را باید به صورت بایت، کلمه یا کلمهٔ مضاعف ذخیره کند. برای رفع این مشکل، یک مشخص‌کنندهٔ اندازه به دستورالعمل بیفزایید:

```
mov    dword [L6], 1       ; store a 1 at L6
```

این دستورالعمل به اسمبلر می‌گوید که عدد ۱ را در کلمهٔ مضاعفی که از L6 آغاز می‌شود ذخیره کند. مشخص‌کننده‌های اندازهٔ دیگر نیز BYTE، WORD، QWORD و

[83] pointer

TWORD[۸۴] هستند.

۶.۳.۱ ورودی و خروجی

ورودی و خروجی از فعالیت‌هایی هستند که به‌شدت به سیستم وابسته هستند، زیرا با سخت‌افزار سیستم سروکار دارند. زبان‌های سطح بالا، مانند C، کتابخانه‌هایی استاندارد از روال‌ها در اختیار برنامه‌نویس می‌گذارند که رابط برنامه‌نویسی ساده و یکنواختی برای ورودی و خروجی فراهم می‌کنند. زبان‌های اسمبلی اما هیچ کتابخانهٔ استانداردی ندارند و برنامه‌های آن‌ها باید یا مستقیماً به سخت‌افزار دسترسی پیدا کنند (که در حالت حفاظت‌شده[۸۵] عملی ممتاز به شمار می‌رود) یا از روال‌های سطح پایینی استفاده کنند که سیستم‌عامل[۸۶] فراهم می‌کند.

پیوند دادن روال‌های اسمبلی با C بسیار رایج است. یکی از مزیت‌های این کار آن است که کد اسمبلی می‌تواند از روال‌های ورودی و خروجی کتابخانهٔ استاندارد C استفاده کند. با این حال، برای این کار باید قاعده‌هایی را که C برای انتقال اطلاعات میان روال‌ها به کار می‌برد بدانیم. این قاعده‌ها پیچیده‌تر از آن هستند که در اینجا بررسی شوند (در ادامهٔ کتاب به آن‌ها خواهیم پرداخت!). از این رو، نویسنده برای ساده کردن ورودی و خروجی، روال‌های ویژهٔ خود را نوشته است که قاعده‌های پیچیدهٔ C را پنهان می‌کنند و رابط بسیار ساده‌تری فراهم می‌آورند. این روال‌ها در جدول ۴.۱ توضیح داده شده‌اند. همهٔ این روال‌ها به جز روال‌های خواندن، مقدار همهٔ ثبات‌ها را حفظ می‌کنند، اما روال‌های خواندن مقدار ثبات EAX را تغییر می‌دهند. برای استفاده از این روال‌ها باید فایلی را در برنامه گنجاند که اطلاعات مورد نیاز اسمبلر برای به‌کارگیری آن‌ها را در خود دارد. برای گنجاندن یک فایل در NASM از رهنمود پیش‌پردازندهٔ `%include` استفاده می‌شود. سطر زیر فایلی را که روال‌های ورودی و خروجی نویسنده به آن نیاز دارند در برنامه می‌گنجاند[۸۷]:

```
%include "asm_io.inc"
```

برای استفاده از یکی از روال‌های چاپ، ابتدا مقدار درست را در EAX بار می‌کنیم و سپس با دستورالعمل CALL آن روال را فرا می‌خوانیم. دستورالعمل CALL معادل

[۸۴] TWORD ناحیه‌ای ده بایتی از حافظه را تعریف می‌کند. کمک‌پردازندهٔ ممیز شناور از این نوع داده استفاده می‌کند.

[85] protected mode

[86] operating system

[۸۷] فایل `asm_io.inc` (و فایل شیء `asm_io` که `asm_io.inc` به آن نیاز دارد) در کدهای نمونهٔ قابل دریافت از صفحهٔ وب این کتاب آموزشی، به نشانی `http://pacman128.github.io/pcasm/` قرار دارند.

print_int	مقدار عدد صحیح ذخیره‌شده در EAX را روی صفحهٔ نمایش چاپ می‌کند.
print_char	نویسه‌ای را که کد ASCII آن در AL ذخیره شده است روی صفحهٔ نمایش چاپ می‌کند.
print_string	محتوای رشته‌ای را که در **آدرس** ذخیره‌شده در EAX قرار دارد روی صفحهٔ نمایش چاپ می‌کند. این رشته باید از نوع رشته‌های C (یعنی پایان‌یافته با نویسهٔ تهی) باشد.
print_nl	یک نویسهٔ سطر جدید روی صفحهٔ نمایش چاپ می‌کند.
read_int	یک عدد صحیح از صفحه‌کلید می‌خواند و آن را در ثبات EAX ذخیره می‌کند.
read_char	یک نویسه از صفحه‌کلید می‌خواند و کد ASCII آن را در ثبات EAX ذخیره می‌کند.

جدول ۴.۱: روال‌های ورودی و خروجی اسمبلی

فراخوانی[88] تابع در زبان‌های سطح بالا است: اجرای برنامه را به بخش دیگری از کد می‌برد و پس از پایان روال، به نقطهٔ آغاز بازمی‌گرداند. برنامهٔ نمونه‌ای که در ادامه آمده است چند نمونه از فراخوانی این روال‌های ورودی و خروجی را نشان می‌دهد.

۷.۳.۱ اشکال‌زدایی

کتابخانهٔ نویسنده روال‌های سودمندی برای اشکال‌زدایی[89] برنامه‌ها نیز دارد. این روال‌ها اطلاعاتی دربارهٔ وضعیت رایانه نمایش می‌دهند، بی‌آنکه آن وضعیت را تغییر دهند. در واقع این روال‌ها **ماکروهایی** هستند که نخست وضعیت کنونی پردازنده را حفظ می‌کنند و سپس یک زیرروال[90] را فرا می‌خوانند. تعریف این ماکروها در همان فایل `asm_io.inc` آمده است که پیش‌تر به آن اشاره شد. ماکروها درست مانند دستورالعمل‌های معمولی به کار می‌روند. عملگرهای ماکروها با ویرگول از هم جدا می‌شوند.

چهار روال اشکال‌زدایی با نام‌های `dump_regs`، `dump_mem`، `dump_stack` و `dump_math` وجود دارد که به ترتیب مقادیر ثبات‌ها، حافظه، پشته و کمک‌پردازندهٔ ریاضی را نمایش می‌دهند.

[88] call
[89] debugging
[90] subroutine

dump_regs این ماکرو مقادیر ثبات‌های رایانه را (به صورت شانزده‌شانزدهی) در stdout (یعنی صفحهٔ نمایش) چاپ می‌کند. همچنین بیت‌هایی را که در ثبات FLAGS[۹۱] یک هستند نمایش می‌دهد؛ برای نمونه، اگر پرچم صفر[۹۲] برابر ۱ باشد، ZF نمایش داده می‌شود و اگر برابر ۰ باشد، نمایش داده نمی‌شود. این ماکرو یک آرگومان[۹۳] صحیح نیز می‌گیرد که آن هم چاپ می‌شود و می‌توان از آن برای تشخیص خروجی فرمان‌های dump_regs گوناگون از یکدیگر استفاده کرد.

dump_mem این ماکرو مقادیر ناحیه‌ای از حافظه را هم به صورت شانزده‌شانزدهی و هم به صورت نویسه‌های ASCII چاپ می‌کند و سه آرگومان می‌گیرد که با ویرگول از هم جدا می‌شوند. آرگومان نخست عددی صحیح است که برای برچسب زدن به خروجی به کار می‌رود (درست مانند آرگومان dump_regs). آرگومان دوم آدرسی است که باید نمایش داده شود (این آدرس می‌تواند یک برچسب باشد) و آرگومان آخر تعداد پاراگراف‌های[۹۴] ۱۶ بایتی است که باید پس از این آدرس نمایش داده شوند. حافظهٔ نمایش‌داده‌شده از نخستین مرز پاراگراف پیش از آدرس درخواستی آغاز می‌شود.

dump_stack این ماکرو مقادیر موجود در پشتهٔ[۹۵] پردازنده را چاپ می‌کند (پشته در فصل ۴ بررسی خواهد شد). پشته به صورت کلمه‌های مضاعف سازمان‌دهی شده است و این روال نیز آن را به همین صورت نمایش می‌دهد. این ماکرو سه آرگومان جداشده با ویرگول می‌گیرد: آرگومان نخست برچسبی صحیح است (همانند dump_regs)، آرگومان دوم تعداد کلمه‌های مضاعفی است که باید **پایین‌تر** از آدرس نگه‌داری‌شده در ثبات EBP نمایش داده شوند و آرگومان سوم تعداد کلمه‌های مضاعفی است که باید **بالاتر** از آدرس موجود در EBP نمایش داده شوند.

dump_math این ماکرو مقادیر ثبات‌های کمک‌پردازندهٔ ریاضی[۹۶] را چاپ می‌کند و یک آرگومان صحیح می‌گیرد که درست مانند آرگومان dump_regs برای برچسب زدن به خروجی به کار می‌رود.

[۹۱] این ثبات در فصل ۲ بررسی می‌شود.

[92] zero flag
[93] argument
[94] paragraph
[95] stack
[96] math coprocessor

```
1 int main()
2 {
3   int ret_status;
4   ret_status = asm_main();
5   return ret_status;
6 }
```

شکل ۶.۱: کد `driver.c`

۴.۱ ساختن یک برنامه

امروزه کمتر پیش می‌آید که برنامه‌ای مستقل را به طور کامل به زبان اسمبلی بنویسند و معمولاً از اسمبلی تنها برای نوشتن برخی روال‌های حساس استفاده می‌شود. چرا؟ زیرا برنامه‌نویسی به یک زبان سطح بالاتر **بسیار** آسان‌تر از برنامه‌نویسی به اسمبلی است. افزون بر این، استفاده از اسمبلی انتقال برنامه به سکوهای دیگر را بسیار دشوار می‌کند. در واقع، به‌ندرت پیش می‌آید که اصلاً از اسمبلی استفاده شود.

پس چرا باید اسمبلی را یاد گرفت؟

۱. گاهی کدی که به زبان اسمبلی نوشته شده است می‌تواند سریع‌تر و کوچک‌تر از کدی باشد که کامپایلر تولید می‌کند.

۲. اسمبلی امکان دسترسی مستقیم به ویژگی‌های سخت‌افزاری سیستم را فراهم می‌کند که استفاده از آن‌ها در یک زبان سطح بالاتر ممکن است دشوار یا حتی ناممکن باشد.

۳. یادگیری برنامه‌نویسی به زبان اسمبلی به درک عمیق‌تری از چگونگی کار رایانه‌ها کمک می‌کند.

۴. یادگیری برنامه‌نویسی به زبان اسمبلی به درک بهتر چگونگی کار کامپایلرها و زبان‌های سطح بالا مانند C کمک می‌کند.

دو نکتهٔ آخر نشان می‌دهند که یادگیری اسمبلی سودمند است، حتی اگر بعداً هرگز به این زبان برنامه ننویسید. در واقع، نویسنده خود به‌ندرت به زبان اسمبلی برنامه می‌نویسد، اما هر روز از ایده‌هایی که از آن آموخته است بهره می‌گیرد.

۱.۴.۱ نخستین برنامه

همهٔ برنامه‌های آغازین این کتاب از برنامهٔ راه‌انداز سادهٔ C که در شکل ۶.۱ آمده است شروع می‌شوند. این برنامه تنها تابع دیگری به نام asm_main را فرا می‌خواند که در واقع روالی است که به زبان اسمبلی نوشته خواهد شد. استفاده از روال راه‌انداز C چند مزیت دارد. نخست آنکه به سیستم C اجازه می‌دهد برنامه را برای اجرای درست در حالت حفاظت‌شده آماده کند؛ همهٔ قطعه‌ها و ثبات‌های قطعهٔ[۹۷] متناظر با آن‌ها را C مقداردهی اولیه می‌کند و کد اسمبلی لازم نیست نگران هیچ‌یک از این کارها باشد. دوم آنکه کتابخانهٔ C نیز در دسترس کد اسمبلی قرار می‌گیرد. روال‌های ورودی و خروجی نویسنده از همین امکان بهره می‌برند و از توابع ورودی و خروجی C (printf و مانند آن) استفاده می‌کنند. در ادامه یک برنامهٔ سادهٔ اسمبلی را می‌بینید.

first.asm

```
1  ; file: first.asm
2  ; First assembly program. This program asks for two integers as
3  ; input and prints out their sum.
4  ;
5  ; To create executable using djgpp:
6  ; nasm -f coff first.asm
7  ; gcc -o first first.o driver.c asm_io.o
8
9  %include "asm_io.inc"
10 ;
11 ; initialized data is put in the .data segment
12 ;
13 segment .data
14 ;
15 ; These labels refer to strings used for output
16 ;
17 prompt1 db    "Enter a number: ", 0       ; don't forget null terminator
18 prompt2 db    "Enter another number: ", 0
19 outmsg1 db    "You entered ", 0
20 outmsg2 db    " and ", 0
21 outmsg3 db    ", the sum of these is ", 0
22
23 ;
24 ; uninitialized data is put in the .bss segment
25 ;
26 segment .bss
27 ;
28 ; These labels refer to double words used to store the inputs
```

[97] segment register

```
29 ;
30 input1  resd 1
31 input2  resd 1
32
33 ;
34 ; code is put in the .text segment
35 ;
36 segment .text
37         global  _asm_main
38 _asm_main:
39         enter   0,0               ; setup routine
40         pusha
41
42         mov     eax, prompt1      ; print out prompt
43         call    print_string
44
45         call    read_int          ; read integer
46         mov     [input1], eax     ; store into input1
47
48         mov     eax, prompt2      ; print out prompt
49         call    print_string
50
51         call    read_int          ; read integer
52         mov     [input2], eax     ; store into input2
53
54         mov     eax, [input1]     ; eax = dword at input1
55         add     eax, [input2]     ; eax += dword at input2
56         mov     ebx, eax          ; ebx = eax
57
58         dump_regs 1                ; print out register values
59         dump_mem  2, outmsg1, 1    ; print out memory
60 ;
61 ; next print out result message as series of steps
62 ;
63         mov     eax, outmsg1
64         call    print_string      ; print out first message
65         mov     eax, [input1]
66         call    print_int         ; print out input1
67         mov     eax, outmsg2
68         call    print_string      ; print out second message
69         mov     eax, [input2]
70         call    print_int         ; print out input2
```

```
71          mov     eax, outmsg3
72          call    print_string      ; print out third message
73          mov     eax, ebx
74          call    print_int         ; print out sum (ebx)
75          call    print_nl          ; print new-line
76
77          popa
78          mov     eax, 0            ; return back to C
79          leave
80          ret
```
first.asm

سطر ۱۳ برنامه بخشی از برنامه را تعریف می‌کند که حافظهٔ مربوط به قطعهٔ داده (با نام `data`.) را مشخص می‌کند. در این قطعه تنها باید داده‌های مقداردهی‌شده تعریف شوند. در سطرهای ۱۷ تا ۲۱ چند رشته اعلان شده‌اند که با کتابخانهٔ C چاپ خواهند شد و از این رو باید با نویسهٔ **تهی** (کد ASCII صفر) پایان یابند. به یاد داشته باشید که میان 0 و '0' تفاوت بزرگی وجود دارد.

داده‌های مقداردهی‌نشده باید در قطعهٔ bss (با نام `bss`. در سطر ۲۶) اعلان شوند. نام این قطعه از یکی از اپراتورهای اسمبلرهای اولیهٔ مبتنی بر یونیکس گرفته شده است که به معنای «بلوکی که با نماد آغاز می‌شود»[۹۸] بود. قطعهٔ پشته نیز وجود دارد که بعداً به آن خواهیم پرداخت.

قطعهٔ کد به دلایل تاریخی `text`. نامیده می‌شود و دستورالعمل‌ها در آن قرار می‌گیرند. توجه کنید که برچسب کد روال اصلی (سطر ۳۸) پیشوندی از جنس زیرخط دارد. این پیشوند بخشی از **قرارداد فراخوانی** C[۹۹] است، قراردادی که قاعده‌هایی را مشخص می‌کند که C هنگام کامپایل کد به کار می‌برد. شناختن این قرارداد برای پیوند دادن C و اسمبلی بسیار مهم است. کل این قرارداد بعداً معرفی خواهد شد، اما فعلاً تنها کافی است بدانید که کامپایلر C به ابتدای همهٔ نمادهای C (یعنی توابع و متغیرهای سراسری[۱۰۰]) یک زیرخط می‌افزاید. (این قاعده به طور خاص مربوط به داس و ویندوز است و کامپایلر C در لینوکس چیزی به ابتدای نام نمادهای C اضافه نمی‌کند.)

رهنمود `global` در سطر ۳۷ به اسمبلر می‌گوید که برچسب `_asm_main` را سراسری کند. برخلاف C، برچسب‌ها به طور پیش‌فرض **دامنهٔ داخلی** دارند، یعنی تنها کدی که در همان پیمانه قرار دارد می‌تواند از آن‌ها استفاده کند. رهنمود `global` به

[98] block started by symbol
[99] C calling convention
[100] global variable

برچسب (یا برچسب‌های) مشخص‌شده **دامنهٔ خارجی** می‌دهد و چنین برچسبی از هر پیمانهٔ برنامه قابل دسترسی است. پیمانهٔ asm_io برچسب‌های print_int و دیگر برچسب‌های مشابه را سراسری اعلان می‌کند و به همین دلیل است که می‌توان آن‌ها را در پیمانهٔ first.asm به کار برد.

۲.۴.۱ وابستگی به کامپایلر

کد اسمبلی بالا ویژهٔ کامپایلر رایگان C/C++ به نام DJGPP است که بر پایهٔ GNU[۱۰۱] ساخته شده است.[۱۰۲] این کامپایلر را می‌توان به رایگان از اینترنت دریافت کرد. DJGPP به رایانهٔ شخصی مبتنی بر پردازندهٔ ۳۸۶ یا بهتر نیاز دارد و در داس، ویندوز 95/98 یا NT اجرا می‌شود. این کامپایلر از فایل‌های شیء[۱۰۳] با قالب COFF (Common Object File Format) استفاده می‌کند. برای اسمبل کردن به این قالب، از گزینهٔ -f coff همراه با nasm استفاده کنید (همان‌گونه که در توضیحات کد بالا نشان داده شده است). پسوند فایل شیء حاصل o خواهد بود.

کامپایلر C لینوکس نیز یک کامپایلر GNU است. برای اینکه کد بالا در لینوکس اجرا شود، کافی است پیشوندهای زیرخط را در سطرهای ۳۷ و ۳۸ حذف کنید. لینوکس برای فایل‌های شیء از قالب ELF (Executable and Linkable Format) استفاده می‌کند. در لینوکس از گزینهٔ -f elf استفاده کنید که آن هم فایل شیئی با پسوند o تولید می‌کند.

فایل‌های نمونهٔ ویژهٔ هر کامپایلر، که از وبگاه نویسنده قابل دریافت هستند، پیشاپیش برای کار با کامپایلر مربوط اصلاح شده‌اند.

Borland C/C++ کامپایلر پرطرفدار دیگری است که برای فایل‌های شیء از قالب OMF مایکروسافت استفاده می‌کند. برای کامپایلرهای بورلند از گزینهٔ -f obj استفاده کنید؛ پسوند فایل شیء در این حالت obj خواهد بود. قالب OMF رهنمودهای segment متفاوتی با قالب‌های دیگر فایل شیء به کار می‌برد. قطعهٔ داده (سطر ۱۳) باید به این صورت تغییر کند:

```
segment _DATA public align=4 class=DATA use32
```

قطعهٔ bss (سطر ۲۶) باید به این صورت تغییر کند:

```
segment _BSS public align=4 class=BSS use32
```

و قطعهٔ text (سطر ۳۶) باید به این صورت تغییر کند:

[۱۰۱] GNU پروژه‌ای از بنیاد نرم‌افزار آزاد است (http://www.fsf.org).
[۱۰۲] http://www.delorie.com/djgpp
[103] object file

```
segment _TEXT public align=1 class=CODE use32
```

افزون بر این، باید سطر جدیدی پیش از سطر ۳۶ افزوده شود:

```
group DGROUP _BSS _DATA
```

کامپایلر Microsoft C/C++ می‌تواند برای فایل‌های شیء از قالب OMF یا قالب Win32 استفاده کند. (اگر فایلی با قالب OMF به آن داده شود، اطلاعات آن را در درون خود به قالب Win32 تبدیل می‌کند.) قالب Win32 اجازه می‌دهد قطعه‌ها درست مانند DJGPP و لینوکس تعریف شوند. برای تولید خروجی در این حالت از گزینهٔ `-f win32` استفاده کنید. پسوند فایل شیء در این حالت نیز `obj` خواهد بود.

۳.۴.۱ اسمبل کردن کد

نخستین گام، اسمبل کردن[۱۰۴] کد است. برای این کار در خط فرمان تایپ کنید:

```
nasm -f object-format first.asm
```

که در آن *object-format* بسته به کامپایلر C مورد استفاده یکی از *coff*، *elf*، *obj* یا *win32* است. (به یاد داشته باشید که برای لینوکس و بورلند باید فایل منبع[۱۰۵] را نیز تغییر داد.)

۴.۴.۱ کامپایل کردن کد C

فایل `driver.c` را با یک کامپایلر C کامپایل کنید. برای DJGPP از فرمان زیر استفاده کنید:

```
gcc -c driver.c
```

گزینهٔ `-c` به این معناست که فایل تنها کامپایل شود و هنوز برای پیوند دادن آن تلاشی صورت نگیرد. همین گزینه در کامپایلرهای لینوکس، بورلند و مایکروسافت نیز کار می‌کند.

---

[104] assemble
[105] source file

۵.۴.۱ پیوند دادن فایل‌های شیء

پیوند دادن فرایندی است که در آن کد ماشین و داده‌های موجود در فایل‌های شیء و فایل‌های کتابخانه با هم ترکیب می‌شوند تا یک فایل اجرایی[۱۰۶] پدید آید. همان‌گونه که در ادامه خواهیم دید، این فرایند پیچیده است.

کد C برای اجرا به کتابخانهٔ استاندارد C و **کد راه‌اندازی**[۱۰۷] ویژه‌ای نیاز دارد. از این رو، **بسیار** آسان‌تر است که اجازه دهیم کامپایلر C پیونددهنده[۱۰۸] را با پارامترهای درست فرا بخواند تا اینکه بکوشیم خودمان مستقیماً پیونددهنده را فرا بخوانیم. برای نمونه، برای پیوند دادن کد نخستین برنامه با DJGPP از فرمان زیر استفاده کنید:

```
gcc -o first driver.o first.o asm_io.o
```

این فرمان فایلی اجرایی به نام `first.exe` (یا در لینوکس، تنها `first`) می‌سازد. در بورلند از فرمان زیر استفاده می‌شود:

```
bcc32 first.obj driver.obj asm_io.obj
```

بورلند نام فایل اجرایی را از نام نخستین فایل فهرست‌شده تعیین می‌کند، بنابراین در مثال بالا نام برنامه `first.exe` خواهد بود.

می‌توان گام‌های کامپایل و پیوند را با هم ترکیب کرد. برای نمونه:

```
gcc -o first driver.c first.o asm_io.o
```

در این حالت `gcc` نخست `driver.c` را کامپایل می‌کند و سپس پیوند را انجام می‌دهد.

۶.۴.۱ درک فایل فهرست اسمبلی

با گزینهٔ `-l` *`listing-file`* می‌توان از `nasm` خواست که یک فایل فهرست با نام داده‌شده بسازد. این فایل نشان می‌دهد که کد چگونه اسمبل شده است. در زیر می‌بینید که سطرهای ۱۷ و ۱۸ (در قطعهٔ داده) در فایل فهرست چگونه نمایش داده می‌شوند. (شماره‌های سطر، شماره‌های فایل فهرست هستند؛ توجه کنید که شمارهٔ سطرها در فایل منبع ممکن است با شمارهٔ سطرها در فایل فهرست یکسان نباشد.)

[106] executable file
[107] startup code
[108] linker

```
48 00000000 456E7465722061206E-     prompt1 db    "Enter a number: ", 0
49 00000009 756D6265723A2000
50 00000011 456E74657220616E6F-     prompt2 db    "Enter another number: ", 0
51 0000001A 74686572206E756D62-
52 00000023 65723A2000
```

ستون نخست هر سطر شمارهٔ سطر و ستون دوم آفست داده در قطعه (به صورت شانزده‌شانزدهی) است. ستون سوم مقادیر شانزده‌شانزدهی خامی را نشان می‌دهد که ذخیره خواهند شد؛ در این مثال، این داده‌های شانزده‌شانزدهی همان کدهای ASCII هستند. سرانجام، متن فایل منبع در همان سطر نمایش داده می‌شود. آفست‌هایی که در ستون دوم فهرست شده‌اند به احتمال بسیار زیاد آفست‌های واقعی داده‌ها در برنامهٔ کامل **نیستند**، زیرا هر پیمانه ممکن است برچسب‌های ویژهٔ خود را در قطعهٔ داده (و نیز در قطعه‌های دیگر) تعریف کند. در گام پیوند (بخش ۵.۴.۱ را ببینید)، همهٔ این تعریف‌های برچسب در قطعه‌های داده با هم ترکیب می‌شوند تا یک قطعهٔ دادهٔ واحد پدید آید و سپس پیونددهنده آفست‌های نهایی جدید را محاسبه می‌کند.

در اینجا بخش کوچکی از قطعهٔ text (سطرهای ۵۴ تا ۵۶ فایل منبع) را در فایل فهرست می‌بینید:

```
94 0000002C A1[00000000]            mov     eax, [input1]
95 00000031 0305[04000000]          add     eax, [input2]
96 00000037 89C3                    mov     ebx, eax
```

ستون سوم کد ماشینی را نشان می‌دهد که از اسمبل کردن تولید شده است. اغلب کد کامل یک دستورالعمل هنوز قابل محاسبه نیست. برای نمونه، در سطر ۹۴ آفست (یا آدرس) input1 تا زمانی که کد پیوند داده نشود معلوم نیست. اسمبلر می‌تواند کد عملیاتی دستورالعمل mov را محاسبه کند (که بر پایهٔ فایل فهرست A1 است)، اما آفست را درون کروشه می‌نویسد، زیرا مقدار دقیق آن هنوز قابل محاسبه نیست. در این مورد، آفست موقت ۰ به کار رفته است، زیرا input1 در ابتدای آن بخشی از قطعهٔ bss قرار دارد که در این فایل تعریف شده است. به یاد داشته باشید که این **به این معنا نیست** که این برچسب در ابتدای قطعهٔ bss نهایی برنامه قرار خواهد گرفت. هنگامی که کد پیوند داده می‌شود، پیونددهنده آفست درست را در این جایگاه می‌نشاند. دستورالعمل‌های دیگر، مانند سطر ۹۶، به هیچ برچسبی ارجاع نمی‌دهند و در این حالت اسمبلر می‌تواند کد ماشین کامل را محاسبه کند.

نمایش بزرگ‌انتها و کوچک‌انتها

اگر با دقت به سطر ۹۵ نگاه کنید، در آفستی که درون کروشه در کد ماشین آمده است چیز بسیار عجیبی به چشم می‌خورد. برچسب `input2` در آفست ۴ قرار دارد (همان‌گونه که در این فایل تعریف شده است)، اما آفستی که در حافظه ظاهر می‌شود 00000004 نیست، بلکه 04000000 است. چرا؟ پردازنده‌های گوناگون، عددهای صحیح چندبایتی را با ترتیب‌های متفاوتی در حافظه ذخیره می‌کنند. دو روش رایج برای ذخیرهٔ عددهای صحیح وجود دارد: **ترتیب بزرگ‌انتها**[۱۰۹] و **ترتیب کوچک‌انتها**[۱۱۰]. ترتیب بزرگ‌انتها روشی است که طبیعی‌تر به نظر می‌رسد: بزرگ‌ترین (یعنی پرارزش‌ترین) بایت نخست ذخیره می‌شود، سپس بایت بزرگ بعدی و به همین ترتیب. برای نمونه، کلمهٔ مضاعف 00000004 به صورت چهار بایت 04 00 00 00 ذخیره می‌شود. رایانه‌های بزرگ IBM، بیشتر پردازنده‌های RISC و پردازنده‌های موتورولا همگی از این روش بزرگ‌انتها استفاده می‌کنند. اما پردازنده‌های مبتنی بر اینتل از روش کوچک‌انتها بهره می‌برند! در این روش، کم‌ارزش‌ترین بایت نخست ذخیره می‌شود؛ بنابراین 00000004 به صورت 00 00 00 04 در حافظه ذخیره می‌شود. این قالب به صورت سخت‌افزاری در پردازنده تعبیه شده است و نمی‌توان آن را تغییر داد. معمولاً برنامه‌نویس لازم نیست نگران این باشد که کدام قالب به کار می‌رود، اما در برخی شرایط این موضوع اهمیت پیدا می‌کند:

واژهٔ Endian مانند *indian* تلفظ می‌شود.

۱. هنگامی که داده‌های دودویی میان رایانه‌های مختلف (از طریق فایل‌ها یا شبکه) منتقل می‌شوند.

۲. هنگامی که داده‌های دودویی به صورت یک عدد صحیح چندبایتی در حافظه نوشته می‌شوند و سپس به صورت بایت‌های جداگانه خوانده می‌شوند، یا **برعکس**.

ترتیب بایت‌ها[۱۱۱] بر ترتیب عناصر آرایه اثری ندارد و نخستین عنصر آرایه همیشه در کمترین آدرس قرار دارد. این نکته دربارهٔ رشته‌ها (که چیزی جز آرایه‌هایی از نویسه‌ها نیستند) نیز صادق است. با این حال، ترتیب بایت‌ها همچنان بر تک‌تک عناصر آرایه‌ها اعمال می‌شود.

[109] big endian
[110] little endian
[111] endianness

skel.asm

```
1  %include "asm_io.inc"
2  segment .data
3  ;
4  ; initialized data is put in the data segment here
5  ;
6
7  segment .bss
8  ;
9  ; uninitialized data is put in the bss segment
10 ;
11
12 segment .text
13         global  _asm_main
14 _asm_main:
15         enter   0,0               ; setup routine
16         pusha
17
18 ;
19 ; code is put in the text segment. Do not modify the code before
20 ; or after this comment.
21 ;
22
23         popa
24         mov     eax, 0            ; return back to C
25         leave
26         ret
```

skel.asm

شکل ۷.۱: برنامهٔ اسکلت

۵.۱ فایل اسکلت

شکل ۷.۱ یک فایل اسکلت[۱۱۲] را نشان می‌دهد که می‌توان از آن به عنوان نقطهٔ شروعی برای نوشتن برنامه‌های اسمبلی استفاده کرد.

[112] skeleton file

فصل ۲

مبانی زبان اسمبلی

۱.۲ کار با اعداد صحیح

۱.۱.۲ نمایش اعداد صحیح

اعداد صحیح[۱] دو گونه هستند: بی‌علامت[۲] و علامت‌دار[۳]. اعداد صحیح بی‌علامت، که هرگز منفی نیستند، به شیوه‌ای کاملاً سرراست و به‌صورت دودویی[۴] نمایش داده می‌شوند. برای نمونه، عدد ۲۰۰ در قالب یک عدد صحیح بی‌علامت یک‌بایتی به‌صورت 11001000 (یا C8 در مبنای شانزده) نمایش داده می‌شود.

اما نمایش اعداد صحیح علامت‌دار، که می‌توانند مثبت یا منفی باشند، پیچیده‌تر است. عدد $56-$ را در نظر بگیرید. عدد $56+$ در قالب یک بایت به‌صورت 00111000 نمایش داده می‌شود. روی کاغذ می‌توان $56-$ را به‌صورت $111000-$ نوشت، ولی این عدد در یک بایت از حافظهٔ رایانه چگونه نمایش داده می‌شود و علامت منفی آن کجا ذخیره می‌شود؟

برای نمایش اعداد صحیح علامت‌دار در حافظهٔ رایانه، سه روش کلی به کار رفته است. همهٔ این روش‌ها باارزش‌ترین بیت عدد را به‌عنوان **بیت علامت**[۵] به کار می‌برند. این بیت برای اعداد مثبت ۰ و برای اعداد منفی ۱ است.

---

[1] integer
[2] unsigned
[3] signed
[4] binary
[5] sign bit

علامت و اندازه

نخستین روش، که ساده‌ترین آن‌ها نیز هست، **علامت و اندازه**[۶] نام دارد. در این روش عدد صحیح از دو بخش تشکیل می‌شود: بخش نخست بیت علامت است و بخش دوم اندازهٔ عدد. بنابراین ۵۶ به‌صورت بایت $\underline{0}0111000$ نمایش داده می‌شود (بیت علامت زیرخط‌دار است) و -56 به‌صورت $\underline{1}0111000$. در این روش بزرگ‌ترین مقدار یک بایت $\underline{0}1111111$ یا $+127$ و کوچک‌ترین مقدار آن $\underline{1}1111111$ یا -127 است و برای قرینه‌کردن یک مقدار کافی است بیت علامت آن را وارونه کنیم. این روش با وجود سادگی، کاستی‌هایی نیز دارد. نخست آنکه صفر دو نمایش ممکن دارد: $+0$ ($\underline{0}0000000$) و -0 ($\underline{1}0000000$). از آنجا که صفر نه مثبت است و نه منفی، هر دو نمایش باید رفتار یکسانی داشته باشند و همین امر منطق محاسبات حسابی را در پردازنده[۷] پیچیده می‌کند. دوم آنکه محاسبات حسابی عادی نیز پیچیده می‌شود. برای نمونه، اگر ۱۰ با -56 جمع شود، این عمل باید به‌صورت تفریق ۵۶ از ۱۰ بازنویسی شود و این نیز منطق پردازنده را پیچیده‌تر می‌کند.

مکمل یک

روش دوم نمایش **مکمل یک**[۸] نام دارد. مکمل یک هر عدد با وارونه‌کردن تک‌تک بیت‌های آن به دست می‌آید؛ به بیان دیگر، مقدار جدید هر بیت برابر است با ۱ منهای مقدار قبلی آن. برای نمونه، مکمل یک عدد $\underline{0}0111000$ ($+56$) برابر با $\underline{1}1000111$ است. در نمادگذاری مکمل یک، محاسبهٔ مکمل یک همان قرینه‌کردن عدد است؛ از این رو $\underline{1}1000111$ نمایش -56 به شمار می‌رود. دقت کنید که بیت علامت با گرفتن مکمل یک به‌طور خودکار تغییر کرد و همان‌گونه که انتظار می‌رود، اگر دو بار مکمل یک بگیریم، عدد اصلی دوباره به دست می‌آید. در این روش نیز، مانند روش نخست، صفر دو نمایش دارد: $\underline{0}0000000$ ($+0$) و $\underline{1}1111111$ (-0). افزون بر این، محاسبات حسابی با اعداد مکمل یک پیچیده است.

برای یافتن مکمل یک یک عدد در مبنای شانزده، ترفند سودمندی وجود دارد که نیازی به تبدیل عدد به دودویی ندارد: کافی است هر رقم شانزده‌شانزدهی را از F (یا ۱۵ در دهدهی) کم کنیم. البته این روش فرض می‌کند که تعداد بیت‌های عدد مضربی از ۴ باشد. برای نمونه، $+56$ در مبنای شانزده به‌صورت 38 نمایش داده می‌شود. اگر برای یافتن مکمل یک آن هر رقم را از F کم کنیم، به C7 در مبنای شانزده می‌رسیم

[6] signed magnitude
[7] CPU
[8] one's complement

که با نتیجهٔ بالا هم‌خوانی دارد.

مکمل دو

دو روشی که تاکنون شرح داده شد در رایانه‌های نخستین به کار می‌رفتند، اما رایانه‌های امروزی روش سومی به نام نمایش **مکمل دو**[9] را به کار می‌برند. مکمل دو هر عدد در دو گام به دست می‌آید:

۱. مکمل یک عدد را پیدا کنید.

۲. یک واحد به نتیجهٔ گام ۱ بیفزایید.

برای نمونه، عدد $\underline{0}0111000$ (۵۶) را در نظر بگیرید. نخست مکمل یک آن محاسبه می‌شود که برابر با $\underline{1}1000111$ است و سپس یک واحد به آن افزوده می‌شود:

$$\begin{array}{rr} & \underline{1}1000111 \\ + & \qquad 1 \\ \hline & \underline{1}1001000 \end{array}$$

در نمادگذاری مکمل دو، محاسبهٔ مکمل دو همان قرینه‌کردن عدد است؛ بنابراین $\underline{1}1001000$ نمایش مکمل دوی $56-$ به شمار می‌رود. از هر روش قرینه‌سازی انتظار می‌رود که با دو بار قرینه‌کردن، عدد اصلی دوباره به دست آید و جالب آنکه مکمل دو این شرط را برآورده می‌کند. برای آزمودن این موضوع، مکمل دوی $\underline{1}1001000$ را با افزودن یک واحد به مکمل یک آن محاسبه می‌کنیم:

$$\begin{array}{rr} & \underline{0}0110111 \\ + & \qquad 1 \\ \hline & \underline{0}0111000 \end{array}$$

هنگام انجام جمع در محاسبهٔ مکمل دو، جمع چپ‌ترین بیت ممکن است یک رقم نقلی[10] تولید کند. این رقم نقلی به کار **نمی‌رود**. به یاد داشته باشید که همهٔ داده‌ها در رایانه اندازهٔ ثابتی (برحسب تعداد بیت) دارند. حاصل جمع دو بایت همیشه یک بایت است، همان‌گونه که حاصل جمع دو کلمه یک کلمه است و به همین ترتیب. این ویژگی برای نمادگذاری مکمل دو اهمیت دارد. برای نمونه، صفر را در قالب یک عدد

[9] two's complement
[10] carry

عدد	نمایش شانزده‌شانزدهی
0	00
1	01
127	7F
-128	80
-127	81
-2	FE
-1	FF

جدول ۱.۲: نمایش مکمل دو

مکمل دوی یک‌بایتی ($\underline{0}0000000$) در نظر بگیرید. محاسبهٔ مکمل دوی آن به این جمع می‌انجامد:

$$\begin{array}{rr} & \underline{1}1111111 \\ + & \qquad 1 \\ \hline c & \underline{0}0000000 \end{array}$$

که در آن c نشان‌دهندهٔ رقم نقلی است. (در ادامه خواهیم دید که این رقم نقلی را چگونه می‌توان تشخیص داد، اما این رقم در نتیجه ذخیره نمی‌شود.) بنابراین در نمادگذاری مکمل دو تنها یک صفر وجود دارد و همین ویژگی، محاسبات حسابی مکمل دو را از روش‌های پیشین ساده‌تر می‌کند.

با نمادگذاری مکمل دو، یک بایت علامت‌دار می‌تواند اعداد -128 تا $+127$ را نمایش دهد. جدول ۱.۲ چند مقدار برگزیده را نشان می‌دهد. اگر ۱۶ بیت به کار رود، می‌توان اعداد علامت‌دار $-32,768$ تا $+32,767$ را نمایش داد؛ در این حالت $+32,767$ به‌صورت 7FFF، $-32,768$ به‌صورت 8000، -128 به‌صورت FF80 و -1 به‌صورت FFFF نمایش داده می‌شود. گسترهٔ اعداد مکمل دوی ۳۲ بیتی نیز تقریباً از -2 میلیارد تا $+2$ میلیارد است.

پردازنده هیچ تصوری ندارد که یک بایت (یا کلمه[11] یا کلمهٔ مضاعف[12]) خاص قرار است چه چیزی را نمایش دهد. زبان اسمبلی[13]، برخلاف زبان‌های سطح بالا[14]، مفهومی به نام نوع داده ندارد و تفسیر داده‌ها به دستورالعملی[15] بستگی دارد که روی آن‌ها به

[11] word
[12] double word
[13] assembly language
[14] high-level language
[15] instruction

کار می‌رود. اینکه مقدار شانزده‌شانزدهی FF نمایانگر -1 علامت‌دار باشد یا $+255$ بی‌علامت، به برنامه‌نویس بستگی دارد. زبان C انواع صحیح علامت‌دار و بی‌علامت را تعریف می‌کند و همین امر به کامپایلر[۱۶] C امکان می‌دهد تا دستورالعمل‌های درستی را برای کار با داده‌ها برگزینند.

۲.۱.۲ گسترش علامت

در اسمبلی همهٔ داده‌ها اندازهٔ مشخصی دارند و کم پیش نمی‌آید که برای به‌کاربردن داده‌ای در کنار داده‌های دیگر، ناچار شویم اندازهٔ آن را تغییر دهیم. در این میان، کاهش اندازه از همه ساده‌تر است.

کاهش اندازهٔ داده‌ها

برای کاهش اندازهٔ داده‌ها کافی است بیت‌های باارزش‌تر آن‌ها را حذف کنیم. نمونهٔ ساده‌ای را در زیر می‌بینید:

```
mov    ax, 0034h      ; ax = 52 (stored in 16 bits)
mov    cl, al         ; cl = lower 8-bits of ax
```

البته اگر عدد را نتوان در اندازهٔ کوچک‌تر به‌درستی نمایش داد، کاهش اندازه نتیجهٔ درستی نخواهد داشت. برای نمونه، اگر مقدار AX برابر با 0134h (یا ۳۰۸ در دهدهی) باشد، کد بالا باز هم CL را برابر با 34h قرار می‌دهد. این روش هم برای اعداد علامت‌دار و هم برای اعداد بی‌علامت کار می‌کند. اعداد علامت‌دار را در نظر بگیرید: اگر AX برابر با FFFFh (یعنی -1 در قالب یک کلمه) باشد، CL برابر با FFh (یعنی -1 در قالب یک بایت) خواهد شد. با این حال، دقت کنید که اگر مقدار AX بی‌علامت بود، این نتیجه درست نبود!

قاعدهٔ اعداد بی‌علامت این است که برای درستی تبدیل، همهٔ بیت‌هایی که حذف می‌شوند باید ۰ باشند. قاعدهٔ اعداد علامت‌دار این است که بیت‌های حذف‌شده یا همگی ۱ باشند یا همگی ۰. افزون بر این، نخستین بیتی که حذف نمی‌شود نیز باید همان مقدار بیت‌های حذف‌شده را داشته باشد، زیرا این بیت، بیت علامت جدید مقدار کوچک‌تر خواهد بود و اهمیت دارد که با بیت علامت اصلی یکسان باشد!

افزایش اندازهٔ داده‌ها

افزایش اندازهٔ داده‌ها از کاهش آن پیچیده‌تر است. بایت شانزده‌شانزدهی FF را در نظر بگیرید. اگر این بایت به یک کلمه گسترش یابد، آن کلمه چه مقداری باید داشته

[16] compiler

باشد؟ پاسخ به تفسیر FF بستگی دارد. اگر FF یک بایت بی‌علامت (۲۵۵ در دهدهی) باشد، کلمهٔ حاصل باید 00FF باشد؛ اما اگر یک بایت علامت‌دار (1− در دهدهی) باشد، کلمهٔ حاصل باید FFFF باشد.

به‌طور کلی، برای گسترش یک عدد بی‌علامت، همهٔ بیت‌هایی را که تازه به عدد افزوده می‌شوند، برابر با ۰ قرار می‌دهیم؛ بنابراین FF به 00FF تبدیل می‌شود. اما برای گسترش یک عدد علامت‌دار باید بیت علامت را **گسترش** دهیم، یعنی بیت‌های جدید رونوشت‌هایی از بیت علامت باشند. از آنجا که بیت علامت FF برابر با ۱ است، بیت‌های جدید نیز باید همگی ۱ باشند تا FFFF به دست آید. اگر عدد علامت‌دار 5A (۹۰ در دهدهی) گسترش یابد، نتیجه 005A خواهد بود.

پردازندهٔ 80386 چند دستورالعمل برای گسترش اعداد فراهم می‌کند. به یاد داشته باشید که رایانه نمی‌داند عدد علامت‌دار است یا بی‌علامت و به‌کاربردن دستورالعمل درست بر عهدهٔ برنامه‌نویس است.

برای اعداد بی‌علامت می‌توان به‌سادگی با یک دستورالعمل MOV بیت‌های بالایی را صفر کرد. برای نمونه، برای گسترش بایت موجود در AL به یک کلمهٔ بی‌علامت در AX چنین می‌نویسیم:

```
mov    ah, 0   ; zero out upper 8-bits
```

با این حال، نمی‌توان با دستورالعمل MOV کلمهٔ بی‌علامت موجود در AX را به یک کلمهٔ مضاعف بی‌علامت در EAX تبدیل کرد. چرا؟ چون هیچ راهی برای مشخص‌کردن ۱۶ بیت بالایی EAX در یک دستورالعمل MOV وجود ندارد. پردازندهٔ 80386 این مشکل را با افزودن دستورالعمل جدیدی به نام MOVZX حل می‌کند. این دستورالعمل دو عملگر[۱۷] دارد. مقصد (عملگر نخست) باید یک ثبات[۱۸] ۱۶ یا ۳۲ بیتی باشد و مبدأ (عملگر دوم) می‌تواند یک ثبات ۸ یا ۱۶ بیتی یا یک بایت یا کلمه از حافظه باشد. محدودیت دیگر این است که مقصد باید از مبدأ بزرگ‌تر باشد (بیشتر دستورالعمل‌ها ایجاب می‌کنند که مبدأ و مقصد هم‌اندازه باشند). چند نمونه را در زیر می‌بینید:

```
movzx  eax, ax      ; extends ax into eax
movzx  eax, al      ; extends al into eax
movzx  ax, al       ; extends al into ax
movzx  ebx, ax      ; extends ax into ebx
```

برای اعداد علامت‌دار در هیچ حالتی راه ساده‌ای برای به‌کاربردن دستورالعمل MOV وجود ندارد. پردازندهٔ 8086 چند دستورالعمل برای گسترش اعداد علامت‌دار فراهم

[17] operand
[18] register

```
1 unsigned char uchar = 0xFF;
2 signed char   schar = 0xFF;
3 int a = (int) uchar;      /* a = 255 (0x000000FF) */
4 int b = (int) schar;      /* b = -1 (0xFFFFFFFF) */
```

شکل ۱.۲:

می‌کرد. دستورالعمل CBW (Convert Byte to Word) علامت ثبات AL را به AX گسترش می‌دهد. عملگرهای این دستورالعمل ضمنی هستند. دستورالعمل CWD (Convert Word to Double word) علامت AX را به DX:AX گسترش می‌دهد. نمادگذاری DX:AX یعنی ثبات‌های DX و AX را یک ثبات ۳۲ بیتی در نظر بگیریم که ۱۶ بیت بالایی آن در DX و بیت‌های پایینی آن در AX است. (به یاد داشته باشید که 8086 هیچ ثبات ۳۲ بیتی نداشت!) پردازندهٔ 80386 چند دستورالعمل جدید به این مجموعه افزود. دستورالعمل CWDE (Convert Word to Double word Extended) علامت AX را به EAX گسترش می‌دهد و دستورالعمل CDQ (Convert Double word to Quad word) علامت EAX را به EDX:EAX (یعنی ۶۴ بیت!) گسترش می‌دهد. سرانجام، دستورالعمل MOVSX همانند MOVZX کار می‌کند، با این تفاوت که قواعد اعداد علامت‌دار را به کار می‌برد.

کاربرد در برنامه‌نویسی C

ANSI C مشخص نمی‌کند که نوع char علامت‌دار است یا نه و تصمیم‌گیری در این باره را به هر کامپایلر واگذار می‌کند. به همین دلیل است که در شکل ۱.۲ نوع این متغیرها به‌صراحت تعریف شده است.

گسترش اعداد صحیح بی‌علامت و علامت‌دار در زبان C نیز رخ می‌دهد. متغیرهای C را می‌توان علامت‌دار یا بی‌علامت اعلان کرد (نوع int علامت‌دار است). کد شکل ۱.۲ را در نظر بگیرید. در سطر ۳، متغیر a با قواعد مقادیر بی‌علامت (با MOVZX) گسترش می‌یابد، اما در سطر ۴ برای b قواعد اعداد علامت‌دار (با MOVSX) به کار می‌رود.

یکی از اشکالات رایج برنامه‌نویسی C مستقیماً به همین موضوع مربوط است. کد شکل ۲.۲ را در نظر بگیرید. پیش‌نمونهٔ تابع fgetc() به این صورت است:

```
int fgetc( FILE * );
```

شاید این پرسش پیش آید که چرا این تابع با آنکه نویسه‌ها را می‌خواند، یک int برمی‌گرداند. دلیل این است که این تابع در حالت عادی یک char برمی‌گرداند که با گسترش صفر به یک مقدار int گسترش یافته است. با این حال، این تابع ممکن است مقداری را برگرداند که نویسه نیست و آن EOF است. EOF یک ماکرو[19] است که

[19] macro

```
char ch;
while( (ch = fgetc(fp)) != EOF ) {
  /* do something with ch */
}
```

شکل ۲.۲:

معمولاً برابر با -1 تعریف می‌شود. بنابراین `fgetc()` یا یک `char` گسترش‌یافته به مقدار `int` برمی‌گرداند (که در مبنای شانزده به شکل $000000xx$ است) یا `EOF` را (که در مبنای شانزده به شکل `FFFFFFFF` است).

مشکل اصلی برنامهٔ شکل ۲.۲ این است که `fgetc()` یک `int` برمی‌گرداند، اما این مقدار در یک `char` ذخیره می‌شود. زبان C بیت‌های مرتبهٔ بالا را کنار می‌گذارد تا مقدار `int` در `char` جا شود. تنها مشکل این است که اعداد `000000FF` و `FFFFFFFF` (در مبنای شانزده) هر دو به بایت `FF` کوتاه می‌شوند. در نتیجه، حلقهٔ while نمی‌تواند میان خواندن بایت `FF` از فایل و رسیدن به پایان فایل تمایز بگذارد.

اینکه کد در این حالت دقیقاً چه رفتاری دارد، به علامت‌دار یا بی‌علامت بودن `char` بستگی دارد. چرا؟ چون در سطر ۲، `ch` با `EOF` مقایسه می‌شود و از آنجا که `EOF` یک مقدار `int` است[۲۰]، `ch` به یک `int` گسترش می‌یابد تا دو مقدار مقایسه‌شونده هم‌اندازه باشند[۲۱]. همان‌گونه که شکل ۱.۲ نشان داد، علامت‌دار یا بی‌علامت بودن متغیر در اینجا اهمیت بسیاری دارد.

اگر `char` بی‌علامت باشد، `FF` به `000000FF` گسترش می‌یابد. این مقدار با `EOF` (`FFFFFFFF`) مقایسه می‌شود و نابرابر از آب درمی‌آید؛ در نتیجه حلقه هرگز پایان نمی‌یابد!

اگر `char` علامت‌دار باشد، `FF` به `FFFFFFFF` گسترش می‌یابد. این بار مقایسه برابری را نشان می‌دهد و حلقه پایان می‌یابد. اما از آنجا که ممکن است بایت `FF` از خود فایل خوانده شده باشد، حلقه ممکن است زودتر از موعد پایان یابد.

راه‌حل این مشکل آن است که متغیر `ch` را به‌جای `char` از نوع `int` تعریف کنیم. با این کار، در سطر ۲ هیچ کوتاه‌سازی یا گسترشی انجام نمی‌شود. درون حلقه نیز کوتاه‌کردن مقدار بی‌خطر است، زیرا در آنجا `ch` **حتماً** یک بایت ساده است.

---

[۲۰] این باور نادرست رایج است که فایل‌ها در انتهای خود نویسه‌ای به نام `EOF` دارند. این باور به هیچ وجه درست **نیست**!

[۲۱] دلیل این الزام را بعداً خواهیم دید.

۳.۱.۲ محاسبات حسابی مکمل دو

همان‌گونه که پیش‌تر دیدیم، دستورالعمل `add` عمل جمع و دستورالعمل `sub` عمل تفریق را انجام می‌دهد. دو بیت از ثبات FLAGS که این دستورالعمل‌ها مقدار آن‌ها را تعیین می‌کنند، پرچم **سرریز**[۲۲] و **پرچم نقلی**[۲۳] هستند. در محاسبات علامت‌دار، اگر نتیجهٔ واقعی عمل بزرگ‌تر از آن باشد که در مقصد جا شود، پرچم سرریز یک می‌شود. پرچم نقلی نیز هنگامی یک می‌شود که در جمع، از باارزش‌ترین بیت (msb) رقم نقلی تولید شود یا در تفریق، باارزش‌ترین بیت رقم قرضی[۲۴] بگیرد. بنابراین از پرچم نقلی می‌توان برای تشخیص سرریز در محاسبات بی‌علامت بهره گرفت. کاربردهای پرچم نقلی در محاسبات علامت‌دار را به‌زودی خواهیم دید. یکی از مزیت‌های بزرگ مکمل دو این است که قواعد جمع و تفریق در آن دقیقاً همان قواعد اعداد صحیح بی‌علامت است؛ از این رو `add` و `sub` را می‌توان هم برای اعداد صحیح علامت‌دار و هم برای اعداد بی‌علامت به کار برد.

$$\begin{array}{rr} 002\mathrm{C} & 44 \\ +\ \mathrm{FFFF} & +\ (-1) \\ \hline 002\mathrm{B} & 43 \end{array}$$

در این جمع یک رقم نقلی تولید می‌شود، اما این رقم بخشی از پاسخ نیست.

ضرب و تقسیم هر کدام دو دستورالعمل متفاوت دارند. نخست به ضرب می‌پردازیم که برای آن یکی از دو دستورالعمل `MUL` یا `IMUL` به کار می‌رود. دستورالعمل `MUL` برای ضرب اعداد بی‌علامت و `IMUL` برای ضرب اعداد صحیح علامت‌دار به کار می‌رود. اما چرا به دو دستورالعمل متفاوت نیاز است؟ چون قواعد ضرب برای اعداد بی‌علامت و اعداد علامت‌دار مکمل دو با هم تفاوت دارند. برای روشن‌شدن موضوع، ضرب بایت FF در خودش را در نظر بگیرید که نتیجهٔ آن یک کلمه است. در ضرب بی‌علامت، این عمل برابر با ۲۵۵ ضرب در ۲۵۵، یعنی ۶۵۰۲۵ (یا FE01 در مبنای شانزده) است، اما در ضرب علامت‌دار برابر با -1 ضرب در -1، یعنی ۱ (یا 0001 در مبنای شانزده) است.

دستورالعمل‌های ضرب چند قالب مختلف دارند. قدیمی‌ترین قالب به این شکل است:

```
mul   source
```

[22] overflow flag
[23] carry flag
[24] borrow

مقصد	مبدأ ۱	مبدأ ۲	عمل
	reg/mem8		AX = AL*source1
	reg/mem16		DX:AX = AX*source1
	reg/mem32		EDX:EAX = EAX*source1
reg16	reg/mem16		dest *= source1
reg32	reg/mem32		dest *= source1
reg16	immed8		dest *= immed8
reg32	immed8		dest *= immed8
reg16	immed16		dest *= immed16
reg32	immed32		dest *= immed32
reg16	reg/mem16	immed8	dest = source1*source2
reg32	reg/mem32	immed8	dest = source1*source2
reg16	reg/mem16	immed16	dest = source1*source2
reg32	reg/mem32	immed32	dest = source1*source2

جدول ۲.۲: دستورالعمل‌های `imul`

در اینجا *source* یا یک ثبات است یا ارجاعی به حافظه و نمی‌تواند یک مقدار بی‌واسطه[۲۵] باشد. اینکه دقیقاً چه ضربی انجام شود، به اندازهٔ عملگر مبدأ بستگی دارد. اگر عملگر یک‌بایتی باشد، در بایت موجود در ثبات AL ضرب می‌شود و نتیجه در ۱۶ بیت AX ذخیره می‌شود. اگر مبدأ ۱۶ بیتی باشد، در کلمهٔ موجود در AX ضرب می‌شود و نتیجهٔ ۳۲ بیتی در DX:AX ذخیره می‌شود و اگر مبدأ ۳۲ بیتی باشد، در EAX ضرب می‌شود و نتیجهٔ ۶۴ بیتی در EDX:EAX ذخیره می‌شود.

دستورالعمل `IMUL` همهٔ قالب‌های `MUL` را دارد و افزون بر آن‌ها، قالب‌های دیگری نیز به آن اضافه شده است. این قالب‌ها دوعملگری و سه‌عملگری هستند:

```
imul   dest, source1
imul   dest, source1, source2
```

جدول ۲.۲ ترکیب‌های ممکن را نشان می‌دهد.

دو دستورالعمل تقسیم `DIV` و `IDIV` هستند که به ترتیب تقسیم صحیح بی‌علامت و علامت‌دار را انجام می‌دهند. قالب کلی آن‌ها چنین است:

```
div   source
```

[25] immediate value

اگر مبدأ ۸ بیتی باشد، AX بر عملگر تقسیم می‌شود و خارج‌قسمت در AL و باقی‌مانده در AH ذخیره می‌شود. اگر مبدأ ۱۶ بیتی باشد، DX:AX بر عملگر تقسیم می‌شود و خارج‌قسمت در AX و باقی‌مانده در DX قرار می‌گیرد. اگر مبدأ ۳۲ بیتی باشد، EDX:EAX بر عملگر تقسیم می‌شود و خارج‌قسمت در EAX و باقی‌مانده در EDX ذخیره می‌شود. دستورالعمل IDIV نیز به همین شیوه کار می‌کند، اما قالب‌های ویژه‌ای مانند قالب‌های ویژهٔ IMUL برای آن وجود ندارد. اگر خارج‌قسمت بزرگ‌تر از آن باشد که در ثبات خود جا شود یا مقسوم‌علیه صفر باشد، اجرای برنامه قطع می‌شود و برنامه پایان می‌یابد. یکی از خطاهای بسیار رایج این است که فراموش کنیم پیش از تقسیم به DX یا EDX مقدار اولیه بدهیم.

دستورالعمل NEG تنها عملگر خود را با محاسبهٔ مکمل دوی آن قرینه می‌کند. عملگر این دستورالعمل می‌تواند هر ثبات یا خانهٔ حافظهٔ ۸، ۱۶ یا ۳۲ بیتی باشد.

۴.۱.۲ برنامهٔ نمونه

math.asm

```
1  %include "asm_io.inc"
2  segment .data          ; Output strings
3  prompt          db    "Enter a number: ", 0
4  square_msg      db    "Square of input is ", 0
5  cube_msg        db    "Cube of input is ", 0
6  cube25_msg      db    "Cube of input times 25 is ", 0
7  quot_msg        db    "Quotient of cube/100 is ", 0
8  rem_msg         db    "Remainder of cube/100 is ", 0
9  neg_msg         db    "The negation of the remainder is ", 0
10
11 segment .bss
12 input   resd 1
13
14 segment .text
15         global  _asm_main
16 _asm_main:
17         enter   0,0               ; setup routine
18         pusha
19
20         mov     eax, prompt
21         call    print_string
22
23         call    read_int
24         mov     [input], eax
```

```
25
26          imul    eax             ; edx:eax = eax * eax
27          mov     ebx, eax        ; save answer in ebx
28          mov     eax, square_msg
29          call    print_string
30          mov     eax, ebx
31          call    print_int
32          call    print_nl
33
34          mov     ebx, eax
35          imul    ebx, [input]    ; ebx *= [input]
36          mov     eax, cube_msg
37          call    print_string
38          mov     eax, ebx
39          call    print_int
40          call    print_nl
41
42          imul    ecx, ebx, 25    ; ecx = ebx*25
43          mov     eax, cube25_msg
44          call    print_string
45          mov     eax, ecx
46          call    print_int
47          call    print_nl
48
49          mov     eax, ebx
50          cdq                     ; initialize edx by sign extension
51          mov     ecx, 100        ; can't divide by immediate value
52          idiv    ecx             ; edx:eax / ecx
53          mov     ecx, eax        ; save quotient into ecx
54          mov     eax, quot_msg
55          call    print_string
56          mov     eax, ecx
57          call    print_int
58          call    print_nl
59          mov     eax, rem_msg
60          call    print_string
61          mov     eax, edx
62          call    print_int
63          call    print_nl
64
65          neg     edx             ; negate the remainder
66          mov     eax, neg_msg
```

```
67          call    print_string
68          mov     eax, edx
69          call    print_int
70          call    print_nl
71
72          popa
73          mov     eax, 0            ; return back to C
74          leave
75          ret
```
math.asm

۵.۱.۲ حساب با دقت گسترش‌یافته

زبان اسمبلی دستورالعمل‌هایی نیز فراهم می‌کند که با آن‌ها می‌توان اعدادی بزرگ‌تر از کلمهٔ مضاعف را جمع یا تفریق کرد. این دستورالعمل‌ها از پرچم نقلی بهره می‌گیرند. همان‌گونه که پیش‌تر گفتیم، دستورالعمل‌های ADD و SUB در صورت تولید رقم نقلی یا رقم قرضی، پرچم نقلی را تغییر می‌دهند. با استفاده از اطلاعاتی که در پرچم نقلی ذخیره می‌شود، می‌توان عمل جمع یا تفریق اعداد بزرگ را به قطعه‌های کوچک‌تری به اندازهٔ کلمهٔ مضاعف (یا کوچک‌تر) شکست و اعداد بزرگ را جمع یا تفریق کرد.

دستورالعمل‌های ADC و SBB همین اطلاعات پرچم نقلی را به کار می‌برند. عملی که ADC انجام می‌دهد چنین است:

operand1 = *operand1* + carry flag + *operand2*

و دستورالعمل SBB این عمل را:

operand1 = *operand1* - carry flag - *operand2*

این دستورالعمل‌ها چگونه به کار می‌روند؟ مجموع دو عدد صحیح ۶۴ بیتی را در نظر بگیرید که در EDX:EAX و EBX:ECX قرار دارند. کد زیر حاصل جمع را در EDX:EAX ذخیره می‌کند:

```
1       add     eax, ecx        ; add lower 32-bits
2       adc     edx, ebx        ; add upper 32-bits and carry from previous sum
```

تفریق نیز بسیار شبیه به جمع است. کد زیر EBX:ECX را از EDX:EAX کم می‌کند:

```
1       sub     eax, ecx        ; subtract lower 32-bits
2       sbb     edx, ebx        ; subtract upper 32-bits and borrow
```

برای اعداد **واقعاً** بزرگ می‌توان از یک حلقه[۲۶] بهره گرفت (بخش ۲.۲ را ببینید). در یک حلقهٔ جمع، بهتر است دستورالعمل ADC در همهٔ تکرارها به کار رود، نه در همهٔ تکرارها به جز نخستین تکرار. برای این کار کافی است درست پیش از آغاز حلقه، دستورالعمل CLC (CLear Carry) را به کار ببریم تا پرچم نقلی مقدار اولیهٔ ۰ بگیرد. وقتی پرچم نقلی ۰ باشد، دستورالعمل‌های ADD و ADC هیچ تفاوتی با هم ندارند. همین ایده را می‌توان برای تفریق نیز به کار برد.

۲.۲ ساختارهای کنترلی

زبان‌های سطح بالا[۲۷] ساختارهای کنترلی[۲۸] سطح بالایی (**برای نمونه**، دستورهای *if* و *while*) در اختیار برنامه‌نویس می‌گذارند که مسیر اجرای برنامه را هدایت می‌کنند. زبان اسمبلی چنین ساختارهای پیچیده‌ای ندارد و به جای آن‌ها از همان *goto* بدنام استفاده می‌کند که اگر نابجا به کار رود، ممکن است به کدی درهم و آشفته، موسوم به «کد اسپاگتی»، بینجامد! با این حال، نوشتن برنامه‌های ساخت‌یافته به زبان اسمبلی **کاملاً** امکان‌پذیر است. روش کار در اصل این است که منطق برنامه را با همان ساختارهای کنترلی آشنای سطح بالا طراحی کنیم و سپس این طرح را، تقریباً همان‌گونه که یک کامپایلر[۲۹] انجام می‌دهد، به دستورالعمل‌های مناسب زبان اسمبلی برگردانیم.

۱.۲.۲ مقایسه‌ها

ساختارهای کنترلی بر اساس مقایسهٔ داده‌ها تصمیم می‌گیرند که چه کاری انجام شود. در اسمبلی، نتیجهٔ مقایسه در ثبات[۳۰] FLAGS ذخیره می‌شود تا بعداً از آن استفاده شود. پردازنده‌های 80x86 برای انجام مقایسه دستورالعمل[۳۱] CMP را فراهم می‌کنند. این دستورالعمل ثبات FLAGS را بر اساس تفاضل دو عملگر[۳۲] خود تنظیم می‌کند؛ یعنی دو عملگر از هم کم می‌شوند و پرچم‌ها[۳۳] بر اساس حاصل تنظیم می‌شوند، اما خود حاصل در هیچ جا ذخیره **نمی‌شود**. بنابراین اگر به حاصل تفریق نیاز

[26] loop
[27] high-level languages
[28] control structures
[29] compiler
[30] register
[31] instruction
[32] operand
[33] flags

دارید، به جای CMP از دستورالعمل SUB استفاده کنید.

در مورد اعداد صحیح بی‌علامت[۳۴]، دو پرچم (یعنی دو بیت از ثبات FLAGS) اهمیت دارند: پرچم صفر[۳۵] (ZF) و پرچم نقلی[۳۶] (CF). اگر تفاضل حاصل صفر باشد، پرچم صفر یک (1) می‌شود. پرچم نقلی نیز در تفریق نقش پرچم رقم قرضی[۳۷] را بازی می‌کند. برای نمونه، مقایسه‌ای مانند زیر را در نظر بگیرید:

```
cmp    vleft, vright
```

در اینجا تفاضل `vleft - vright` محاسبه می‌شود و پرچم‌ها متناسب با آن تنظیم می‌شوند. حال اگر این تفاضل صفر باشد، یعنی اگر `vleft = vright` باشد، ZF یک (**یعنی** 1) و CF صفر (**یعنی** 0) می‌شود. اگر `vleft > vright` باشد، هر دو پرچم ZF و CF صفر می‌شوند (زیرا رقم قرضی وجود ندارد) و اگر `vleft < vright` باشد، ZF صفر و CF یک می‌شود (زیرا رقم قرضی پیش می‌آید).

در مورد اعداد صحیح علامت‌دار[۳۸]، سه پرچم اهمیت دارند: پرچم صفر (ZF)، پرچم سرریز[۳۹] (OF) و پرچم علامت[۴۰] (SF). هرگاه حاصل یک عمل سرریز (یا زیرریز) کند، پرچم سرریز یک می‌شود و هرگاه حاصل عمل منفی باشد، پرچم علامت یک می‌شود. اگر `vleft = vright` باشد، ZF یک می‌شود (درست مانند اعداد صحیح بی‌علامت). اگر `vleft > vright` باشد، ZF صفر می‌شود و $SF = OF$ خواهد بود و اگر `vleft < vright` باشد، ZF صفر می‌شود و $SF \neq OF$ خواهد بود.

چرا اگر `vleft > vright` باشد، $SF = OF$ می‌شود؟ اگر سرریزی رخ ندهد، تفاضل مقدار درستی خواهد داشت و ناگزیر نامنفی است؛ پس $SF = OF = 0$ است. اما اگر سرریز رخ دهد، تفاضل مقدار درستی نخواهد داشت (و در واقع منفی خواهد بود)؛ پس $SF = OF = 1$ است.

فراموش نکنید که ثبات FLAGS را تنها CMP تغییر نمی‌دهد و دستورالعمل‌های دیگر نیز می‌توانند آن را تغییر دهند.

۲.۲.۲ دستورالعمل‌های انشعاب

دستورالعمل‌های انشعاب[۴۱] می‌توانند اجرای برنامه را به هر نقطهٔ دلخواهی از آن منتقل کنند؛ به بیان دیگر، کاری شبیه *goto* انجام می‌دهند. انشعاب دو نوع است:

[34] unsigned
[35] zero flag
[36] carry flag
[37] borrow
[38] signed
[39] overflow flag
[40] sign flag
[41] branch instructions

غیرشرطی[۴۲] و شرطی[۴۳]. انشعاب غیرشرطی درست مانند goto است و همیشه انجام می‌شود. اما انشعاب شرطی بسته به وضعیت پرچم‌های ثبات FLAGS ممکن است انجام بشود یا نشود و اگر انجام نشود، کنترل به دستورالعمل بعدی می‌رسد.

دستورالعمل JMP (کوتاه‌شدهٔ *jump* به معنای پرش[۴۴]) انشعاب‌های غیرشرطی را انجام می‌دهد. تنها آرگومان[۴۵] این دستورالعمل معمولاً یک **برچسب کد**[۴۶] است که به دستورالعمل مقصد اشاره می‌کند. اسمبلر[۴۷] یا پیونددهنده[۴۸] این برچسب را با آدرس درست آن دستورالعمل جایگزین می‌کند. این هم یکی دیگر از کارهای ملال‌آوری است که اسمبلر انجام می‌دهد تا زندگی برنامه‌نویس آسان‌تر شود. باید توجه داشت که دستوری که بلافاصله پس از دستورالعمل JMP می‌آید هرگز اجرا نخواهد شد، مگر آنکه دستورالعمل دیگری به آن انشعاب کند!

دستورالعمل پرش چند گونهٔ مختلف دارد:

SHORT برد این پرش بسیار محدود است و تنها می‌تواند ۱۲۸ بایت در حافظه به بالا یا پایین جابه‌جا شود. مزیت این نوع پرش آن است که از انواع دیگر حافظهٔ کمتری مصرف می‌کند، زیرا **جابه‌جایی**[۴۹] پرش را در یک بایت علامت‌دار ذخیره می‌کند. جابه‌جایی نشان می‌دهد که چند بایت باید به جلو یا عقب رفت (جابه‌جایی با EIP جمع می‌شود). برای مشخص کردن یک پرش کوتاه، کلیدواژهٔ SHORT را درست پیش از برچسب در دستورالعمل JMP قرار دهید.

NEAR این پرش نوع پیش‌فرض برای انشعاب‌های غیرشرطی و شرطی است و با آن می‌توان به هر مکانی در یک قطعه[۵۰] پرش کرد. در واقع پردازندهٔ 80386 از دو نوع پرش نزدیک پشتیبانی می‌کند. نوع اول برای جابه‌جایی از دو بایت استفاده می‌کند و به این ترتیب امکان می‌دهد تقریباً ۳۲۰۰۰ بایت به بالا یا پایین جابه‌جا شویم. نوع دوم برای جابه‌جایی چهار بایت به کار می‌برد و طبیعتاً امکان می‌دهد به هر مکانی در قطعهٔ کد برویم. در حالت حفاظت‌شدهٔ[۵۱] 386، نوع چهاربایتی پیش‌فرض است و نوع دوبایتی را می‌توان با قراردادن کلیدواژهٔ WORD پیش از

[42] unconditional
[43] conditional
[44] jump
[45] argument
[46] code label
[47] assembler
[48] linker
[49] displacement
[50] segment
[51] protected mode

JZ	تنها در صورتی انشعاب می‌کند که ZF یک باشد
JNZ	تنها در صورتی انشعاب می‌کند که ZF صفر باشد
JO	تنها در صورتی انشعاب می‌کند که OF یک باشد
JNO	تنها در صورتی انشعاب می‌کند که OF صفر باشد
JS	تنها در صورتی انشعاب می‌کند که SF یک باشد
JNS	تنها در صورتی انشعاب می‌کند که SF صفر باشد
JC	تنها در صورتی انشعاب می‌کند که CF یک باشد
JNC	تنها در صورتی انشعاب می‌کند که CF صفر باشد
JP	تنها در صورتی انشعاب می‌کند که PF یک باشد
JNP	تنها در صورتی انشعاب می‌کند که PF صفر باشد

جدول ۳.۲: انشعاب‌های شرطی ساده

برچسب در دستورالعمل JMP مشخص کرد.

FAR این پرش امکان می‌دهد کنترل به قطعهٔ کد دیگری منتقل شود؛ کاری که در حالت حفاظت‌شدهٔ 386 بسیار به‌ندرت پیش می‌آید.

برچسب‌های معتبر کد از همان قواعد برچسب‌های داده پیروی می‌کنند. برای تعریف یک برچسب کد، آن را در قطعهٔ کد و در جلوی دستوری که برچسب می‌خورد قرار می‌دهیم. در محل تعریف برچسب، یک دونقطه در پایان آن گذاشته می‌شود، اما این دونقطه جزء نام برچسب **نیست**.

دستورالعمل‌های انشعاب شرطی بسیار متنوع هستند. این دستورالعمل‌ها نیز یک برچسب کد را به عنوان تنها عملگر خود می‌گیرند. ساده‌ترین آن‌ها فقط به یک پرچم از ثبات FLAGS نگاه می‌کنند تا تصمیم بگیرند انشعاب انجام شود یا نه. فهرست این دستورالعمل‌ها را در جدول ۳.۲ می‌بینید. (PF همان **پرچم توازن**[۵۲] است که زوج یا فرد بودن تعداد بیت‌های یک را در ۸ بیت پایینی حاصل نشان می‌دهد.)

برای نمونه، شبه‌کد[۵۳] زیر را در نظر بگیرید:

```
if ( EAX == 0 )
  EBX = 1;
else
  EBX = 2;
```

این شبه‌کد را می‌توان به این صورت به اسمبلی نوشت:

[52] parity flag
[53] pseudo-code

```
1         cmp     eax, 0              ; set flags (ZF set if eax - 0 = 0)
2         jz      thenblock           ; if ZF is set branch to thenblock
3         mov     ebx, 2              ; ELSE part of IF
4         jmp     next                ; jump over THEN part of IF
5   thenblock:
6         mov     ebx, 1              ; THEN part of IF
7   next:
```

اما پیاده‌سازی مقایسه‌های دیگر با انشعاب‌های شرطی جدول ۳.۲ به این آسانی نیست. برای روشن شدن موضوع، شبه‌کد زیر را در نظر بگیرید:

```
if ( EAX >= 5 )
  EBX = 1;
else
  EBX = 2;
```

اگر EAX بزرگ‌تر از پنج یا مساوی آن باشد، ZF ممکن است یک یا صفر باشد و SF با OF برابر خواهد بود. کد اسمبلی زیر وجود این شرایط را بررسی می‌کند (با این فرض که EAX علامت‌دار است):

```
1         cmp     eax, 5
2         js      signon              ; goto signon if SF = 1
3         jo      elseblock           ; goto elseblock if OF = 1 and SF = 0
4         jmp     thenblock           ; goto thenblock if SF = 0 and OF = 0
5   signon:
6         jo      thenblock           ; goto thenblock if SF = 1 and OF = 1
7   elseblock:
8         mov     ebx, 2
9         jmp     next
10  thenblock:
11        mov     ebx, 1
12  next:
```

کد بالا بسیار دست‌وپاگیر است. خوشبختانه پردازنده‌های 80x86 دستورالعمل‌های انشعاب دیگری نیز دارند که این نوع آزمون‌ها را **بسیار** آسان‌تر می‌کنند. از هر یک از این دستورالعمل‌ها یک نسخهٔ علامت‌دار و یک نسخهٔ بی‌علامت وجود دارد که در جدول ۴.۲ آمده‌اند. انشعاب‌های «مساوی» و «نامساوی» (JE و JNE) برای اعداد صحیح علامت‌دار و بی‌علامت یکسان هستند. (در واقع JE و JNE به ترتیب دقیقاً همان JZ و JNZ هستند.) هر یک از دیگر دستورالعمل‌های انشعاب دو نام مترادف دارد. برای نمونه، JL و JNGE را در نظر بگیرید. JL کوتاه‌شدهٔ عبارت

بی‌علامت		علامت‌دار	
JE	انشعاب اگر vleft = vright	JE	انشعاب اگر vleft = vright
JNE	انشعاب اگر vleft ≠ vright	JNE	انشعاب اگر vleft ≠ vright
JB, JNAE	انشعاب اگر vleft < vright	JL, JNGE	انشعاب اگر vleft < vright
JBE, JNA	انشعاب اگر vleft ≤ vright	JLE, JNG	انشعاب اگر vleft ≤ vright
JA, JNBE	انشعاب اگر vleft > vright	JG, JNLE	انشعاب اگر vleft > vright
JAE, JNB	انشعاب اگر vleft ≥ vright	JGE, JNL	انشعاب اگر vleft ≥ vright

جدول ۴.۲: دستورالعمل‌های مقایسهٔ علامت‌دار و بی‌علامت

jump less than به معنای «پرش اگر کوچک‌تر باشد» است و JNGE کوتاه‌شدهٔ عبارت jump not greater than or equal to به معنای «پرش اگر بزرگ‌تر یا مساوی نباشد». این دو در واقع یک دستورالعمل هستند، زیرا:

$$x < y \Longrightarrow \mathbf{not}(x \geq y)$$

در انشعاب‌های بی‌علامت، به جای L و G از A به معنای *above* (بالاتر) و B به معنای *below* (پایین‌تر) استفاده می‌شود.

با این دستورالعمل‌های انشعاب جدید، شبه‌کد بالا را می‌توان بسیار ساده‌تر به اسمبلی برگرداند:

```
1        cmp    eax, 5
2        jge    thenblock
3        mov    ebx, 2
4        jmp    next
5  thenblock:
6        mov    ebx, 1
7  next:
```

۳.۲.۲ دستورالعمل‌های حلقه

پردازنده‌های 80x86 چند دستورالعمل دارند که برای پیاده‌سازی حلقه‌هایی[۵۴] شبیه *for* طراحی شده‌اند. هر یک از این دستورالعمل‌ها یک برچسب کد را به عنوان تنها عملگر خود می‌گیرد.

LOOP یک واحد از ECX کم می‌کند و اگر $ECX \neq 0$ باشد، به برچسب انشعاب می‌کند.

[54] loops

`LOOPE, LOOPZ` یک واحد از ECX کم می‌کند (بی‌آنکه ثبات FLAGS تغییر کند) و اگر $ECX \neq 0$ و $ZF = 1$ باشد، انشعاب می‌کند.

`LOOPNE, LOOPNZ` یک واحد از ECX کم می‌کند (FLAGS بدون تغییر می‌ماند) و اگر $ECX \neq 0$ و $ZF = 0$ باشد، انشعاب می‌کند.

دو دستورالعمل حلقهٔ آخر برای حلقه‌های جست‌وجوی ترتیبی سودمند هستند. برای نمونه، شبه‌کد زیر را در نظر بگیرید:

```
sum = 0;
for( i=10; i >0; i-- )
  sum += i;
```

این شبه‌کد را می‌توان به این صورت به اسمبلی برگرداند:

```
1        mov    eax, 0          ; eax is sum
2        mov    ecx, 10         ; ecx is i
3  loop_start:
4        add    eax, ecx
5        loop   loop_start
```

۳.۲ ترجمهٔ ساختارهای کنترلی استاندارد

در این بخش می‌بینیم که ساختارهای کنترلی استاندارد زبان‌های سطح بالا را چگونه می‌توان به زبان اسمبلی[۵۵] پیاده‌سازی کرد.

۱.۳.۲ دستور if

شبه‌کد زیر را در نظر بگیرید:

```
if ( condition )
  then_block;
else
  else_block;
```

این ساختار را می‌توان به این صورت پیاده‌سازی کرد:

```
1        ; code to set FLAGS
2        jxx    else_block    ; select xx so that branches if condition false
3        ; code for then block
```

[55] assembly language

```
4       jmp    endif
5  else_block:
6       ; code for else block
7  endif:
```

اگر بخش else وجود نداشته باشد، می‌توان انشعاب به else_block را با انشعاب به endif جایگزین کرد:

```
1       ; code to set FLAGS
2       jxx    endif          ; select xx so that branches if condition false
3       ; code for then block
4  endif:
```

۲.۳.۲ حلقه‌های while

حلقهٔ *while* حلقه‌ای است که شرط آن در ابتدای حلقه آزموده می‌شود:

```
while( condition ) {
  body of loop;
}
```

این حلقه را می‌توان به صورت زیر ترجمه کرد:

```
1  while:
2       ; code to set FLAGS based on condition
3       jxx    endwhile       ; select xx so that branches if false
4       ; body of loop
5       jmp    while
6  endwhile:
```

۳.۳.۲ حلقه‌های do while

در مقابل، حلقهٔ *do while* حلقه‌ای است که شرط آن در انتهای حلقه آزموده می‌شود:

```
do {
  body of loop;
} while( condition );
```

این حلقه را نیز می‌توان به صورت زیر ترجمه کرد:

```
1  do:
2       ; body of loop
3       ; code to set FLAGS based on condition
4       jxx    do          ; select xx so that branches if true
```

```
1  unsigned guess;   /* current guess for prime      */
2  unsigned factor;  /* possible factor of guess     */
3  unsigned limit;   /* find primes up to this value */
4
5   printf("Find primes up to: ");
6  scanf("%u", &limit);
7   printf("2\n");    /* treat first two primes as  */
8   printf("3\n");    /* special case               */
9  guess = 5;        /* initial guess */
10 while ( guess <= limit ) {
11   /* look for a factor of guess */
12    factor = 3;
13    while ( factor*factor < guess &&
14            guess % factor != 0 )
15      factor += 2;
16    if ( guess % factor != 0 )
17      printf("%d\n", guess);
18    guess += 2;   /* only look at odd numbers */
19 }
```

شکل ۳.۲:

۴.۲ مثال: یافتن اعداد اول

در این بخش برنامه‌ای را بررسی می‌کنیم که اعداد اول[۵۶] را پیدا می‌کند. به یاد داریم که عدد اول تنها بر ۱ و خودش بخش‌پذیر است. فرمولی برای یافتن این اعداد وجود ندارد؛ از این رو، روش اصلی این برنامه آن است که مقسوم‌علیه‌های همهٔ اعداد فرد[۵۷] کوچک‌تر از یک حد معین را جست‌وجو کند. اگر برای یک عدد فرد هیچ مقسوم‌علیهی پیدا نشود، آن عدد اول است. شکل ۳.۲ الگوریتم اصلی این کار را به زبان C نشان می‌دهد.

نسخهٔ اسمبلی این برنامه چنین است:

prime.asm

```
1  %include "asm_io.inc"
2  segment .data
3  Message         db      "Find primes up to: ", 0
4
```

[56] prime numbers

[۵۷] عدد ۲ تنها عدد اول زوج است.

```
5  segment .bss
6  Limit           resd    1               ; find primes up to this limit
7  Guess           resd    1               ; the current guess for prime
8
9  segment .text
10         global  _asm_main
11 _asm_main:
12         enter   0,0               ; setup routine
13         pusha
14
15         mov     eax, Message
16         call    print_string
17         call    read_int            ; scanf("%u", & limit );
18         mov     [Limit], eax
19
20         mov     eax, 2              ; printf("2\n");
21         call    print_int
22         call    print_nl
23         mov     eax, 3              ; printf("3\n");
24         call    print_int
25         call    print_nl
26
27         mov     dword [Guess], 5    ; Guess = 5;
28 while_limit:                        ; while ( Guess <= Limit )
29         mov     eax,[Guess]
30         cmp     eax, [Limit]
31         jnbe    end_while_limit     ; use jnbe since numbers are unsigned
32
33         mov     ebx, 3              ; ebx is factor = 3;
34 while_factor:
35         mov     eax,ebx
36         mul     eax                 ; edx:eax = eax*eax
37         jo      end_while_factor    ; if answer won't fit in eax alone
38         cmp     eax, [Guess]
39         jnb     end_while_factor    ; if !(factor*factor < guess)
40         mov     eax,[Guess]
41         mov     edx,0
42         div     ebx                 ; edx = edx:eax % ebx
43         cmp     edx, 0
44         je      end_while_factor    ; if !(guess % factor != 0)
45
46         add     ebx,2               ; factor += 2;
```

```
47          jmp     while_factor
48  end_while_factor:
49          je      end_if              ; if !(guess % factor != 0)
50          mov     eax,[Guess]         ; printf("%u\n")
51          call    print_int
52          call    print_nl
53  end_if:
54          add     dword [Guess], 2    ; guess += 2
55          jmp     while_limit
56  end_while_limit:
57
58          popa
59          mov     eax, 0            ; return back to C
60          leave
61          ret
```

prime.asm

فصل ۳

عملیات بیتی

۱.۳ عملیات جابه‌جایی

زبان اسمبلی به برنامه‌نویس اجازه می‌دهد تک‌تک بیت‌های داده را دست‌کاری کند. یکی از رایج‌ترین عملیات بیتی، **جابه‌جایی**[1] نام دارد که در آن جایگاه بیت‌های یک داده تغییر می‌کند. جابه‌جایی می‌تواند به سمت چپ، یعنی به سوی پرارزش‌ترین بیت‌ها، یا به سمت راست، یعنی به سوی کم‌ارزش‌ترین بیت‌ها، انجام شود.

۱.۱.۳ جابه‌جایی‌های منطقی

جابه‌جایی منطقی[2] ساده‌ترین نوع جابه‌جایی است و به سرراست‌ترین شکل ممکن انجام می‌شود. شکل ۱.۳ نمونه‌ای از جابه‌جایی یک عدد تک‌بایتی را نشان می‌دهد.

عدد اصلی	1	1	1	0	1	0	1	0
جابه‌جایی به چپ	1	1	0	1	0	1	0	0
جابه‌جایی به راست	0	1	1	1	0	1	0	1

شکل ۱.۳: جابه‌جایی‌های منطقی

دقت کنید که بیت‌های تازه‌ای که وارد می‌شوند همیشه صفر هستند. برای جابه‌جایی منطقی به چپ و به راست به ترتیب از دستورالعمل‌های SHL و SHR استفاده می‌شود. این دستورالعمل‌ها امکان جابه‌جایی به هر تعداد موقعیت را فراهم می‌کنند و

[1] shift
[2] logical shift

تعداد موقعیت‌ها را می‌توان یا به صورت یک ثابت مشخص کرد یا در ثبات CL قرار داد. آخرین بیتی که از داده بیرون رانده می‌شود در پرچم نقلی[۳] ذخیره می‌شود. در ادامه چند نمونه کد آمده است:

```
1  mov   ax, 0C123H
2  shl   ax, 1          ; shift 1 bit to left,   ax = 8246H, CF = 1
3  shr   ax, 1          ; shift 1 bit to right,  ax = 4123H, CF = 0
4  shr   ax, 1          ; shift 1 bit to right,  ax = 2091H, CF = 1
5  mov   ax, 0C123H
6  shl   ax, 2          ; shift 2 bits to left,  ax = 048CH, CF = 1
7  mov   cl, 3
8  shr   ax, cl         ; shift 3 bits to right, ax = 0091H, CF = 1
```

۲.۱.۳ کاربرد جابه‌جایی‌ها

رایج‌ترین کاربرد عملیات جابه‌جایی، ضرب و تقسیم سریع است. به یاد دارید که در دستگاه دهدهی، ضرب و تقسیم در توانی از ده بسیار ساده است و تنها کافی است رقم‌ها را جابه‌جا کنیم. همین موضوع در دستگاه دودویی برای توان‌های دو صادق است. برای نمونه، اگر بخواهیم عدد دودویی 1011_2 (یعنی ۱۱ در مبنای ده) را دو برابر کنیم، کافی است آن را یک بار به چپ جابه‌جا کنیم تا 10110_2 (یعنی ۲۲) به دست آید. به همین ترتیب، خارج قسمت تقسیم بر توانی از دو، حاصل یک جابه‌جایی به راست است. برای تقسیم بر ۲ یک جابه‌جایی به راست کافی است، برای تقسیم بر ۴ (2^2) باید دو موقعیت به راست جابه‌جا کرد، برای تقسیم بر ۸ (2^3) سه موقعیت و **الی آخر**. دستورالعمل‌های جابه‌جایی بسیار ابتدایی هستند و **بسیار** سریع‌تر از دستورالعمل‌های متناظر MUL و DIV اجرا می‌شوند!

البته از جابه‌جایی‌های منطقی فقط می‌توان برای ضرب و تقسیم مقادیر بی‌علامت استفاده کرد و این روش در حالت کلی برای مقادیر علامت‌دار درست کار نمی‌کند. مقدار دوبایتی FFFF را در نظر بگیرید که به صورت علامت‌دار برابر -1 است. اگر این مقدار یک بار به صورت منطقی به راست جابه‌جا شود، حاصل 7FFF خواهد بود که برابر $+32767$ است! از این رو برای مقادیر علامت‌دار باید از نوع دیگری از جابه‌جایی استفاده کرد.

[۳] carry flag

۳.۱.۳ جابه‌جایی‌های حسابی

جابه‌جایی‌های حسابی[۴] به این منظور طراحی شده‌اند که بتوان اعداد علامت‌دار را به سرعت در توان‌هایی از ۲ ضرب یا بر آن‌ها تقسیم کرد. این جابه‌جایی‌ها تضمین می‌کنند که با بیت علامت[۵] به درستی رفتار شود.

SAL جابه‌جایی حسابی به چپ (Shift Arithmetic Left): این دستورالعمل تنها نام دیگری برای SHL است و دقیقاً به همان کد ماشین SHL اسمبل می‌شود. تا زمانی که بیت علامت بر اثر جابه‌جایی تغییر نکند، نتیجه درست خواهد بود.

SAR جابه‌جایی حسابی به راست (Shift Arithmetic Right): این یک دستورالعمل تازه است که بیت علامت عملگر[۶] خود، یعنی پرارزش‌ترین بیت (msb)، را جابه‌جا نمی‌کند. بیت‌های دیگر به روال معمول جابه‌جا می‌شوند، با این تفاوت که بیت‌های تازه‌ای که از سمت چپ وارد می‌شوند رونوشتی از بیت علامت هستند؛ یعنی اگر بیت علامت ۱ باشد، بیت‌های تازه نیز ۱ خواهند بود. بنابراین اگر یک بایت با این دستورالعمل جابه‌جا شود، تنها ۷ بیت پایینی آن جابه‌جا می‌شوند. در اینجا نیز مانند دیگر جابه‌جایی‌ها، آخرین بیتی که بیرون رانده می‌شود در پرچم نقلی ذخیره می‌شود.

```
1        mov    ax, 0C123H
2        sal    ax, 1           ; ax = 8246H, CF = 1
3        sal    ax, 1           ; ax = 048CH, CF = 1
4        sar    ax, 2           ; ax = 0123H, CF = 0
```

۴.۱.۳ جابه‌جایی‌های چرخشی

دستورالعمل‌های چرخش[۷] مانند جابه‌جایی‌های منطقی عمل می‌کنند، با این تفاوت که بیت‌هایی که از یک سر داده بیرون می‌افتند از سر دیگر دوباره وارد می‌شوند. به بیان دیگر، با داده مانند یک ساختار دایره‌ای رفتار می‌شود. ساده‌ترین دستورالعمل‌های چرخش، ROL و ROR هستند که به ترتیب چرخش به چپ و به راست را انجام می‌دهند. این جابه‌جایی‌ها نیز مانند بقیه، رونوشتی از آخرین بیتی را که چرخانده شده است در پرچم نقلی نگه می‌دارند.

---

[4] arithmetic shift
[5] sign bit
[6] operand
[7] rotate

```
1       mov    ax, 0C123H
2       rol    ax, 1           ; ax = 8247H, CF = 1
3       rol    ax, 1           ; ax = 048FH, CF = 1
4       rol    ax, 1           ; ax = 091EH, CF = 0
5       ror    ax, 2           ; ax = 8247H, CF = 1
6       ror    ax, 1           ; ax = C123H, CF = 1
```

دو دستورالعمل چرخش دیگر نیز به نام‌های RCL و RCR وجود دارند که بیت‌های داده را همراه با پرچم نقلی می‌چرخانند. برای نمونه، اگر ثبات AX با این دستورالعمل‌ها چرخانده شود، مجموعهٔ ۱۷ بیتی متشکل از AX و پرچم نقلی چرخش می‌یابد.

```
1       mov    ax, 0C123H
2       clc                    ; clear the carry flag (CF = 0)
3       rcl    ax, 1           ; ax = 8246H, CF = 1
4       rcl    ax, 1           ; ax = 048DH, CF = 1
5       rcl    ax, 1           ; ax = 091BH, CF = 0
6       rcr    ax, 2           ; ax = 8246H, CF = 1
7       rcr    ax, 1           ; ax = C123H, CF = 0
```

۵.۱.۳ یک کاربرد ساده

قطعه کد زیر تعداد بیت‌های «روشن»، یعنی بیت‌هایی را که مقدارشان ۱ است، در ثبات EAX می‌شمارد.

```
1       mov    bl, 0           ; bl will contain the count of ON bits
2       mov    ecx, 32         ; ecx is the loop counter
3   count_loop:
4       shl    eax, 1          ; shift bit into carry flag
5       jnc    skip_inc        ; if CF == 0, goto skip_inc
6       inc    bl
7   skip_inc:
8       loop   count_loop
```

کد بالا مقدار اولیهٔ EAX را از بین می‌برد، زیرا در پایان حلقه EAX برابر صفر می‌شود. اگر بخواهیم مقدار EAX حفظ شود، می‌توانیم خط ۴ را با `rol eax, 1` جایگزین کنیم.

X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1

جدول ۱.۳: عمل AND

	1	0	1	0	1	0	1	0
AND	1	1	0	0	1	0	0	1
	1	0	0	0	1	0	0	0

شکل ۲.۳: اعمال AND روی یک بایت

۲.۳ عملیات بیتی بولی

چهار اپراتور[۸] بولی رایج وجود دارد: AND، OR، XOR و NOT. **جدول درستی**[۹] نتیجهٔ هر یک از این عملیات را به ازای تمام مقادیر ممکن عملگرهای آن نشان می‌دهد.

۱.۲.۳ عمل AND

همان‌طور که جدول درستی در جدول ۱.۳ نشان می‌دهد، حاصل AND دو بیت تنها زمانی ۱ است که هر دو بیت ۱ باشند و در غیر این صورت نتیجه ۰ خواهد بود.

پردازنده‌ها این عملیات را به صورت دستورالعمل‌هایی پشتیبانی می‌کنند که به طور مستقل و موازی روی همهٔ بیت‌های داده عمل می‌کنند. برای نمونه، اگر محتوای AL و BL با هم AND شوند، همان‌طور که شکل ۲.۳ نشان می‌دهد، عمل پایه‌ای AND روی هر یک از ۸ جفت بیت متناظر در این دو ثبات جداگانه اعمال می‌شود. در ادامه یک نمونه کد آمده است:

```
1        mov    ax, 0C123H
2        and    ax, 82F6H          ; ax = 8022H
```

X	Y	X OR Y
0	0	0
0	1	1
1	0	1
1	1	1

جدول ۲.۳: عمل OR

X	Y	X XOR Y
0	0	0
0	1	1
1	0	1
1	1	0

جدول ۳.۳: عمل XOR

۲.۲.۳ عمل OR

حاصل OR فراگیر[10] دو بیت تنها زمانی ۰ است که هر دو بیت ۰ باشند و در غیر این صورت نتیجه ۱ است؛ جدول درستی در جدول ۲.۳ همین را نشان می‌دهد. نمونه کد زیر را ببینید:

```
1       mov     ax, 0C123H
2       or      ax, 0E831H          ; ax = E933H
```

۳.۲.۳ عمل XOR

حاصل OR انحصاری[11] دو بیت، اگر و تنها اگر دو بیت با هم برابر باشند، ۰ است و در غیر این صورت ۱ خواهد بود؛ جدول درستی در جدول ۳.۳ این را نشان می‌دهد. نمونه کد زیر را ببینید:

```
1       mov     ax, 0C123H
2       xor     ax, 0E831H          ; ax = 2912H
```

[8] operator
[9] truth table
[10] inclusive OR
[11] exclusive OR

X	NOT X
0	1
1	0

جدول ۴.۳: عمل NOT

۴.۲.۳ عمل NOT

عمل NOT یک عمل **یگانی**[۱۲] است؛ یعنی برخلاف عملیات **دوتایی**[۱۳] مانند AND که روی دو عملگر کار می‌کنند، تنها روی یک عملگر عمل می‌کند. همان‌طور که جدول درستی در جدول ۴.۳ نشان می‌دهد، NOT یک بیت برابر با مقدار مخالف همان بیت است. نمونه کد زیر را ببینید:

```
1    mov    ax, 0C123H
2    not    ax                ; ax = 3EDCH
```

توجه کنید که NOT در واقع مکمل یک[۱۴] عدد را به دست می‌دهد. همچنین دستورالعمل NOT، برخلاف دیگر عملیات بیتی، هیچ‌یک از بیت‌های ثبات FLAGS را تغییر نمی‌دهد.

۵.۲.۳ دستورالعمل TEST

دستورالعمل TEST یک عمل AND انجام می‌دهد، اما نتیجه را ذخیره نمی‌کند و تنها ثبات FLAGS را بر اساس نتیجه‌ای که به دست می‌آمد تنظیم می‌کند. این رفتار بسیار شبیه دستورالعمل CMP است که یک تفریق انجام می‌دهد ولی فقط FLAGS را تنظیم می‌کند. برای نمونه، اگر نتیجه صفر باشد، ZF یک می‌شود.

۶.۲.۳ کاربردهای عملیات بیتی

عملیات بیتی برای دست‌کاری تک‌تک بیت‌های داده، بدون تغییر دادن بیت‌های دیگر، بسیار سودمند هستند. جدول ۵.۳ سه کاربرد رایج این عملیات را نشان می‌دهد و کد زیر نمونه‌ای از پیاده‌سازی همین ایده‌هاست.

```
1    mov    ax, 0C123H
2    or     ax, 8          ; turn on bit 3,   ax = C12BH
```

[12] unary
[13] binary
[14] one's complement

روشن کردن بیت i	عدد را با 2^i، یعنی عدد دودویی‌ای که تنها بیت i آن روشن است، OR کنید
خاموش کردن بیت i	عدد را با عدد دودویی‌ای که تنها بیت i آن خاموش است AND کنید. به این عملگر اغلب **ماسک** گفته می‌شود
معکوس کردن بیت i	عدد را با $XOR\ 2^i$ کنید

جدول ۵.۳: کاربردهای عملیات بولی

```
3       and     ax, 0FFDFH      ; turn off bit 5,  ax = C10BH
4       xor     ax, 8000H       ; invert bit 15,   ax = 410BH
5       or      ax, 0F00H       ; turn on nibble,  ax = 4F0BH
6       and     ax, 0FFF0H      ; turn off nibble, ax = 4F00H
7       xor     ax, 0F00FH      ; invert nibbles,  ax = BF0FH
8       xor     ax, 0FFFFH      ; 1's complement,  ax = 40F0H
```

از عمل AND می‌توان برای یافتن باقی‌ماندهٔ تقسیم بر توانی از دو نیز استفاده کرد. برای به دست آوردن باقی‌ماندهٔ تقسیم بر 2^i، کافی است عدد را با ماسکی[۱۵] برابر $AND\ 2^i - 1$ کنیم. این ماسک از بیت ۰ تا بیت $i - 1$ شامل یک است و باقی‌مانده دقیقاً در همین بیت‌ها قرار دارد. بنابراین حاصل AND این بیت‌ها را نگه می‌دارد و بقیه را صفر می‌کند. قطعه کد زیر خارج قسمت و باقی‌ماندهٔ تقسیم ۱۰۰ بر ۱۶ را به دست می‌آورد.

```
1       mov     eax, 100        ; 100 = 64H
2       mov     ebx, 0000000FH  ; mask = 16 - 1 = 15 or F
3       and     ebx, eax        ; ebx = remainder = 4
4       shr     eax, 4          ; eax = quotient of eax/2^4 = 6
```

با کمک ثبات CL می‌توان بیت‌های دلخواهی از داده را تغییر داد. در نمونهٔ زیر یک بیت دلخواه در EAX یک (روشن) می‌شود و شمارهٔ بیت مورد نظر در BH ذخیره شده است.

```
1       mov     cl, bh          ; first build the number to OR with
2       mov     ebx, 1
3       shl     ebx, cl         ; shift left cl times
4       or      eax, ebx        ; turn on bit
```

[15] mask

```
1       mov    bl, 0           ; bl will contain the count of ON bits
2       mov    ecx, 32         ; ecx is the loop counter
3  count_loop:
4       shl    eax, 1          ; shift bit into carry flag
5       adc    bl, 0           ; add just the carry flag to bl
6       loop   count_loop
```

شکل ۳.۳: شمارش بیت‌ها با ADC

خاموش کردن یک بیت کمی دشوارتر است:

```
1       mov    cl, bh          ; first build the number to AND with
2       mov    ebx, 1
3       shl    ebx, cl         ; shift left cl times
4       not    ebx             ; invert bits
5       and    eax, ebx        ; turn off bit
```

نوشتن کدی که یک بیت دلخواه را معکوس کند، به عنوان تمرین به خواننده واگذار می‌شود.

دیدن دستورالعمل معماگونهٔ زیر در برنامه‌های 80x86 چندان هم غیرعادی نیست:

```
xor    eax, eax          ; eax = 0
```

حاصل XOR هر عدد با خودش همیشه صفر است و این دستورالعمل از آن رو به کار می‌رود که کد ماشین آن از کد ماشین دستورالعمل MOV معادل کوچک‌تر است.

۳.۳ پرهیز از انشعاب‌های شرطی

پردازنده‌های امروزی برای اجرای هر چه سریع‌تر کد از روش‌های بسیار پیچیده‌ای بهره می‌گیرند. یکی از روش‌های رایج، **اجرای گمانه‌زنانه**[۱۶] نام دارد که از توانایی پردازش موازی پردازنده برای اجرای هم‌زمان چند دستورالعمل استفاده می‌کند. اما انشعاب‌های شرطی[۱۷] این ایده را با مشکل روبه‌رو می‌کنند، زیرا پردازنده در حالت کلی نمی‌داند که انشعاب انجام خواهد شد یا نه، و بسته به اینکه انشعاب انجام شود یا نشود، مجموعهٔ متفاوتی از دستورالعمل‌ها اجرا خواهد شد. از این رو پردازنده‌ها می‌کوشند

[16] speculative execution
[17] conditional branch

انجام شدن یا نشدن انشعاب را پیش‌بینی کنند و اگر پیش‌بینی نادرست باشد، پردازنده وقت خود را صرف اجرای کد نادرست کرده است.

یک راه برای پرهیز از این مشکل آن است که تا جای ممکن از انشعاب‌های شرطی استفاده نکنیم. نمونه کد بخش ۵.۱.۳ مثال ساده‌ای از موقعیتی است که می‌توان چنین کرد. در آن مثال، بیت‌های «روشن» ثبات EAX شمرده می‌شد و برای رد شدن از دستورالعمل INC از یک انشعاب استفاده شده بود. شکل ۳.۳ نشان می‌دهد که چگونه می‌توان با به کار بردن دستورالعمل ADC و افزودن مستقیم پرچم نقلی، این انشعاب را حذف کرد.

دستورالعمل‌های SET*xx* نیز راهی برای حذف انشعاب‌ها در برخی موارد فراهم می‌کنند. این دستورالعمل‌ها مقدار یک ثبات بایتی یا یک خانهٔ حافظه را بر اساس وضعیت ثبات FLAGS برابر صفر یا یک قرار می‌دهند. نویسه‌هایی که پس از SET می‌آیند همان نویسه‌هایی هستند که در انشعاب‌های شرطی به کار می‌روند. اگر شرط متناظر با SET*xx* برقرار باشد، مقدار ذخیره‌شده یک و در غیر این صورت صفر خواهد بود. برای نمونه:

```
setz   al        ; AL = 1 if Z flag is set, else 0
```

با استفاده از این دستورالعمل‌ها می‌توان روش‌های هوشمندانه‌ای ابداع کرد که مقادیر را بدون هیچ انشعابی محاسبه می‌کنند.

برای نمونه، مسئلهٔ یافتن بیشینهٔ دو مقدار را در نظر بگیرید. روش متداول برای حل این مسئله آن است که از یک CMP استفاده کنیم و سپس با یک انشعاب شرطی، بر اساس اینکه کدام مقدار بزرگ‌تر است، عمل کنیم. برنامهٔ نمونهٔ زیر نشان می‌دهد که چگونه می‌توان بیشینه را بدون هیچ انشعابی پیدا کرد.

```
1  ; file: max.asm
2  %include "asm_io.inc"
3  segment .data
4
5  message1 db "Enter a number: ",0
6  message2 db "Enter another number: ", 0
7  message3 db "The larger number is: ", 0
8
9  segment .bss
10
11 input1  resd    1        ; first number entered
12
13 segment .text
```

```
14          global  _asm_main
15  _asm_main:
16          enter   0,0               ; setup routine
17          pusha
18
19          mov     eax, message1     ; print out first message
20          call    print_string
21          call    read_int          ; input first number
22          mov     [input1], eax
23
24          mov     eax, message2     ; print out second message
25          call    print_string
26          call    read_int          ; input second number (in eax)
27
28          xor     ebx, ebx          ; ebx = 0
29          cmp     eax, [input1]     ; compare second and first number
30          setg    bl                ; ebx = (input2 > input1) ?          1 : 0
31          neg     ebx               ; ebx = (input2 > input1) ? 0xFFFFFFFF : 0
32          mov     ecx, ebx          ; ecx = (input2 > input1) ? 0xFFFFFFFF : 0
33          and     ecx, eax          ; ecx = (input2 > input1) ?     input2 : 0
34          not     ebx               ; ebx = (input2 > input1) ?          0 : 0xFFFFFFFF
35          and     ebx, [input1]     ; ebx = (input2 > input1) ?          0 : input1
36          or      ecx, ebx          ; ecx = (input2 > input1) ?     input2 : input1
37
38          mov     eax, message3     ; print out result
39          call    print_string
40          mov     eax, ecx
41          call    print_int
42          call    print_nl
43
44          popa
45          mov     eax, 0            ; return back to C
46          leave
47          ret
```

ترفند کار در ساختن یک ماسک بیتی است که بتوان با آن مقدار درست را به عنوان بیشینه برگزید. دستورالعمل SETG در خط ۳۰ اگر ورودی دوم بیشینه باشد BL را برابر ۱ و در غیر این صورت برابر ۰ قرار می‌دهد. این هنوز همان ماسک بیتی مورد نظر نیست. برای ساختن ماسک لازم، خط ۳۱ دستورالعمل NEG را روی کل ثبات EBX اعمال می‌کند (توجه کنید که EBX پیش‌تر صفر شده بود). اگر EBX برابر ۰ باشد،

این کار هیچ اثری ندارد، اما اگر EBX برابر ۱ باشد، حاصل نمایش مکمل دو[۱۸] عدد -1، یعنی 0xFFFFFFFF، خواهد بود که دقیقاً همان ماسک بیتی لازم است. بقیهٔ کد با استفاده از این ماسک بیتی، ورودی درست را به عنوان بیشینه انتخاب می‌کند.

ترفند دیگر، استفاده از دستورالعمل DEC است. اگر در کد بالا NEG را با DEC جایگزین کنیم، باز هم نتیجه یا ۰ یا 0xFFFFFFFF خواهد بود، با این تفاوت که این مقادیر نسبت به حالت استفاده از دستورالعمل NEG برعکس هستند.

۴.۳ دست‌کاری بیت‌ها در C

۱.۴.۳ اپراتورهای بیتی C

زبان C، برخلاف برخی زبان‌های سطح بالا، اپراتورهایی برای عملیات بیتی در اختیار برنامه‌نویس می‌گذارد. عمل AND با اپراتور دوتایی & نمایش داده می‌شود[۱۹]، عمل OR با اپراتور دوتایی |، عمل XOR با اپراتور دوتایی ^ و سرانجام عمل NOT با اپراتور یگانی ~.

عملیات جابه‌جایی در C با اپراتورهای دوتایی >> و << انجام می‌شود؛ اپراتور >> جابه‌جایی به چپ و اپراتور << جابه‌جایی به راست را انجام می‌دهد. این اپراتورها دو عملگر می‌گیرند: عملگر سمت چپ مقداری است که باید جابه‌جا شود و عملگر سمت راست تعداد بیت‌هایی است که باید جابه‌جا شوند. اگر مقدار مورد نظر از یک نوع بی‌علامت باشد، جابه‌جایی منطقی انجام می‌شود و اگر از یک نوع علامت‌دار (مانند int) باشد، جابه‌جایی حسابی به کار می‌رود. در ادامه نمونه کدی به زبان C آمده است که از این اپراتورها استفاده می‌کند:

```
1  short int s;          /* assume that short int is 16-bit */
2  short unsigned u;
3  s = -1;               /* s = 0xFFFF (2's complement) */
4  u = 100;              /* u = 0x0064 */
5  u = u | 0x0100;       /* u = 0x0164 */
6  s = s & 0xFFF0;       /* s = 0xFFF0 */
7  s = s ^ u;            /* s = 0xFE94 */
8  u = u << 3;           /* u = 0x0B20 (logical shift ) */
9  s = s >> 2;           /* s = 0xFFA5 (arithmetic shift ) */
```

[18] two's complement

[۱۹] این اپراتور با اپراتور دوتایی && و اپراتور یگانی & تفاوت دارد!

ماکرو	معنا
S_IRUSR	کاربر می‌تواند بخواند
S_IWUSR	کاربر می‌تواند بنویسد
S_IXUSR	کاربر می‌تواند اجرا کند
S_IRGRP	گروه می‌تواند بخواند
S_IWGRP	گروه می‌تواند بنویسد
S_IXGRP	گروه می‌تواند اجرا کند
S_IROTH	دیگران می‌توانند بخوانند
S_IWOTH	دیگران می‌توانند بنویسند
S_IXOTH	دیگران می‌توانند اجرا کنند

جدول ۶.۳: ماکروهای مجوز فایل در POSIX

۲.۴.۳ به کارگیری اپراتورهای بیتی در C

در زبان C نیز اپراتورهای بیتی برای همان مقاصدی به کار می‌روند که در زبان اسمبلی به کار می‌روند. این اپراتورها امکان دست‌کاری تک‌تک بیت‌های داده را فراهم می‌کنند و می‌توان از آن‌ها برای ضرب و تقسیم سریع استفاده کرد. در واقع یک کامپایلر هوشمند C برای ضربی مانند `x *= 2` به طور خودکار از جابه‌جایی استفاده می‌کند.

بسیاری از APIهای[۲۰] سیستم‌عامل‌ها (مانند *POSIX*[۲۱] و Win32) توابعی دارند که عملگرهای آن‌ها داده‌هایی را به صورت بیتی کدگذاری‌شده در خود دارند. برای نمونه، سیستم‌های POSIX مجوزهای فایل را برای سه دستهٔ متفاوت از کاربران نگه می‌دارند: **کاربر** (که نام بهتر آن **مالک** است)، **گروه** و **دیگران**. به هر دسته از کاربران می‌توان مجوز خواندن، نوشتن و یا اجرای فایل را داد. برای تغییر مجوزهای یک فایل، برنامه‌نویس C باید تک‌تک بیت‌ها را دست‌کاری کند و POSIX برای کمک به این کار چندین ماکرو[۲۲] تعریف کرده است (جدول ۶.۳ را ببینید). برای تنظیم مجوزهای فایل می‌توان از تابع `chmod` استفاده کرد. این تابع دو پارامتر می‌گیرد: رشته‌ای شامل نام فایلی که باید روی آن عمل شود، و یک عدد صحیح[۲۳] که بیت‌های متناظر با مجوزهای

[۲۰] واسط برنامه‌نویسی کاربردی (Application Programming Interface)

[۲۱] این نام به معنای «واسط سیستم‌عامل قابل‌حمل برای محیط‌های رایانه‌ای» است و به استانداردی اشاره دارد که IEEE بر پایهٔ یونیکس تدوین کرده است. شکل کامل نام چنین است: Portable Operating System Interface for Computer Environments.

[22] macro

[۲۳] در واقع پارامتری از نوع `mode_t` که با typedef به یک نوع صحیح تعریف شده است.

مورد نظر در آن یک شده‌اند. برای نمونه، کد زیر مجوزها را طوری تنظیم می‌کند که مالک فایل بتواند آن را بخواند و در آن بنویسد، کاربران گروه بتوانند فایل را بخوانند و دیگران هیچ دسترسی‌ای نداشته باشند.

```
chmod("foo", S_IRUSR | S_IWUSR | S_IRGRP );
```

برای به دست آوردن بیت‌های مجوز کنونی فایل می‌توان از تابع `stat` در POSIX استفاده کرد. اگر این تابع همراه با تابع `chmod` به کار رود، می‌توان برخی از مجوزها را بدون تغییر دادن بقیه اصلاح کرد. در نمونهٔ زیر، دسترسی نوشتن از دیگران گرفته می‌شود و دسترسی خواندن به مالک فایل افزوده می‌شود، بی‌آنکه مجوزهای دیگر تغییری کنند.

```
1 struct stat file_stats ;      /* struct used by stat() */
2 stat("foo", &file_stats );   /* read file info.
3                                 file_stats.st_mode holds permission bits */
4 chmod("foo", ( file_stats.st_mode & ~S_IWOTH) | S_IRUSR);
```

۵.۳ نمایش‌های بزرگ‌انتها و کوچک‌انتها

در فصل ۱ با مفهوم نمایش بزرگ‌انتها و کوچک‌انتهای داده‌های چندبایتی آشنا شدیم. با این حال، نویسنده دریافته است که این موضوع بسیاری از افراد را سردرگم می‌کند؛ از این رو در این بخش آن را با جزئیات بیشتری بررسی می‌کنیم.

به یاد دارید که ترتیب بایت‌ها[۲۴] به ترتیبی اشاره دارد که تک‌تک بایت‌های (و **نه** بیت‌های) یک عنصر دادهٔ چندبایتی در حافظه ذخیره می‌شوند. ترتیب بزرگ‌انتها[۲۵] سرراست‌ترین روش است و در آن ابتدا پرارزش‌ترین بایت، سپس بایت پرارزش بعدی و به همین ترتیب تا آخر ذخیره می‌شود؛ به بیان دیگر، بیت‌های **بزرگ** زودتر ذخیره می‌شوند. در ترتیب کوچک‌انتها[۲۶]، بایت‌ها به ترتیب عکس ذخیره می‌شوند، یعنی ابتدا کم‌ارزش‌ترین بایت. خانوادهٔ پردازنده‌های x86 از نمایش کوچک‌انتها استفاده می‌کند.

برای نمونه، کلمهٔ مضاعفی را در نظر بگیرید که عدد 12345678_{16} را نمایش می‌دهد. در نمایش بزرگ‌انتها، بایت‌ها به صورت 78 56 34 12 و در نمایش کوچک‌انتها به صورت 12 34 56 78 ذخیره می‌شوند.

احتمالاً خواننده هم‌اکنون از خود می‌پرسد که چرا یک طراح تراشهٔ عاقل باید از نمایش کوچک‌انتها استفاده کند؟ آیا مهندسان اینتل آن‌قدر بی‌رحم بودند که این

[24] endianness
[25] big endian
[26] little endian

```
unsigned short word = 0x1234;  /* assumes sizeof(short) == 2 */
unsigned char * p = (unsigned char *) &word;

if ( p[0] == 0x12 )
   printf ("Big Endian Machine\n");
else
   printf (" Little  Endian Machine\n");
```

شکل ۴.۳: چگونگی تعیین ترتیب بایت‌ها

نمایش گیج‌کننده را به انبوهی از برنامه‌نویسان تحمیل کنند؟ به نظر می‌رسد پردازنده برای ذخیرهٔ وارونهٔ بایت‌ها در حافظه (و برگرداندن آن‌ها به ترتیب درست هنگام خواندن دوباره) باید کار اضافه‌ای انجام دهد. اما پاسخ این است که پردازنده برای نوشتن و خواندن حافظه با قالب کوچک‌انتها هیچ کار اضافه‌ای انجام نمی‌دهد. باید توجه داشت که پردازنده از مدارهای الکترونیکی فراوانی تشکیل شده است که صرفاً روی مقادیر بیتی کار می‌کنند و بیت‌ها (و بایت‌ها) درون پردازنده هیچ ترتیب الزامی‌ای ندارند.

ثبات دوبایتی AX را در نظر بگیرید که می‌توان آن را به دو ثبات تک‌بایتی AH و AL تجزیه کرد. در پردازنده مدارهایی وجود دارند که مقادیر AH و AL را نگه می‌دارند، اما این مدارها در پردازنده ترتیب خاصی ندارند؛ یعنی مدارهای AH نه پیش از مدارهای AL قرار دارند و نه پس از آن‌ها. دستورالعمل mov که مقدار AX را در حافظه رونویسی می‌کند، ابتدا مقدار AL و سپس AH را رونویسی می‌کند و این کار برای پردازنده به هیچ وجه دشوارتر از آن نیست که ابتدا AH را ذخیره کند.

همین استدلال دربارهٔ تک‌تک بیت‌های یک بایت نیز صادق است. این بیت‌ها نیز در مدارهای پردازنده (و در واقع حافظه) ترتیب خاصی ندارند. با این حال، از آنجا که در پردازنده یا حافظه نمی‌توان تک‌تک بیت‌ها را آدرس‌دهی کرد، هیچ راهی برای دانستن ترتیبی که پردازنده ظاهراً بیت‌ها را در درون خود نگه می‌دارد وجود ندارد و اهمیتی هم ندارد.

کد C در شکل ۴.۳ نشان می‌دهد که چگونه می‌توان ترتیب بایت‌های یک پردازنده را تعیین کرد. اشاره‌گر p با متغیر word مانند یک آرایهٔ نویسه‌ای دوعنصری رفتار می‌کند. بنابراین p[0] برابر با نخستین بایت word در حافظه خواهد بود که به ترتیب بایت‌های پردازنده بستگی دارد.

۱.۵.۳ چه زمانی ترتیب کوچک‌انتها و بزرگ‌انتها اهمیت دارد؟

در برنامه‌نویسی‌های معمولی، ترتیب بایت‌های پردازنده اهمیتی ندارد و رایج‌ترین موقعیتی که این موضوع اهمیت پیدا می‌کند، هنگام انتقال داده‌های دودویی میان سیستم‌های رایانه‌ای متفاوت است. این انتقال معمولاً یا از راه نوعی رسانهٔ فیزیکی داده (مانند دیسک) انجام می‌شود یا از راه شبکه. اما از آنجا که داده‌های ASCII تک‌بایتی هستند، ترتیب بایت‌ها برای آن‌ها مسئله‌ای ایجاد نمی‌کند.

با پیدایش مجموعه‌نویسه‌های چندبایتی مانند یونیکد، ترتیب بایت‌ها حتی برای داده‌های متنی نیز اهمیت یافته است. یونیکد از هر دو ترتیب پشتیبانی می‌کند و سازوکاری برای مشخص کردن ترتیبی که برای نمایش داده‌ها به کار رفته است در اختیار می‌گذارد.

همهٔ سرآیندهای داخلی TCP/IP اعداد صحیح را با قالب بزرگ‌انتها ذخیره می‌کنند که **ترتیب بایتی شبکه**[۲۷] نامیده می‌شود. کتابخانه‌های TCP/IP توابعی به زبان C فراهم می‌کنند تا بتوان مسائل مربوط به ترتیب بایت‌ها را به شکلی قابل‌حمل مدیریت کرد. برای نمونه، تابع `htonl()` یک کلمهٔ مضاعف (یا عدد صحیح بلند) را از قالب **میزبان** به قالب **شبکه** تبدیل می‌کند و تابع `ntohl()` تبدیل عکس آن را انجام می‌دهد.[۲۸] در یک سیستم بزرگ‌انتها، این دو تابع ورودی خود را بدون تغییر بازمی‌گردانند. به این ترتیب می‌توان برنامه‌های شبکه‌ای نوشت که صرف نظر از ترتیب بایت‌های سیستم، روی هر سیستمی به درستی کامپایل و اجرا شوند. برای آگاهی بیشتر دربارهٔ ترتیب بایت‌ها و برنامه‌نویسی شبکه، کتاب بسیار خوب W. Richard Steven با عنوان *UNIX Network Programming* را ببینید.

شکل ۵.۳ یک تابع C را نشان می‌دهد که ترتیب بایت‌های یک کلمهٔ مضاعف را وارونه می‌کند. پردازندهٔ 486 دستورالعمل ماشین تازه‌ای به نام `BSWAP` معرفی کرد که بایت‌های هر ثبات ۳۲ بیتی را وارونه می‌کند. برای نمونه:

```
bswap   edx          ; swap bytes of edx
```

این دستورالعمل را نمی‌توان روی ثبات‌های ۱۶ بیتی به کار برد. با این حال، برای جابه‌جا کردن بایت‌های آن دسته از ثبات‌های ۱۶ بیتی که به ثبات‌های ۸ بیتی تجزیه‌پذیر هستند، می‌توان از دستورالعمل `XCHG` استفاده کرد. برای نمونه:

```
xchg    ah,al        ; swap bytes of ax
```

---

[27] network byte order

[۲۸] در واقع وارونه کردن ترتیب بایت‌های یک عدد صحیح صرفاً بایت‌های آن را وارونه می‌کند؛ بنابراین تبدیل از بزرگ‌انتها به کوچک‌انتها و از کوچک‌انتها به بزرگ‌انتها یک عمل واحد است و هر دو تابع در واقع یک کار انجام می‌دهند.

```
1  unsigned invert_endian( unsigned x )
2  {
3    unsigned invert;
4    const unsigned char * xp = (const unsigned char *) &x;
5    unsigned char * ip = (unsigned char *) & invert;
6
7    ip[0] = xp[3];    /* reverse the individual bytes */
8    ip[1] = xp[2];
9    ip[2] = xp[1];
10   ip[3] = xp[0];
11
12   return invert;    /* return the bytes reversed */
13 }
```

شکل ۵.۳: تابع `invert_endian`

۶.۳ شمارش بیت‌ها

پیش‌تر روشی سرراست برای شمردن تعداد بیت‌های «روشن» یک کلمهٔ مضاعف ارائه شد. در این بخش، به عنوان تمرینی برای به کار بستن عملیات بیتی که در این فصل بررسی شد، روش‌های دیگری را بررسی می‌کنیم که کمتر مستقیم هستند.

۱.۶.۳ روش اول

روش اول بسیار ساده است، اما در نگاه نخست بدیهی به نظر نمی‌رسد. کد آن در شکل ۶.۳ آمده است.

این روش چگونه کار می‌کند؟ در هر تکرار حلقه، یک بیت در `data` خاموش می‌شود و هنگامی که همهٔ بیت‌ها خاموش شوند، یعنی `data` برابر صفر شود، حلقه متوقف می‌شود. بنابراین تعداد تکرارهای لازم برای صفر کردن `data` برابر با تعداد بیت‌های روشن در مقدار اولیهٔ `data` است.

خاموش شدن یک بیت از `data` در خط ۶ رخ می‌دهد، اما این کار چگونه انجام می‌شود؟ شکل کلی نمایش دودویی `data` و راست‌ترین ۱ را در این نمایش در نظر بگیرید. بنا به تعریف، همهٔ بیت‌های پس از این ۱ باید صفر باشند. حال نمایش دودویی `data - 1` چه خواهد بود؟ بیت‌های سمت چپ راست‌ترین ۱ همان بیت‌های `data` خواهند بود، اما از جایگاه راست‌ترین ۱ به بعد، بیت‌ها مکمل بیت‌های اولیهٔ `data` می‌شوند. برای نمونه:

```
1  int count_bits( unsigned int data )
2  {
3    int cnt = 0;
4
5    while( data != 0 ) {
6      data = data & (data − 1);
7      cnt++;
8    }
9    return cnt;
10 }
```

شکل ۶.۳: شمارش بیت‌ها: روش اول

$$\begin{aligned} \texttt{data} &= \text{xxxxx10000} \\ \texttt{data - 1} &= \text{xxxxx01111} \end{aligned}$$

که در آن xها در هر دو عدد یکسان هستند. هنگامی که data با 1 - data AND شود، راست‌ترین ۱ در data صفر می‌شود و همهٔ بیت‌های دیگر بدون تغییر باقی می‌مانند.

۲.۶.۳ روش دوم

برای شمردن بیت‌های یک کلمهٔ مضاعف دلخواه می‌توان از جدول جست‌وجو[۲۹] نیز استفاده کرد. روش سرراست آن است که تعداد بیت‌های هر کلمهٔ مضاعف را از پیش محاسبه کنیم و در یک آرایه نگه داریم، اما این روش دو مشکل به هم مرتبط دارد: تعداد مقادیر ممکن برای یک کلمهٔ مضاعف تقریباً **۴ میلیارد** است! این یعنی آرایه بسیار بزرگ خواهد شد و مقداردهی اولیهٔ آن نیز زمان بسیار زیادی خواهد گرفت. (در واقع اگر قرار نباشد آرایه بیش از ۴ میلیارد بار به کار رود، زمانی که صرف مقداردهی اولیهٔ آن می‌شود از زمان لازم برای محاسبهٔ مستقیم تعداد بیت‌ها با روش اول بیشتر خواهد بود!)

روش واقع‌بینانه‌تر آن است که تعداد بیت‌های همهٔ مقادیر ممکن یک بایت را از پیش محاسبه کنیم و در یک آرایه ذخیره کنیم. سپس کلمهٔ مضاعف را به چهار مقدار بایتی تقسیم می‌کنیم، تعداد بیت‌های هر یک از این چهار بایت را از آرایه می‌خوانیم و آن‌ها را با هم جمع می‌کنیم تا تعداد بیت‌های کلمهٔ مضاعف اصلی به دست آید.

[۲۹] lookup table

```
1  static unsigned char byte_bit_count[256]; /* lookup table */
2
3  void initialize_count_bits ()
4  {
5    int cnt, i, data;
6
7    for( i = 0; i < 256; i++ ) {
8      cnt = 0;
9      data = i;
10     while( data != 0 ) {          /* method one */
11       data = data & (data - 1);
12       cnt++;
13     }
14     byte_bit_count[i] = cnt;
15   }
16 }
17
18 int count_bits( unsigned int data )
19 {
20   const unsigned char * byte = ( unsigned char *) & data;
21
22   return byte_bit_count[byte[0]] + byte_bit_count[byte[1]] +
23          byte_bit_count[byte[2]] + byte_bit_count[byte[3]];
24 }
```

شکل ۷.۳: روش دوم

شکل ۷.۳ کد پیاده‌سازی این روش را نشان می‌دهد.

تابع `initialize_count_bits` که وظیفهٔ مقداردهی اولیهٔ آرایهٔ سراسری `byte_bit_count` را بر عهده دارد، باید پیش از نخستین فراخوانی تابع `count_bits` فراخوانی شود. تابع `count_bits` به متغیر `data` نه به چشم یک کلمهٔ مضاعف، بلکه به چشم آرایه‌ای از چهار بایت نگاه می‌کند و اشاره‌گر `dword` نقش اشاره‌گری به این آرایهٔ چهاربایتی را دارد. بنابراین `dword[0]` یکی از بایت‌های `data` است که بسته به اینکه سخت‌افزار کوچک‌انتها باشد یا بزرگ‌انتها، به ترتیب کم‌ارزش‌ترین یا پرارزش‌ترین بایت خواهد بود. البته می‌توان برای یافتن مقدار پرارزش‌ترین بایت از ساختاری مانند

(data >> 24) & 0x000000FF

و برای بایت‌های دیگر از ساختارهای مشابه استفاده کرد، اما این ساختارها از ارجاع به

```
1  int count_bits(unsigned int x )
2  {
3    static unsigned int mask[] = { 0x55555555,
4                                   0x33333333,
5                                   0x0F0F0F0F,
6                                   0x00FF00FF,
7                                   0x0000FFFF };
8    int i;
9    int shift;    /* number of positions to shift to right */
10
11   for( i=0, shift=1; i < 5; i++, shift *= 2 )
12     x = (x & mask[i]) + ( (x >> shift) & mask[i] );
13   return x;
14 }
```

شکل ۸.۳: روش سوم

آرایه کندتر هستند.

نکتهٔ پایانی اینکه به آسانی می‌شد برای محاسبهٔ مجموع در خط‌های ۲۲ و ۲۳ از یک حلقهٔ `for` استفاده کرد، اما حلقهٔ `for` سربار مقداردهی اولیهٔ اندیس حلقه، مقایسهٔ اندیس پس از هر تکرار و افزایش اندیس را به همراه دارد. از این رو محاسبهٔ مجموع به صورت جمع صریح چهار مقدار سریع‌تر خواهد بود. در واقع یک کامپایلر هوشمند، نسخهٔ دارای حلقهٔ `for` را خود به جمع صریح تبدیل می‌کند. این فرایند کاهش یا حذف تکرارهای حلقه، یکی از روش‌های بهینه‌سازی کامپایلر است که **باز کردن حلقه**[۳۰] نامیده می‌شود.

۳.۶.۳ روش سوم

روش هوشمندانهٔ دیگری نیز برای شمردن بیت‌های روشن داده وجود دارد. در این روش، یک‌ها و صفرهای داده به معنای واقعی کلمه با هم جمع می‌شوند و این مجموع باید برابر با تعداد یک‌های داده باشد. برای نمونه، شمردن یک‌های بایتی را در نظر بگیرید که در متغیری به نام `data` ذخیره شده است. گام نخست انجام عمل زیر است:

```
data = (data & 0x55) + ((data >> 1) & 0x55);
```

این عمل چه می‌کند؟ ثابت شانزده‌شانزدهی `0x55` در مبنای دو برابر 01010101 است. در عملگر اول جمع، `data` با این ثابت AND می‌شود و بیت‌های واقع در

[30] loop unrolling

جایگاه‌های فرد جدا می‌شوند. عملگر دوم، یعنی ((data >> 1) & 0x55)، ابتدا همهٔ بیت‌های واقع در جایگاه‌های زوج را به یک جایگاه فرد منتقل می‌کند و سپس با همان ماسک همین بیت‌ها را جدا می‌کند. اکنون عملگر اول شامل بیت‌های فرد و عملگر دوم شامل بیت‌های زوج data است و هنگامی که این دو عملگر با هم جمع شوند، بیت‌های زوج و فرد data با هم جمع می‌شوند. برای نمونه، اگر data برابر 10110011_2 باشد، آن‌گاه:

data & 01010101_2		00	01	00	01
+ (data >> 1) & 01010101_2	یا +	01	01	00	01
		01	10	00	10

جمع سمت راست، بیت‌هایی را که واقعاً با هم جمع می‌شوند نشان می‌دهد. بیت‌های بایت به چهار میدان دوبیتی تقسیم شده‌اند تا روشن شود که در واقع چهار جمع مستقل انجام می‌شود. از آنجا که هیچ‌یک از این مجموع‌ها بیش از دو نمی‌شود، امکان ندارد مجموعی از میدان خود سرریز کند و مجموع میدان دیگری را خراب کند.

البته هنوز تعداد کل بیت‌ها محاسبه نشده است، اما می‌توان همان روشی را که در بالا به کار رفت، در چند گام مشابه برای محاسبهٔ مجموع کل به کار برد. گام بعدی چنین خواهد بود:

```
data = (data & 0x33) + ((data >> 2) & 0x33);
```

اکنون مثال بالا را ادامه می‌دهیم (به یاد داشته باشید که data اکنون برابر 01100010_2 است):

data & 00110011_2		0010	0010
+ (data >> 2) & 00110011_2	یا +	0001	0000
		0011	0010

این بار دو میدان چهاربیتی داریم که به طور مستقل با هم جمع می‌شوند.

در گام بعد، این دو مجموع بیتی با هم جمع می‌شوند تا نتیجهٔ نهایی به دست آید:

```
data = (data & 0x0F) + ((data >> 4) & 0x0F);
```

با ادامهٔ همان مثال (که در آن data برابر 00110010_2 است) خواهیم داشت:

data & 00001111_2		00000010
+ (data >> 4) & 00001111_2	یا +	00000011
		00000101

اکنون data برابر ۵ است که نتیجهٔ درست است. شکل ۸.۳ پیاده‌سازی این روش را برای شمردن بیت‌های یک کلمهٔ مضاعف نشان می‌دهد. این پیاده‌سازی برای محاسبهٔ مجموع از یک حلقهٔ for استفاده می‌کند. باز کردن حلقه سرعت را بیشتر می‌کرد، اما با وجود حلقه روشن‌تر دیده می‌شود که این روش چگونه به اندازه‌های مختلف داده تعمیم می‌یابد.

فصل ۴

زیربرنامه‌ها

در این فصل خواهیم دید که چگونه می‌توان به کمک زیربرنامه‌ها[۱] برنامه‌هایی پیمانه‌ای[۲] ساخت و میان اسمبلی و زبان‌های سطح بالایی[۳] مانند C ارتباط برقرار کرد. توابع و رویه‌ها در واقع همان زیربرنامه‌ها در زبان‌های سطح بالا هستند.

کدی که زیربرنامه را فراخوانی می‌کند و خود زیربرنامه باید دربارهٔ شیوهٔ انتقال داده‌ها میان خود به توافق برسند. به این قواعدِ انتقال داده، **قراردادهای فراخوانی**[۴] گفته می‌شود. بخش بزرگی از این فصل به قراردادهای فراخوانی استاندارد C می‌پردازد که با آن‌ها می‌توان زیربرنامه‌های اسمبلی را به برنامه‌های C پیوند زد. این قرارداد (و بسیاری از قراردادهای دیگر) اغلب به‌جای خود داده‌ها، آدرس آن‌ها (**یعنی** اشاره‌گرها[۵]) را منتقل می‌کند تا زیربرنامه بتواند به داده‌های موجود در حافظه دسترسی پیدا کند.

۱.۴ آدرس‌دهی غیرمستقیم

آدرس‌دهی غیرمستقیم[۶] به ثبات‌ها[۷] این امکان را می‌دهد که مانند متغیرهای اشاره‌گر عمل کنند. برای آنکه نشان دهیم ثباتی باید به‌طور غیرمستقیم و در نقش اشاره‌گر به کار رود، آن را درون کروشه ([]) قرار می‌دهیم. به نمونهٔ زیر توجه کنید:

[1] subprogram
[2] modular
[3] high level language
[4] calling conventions
[5] pointer
[6] indirect addressing
[7] register

```
1  mov    ax, [Data]     ; normal direct memory addressing of a word
2  mov    ebx, Data      ; ebx = & Data
3  mov    ax, [ebx]      ; ax = *ebx
```

از آنجا که AX یک کلمه[8] را در خود جای می‌دهد، خط ۳ کلمه‌ای را می‌خواند که از آدرس ذخیره‌شده در EBX آغاز می‌شود. اگر به‌جای AX از AL استفاده می‌شد، تنها یک بایت خوانده می‌شد. باید توجه داشت که ثبات‌ها برخلاف متغیرهای زبان C نوع ندارند. اینکه فرض شود EBX به چه چیزی اشاره می‌کند، یکسره به دستورالعمل‌هایی[9] بستگی دارد که به کار رفته‌اند؛ حتی خودِ اینکه EBX اشاره‌گر است نیز تنها از روی همین دستورالعمل‌ها معلوم می‌شود. از این رو اگر EBX نادرست به کار رود، اغلب اسمبلر[10] هیچ خطایی گزارش نمی‌کند، اما برنامه درست کار نخواهد کرد. این یکی از دلایل فراوانی است که برنامه‌نویسی اسمبلی را از برنامه‌نویسی سطح بالا خطاپذیرتر می‌کند.

همهٔ ثبات‌های همه‌منظورهٔ[11] ۳۲ بیتی (EAX، EBX، ECX و EDX) و نیز ثبات‌های اندیس[12] (ESI و EDI) را می‌توان در آدرس‌دهی غیرمستقیم به کار برد، اما ثبات‌های ۱۶ بیتی و ۸ بیتی را در حالت کلی نمی‌توان برای این کار استفاده کرد.

۲.۴ نمونه‌ای ساده از زیربرنامه

زیربرنامه واحد مستقلی از کد است که می‌توان آن را از بخش‌های گوناگون یک برنامه به کار گرفت؛ به بیان دیگر، زیربرنامه چیزی شبیه تابع در زبان C است. برای فراخواندن زیربرنامه می‌توان از یک پرش[13] استفاده کرد، اما بازگشت از آن دردسرساز است. اگر قرار باشد زیربرنامه از بخش‌های مختلف برنامه به کار رود، باید پس از پایان کار به همان بخشی از کد بازگردد که آن را فراخوانده است. بنابراین نمی‌توان مقصد پرش بازگشت را به‌صورت ثابت و با یک برچسب[14] مشخص در کد نوشت. کد زیر نشان می‌دهد که چگونه می‌توان این مشکل را با شکل غیرمستقیم دستورالعمل JMP حل کرد. در این شکل از دستورالعمل، مقصد پرش از روی مقدار یک ثبات تعیین می‌شود (به این ترتیب، ثبات در اینجا نقشی بسیار شبیه به یک **اشاره‌گر تابع**[15] در C دارد).

[8] word
[9] instruction
[10] assembler
[11] general purpose
[12] index
[13] jump
[14] label
[15] function pointer

در ادامه، نخستین برنامهٔ فصل ۱ را می‌بینید که این بار با استفاده از یک زیربرنامه بازنویسی شده است.

sub1.asm

```
; file: sub1.asm
; Subprogram example program
%include "asm_io.inc"

segment .data
prompt1 db    "Enter a number: ", 0       ; don't forget null terminator
prompt2 db    "Enter another number: ", 0
outmsg1 db    "You entered ", 0
outmsg2 db    " and ", 0
outmsg3 db    ", the sum of these is ", 0

segment .bss
input1  resd 1
input2  resd 1

segment .text
        global  _asm_main
_asm_main:
        enter   0,0               ; setup routine
        pusha

        mov     eax, prompt1      ; print out prompt
        call    print_string

        mov     ebx, input1       ; store address of input1 into ebx
        mov     ecx, ret1         ; store return address into ecx
        jmp     short get_int     ; read integer
ret1:
        mov     eax, prompt2      ; print out prompt
        call    print_string

        mov     ebx, input2
        mov     ecx, $ + 7        ; ecx = this address + 7
        jmp     short get_int

        mov     eax, [input1]     ; eax = dword at input1
        add     eax, [input2]     ; eax += dword at input2
        mov     ebx, eax          ; ebx = eax

```

```
40          mov     eax, outmsg1
41          call    print_string      ; print out first message
42          mov     eax, [input1]
43          call    print_int         ; print out input1
44          mov     eax, outmsg2
45          call    print_string      ; print out second message
46          mov     eax, [input2]
47          call    print_int         ; print out input2
48          mov     eax, outmsg3
49          call    print_string      ; print out third message
50          mov     eax, ebx
51          call    print_int         ; print out sum (ebx)
52          call    print_nl          ; print new-line
53
54          popa
55          mov     eax, 0            ; return back to C
56          leave
57          ret
58  ; subprogram get_int
59  ; Parameters:
60  ;   ebx - address of dword to store integer into
61  ;   ecx - address of instruction to return to
62  ; Notes:
63  ;   value of eax is destroyed
64  get_int:
65          call    read_int
66          mov     [ebx], eax          ; store input into memory
67          jmp     ecx                 ; jump back to caller
```

sub1.asm

زیربرنامهٔ get_int از یک قرارداد فراخوانی ساده و مبتنی بر ثبات استفاده می‌کند. این زیربرنامه انتظار دارد که ثبات EBX آدرس کلمهٔ مضاعفی (DWORD) را در خود داشته باشد که عدد ورودی باید در آن ذخیره شود، و ثبات ECX نیز آدرس دستورالعملی را نگه دارد که پس از پایان کار باید به آن بازگشت. در خطوط ۲۵ تا ۲۸ برای محاسبهٔ این آدرس بازگشت[۱۶] از برچسب ret1 استفاده شده است، در حالی که در خطوط ۳۲ تا ۳۴ همین آدرس با اپراتور[۱۷] $ محاسبه می‌شود. اپراتور $ آدرس جاری همان خطی را برمی‌گرداند که در آن آمده است؛ از این رو عبارت $ + 7 آدرس دستورالعمل MOV در خط ۳۶ را به دست می‌دهد.

[16] return address
[17] operator

هر دو روش محاسبهٔ آدرس بازگشت دست‌وپاگیر هستند. روش نخست ایجاب می‌کند که برای هر فراخوانی زیربرنامه یک برچسب تعریف شود. روش دوم به برچسب نیازی ندارد، اما دقت فراوانی می‌طلبد؛ برای نمونه، اگر به‌جای پرش کوتاه[۱۸] از پرش نزدیک[۱۹] استفاده می‌شد، عددی که باید به $ افزوده شود دیگر ۷ نبود! خوشبختانه راه بسیار ساده‌تری برای فراخواندن زیربرنامه‌ها وجود دارد که از **پشته** بهره می‌گیرد.

۳.۴ پشته

بسیاری از پردازنده‌ها به‌طور داخلی از پشته[۲۰] پشتیبانی می‌کنند. پشته فهرستی از نوع «آخرین ورودی، نخستین خروجی» (*LIFO*[۲۱]) است و در عمل، ناحیه‌ای از حافظه است که به همین شیوه سازمان‌دهی شده است. دستورالعمل PUSH داده‌ای را به پشته می‌افزاید و دستورالعمل POP داده‌ای را از آن برمی‌دارد. داده‌ای که برداشته می‌شود همواره آخرین داده‌ای است که افزوده شده است (و به همین دلیل است که پشته را فهرست «آخرین ورودی، نخستین خروجی» می‌نامند).

ثبات قطعهٔ[۲۲] SS قطعه‌ای را مشخص می‌کند که پشته در آن قرار دارد (این قطعه معمولاً همان قطعه‌ای است که داده‌ها در آن ذخیره می‌شوند). ثبات ESP آدرس داده‌ای را در خود دارد که اگر عمل برداشتن از پشته انجام شود، از پشته خارج خواهد شد؛ به همین دلیل گفته می‌شود که این داده در **بالای**[۲۳] پشته قرار دارد. داده‌ها را تنها می‌توان در واحدهای کلمهٔ مضاعف[۲۴] به پشته افزود، یعنی نمی‌توان یک بایت تنها را در پشته قرار داد.

دستورالعمل PUSH برای قراردادن یک کلمهٔ مضاعف[۲۵] در پشته، نخست ۴ واحد از ESP کم می‌کند و سپس کلمهٔ مضاعف را در [ESP] ذخیره می‌کند. در مقابل، دستورالعمل POP کلمهٔ مضاعف موجود در [ESP] را می‌خواند و سپس ۴ واحد به ESP می‌افزاید. کد زیر نحوهٔ کار این دستورالعمل‌ها را نشان می‌دهد و فرض را بر این می‌گذارد که مقدار اولیهٔ ESP برابر با 1000H است.

```
1       push   dword 1     ; 1 stored at 0FFCh, ESP = 0FFCh
```

[18] short jump
[19] near jump
[20] stack
[21] Last-In First-Out
[22] segment register
[23] top
[24] double word

[۲۵] در واقع می‌توان کلمه‌ها را نیز در پشته قرار داد، اما در حالت حفاظت‌شدهٔ ۳۲ بیتی بهتر است تنها با کلمه‌های مضاعف در پشته کار کنیم.

```
2       push    dword 2     ; 2 stored at 0FF8h, ESP = 0FF8h
3       push    dword 3     ; 3 stored at 0FF4h, ESP = 0FF4h
4       pop     eax         ; EAX = 3, ESP = 0FF8h
5       pop     ebx         ; EBX = 2, ESP = 0FFCh
6       pop     ecx         ; ECX = 1, ESP = 1000h
```

پشته جای مناسبی برای نگهداری موقت داده‌ها است. افزون بر این، از پشته برای فراخوانی زیربرنامه‌ها، ارسال پارامترها و نگهداری متغیرهای محلی نیز استفاده می‌شود.

پردازندهٔ 80x86 دستورالعمل PUSHA را نیز در اختیار می‌گذارد که مقادیر ثبات‌های EAX، EBX، ECX، EDX، ESI، EDI و EBP را (البته نه به همین ترتیب) در پشته قرار می‌دهد. با دستورالعمل POPA نیز می‌توان همهٔ آن‌ها را دوباره از پشته برداشت.

۴.۴ دستورالعمل‌های CALL و RET

پردازندهٔ 80x86 دو دستورالعمل فراهم می‌کند که با بهره‌گیری از پشته، فراخوانی زیربرنامه‌ها را سریع و آسان می‌سازند. دستورالعمل CALL یک پرش غیرشرطی به زیربرنامه انجام می‌دهد و در همان حال، آدرس دستورالعمل بعدی را در پشته **قرار می‌دهد**. دستورالعمل RET نیز آدرسی را از پشته **برمی‌دارد** و به همان آدرس پرش می‌کند. هنگام استفاده از این دستورالعمل‌ها بسیار اهمیت دارد که پشته به‌درستی مدیریت شود تا دستورالعمل RET عدد درستی را از پشته بردارد!

برنامهٔ پیشین را می‌توان با این دستورالعمل‌های تازه بازنویسی کرد؛ کافی است خطوط ۲۵ تا ۳۴ را به این صورت تغییر دهیم:

```
        mov     ebx, input1
        call    get_int

        mov     ebx, input2
        call    get_int
```

و زیربرنامهٔ get_int را نیز به شکل زیر درآوریم:

```
get_int:
        call    read_int
        mov     [ebx], eax
        ret
```

استفاده از CALL و RET چند مزیت دارد:

- ساده‌تر است!
- فراخوانی‌های تودرتوی زیربرنامه‌ها را به‌آسانی ممکن می‌سازد. توجه کنید که get_int خود read_int را فراخوانی می‌کند و این فراخوانی آدرس دیگری را در پشته قرار می‌دهد. در پایان کد read_int یک دستورالعمل RET وجود دارد که آدرس بازگشت را از پشته برمی‌دارد و به کد get_int بازمی‌گردد. سپس هنگامی که RET زیربرنامهٔ get_int اجرا می‌شود، آدرس بازگشتی را از پشته برمی‌دارد که به asm_main بازمی‌گردد. این سازوکار به لطف خاصیت «آخرین ورودی، نخستین خروجی» (LIFO) پشته به‌درستی کار می‌کند.

به یاد داشته باشید که برداشتن همهٔ داده‌هایی که در پشته قرار داده شده‌اند **بسیار** مهم است. برای نمونه، کد زیر را در نظر بگیرید:

```
1 get_int:
2       call   read_int
3       mov    [ebx], eax
4       push   eax
5       ret                 ; pops off EAX value, not return address!!
```

این کد به‌درستی بازنمی‌گردد!

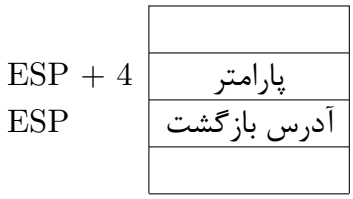
۵.۴ قراردادهای فراخوانی

هنگامی که زیربرنامه‌ای فراخوانده می‌شود، کد فراخواننده و خود زیربرنامه (یعنی **فراخوانده‌شده**[۲۶]) باید دربارهٔ شیوهٔ انتقال داده‌ها میان خود به توافق برسند. زبان‌های سطح بالا برای انتقال داده‌ها روش‌های استانداردی دارند که **قراردادهای فراخوانی** نامیده می‌شوند. برای آنکه کد سطح بالا بتواند با زبان اسمبلی ارتباط برقرار کند، کد اسمبلی باید از همان قراردادهای زبان سطح بالا پیروی کند. قراردادهای فراخوانی ممکن است از کامپایلری به کامپایلر دیگر متفاوت باشند یا بسته به نحوهٔ کامپایل کد (**برای نمونه**، روشن یا خاموش بودن بهینه‌سازی‌ها) تغییر کنند. با این حال، یک قرارداد همگانی وجود دارد و آن اینکه کد با دستورالعمل CALL فراخوانده می‌شود و با RET بازمی‌گردد.

همهٔ کامپایلرهای C رایانه‌های شخصی از یک قرارداد فراخوانی مشترک پشتیبانی می‌کنند که در ادامهٔ این فصل گام‌به‌گام شرح داده خواهد شد. این قراردادها امکان ساختن زیربرنامه‌های **بازورودپذیر**[۲۷] را فراهم می‌کنند. زیربرنامهٔ بازورودپذیر را می‌توان

[26] callee

[27] reentrant



شکل ۱.۴:

با اطمینان در هر نقطه‌ای از برنامه (حتی درون خود همان زیربرنامه) فراخوانی کرد.

۱.۵.۴ ارسال پارامترها از طریق پشته

پارامترهای[۲۸] زیربرنامه را می‌توان از طریق پشته ارسال کرد؛ به این صورت که آن‌ها پیش از اجرای دستورالعمل CALL در پشته قرار داده می‌شوند. درست مانند زبان C، اگر قرار باشد زیربرنامه پارامتری را تغییر دهد، باید **آدرس** داده ارسال شود، نه **مقدار** آن. همچنین اگر اندازهٔ پارامتر کمتر از یک کلمهٔ مضاعف باشد، باید پیش از قرار گرفتن در پشته به کلمهٔ مضاعف تبدیل شود.

در آدرس‌دهی غیرمستقیم، پردازندهٔ 80x86 بسته به اینکه چه ثبات‌هایی در عبارت آدرس‌دهی به کار رفته باشند، به قطعه‌های متفاوتی دسترسی پیدا می‌کند. ESP (و EBP) از قطعهٔ پشته استفاده می‌کنند، در حالی که EAX، EBX، ECX و EDX از قطعهٔ داده بهره می‌گیرند. البته این نکته در بیشتر برنامه‌های حالت حفاظت‌شده معمولاً اهمیتی ندارد، زیرا در این برنامه‌ها قطعهٔ داده و قطعهٔ پشته یکی هستند.

زیربرنامه پارامترهای موجود در پشته را از آن برنمی‌دارد، بلکه در همان پشته به آن‌ها دسترسی پیدا می‌کند. چرا؟

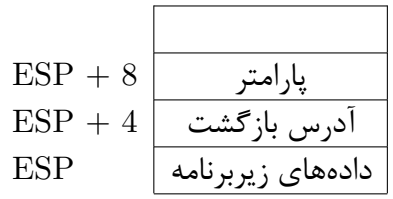
- از آنجا که پارامترها باید پیش از دستورالعمل CALL در پشته قرار بگیرند، برای برداشتن آن‌ها ابتدا باید آدرس بازگشت از پشته برداشته شود (و سپس دوباره در آن قرار داده شود).
- اغلب لازم است پارامترها در چند جای زیربرنامه به کار روند. معمولاً نمی‌توان آن‌ها را در تمام طول زیربرنامه در یک ثبات نگه داشت و ناچار باید در حافظه ذخیره شوند. باقی گذاشتن پارامترها در پشته، نسخه‌ای از داده‌ها را در حافظه نگه می‌دارد که در هر نقطه از زیربرنامه می‌توان به آن دسترسی داشت.

زیربرنامه‌ای را در نظر بگیرید که تنها یک پارامتر را از طریق پشته دریافت می‌کند. هنگامی که این زیربرنامه فراخوانده می‌شود، پشته به شکل ۱.۴ خواهد بود و می‌توان با آدرس‌دهی غیرمستقیم ([ESP+4] [۲۹]) به پارامتر دسترسی داشت.

اگر درون زیربرنامه نیز از پشته برای ذخیرهٔ داده‌ها استفاده شود، عددی که باید به ESP افزوده شود تغییر می‌کند. برای نمونه، شکل ۲.۴ وضعیت پشته را پس از قرار

[28] parameter

[۲۹] افزودن یک ثابت به ثبات در آدرس‌دهی غیرمستقیم مجاز است. عبارت‌های پیچیده‌تری نیز امکان‌پذیر هستند که در فصل بعد به آن‌ها پرداخته می‌شود.



شکل ۲.۴:

```
1  subprogram_label:
2        push   ebp           ; save original EBP value on stack
3        mov    ebp, esp      ; new EBP = ESP
4  ; subprogram code
5        pop    ebp           ; restore original EBP value
6        ret
```

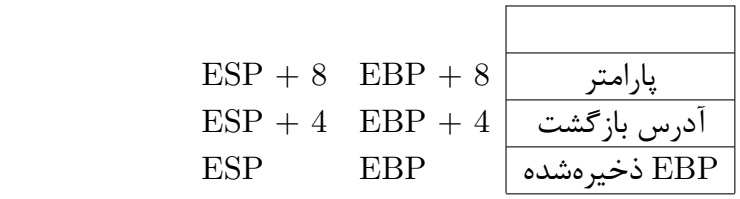
شکل ۳.۴: شکل کلی زیربرنامه

دادن یک کلمهٔ مضاعف (DWORD) در آن نشان می‌دهد. اکنون پارامتر در `ESP + 8` قرار دارد، نه در `ESP + 4`. بنابراین استفاده از ESP برای ارجاع به پارامترها می‌تواند بسیار خطاپذیر باشد. برای حل این مشکل، پردازندهٔ 80386 ثبات دیگری به نام EBP در اختیار می‌گذارد که تنها کاربرد آن ارجاع به داده‌های موجود در پشته است. قرارداد فراخوانی C حکم می‌کند که زیربرنامه نخست مقدار EBP را در پشته ذخیره کند و سپس EBP را برابر با ESP قرار دهد. به این ترتیب، ESP می‌تواند با قرار دادن داده‌ها در پشته یا برداشتن آن‌ها تغییر کند، بی‌آنکه EBP دستخوش تغییر شود. در پایان زیربرنامه باید مقدار اولیهٔ EBP بازگردانده شود (و به همین دلیل است که این مقدار در آغاز زیربرنامه ذخیره می‌شود). شکل ۳.۴ شکل کلی زیربرنامه‌ای را نشان می‌دهد که از این قراردادها پیروی می‌کند.

خطوط ۲ و ۳ در شکل ۳.۴ **پیش‌درآمد**[۳۰] کلی زیربرنامه را تشکیل می‌دهند و خطوط ۵ و ۶ نیز **پایان‌بخش**[۳۱] آن را می‌سازند. شکل ۴.۴ وضعیت پشته را درست پس از اجرای پیش‌درآمد نشان می‌دهد. اکنون می‌توان در هر جای زیربرنامه با `[EBP + 8]` به پارامتر دسترسی داشت، بی‌آنکه نگران باشیم زیربرنامه چه چیزهای دیگری را در پشته قرار داده است.

پس از پایان کار زیربرنامه، پارامترهایی که در پشته قرار داده شده بودند باید از آن

[30] prologue
[31] epilogue



شکل ۴.۴:

```
1       push    dword 1         ; pass 1 as parameter
2       call    fun
3       add     esp, 4          ; remove parameter from stack
```

شکل ۵.۴: نمونه‌ای از فراخوانی زیربرنامه

حذف شوند. قرارداد فراخوانی C مقرر می‌کند که این کار بر عهدهٔ کد فراخواننده[۳۲] است. قراردادهای دیگر در این زمینه متفاوت هستند؛ برای نمونه، قرارداد فراخوانی پاسکال مقرر می‌کند که خود زیربرنامه باید پارامترها را حذف کند. (شکل دیگری از دستورالعمل RET وجود دارد که انجام این کار را آسان می‌کند.) برخی از کامپایلرهای C نیز از این قرارداد پشتیبانی می‌کنند و برای آنکه کامپایلر از آن استفاده کند، کلیدواژهٔ `pascal` در پیش‌نمونه[۳۳] و تعریف تابع به کار می‌رود. در واقع قرارداد `stdcall` نیز، که توابع C رابط برنامه‌نویسی (API) ویندوز مایکروسافت از آن استفاده می‌کنند، به همین شیوه عمل می‌کند. مزیت این روش چیست؟ این روش اندکی از قرارداد C کارآمدتر است. پس چرا همهٔ توابع C از این قرارداد استفاده نمی‌کنند؟ دلیل آن است که زبان C در حالت کلی اجازه می‌دهد تابع تعداد متغیری آرگومان[۳۴] داشته باشد (**برای نمونه**، توابع `printf` و `scanf`). در این گونه توابع، عمل حذف پارامترها از پشته از یک فراخوانی تابع به فراخوانی بعدی فرق می‌کند. قرارداد C این امکان را فراهم می‌کند که دستورالعمل‌های انجام‌دهندهٔ این عمل به‌آسانی از فراخوانی‌ای به فراخوانی دیگر تغییر کنند، در حالی که قراردادهای پاسکال و stdcall انجام این عمل را بسیار دشوار می‌سازند. از این رو قرارداد پاسکال (همانند خود زبان پاسکال) چنین توابعی را مجاز نمی‌داند. ویندوز مایکروسافت می‌تواند از این قرارداد استفاده کند، زیرا هیچ‌یک از توابع API آن تعداد متغیری آرگومان نمی‌گیرد.

شکل ۵.۴ نشان می‌دهد که زیربرنامه‌ای که از قرارداد فراخوانی C پیروی می‌کند

[32] caller
[33] prototype
[34] argument

چگونه فراخوانده می‌شود. خط ۳ با دست‌کاری مستقیم اشاره‌گر پشته[۳۵]، پارامتر را از پشته حذف می‌کند. برای این کار می‌شد از دستورالعمل POP نیز استفاده کرد، اما در آن صورت نتیجهٔ بی‌فایدهٔ آن باید در یک ثبات ذخیره می‌شد. در واقع بسیاری از کامپایلرها در همین مورد خاص، برای حذف پارامتر از دستورالعمل POP ECX استفاده می‌کنند. کامپایلر POP را به ADD ترجیح می‌دهد، زیرا دستورالعمل ADD بایت‌های بیشتری اشغال می‌کند. با این حال، POP مقدار ECX را نیز تغییر می‌دهد! در ادامه نمونه برنامهٔ دیگری را می‌بینید که دو زیربرنامه دارد و هر دو از قراردادهای فراخوانی C که شرح داده شد پیروی می‌کنند. خط ۵۴ (و چند خط دیگر) نشان می‌دهد که در یک فایل منبع[۳۶] می‌توان چندین قطعهٔ داده و متن تعریف کرد. این قطعه‌ها در فرایند پیونددادن[۳۷] با هم ترکیب می‌شوند و یک قطعهٔ دادهٔ واحد و یک قطعهٔ متن واحد را می‌سازند. تقسیم داده‌ها و کد به قطعه‌های جداگانه این امکان را فراهم می‌کند که داده‌هایی که زیربرنامه از آن‌ها استفاده می‌کند در نزدیکی کد همان زیربرنامه تعریف شوند.

sub3.asm

```
1  %include "asm_io.inc"
2
3  segment .data
4  sum     dd   0
5
6  segment .bss
7  input   resd 1
8
9  ;
10 ; pseudo-code algorithm
11 ; i = 1;
12 ; sum = 0;
13 ; while( get_int(i, &input), input != 0 ) {
14 ;   sum += input;
15 ;   i++;
16 ; }
17 ; print_sum(num);
18 segment .text
19         global  _asm_main
20 _asm_main:
21         enter   0,0               ; setup routine
```

[35] stack pointer
[36] source file
[37] linking

```
22          pusha
23
24          mov     edx, 1              ; edx is 'i' in pseudo-code
25  while_loop:
26          push    edx                 ; save i on stack
27          push    dword input         ; push address of input on stack
28          call    get_int
29          add     esp, 8              ; remove i and &input from stack
30
31          mov     eax, [input]
32          cmp     eax, 0
33          je      end_while
34
35          add     [sum], eax          ; sum += input
36
37          inc     edx
38          jmp     short while_loop
39
40  end_while:
41          push    dword [sum]         ; push value of sum onto stack
42          call    print_sum
43          pop     ecx                 ; remove [sum] from stack
44
45          popa
46          leave
47          ret
48
49  ; subprogram get_int
50  ; Parameters (in order pushed on stack)
51  ;   number of input (at [ebp + 12])
52  ;   address of word to store input into (at [ebp + 8])
53  ; Notes:
54  ;   values of eax and ebx are destroyed
55  segment .data
56  prompt  db      ") Enter an integer number (0 to quit): ", 0
57
58  segment .text
59  get_int:
60          push    ebp
61          mov     ebp, esp
62
63          mov     eax, [ebp + 12]
```

```
64          call    print_int
65
66          mov     eax, prompt
67          call    print_string
68
69          call    read_int
70          mov     ebx, [ebp + 8]
71          mov     [ebx], eax          ; store input into memory
72
73          pop     ebp
74          ret                         ; jump back to caller
75
76  ; subprogram print_sum
77  ; prints out the sum
78  ; Parameter:
79  ;   sum to print out (at [ebp+8])
80  ; Note: destroys value of eax
81  ;
82  segment .data
83  result  db      "The sum is ", 0
84
85  segment .text
86  print_sum:
87          push    ebp
88          mov     ebp, esp
89
90          mov     eax, result
91          call    print_string
92
93          mov     eax, [ebp+8]
94          call    print_int
95          call    print_nl
96
97          pop     ebp
98          ret
```

sub3.asm

```
1 subprogram_label:
2       push   ebp                ; save original EBP value on stack
3       mov    ebp, esp           ; new EBP = ESP
4       sub    esp, LOCAL_BYTES   ; = # bytes needed by locals
5 ; subprogram code
6       mov    esp, ebp           ; deallocate locals
7       pop    ebp                ; restore original EBP value
8       ret
```

شکل ۶.۴: شکل کلی زیربرنامه همراه با متغیرهای محلی

۲.۵.۴ متغیرهای محلی در پشته

پشته جای مناسبی برای نگهداری متغیرهای محلی[۳۸] است و زبان C نیز متغیرهای معمولی (یا به اصطلاح C، متغیرهای **خودکار**[۳۹]) را دقیقاً در همین جا ذخیره می‌کند. اگر بخواهیم زیربرنامه‌ها بازورودپذیر باشند، استفاده از پشته برای متغیرها اهمیت زیادی دارد. زیربرنامهٔ بازورودپذیر در هر جایی که فراخوانده شود، حتی درون خود همان زیربرنامه، درست کار می‌کند؛ به بیان دیگر، زیربرنامه‌های بازورودپذیر را می‌توان به‌صورت **بازگشتی**[۴۰] فراخوانی کرد. استفاده از پشته برای متغیرها در مصرف حافظه نیز صرفه‌جویی می‌کند. داده‌هایی که در پشته ذخیره نمی‌شوند از آغاز تا پایان اجرای برنامه حافظه اشغال می‌کنند (زبان C این نوع متغیرها را **سراسری**[۴۱] یا **ایستا**[۴۲] می‌نامد)، اما داده‌های ذخیره‌شده در پشته تنها هنگامی حافظه مصرف می‌کنند که زیربرنامه‌ای که برای آن تعریف شده‌اند فعال باشد.

متغیرهای محلی در پشته درست پس از مقدار ذخیره‌شدهٔ EBP قرار می‌گیرند و برای تخصیص آن‌ها، در پیش‌درآمد زیربرنامه تعداد بایت‌های موردنیاز از ESP کم می‌شود. شکل ۶.۴ اسکلت جدید زیربرنامه را نشان می‌دهد. برای دسترسی به متغیرهای محلی نیز از ثبات EBP استفاده می‌شود. برای نمونه، تابع C شکل ۷.۴ را در نظر بگیرید؛ شکل ۸.۴ نشان می‌دهد که زیربرنامهٔ معادل آن را چگونه می‌توان به زبان اسمبلی نوشت.

شکل ۹.۴ وضعیت پشته را پس از اجرای پیش‌درآمد برنامهٔ شکل ۸.۴ نشان می‌دهد.

ENTER و LEAVE با آنکه پیش‌درآمد و پایان‌بخش را ساده می‌کنند، کمتر به کار می‌روند. چرا؟ چون از دستورالعمل‌های ساده‌تر معادل خود کندتر هستند! این نمونه نشان می‌دهد که نمی‌توان همواره فرض کرد یک دستورالعمل تنها از چند دستورالعمل پیاپی سریع‌تر است.

[38] local variables
[39] automatic
[40] recursive
[41] global
[42] static

```
1  void calc_sum( int n, int * sump )
2  {
3    int i, sum = 0;
4
5    for( i=1; i <= n; i++ )
6      sum += i;
7    *sump = sum;
8  }
```

شکل ۷.۴: نسخهٔ C محاسبهٔ مجموع

این بخش از پشته که پارامترها، اطلاعات بازگشت و فضای ذخیرهٔ متغیرهای محلی را در بر می‌گیرد، **قاب پشته**[۴۳] نامیده می‌شود. هر بار که تابعی در C فراخوانده می‌شود، قاب پشتهٔ تازه‌ای در پشته ساخته می‌شود.

پیش‌درآمد و پایان‌بخش زیربرنامه را می‌توان با دو دستورالعمل ویژه، که دقیقاً برای همین منظور طراحی شده‌اند، ساده‌تر کرد. دستورالعمل ENTER کار کد پیش‌درآمد را انجام می‌دهد و دستورالعمل LEAVE نیز کار پایان‌بخش را بر عهده دارد. دستورالعمل ENTER دو عملگر بی‌واسطه[۴۴] می‌گیرد. در قرارداد فراخوانی C، عملگر دوم همواره ۰ است و عملگر نخست تعداد بایت‌هایی را مشخص می‌کند که متغیرهای محلی به آن نیاز دارند. دستورالعمل LEAVE هیچ عملگری ندارد. شکل ۱۰.۴ نحوهٔ استفاده از این دستورالعمل‌ها را نشان می‌دهد. توجه کنید که اسکلت برنامه (شکل ۷.۱) نیز از ENTER و LEAVE استفاده می‌کند.

۶.۴ برنامه‌های چندپیمانه‌ای

برنامهٔ چندپیمانه‌ای[۴۵] برنامه‌ای است که از بیش از یک فایل شیء[۴۶] ساخته شده باشد. در واقع همهٔ برنامه‌هایی که تاکنون دیدیم چندپیمانه‌ای بودند، زیرا هر کدام از فایل شیء برنامهٔ راه‌انداز C و فایل شیء اسمبلی (به‌علاوهٔ فایل‌های شیء کتابخانهٔ C) تشکیل می‌شدند. به یاد دارید که پیونددهنده[۴۷] فایل‌های شیء را با هم ترکیب می‌کند و از آن‌ها یک برنامهٔ اجرایی واحد می‌سازد. برای این کار، پیونددهنده باید هر

[43] stack frame
[44] immediate operands
[45] multi-module program
[46] object file
[47] linker

```
1  cal_sum:
2        push   ebp
3        mov    ebp, esp
4        sub    esp, 4               ; make room for local sum
5
6        mov    dword [ebp - 4], 0   ; sum = 0
7        mov    ebx, 1               ; ebx (i) = 1
8  for_loop:
9        cmp    ebx, [ebp+8]         ; is i <= n?
10       jnle   end_for
11
12       add    [ebp-4], ebx         ; sum += i
13       inc    ebx
14       jmp    short for_loop
15
16 end_for:
17       mov    ebx, [ebp+12]        ; ebx = sump
18       mov    eax, [ebp-4]         ; eax = sum
19       mov    [ebx], eax           ; *sump = sum;
20
21       mov    esp, ebp
22       pop    ebp
23       ret
```

شکل ۸.۴: نسخهٔ اسمبلی محاسبهٔ مجموع

ارجاعی را که در یک پیمانه[۴۸] (**یعنی** یک فایل شیء) به برچسبی داده شده است با تعریف همان برچسب در پیمانه‌ای دیگر تطبیق دهد. بنابراین اگر پیمانهٔ A بخواهد از برچسبی استفاده کند که در پیمانهٔ B تعریف شده است، باید رهنمود[۴۹] `extern` را به کار ببرد. پس از رهنمود `extern` فهرستی از برچسب‌ها می‌آید که با ویرگول از هم جدا شده‌اند. این رهنمود به اسمبلر می‌گوید که این برچسب‌ها را **خارجی**[۵۰] در نظر بگیرد، به بیان دیگر، این برچسب‌ها در همین پیمانه قابل استفاده هستند، اما در پیمانه‌ای دیگر تعریف شده‌اند. برای نمونه، فایل `asm_io.inc` روال‌هایی مانند `read_int` را به صورت خارجی تعریف می‌کند.

[48] module
[49] directive
[50] external

ESP + 16	EBP + 12	sump
ESP + 12	EBP + 8	n
ESP + 8	EBP + 4	آدرس بازگشت
ESP + 4	EBP	EBP ذخیره‌شده
ESP	EBP - 4	sum

شکل ۹.۴:

```
1  subprogram_label:
2        enter  LOCAL_BYTES, 0     ; = # bytes needed by locals
3  ; subprogram code
4        leave
5        ret
```

شکل ۱۰.۴: شکل کلی زیربرنامه همراه با متغیرهای محلی، با استفاده از ENTER و LEAVE

در اسمبلی، برچسب‌ها به طور پیش‌فرض از بیرون پیمانه در دسترس نیستند. اگر قرار باشد برچسبی از پیمانه‌هایی غیر از پیمانهٔ تعریف‌کنندهٔ آن در دسترس باشد، باید در پیمانهٔ خود **سراسری**[۵۱] اعلان شود و این کار را رهنمود global انجام می‌دهد. سطر ۱۳ از برنامهٔ اسکلت در شکل ۷.۱ نشان می‌دهد که برچسب _asm_main به صورت سراسری تعریف شده است. اگر این اعلان نبود، با خطای پیونددهنده روبه‌رو می‌شدیم. چرا؟ زیرا کد C نمی‌توانست به برچسب **داخلی** _asm_main ارجاع دهد.

در ادامه، کد مثال قبل را می‌بینید که این بار با دو پیمانه بازنویسی شده است. دو زیربرنامهٔ get_int و print_sum در فایل منبعی جدا از روال _asm_main قرار دارند.

main4.asm

```
1  %include "asm_io.inc"
2
3  segment .data
4  sum     dd   0
5
6  segment .bss
7  input   resd 1
8
```

[۵۱] global

```
segment .text
        global  _asm_main
        extern  get_int, print_sum
_asm_main:
        enter   0,0               ; setup routine
        pusha

        mov     edx, 1            ; edx is 'i' in pseudo-code
while_loop:
        push    edx               ; save i on stack
        push    dword input       ; push address on input on stack
        call    get_int
        add     esp, 8            ; remove i and &input from stack

        mov     eax, [input]
        cmp     eax, 0
        je      end_while

        add     [sum], eax        ; sum += input

        inc     edx
        jmp     short while_loop

end_while:
        push    dword [sum]       ; push value of sum onto stack
        call    print_sum
        pop     ecx               ; remove [sum] from stack

        popa
        leave
        ret
```

main4.asm

sub4.asm

```
%include "asm_io.inc"

segment .data
prompt  db      ") Enter an integer number (0 to quit): ", 0

segment .text
        global  get_int, print_sum
get_int:
        enter   0,0

```

```
11          mov     eax, [ebp + 12]
12          call    print_int
13
14          mov     eax, prompt
15          call    print_string
16
17          call    read_int
18          mov     ebx, [ebp + 8]
19          mov     [ebx], eax          ; store input into memory
20
21          leave
22          ret                         ; jump back to caller
23
24  segment .data
25  result  db      "The sum is ", 0
26
27  segment .text
28  print_sum:
29          enter   0,0
30
31          mov     eax, result
32          call    print_string
33
34          mov     eax, [ebp+8]
35          call    print_int
36          call    print_nl
37
38          leave
39          ret
```

sub4.asm

در مثال بالا فقط برچسب‌های کد سراسری به کار رفته‌اند، اما برچسب‌های دادهٔ سراسری نیز دقیقاً به همین شیوه کار می‌کنند.

۷.۴ پیوند اسمبلی با C

امروزه برنامه‌های بسیار کمی به طور کامل با اسمبلی نوشته می‌شوند. کامپایلرها در تبدیل کد سطح بالا به کد ماشین کارآمد بسیار توانا هستند و چون نوشتن کد به یک زبان سطح بالا بسیار آسان‌تر است، این زبان‌ها محبوبیت بیشتری دارند. افزون بر این، قابلیت حمل[۵۲] کد سطح بالا **بسیار** بیشتر از اسمبلی است!

[52] portability

هنگامی هم که از اسمبلی استفاده می‌شود، معمولاً فقط بخش‌های کوچکی از کد به آن نوشته می‌شود. این کار به دو روش انجام‌پذیر است: فراخوانی زیرروال‌های اسمبلی از C، یا اسمبلی درون‌خطی[۵۳]. اسمبلی درون‌خطی به برنامه‌نویس اجازه می‌دهد که دستورات اسمبلی را مستقیماً درون کد C قرار دهد. این روش می‌تواند بسیار راحت باشد، اما معایبی هم دارد. کد اسمبلی باید با قالبی نوشته شود که کامپایلر به کار می‌برد و در حال حاضر هیچ کامپایلری از قالب NASM پشتیبانی نمی‌کند. از سوی دیگر، کامپایلرهای مختلف قالب‌های متفاوتی می‌خواهند: بورلند و مایکروسافت قالب MASM را لازم دارند، در حالی که DJGPP و gcc لینوکس قالب GAS[۵۴] را می‌خواهند. در مقابل، روش فراخوانی زیرروال اسمبلی روی رایانه‌های شخصی بسیار استانداردتر است.

روال‌های اسمبلی معمولاً به دلایل زیر همراه با C به کار می‌روند:

- به دسترسی مستقیم به امکانات سخت‌افزاری رایانه نیاز است که دسترسی به آن‌ها از طریق C دشوار یا ناممکن است.
- روال باید تا حد ممکن سریع باشد و برنامه‌نویس می‌تواند کد را با دست بهتر از کامپایلر بهینه کند.

دلیل دوم دیگر به اندازهٔ گذشته معتبر نیست. فناوری کامپایلرها در طول سال‌ها پیشرفت کرده است و کامپایلرها اغلب می‌توانند کد بسیار کارآمدی تولید کنند (به‌ویژه اگر بهینه‌سازی‌های کامپایلر فعال باشند). معایب روال‌های اسمبلی نیز کاهش قابلیت حمل و خوانایی است.

بیشتر قراردادهای فراخوانی[۵۵] C را پیش‌تر مشخص کرده‌ایم، اما چند ویژگی دیگر نیز هست که باید شرح داده شود.

۱.۷.۴ ذخیره‌سازی ثبات‌ها

کلیدواژهٔ `register` را می‌توان در اعلان یک متغیر C به کار برد تا به کامپایلر پیشنهاد شود که برای این متغیر به جای یک خانهٔ حافظه از یک ثبات استفاده کند. چنین متغیرهایی را متغیرهای ثباتی می‌نامند. کامپایلرهای امروزی این کار را بدون نیاز به هیچ پیشنهادی به طور خودکار انجام می‌دهند.

نخست باید بدانیم که C فرض می‌کند یک زیرروال مقادیر ثبات‌های[۵۶] EBX، ESI، EDI، EBP، CS، DS، SS و ES را حفظ می‌کند. معنای این فرض آن نیست که زیرروال نمی‌تواند این ثبات‌ها را در درون خود تغییر دهد، بلکه به این معناست که اگر مقدار آن‌ها را تغییر داد، باید پیش از بازگشت، مقادیر اولیهٔ آن‌ها را بازگرداند. مقادیر

[53] inline assembly

[54] GAS اسمبلری است که همهٔ کامپایلرهای GNU از آن استفاده می‌کنند. این اسمبلر نحو AT&T را به کار می‌برد که با نحوهای نسبتاً مشابه MASM، TASM و NASM تفاوت بسیاری دارد.

[55] calling conventions

[56] registers

```
1  segment .data
2  x            dd     0
3  format       db     "x = %d\n", 0
4
5  segment .text
6  ...
7        push   dword [x]     ; push x's value
8        push   dword format  ; push address of format string
9        call   _printf       ; note underscore!
10       add    esp, 8        ; remove parameters from stack
```

شکل ۱۱.۴: فراخوانی printf

EBP + 12	مقدار x
EBP + 8	آدرس رشتهٔ قالب
EBP + 4	آدرس بازگشت
EBP	EBP ذخیره‌شده

شکل ۱۲.۴: پشته درون printf

EBX، ESI و EDI باید دست‌نخورده بمانند، زیرا C از این ثبات‌ها برای **متغیرهای ثباتی**[۵۷] استفاده می‌کند. معمولاً برای ذخیرهٔ مقادیر اولیهٔ این ثبات‌ها از پشته استفاده می‌شود.

۲.۷.۴ برچسب‌های توابع

بیشتر کامپایلرهای C یک نویسهٔ زیرخط (_) به ابتدای نام توابع و متغیرهای سراسری یا ایستا اضافه می‌کنند. برای نمونه، به تابعی به نام f برچسب _f داده می‌شود. بنابراین اگر قرار است این تابع یک روال اسمبلی باشد، برچسب آن **باید** _f باشد، نه f. کامپایلر gcc لینوکس هیچ نویسه‌ای به نام‌ها اضافه **نمی‌کند**، از این رو در فایل‌های اجرایی ELF لینوکس کافی است برای تابع C به نام f از همان برچسب f استفاده شود. با این حال، gcc در DJGPP زیرخط را اضافه می‌کند. توجه کنید که در برنامهٔ اسکلت اسمبلی (شکل ۷.۱) برچسب روال اصلی _asm_main است.

[57] register variables

۳.۷.۴ ارسال پارامترها

در قرارداد فراخوانی C، آرگومان‌های[۵۸] یک تابع به ترتیبی **وارون** ترتیب ظاهرشدن آن‌ها در فراخوانی تابع در پشته قرار داده می‌شوند.

دستور C زیر را در نظر بگیرید: `printf("x = %d\n",x);` شکل ۱۱.۴ نشان می‌دهد که این دستور چگونه کامپایل می‌شود (به صورت معادل آن در قالب NASM) و شکل ۱۲.۴ وضعیت پشته را پس از اجرای پیش‌درآمد[۵۹] تابع `printf` نشان می‌دهد. تابع `printf` یکی از توابع کتابخانهٔ C است که می‌تواند هر تعداد آرگومانی را بپذیرد و قواعد قرارداد فراخوانی C در واقع به طور ویژه برای امکان‌پذیر ساختن چنین توابعی نوشته شده‌اند. از آنجا که آدرس رشتهٔ قالب آخر از همه در پشته قرار داده می‌شود، جای آن در پشته، هر تعداد پارامتر که به تابع ارسال شده باشد، **همیشه** `EBP + 8` خواهد بود. بنابراین کد `printf` می‌تواند با بررسی رشتهٔ قالب تعیین کند که چند پارامتر باید ارسال شده باشد و سپس آن‌ها را در پشته جست‌وجو کند.

برای پردازش تعداد دلخواهی آرگومان در C نیازی به استفاده از اسمبلی نیست. فایل سرآیند `stdarg.h` ماکروهایی تعریف می‌کند که با آن‌ها می‌توان این آرگومان‌ها را به شکلی قابل حمل پردازش کرد. برای جزئیات بیشتر به هر کتاب خوبی دربارهٔ C مراجعه کنید.

البته اگر اشتباهی رخ دهد و برای نمونه `printf("x = %d\n")` نوشته شود، کد `printf` باز هم مقدار کلمهٔ مضاعفِ موجود در `[EBP + 12]` را چاپ می‌کند، اما این مقدار دیگر مقدار `x` نخواهد بود!

۴.۷.۴ محاسبهٔ آدرس متغیرهای محلی

یافتن آدرس برچسبی که در قطعه‌های `data` یا `bss` تعریف شده، ساده است و در اصل پیونددهنده این کار را انجام می‌دهد. اما محاسبهٔ آدرس یک متغیر محلی[۶۰] (یا پارامتر) در پشته به این سادگی نیست، در حالی که هنگام فراخوانی زیرروال‌ها بسیار به آن نیاز پیدا می‌کنیم. حالتی را در نظر بگیرید که بخواهیم آدرس یک متغیر (آن را `x` بنامیم) را به یک تابع (آن را `foo` بنامیم) ارسال کنیم. اگر `x` در پشته در $EBP - 8$ قرار داشته باشد، نمی‌توان صرفاً از دستور زیر استفاده کرد:

```
mov    eax, ebp - 8
```

چرا؟ زیرا مقداری که `MOV` در EAX ذخیره می‌کند باید به دست اسمبلر محاسبه شود (یعنی در نهایت باید یک ثابت باشد). با این حال، دستورالعملی[۶۱] وجود دارد که همین محاسبهٔ مورد نظر را انجام می‌دهد. نام این دستورالعمل `LEA` (کوتاه‌شدهٔ

[58] arguments
[59] prologue
[60] local variable
[61] instruction

Load Effective Address، یعنی «بارکردن آدرس مؤثر») است. دستور زیر آدرس x را محاسبه می‌کند و در EAX قرار می‌دهد:

```
lea    eax, [ebp - 8]
```

اکنون EAX آدرس x را در خود دارد و می‌توان آن را هنگام فراخوانی تابع foo در پشته قرار داد. دچار اشتباه نشوید: شاید به نظر برسد که این دستورالعمل داده‌های موجود در [EBP−8] را می‌خواند، اما چنین **نیست**. دستورالعمل LEA **هرگز** حافظه را نمی‌خواند! این دستورالعمل تنها آدرسی را محاسبه می‌کند که دستورالعملی دیگر از آن می‌خواند و این آدرس را در نخستین عملگر[۶۲] ثباتی خود ذخیره می‌کند. چون LEA در عمل هیچ حافظه‌ای را نمی‌خواند، هیچ مشخص‌کنندهٔ اندازهٔ حافظه‌ای (**مانند** dword) نه لازم است و نه مجاز.

۵.۷.۴ بازگرداندن مقادیر

توابع C که از نوع void نیستند، مقداری را بازمی‌گردانند و قرارداد فراخوانی C چگونگی انجام این کار را مشخص می‌کند. مقادیر بازگشتی از طریق ثبات‌ها منتقل می‌شوند. همهٔ انواع صحیح (char، int، enum و **مانند آن‌ها**) در ثبات EAX بازگردانده می‌شوند. اگر این مقادیر کوچک‌تر از ۳۲ بیت باشند، هنگام ذخیره در EAX به ۳۲ بیت گسترش می‌یابند. (چگونگی این گسترش به علامت‌دار یا بی‌علامت بودن نوع آن‌ها بستگی دارد.) مقادیر ۶۴ بیتی در جفت‌ثبات EDX:EAX بازگردانده می‌شوند. مقادیر اشاره‌گر[۶۳] نیز در EAX ذخیره می‌شوند. مقادیر ممیز شناور[۶۴] هم در ثبات ST0 کمک‌پردازندهٔ ریاضی[۶۵] قرار می‌گیرند. (این ثبات در فصل ممیز شناور بررسی می‌شود.)

۶.۷.۴ قراردادهای فراخوانی دیگر

قواعدی که تا اینجا گفته شد، قرارداد فراخوانی استاندارد C را توصیف می‌کنند که همهٔ کامپایلرهای C برای 80x86 از آن پشتیبانی می‌کنند. کامپایلرها اغلب از قراردادهای فراخوانی دیگری نیز پشتیبانی می‌کنند. هنگام پیوند با زبان اسمبلی، **بسیار** مهم است بدانید کامپایلر هنگام فراخوانی تابع شما از کدام قرارداد فراخوانی استفاده می‌کند. معمولاً پیش‌فرض، استفاده از قرارداد فراخوانی استاندارد است، اما همیشه

[62] operand
[63] pointer
[64] floating point
[65] math coprocessor

چنین نیست[۶۶]. کامپایلرهایی که چند قرارداد را به کار می‌برند، اغلب گزینه‌هایی در خط فرمان دارند که با آن‌ها می‌توان قرارداد پیش‌فرض را تغییر داد. این کامپایلرها همچنین افزونه‌هایی بر نحو C فراهم می‌کنند تا بتوان قرارداد فراخوانی هر تابع را به طور صریح تعیین کرد. با این حال، این افزونه‌ها استاندارد نیستند و ممکن است از کامپایلری به کامپایلر دیگر فرق کنند.

کامپایلر GCC قراردادهای فراخوانی مختلفی را مجاز می‌داند و قرارداد یک تابع را می‌توان با استفاده از افزونهٔ `__attribute__` به طور صریح اعلان کرد. برای نمونه، برای اعلان تابعی از نوع `void` به نام `f` که از قرارداد فراخوانی استاندارد استفاده می‌کند و یک پارامتر[۶۷] `int` می‌گیرد، نحو زیر را برای پیش‌نمونهٔ آن به کار ببرید:

```
void f( int ) __attribute__((cdecl));
```

GCC از قرارداد فراخوانی *standard call* نیز پشتیبانی می‌کند. تابع بالا را می‌توان با جایگزین کردن `cdecl` با `stdcall` طوری اعلان کرد که از این قرارداد استفاده کند. تفاوت `stdcall` و `cdecl` در این است که در `stdcall` خود زیرروال باید پارامترها را از پشته بردارد (همان‌گونه که قرارداد فراخوانی پاسکال عمل می‌کند). بنابراین قرارداد `stdcall` را فقط می‌توان برای توابعی به کار برد که تعداد ثابتی آرگومان می‌گیرند (**یعنی** توابعی که مانند `printf` و `scanf` نیستند).

GCC همچنین از صفت دیگری به نام `regparm` پشتیبانی می‌کند که به کامپایلر می‌گوید برای ارسال حداکثر ۳ آرگومان صحیح به یک تابع، به جای پشته از ثبات‌ها استفاده کند. این نوع بهینه‌سازی رایج است و بسیاری از کامپایلرها از آن پشتیبانی می‌کنند.

بورلند و مایکروسافت برای اعلان قراردادهای فراخوانی نحو مشترکی به کار می‌برند و کلیدواژه‌های `__cdecl` و `__stdcall` را به C می‌افزایند. این کلیدواژه‌ها نقش توصیف‌کنندهٔ تابع را دارند و در پیش‌نمونه، بلافاصله پیش از نام تابع می‌آیند. برای نمونه، تابع `f` بالا در بورلند و مایکروسافت به صورت زیر تعریف می‌شود:

```
void __cdecl f( int );
```

هر یک از قراردادهای فراخوانی مزایا و معایبی دارد. مزیت‌های اصلی قرارداد `cdecl` سادگی و انعطاف‌پذیری بسیار آن است. این قرارداد را می‌توان برای هر نوع تابع C و با هر کامپایلر C به کار برد، در حالی که استفاده از قراردادهای دیگر ممکن است قابلیت حمل زیرروال را محدود کند. عیب اصلی آن این است که ممکن است از برخی

[۶۶] کامپایلر Watcom C نمونه‌ای از کامپایلرهایی است که به طور پیش‌فرض از قرارداد استاندارد استفاده **نمی‌کنند**. برای جزئیات بیشتر، فایل کد منبع نمونه برای Watcom را ببینید.

[67] parameter

قراردادهای دیگر کندتر باشد و حافظهٔ بیشتری مصرف کند (زیرا هر بار فراخوانی تابع به کدی برای برداشتن پارامترها از پشته نیاز دارد).

مزیت قرارداد `stdcall` این است که حافظهٔ کمتری از `cdecl` مصرف می‌کند، چون پس از دستورالعمل `CALL` نیازی به پاک‌سازی پشته نیست. عیب اصلی آن این است که نمی‌توان آن را برای توابعی با تعداد آرگومان متغیر به کار برد.

مزیت قراردادی که پارامترهای صحیح را از طریق ثبات‌ها ارسال می‌کند، سرعت آن است. عیب اصلی آن پیچیده‌تر بودن این قرارداد است، زیرا ممکن است برخی پارامترها در ثبات‌ها و برخی دیگر در پشته باشند.

۷.۷.۴ مثال‌ها

در ادامه مثالی آمده است که نشان می‌دهد چگونه می‌توان یک روال اسمبلی را با یک برنامهٔ C پیوند داد. (توجه کنید که این برنامه از برنامهٔ اسکلت اسمبلی (شکل ۷.۱) یا پیمانهٔ driver.c استفاده نمی‌کند.)

main5.c

```c
#include <stdio.h>
/* prototype for assembly routine */
void calc_sum( int, int * ) __attribute__((cdecl));

int main( void )
{
  int n, sum;

   printf ("Sum integers up to: ");
  scanf("%d", &n);
  calc_sum(n, &sum);
   printf ("Sum is %d\n", sum);
  return 0;
}
```

main5.c

sub5.asm

```
; subroutine _calc_sum
; finds the sum of the integers 1 through n
; Parameters:
;   n    - what to sum up to (at [ebp + 8])
```

```
5  ;   sump - pointer to int to store sum into (at [ebp + 12])
6  ; pseudo C code:
7  ; void calc_sum( int n, int * sump )
8  ; {
9  ;   int i, sum = 0;
10 ;   for( i=1; i <= n; i++ )
11 ;     sum += i;
12 ;   *sump = sum;
13 ; }
14
15 segment .text
16         global  _calc_sum
17 ;
18 ; local variable:
19 ;   sum at [ebp-4]
20 _calc_sum:
21         enter   4,0               ; make room for sum on stack
22         push    ebx               ; IMPORTANT!
23
24         mov     dword [ebp-4],0   ; sum = 0
25         dump_stack 1, 2, 4        ; print out stack from ebp-8 to ebp+16
26         mov     ecx, 1            ; ecx is i in pseudocode
27 for_loop:
28         cmp     ecx, [ebp+8]      ; cmp i and n
29         jnle    end_for           ; if not i <= n, quit
30
31         add     [ebp-4], ecx      ; sum += i
32         inc     ecx
33         jmp     short for_loop
34
35 end_for:
36         mov     ebx, [ebp+12]     ; ebx = sump
37         mov     eax, [ebp-4]      ; eax = sum
38         mov     [ebx], eax
39
40         pop     ebx               ; restore ebx
41         leave
42         ret
```

sub5.asm

چرا سطر ۲۲ از sub5.asm تا این اندازه مهم است؟ زیرا قرارداد فراخوانی C ایجاب می‌کند که مقدار EBX پس از فراخوانی تابع تغییر نکرده باشد. اگر این کار انجام نشود، به احتمال بسیار زیاد برنامه درست کار نخواهد کرد.

```
Sum integers up to: 10
Stack Dump # 1
EBP = BFFFFB70 ESP = BFFFFB68
 +16  BFFFFB80  080499EC
 +12  BFFFFB7C  BFFFFB80
  +8  BFFFFB78  0000000A
  +4  BFFFFB74  08048501
  +0  BFFFFB70  BFFFFB88
  -4  BFFFFB6C  00000000
  -8  BFFFFB68  4010648C
Sum is 55
```

شکل ۱۳.۴: نمونه‌ای از اجرای برنامهٔ sub5

سطر ۲۵ چگونگی کار ماکروی[۶۸] dump_stack را نشان می‌دهد. به یاد دارید که پارامتر اول این ماکرو تنها یک برچسب عددی است و پارامترهای دوم و سوم به ترتیب تعیین می‌کنند که چند کلمهٔ مضاعف پایین‌تر و بالاتر از EBP نمایش داده شود. شکل ۱۳.۴ نمونه‌ای از اجرای برنامه را نشان می‌دهد. در این خروجی می‌توان دید که آدرس کلمهٔ مضاعفی که مجموع باید در آن ذخیره شود BFFFFB80 است (در EBP + 12)، عددی که مجموع تا آن محاسبه می‌شود 0000000A است (در EBP + 8)، آدرس بازگشت روال 08048501 است (در EBP + 4)، مقدار ذخیره‌شدهٔ EBP برابر BFFFFB88 است (در EBP)، مقدار متغیر محلی ۰ است (در EBP - 4) و سرانجام مقدار ذخیره‌شدهٔ EBX برابر 4010648C است (در EBP - 8).

می‌توان تابع calc_sum را طوری بازنویسی کرد که به جای استفاده از یک پارامتر اشاره‌گر، مجموع را به عنوان مقدار بازگشتی خود برگرداند. از آنجا که مجموع مقداری صحیح است، باید آن را در ثبات EAX باقی گذاشت. در این صورت سطر ۱۱ فایل main5.c به صورت زیر تغییر می‌کند:

```
sum = calc_sum(n);
```

همچنین باید پیش‌نمونهٔ calc_sum را نیز تغییر داد. کد اسمبلی اصلاح‌شده در زیر آمده است:

sub6.asm

```
1 ; subroutine _calc_sum
2 ; finds the sum of the integers 1 through n
```

[۶۸] macro

```
3  ; Parameters:
4  ;   n    - what to sum up to (at [ebp + 8])
5  ; Return value:
6  ;   value of sum
7  ; pseudo C code:
8  ; int calc_sum( int n )
9  ; {
10 ;   int i, sum = 0;
11 ;   for( i=1; i <= n; i++ )
12 ;     sum += i;
13 ;   return sum;
14 ; }
15 segment .text
16         global  _calc_sum
17 ;
18 ; local variable:
19 ;   sum at [ebp-4]
20 _calc_sum:
21         enter   4,0               ; make room for sum on stack
22
23         mov     dword [ebp-4],0   ; sum = 0
24         mov     ecx, 1            ; ecx is i in pseudocode
25 for_loop:
26         cmp     ecx, [ebp+8]      ; cmp i and n
27         jnle    end_for           ; if not i <= n, quit
28
29         add     [ebp-4], ecx      ; sum += i
30         inc     ecx
31         jmp     short for_loop
32
33 end_for:
34         mov     eax, [ebp-4]      ; eax = sum
35
36         leave
37         ret
```

sub6.asm

۸.۷.۴ فراخوانی توابع C از اسمبلی

یکی از مزیت‌های بزرگ پیوند C و اسمبلی این است که کد اسمبلی می‌تواند به کتابخانهٔ بزرگ C و توابعی که کاربر نوشته است دسترسی داشته باشد. برای نمونه، فرض کنید بخواهیم تابع scanf را فراخوانی کنیم تا یک عدد صحیح را از صفحه‌کلید

```
segment .data
format       db "%d", 0

segment .text
...
      lea    eax, [ebp-16]
      push   eax
      push   dword format
      call   _scanf
      add    esp, 8
...
```

شکل ۱۴.۴: فراخوانی scanf از اسمبلی

بخوانیم. شکل ۱۴.۴ کد این کار را نشان می‌دهد. نکتهٔ بسیار مهمی که باید به خاطر داشت این است که scanf مو به مو از استاندارد فراخوانی C پیروی می‌کند. این یعنی مقادیر ثبات‌های EBX، ESI و EDI را حفظ می‌کند، اما ممکن است ثبات‌های EAX، ECX و EDX را تغییر دهد! در واقع EAX قطعاً تغییر خواهد کرد، زیرا مقدار بازگشتی فراخوانی scanf را در خود خواهد داشت. برای دیدن نمونه‌های دیگری از پیوند با C، به کد asm_io.asm نگاه کنید که برای ساختن asm_io.obj به کار رفته است.

۸.۴ زیربرنامه‌های بازورودپذیر و بازگشتی

یک زیربرنامهٔ بازورودپذیر[۶۹] باید ویژگی‌های زیر را داشته باشد:

- هیچ دستورالعملی از کد را تغییر ندهد. در یک زبان سطح بالا انجام چنین کاری دشوار است، اما در اسمبلی برای یک برنامه چندان سخت نیست که بخواهد کد خود را تغییر دهد. برای نمونه:

```
mov    word [cs:$+7], 5      ; copy 5 into the word 7 bytes ahead
add    ax, 2                 ; previous statement changes 2 to 5!
```

این کد در حالت حقیقی[۷۰] کار می‌کند، اما در سیستم‌عامل‌هایی که در حالت

[69] reentrant
[70] real mode

حفاظت‌شده[۷۱] اجرا می‌شوند، قطعهٔ کد[۷۲] به عنوان فقط‌خواندنی علامت‌گذاری شده است، از این رو در این سیستم‌ها به محض اجرای سطر اول کد بالا، برنامه متوقف می‌شود. چنین روشی از برنامه‌نویسی به دلایل بسیاری نادرست است: گیج‌کننده است، نگهداری آن دشوار است و اجازهٔ اشتراک کد را نمی‌دهد (توضیح آن را در ادامه ببینید).

- داده‌های سراسری (مانند داده‌های قطعه‌های `data` و `bss`) را تغییر ندهد. همهٔ متغیرها در پشته[۷۳] ذخیره می‌شوند.

نوشتن کد بازورودپذیر چند مزیت دارد:

- یک زیربرنامهٔ بازورودپذیر را می‌توان به صورت بازگشتی[۷۴] فراخوانی کرد.
- یک برنامهٔ بازورودپذیر را می‌توان میان چند فرایند به اشتراک گذاشت. در بسیاری از سیستم‌عامل‌های چندوظیفه‌ای[۷۵]، اگر چند نمونه از یک برنامه در حال اجرا باشد، تنها **یک** نسخه از کد آن در حافظه قرار دارد. کتابخانه‌های مشترک و DLLها (**کتابخانه‌های پیوند پویا**[۷۶]) نیز از همین ایده استفاده می‌کنند.
- زیربرنامه‌های بازورودپذیر در برنامه‌های **چندریسمانی**[۷۷] بسیار بهتر کار می‌کنند. ویندوز 9x/NT و بیشتر سیستم‌عامل‌های شبه‌یونیکس (سولاریس، لینوکس و **مانند آن‌ها**) از برنامه‌های چندریسمانی پشتیبانی می‌کنند.

۱.۸.۴ زیربرنامه‌های بازگشتی

این نوع زیربرنامه‌ها خودشان را فراخوانی می‌کنند. بازگشت[۷۸] می‌تواند **مستقیم** یا **غیرمستقیم** باشد. بازگشت مستقیم هنگامی رخ می‌دهد که یک زیربرنامه، مثلاً `foo`، درون بدنهٔ خود `foo` را فراخوانی کند. اما در بازگشت غیرمستقیم، زیربرنامه مستقیماً به دست خود فراخوانی نمی‌شود، بلکه زیربرنامهٔ دیگری که خود آن را فراخوانی کرده

[71] protected mode
[72] code segment
[73] stack
[74] recursive
[75] multi-tasking
[76] Dynamic Link Libraries
[۷۷] برنامهٔ چندریسمانی (multi-threaded) چند ریسمان اجرایی (thread) دارد، به بیان دیگر، خود برنامه چندوظیفه‌ای است.
[78] recursion

```
1  ; finds n!
2  segment .text
3        global _fact
4  _fact:
5        enter  0,0
6
7        mov    eax, [ebp+8]    ; eax = n
8        cmp    eax, 1
9        jbe    term_cond       ; if n <= 1, terminate
10       dec    eax
11       push   eax
12       call   _fact           ; eax = fact(n-1)
13       pop    ecx             ; answer in eax
14       mul    dword [ebp+8]   ; edx:eax = eax * [ebp+8]
15       jmp    short end_fact
16 term_cond:
17       mov    eax, 1
18 end_fact:
19       leave
20       ret
```

شکل ۱۵.۴: تابع بازگشتی فاکتوریل

است، دوباره آن را فرا می‌خواند. برای نمونه، زیربرنامهٔ foo ممکن است bar را فراخوانی کند و bar نیز foo را.

زیربرنامه‌های بازگشتی باید یک **شرط پایان**[۷۹] داشته باشند. هنگامی که این شرط برقرار شود، دیگر فراخوانی بازگشتی انجام نمی‌شود. اگر یک روال بازگشتی شرط پایان نداشته باشد یا این شرط هرگز برقرار نشود، بازگشت هرگز پایان نمی‌یابد (بسیار شبیه به یک حلقهٔ بی‌پایان).

شکل ۱۵.۴ تابعی را نشان می‌دهد که فاکتوریل را به صورت بازگشتی محاسبه می‌کند. این تابع را می‌توان از C به این صورت فراخوانی کرد:

```
x = fact(3);          /* find 3! */
```

شکل ۱۶.۴ وضعیت پشته را در عمیق‌ترین نقطهٔ آن برای فراخوانی بالا نشان می‌دهد.

شکل‌های ۱۷.۴ و ۱۸.۴ مثال بازگشتی پیچیده‌تر دیگری را به ترتیب به زبان C و

[79] termination condition



شکل ۱۶.۴: قاب‌های پشته برای تابع فاکتوریل

```
1 void f( int x )
2 {
3   int i;
4   for( i=0; i < x; i++ ) {
5     printf ("%d\n", i);
6     f(i);
7   }
8 }
```

شکل ۱۷.۴: مثالی دیگر (نسخهٔ C)

اسمبلی نشان می‌دهند. خروجی `f(3)` چیست؟ توجه کنید که دستورالعمل `ENTER` برای هر فراخوانی بازگشتی یک `i` جدید در پشته ایجاد می‌کند. بنابراین هر نمونهٔ بازگشتی از `f` متغیر `i` مستقل خود را دارد. اگر `i` به صورت یک کلمهٔ مضاعف در قطعهٔ `data` تعریف می‌شد، برنامه دیگر به همین شکل کار نمی‌کرد.

۲.۸.۴ مروری بر انواع ذخیره‌سازی متغیرها در C

زبان C چند نوع ذخیره‌سازی برای متغیرها فراهم می‌کند:

سراسری (global) این متغیرها بیرون از هر تابعی تعریف می‌شوند و در خانه‌های ثابتی از حافظه (در قطعه‌های `data` یا `bss`) ذخیره می‌شوند و از آغاز تا پایان برنامه وجود دارند. به طور پیش‌فرض، همهٔ توابع برنامه می‌توانند به آن‌ها دسترسی داشته باشند، اما اگر به صورت `static` اعلان شوند، تنها توابع همان

```
%define i ebp-4
%define x ebp+8          ; useful macros
segment .data
format       db "%d", 10, 0     ; 10 = '\n'
segment .text
      global _f
      extern _printf
_f:
      enter  4,0           ; allocate room on stack for i

      mov    dword [i], 0  ; i = 0
lp:
      mov    eax, [i]      ; is i < x?
      cmp    eax, [x]
      jnl    quit

      push   eax           ; call printf
      push   format
      call   _printf
      add    esp, 8

      push   dword [i]     ; call f
      call   _f
      pop    eax

      inc    dword [i]     ; i++
      jmp    short lp
quit:
      leave
      ret
```

شکل ۱۸.۴: مثالی دیگر (نسخهٔ اسمبلی)

پیمانه به آن‌ها دسترسی دارند (**یعنی** به زبان اسمبلی، برچسب آن‌ها داخلی است، نه خارجی).

ایستا (static) این‌ها متغیرهای **محلی** یک تابع هستند که به صورت `static` اعلان شده‌اند. (متأسفانه C کلیدواژهٔ `static` را برای دو هدف متفاوت به کار می‌برد!) این متغیرها نیز در خانه‌های ثابتی از حافظه (در `data` یا `bss`) ذخیره می‌شوند، اما تنها در همان توابعی که در آن‌ها تعریف شده‌اند مستقیماً در دسترس هستند.

خودکار (automatic) این نوع پیش‌فرض برای متغیری در C است که درون یک تابع تعریف شود. این متغیرها هنگام فراخوانی تابعی که در آن تعریف شده‌اند در پشته تخصیص می‌یابند و هنگام بازگشت تابع آزاد می‌شوند. بنابراین جای ثابتی در حافظه ندارند.

ثباتی (register) این کلیدواژه از کامپایلر می‌خواهد که برای دادهٔ این متغیر از یک ثبات استفاده کند. این فقط یک **درخواست** است و کامپایلر **ملزم** به پذیرفتن آن نیست. اگر در جایی از برنامه از آدرس این متغیر استفاده شود، درخواست پذیرفته نخواهد شد (زیرا ثبات‌ها آدرس ندارند). همچنین تنها انواع صحیح ساده می‌توانند مقادیر ثباتی باشند و انواع ساختاری نمی‌توانند، چون در یک ثبات جا نمی‌شوند! کامپایلرهای C اغلب متغیرهای خودکار معمولی را بدون هیچ اشاره‌ای از سوی برنامه‌نویس به طور خودکار به متغیرهای ثباتی تبدیل می‌کنند.

فرّار (volatile) این کلیدواژه به کامپایلر می‌گوید که مقدار متغیر ممکن است در هر لحظه تغییر کند. معنای این سخن آن است که کامپایلر نمی‌تواند دربارهٔ زمان تغییر متغیر هیچ فرضی داشته باشد. کامپایلر اغلب مقدار یک متغیر را به طور موقت در یک ثبات نگه می‌دارد و در بخشی از کد، به جای متغیر از آن ثبات استفاده می‌کند، اما با متغیرهای `volatile` نمی‌تواند چنین بهینه‌سازی‌هایی انجام دهد. نمونهٔ رایج یک متغیر فرّار، متغیری است که دو ریسمان یک برنامهٔ چندریسمانی می‌توانند آن را تغییر دهند. کد زیر را در نظر بگیرید:

```
1 x = 10;
2 y = 20;
3 z = x;
```

اگر ریسمان دیگری بتواند `x` را تغییر دهد، ممکن است آن ریسمان `x` را میان سطرهای ۱ و ۳ تغییر دهد، به طوری که `z` دیگر ۱۰ نباشد. اما اگر `x` فرّار اعلان

نشده باشد، کامپایلر ممکن است فرض کند که `x` تغییری نکرده است و `z` را برابر ۱۰ قرار دهد.

کاربرد دیگر `volatile` این است که از استفادهٔ کامپایلر از یک ثبات برای یک متغیر جلوگیری کند.

فصل ۵

آرایه‌ها

۱.۵ مقدمه

آرایه[1] فهرستی از داده‌هاست که به صورت یک بلوک پیوسته در حافظه قرار می‌گیرد. همهٔ عناصر[2] این فهرست باید از یک نوع باشند و برای ذخیره شدن دقیقاً به تعداد یکسانی بایت از حافظه نیاز داشته باشند. به لطف همین ویژگی‌ها، آرایه‌ها امکان می‌دهند که به هر داده با دانستن جایگاه آن (یا همان اندیس[3] آن) در آرایه به‌سرعت دسترسی داشته باشیم. در واقع آدرس هر عنصر را می‌توان با دانستن سه چیز محاسبه کرد:

- آدرس نخستین عنصر آرایه؛
- تعداد بایت‌های هر عنصر؛
- اندیس آن عنصر.

ساده‌تر است که اندیس نخستین عنصر آرایه را، درست مانند زبان C، صفر در نظر بگیریم. البته می‌توان اندیس نخست را مقدار دیگری هم گرفت، اما این کار محاسبات را پیچیده‌تر می‌کند.

۱.۱.۵ تعریف آرایه‌ها

[1] array
[2] element
[3] index

```
segment .data
; define array of 10 double words initialized to 1,2,..,10
a1          dd   1, 2, 3, 4, 5, 6, 7, 8, 9, 10
; define array of 10 words initialized to 0
a2          dw   0, 0, 0, 0, 0, 0, 0, 0, 0, 0
; same as before using TIMES
a3          times 10 dw 0
; define array of bytes with 200 0's and then 100 1's
a4          times 200 db 0
            times 100 db 1

segment .bss
; define an array of 10 uninitialized double words
a5          resd  10
; define an array of 100 uninitialized words
a6          resw  100
```

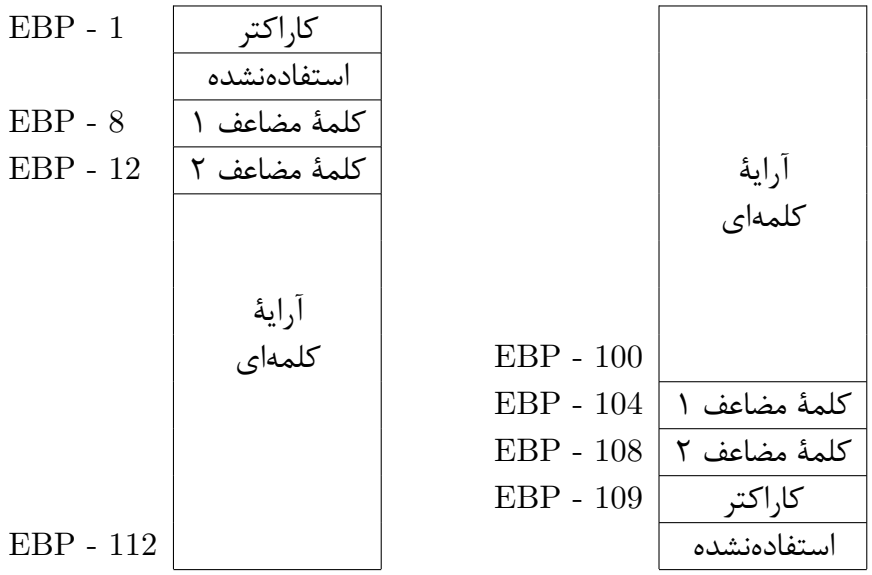
شکل ۱.۵: تعریف آرایه‌ها

تعریف آرایه در قطعه‌های `data` و `bss`

برای تعریف یک آرایهٔ مقداردهی‌شده در قطعهٔ[۴] `data` از همان رهنمودهای[۵] معمول `db`، `dw` و مانند آن‌ها استفاده می‌کنیم. افزون بر این، NASM رهنمود سودمندی به نام `TIMES` در اختیار ما می‌گذارد که با آن می‌توان یک عبارت را بارها تکرار کرد، بی‌آنکه لازم باشد آن را دستی چندین بار نوشت. شکل ۱.۵ چند نمونه از این تعریف‌ها را نشان می‌دهد.

آرایهٔ مقداردهی‌نشده را نیز در قطعهٔ `bss` و با رهنمودهای `resb`، `resw` و مانند آن‌ها تعریف می‌کنیم. به یاد دارید که این رهنمودها عملگری دارند که مشخص می‌کند چند واحد از حافظه باید رزرو شود. نمونه‌هایی از این نوع تعریف نیز در شکل ۱.۵ آمده است.

[4] segment
[5] directive



شکل ۲.۵: دو چیدمان ممکن در پشته

تعریف آرایه به صورت متغیر محلی در پشته

برای تعریف یک آرایهٔ محلی در پشته[6] هیچ راه مستقیمی وجود ندارد. در این حالت نیز مانند گذشته، مجموع بایت‌هایی را که **همهٔ** متغیرهای محلی[7]، از جمله آرایه‌ها، لازم دارند حساب می‌کنیم و این مقدار را از ESP کم می‌کنیم، خواه به طور مستقیم و خواه با دستورالعمل ENTER. برای نمونه، اگر تابعی به یک متغیر کاراکتری، دو عدد صحیح از نوع کلمهٔ مضاعف و یک آرایهٔ ۵۰ عنصری از کلمه‌ها نیاز داشته باشد، به $1 + 2 \times 4 + 50 \times 2 = 109$ بایت حافظه احتیاج خواهد داشت. با این حال، عددی که از ESP کم می‌شود باید مضربی از چهار باشد (در اینجا ۱۱۲) تا ESP همچنان روی مرز[8] کلمهٔ مضاعف باقی بماند. متغیرها را می‌توان به چند شکل در این ۱۰۹ بایت چید و شکل ۲.۵ دو چیدمان ممکن را نشان می‌دهد. بخش استفاده‌نشده در چیدمان نخست برای این است که کلمه‌های مضاعف روی مرز کلمهٔ مضاعف قرار بگیرند و در نتیجه دسترسی به حافظه سریع‌تر انجام شود.

[6] stack
[7] local variable
[8] boundary

۲.۱.۵ دسترسی به عناصر آرایه

زبان اسمبلی، برخلاف C، اپراتوری[9] مانند [ ] ندارد. بنابراین برای دسترسی به یک عنصر آرایه باید آدرس آن را خودمان محاسبه کنیم. دو تعریف آرایهٔ زیر را در نظر بگیرید:

```
array1      db      5, 4, 3, 2, 1      ; array of bytes
array2      dw      5, 4, 3, 2, 1      ; array of words
```

اکنون چند نمونه از کار با این آرایه‌ها را ببینید:

```
1       mov     al, [array1]            ; al = array1[0]
2       mov     al, [array1 + 1]        ; al = array1[1]
3       mov     [array1 + 3], al        ; array1[3] = al
4       mov     ax, [array2]            ; ax = array2[0]
5       mov     ax, [array2 + 2]        ; ax = array2[1] (NOT array2[2]!)
6       mov     [array2 + 6], ax        ; array2[3] = ax
7       mov     ax, [array2 + 1]        ; ax = ??
```

در سطر ۵ به عنصر شمارهٔ ۱ آرایهٔ کلمه‌ای ارجاع شده است، نه به عنصر شمارهٔ ۲. دلیل آن چیست؟ هر کلمه دو بایت جا می‌گیرد، پس برای رفتن به عنصر بعدی یک آرایهٔ کلمه‌ای باید دو بایت جلو رفت، نه یک بایت. سطر ۷ نیز یک بایت از عنصر نخست و یک بایت از عنصر دوم را می‌خواند. در C، کامپایلر در عبارت‌هایی که از حساب اشاره‌گرها[10] استفاده می‌کنند به نوع اشاره‌گر نگاه می‌کند تا بفهمد چند بایت باید جابه‌جا شود، و به این ترتیب برنامه‌نویس درگیر این کار نمی‌شود. اما در اسمبلی این بر عهدهٔ خود برنامه‌نویس است که هنگام رفتن از یک عنصر به عنصر دیگر، اندازهٔ عناصر آرایه را در نظر بگیرد.

شکل ۳.۵ تکه‌کدی را نشان می‌دهد که همهٔ عناصر array1 از مثال قبل را با هم جمع می‌کند. در سطر ۷، AX با DX جمع می‌شود. شاید بپرسید چرا AL نه؟ دلیل نخست این است که دو عملگر[11] دستورالعمل ADD باید هم‌اندازه باشند. دلیل دوم این است که جمع چند بایت به آسانی ممکن است حاصلی بدهد که در یک بایت جا نشود. با به کار بردن DX، حاصل جمع می‌تواند تا ۶۵۵۳۵ برسد. با این حال باید توجه داشت که AH نیز در این جمع شرکت می‌کند و به همین دلیل است که در

[9] operator
[10] pointer arithmetic
[11] operand

```
1       mov     ebx, array1             ; ebx = address of array1
2       mov     dx, 0                   ; dx will hold sum
3       mov     ah, 0                   ; ?
4       mov     ecx, 5
5   lp:
6       mov     al, [ebx]               ; al = *ebx
7       add     dx, ax                  ; dx += ax (not al!)
8       inc     ebx                     ; bx++
9       loop    lp
```

شکل ۳.۵: جمع عناصر یک آرایه (نسخهٔ ۱)

```
1       mov     ebx, array1             ; ebx = address of array1
2       mov     dx, 0                   ; dx will hold sum
3       mov     ecx, 5
4   lp:
5       add     dl, [ebx]               ; dl += *ebx
6       jnc     next                    ; if no carry goto next
7       inc     dh                      ; inc dh
8   next:
9       inc     ebx                     ; bx++
10      loop    lp
```

شکل ۴.۵: جمع عناصر یک آرایه (نسخهٔ ۲)

سطر ۳ مقدار AH صفر می‌شود.[۱۲]

شکل‌های ۴.۵ و ۵.۵ دو روش دیگر برای محاسبهٔ این مجموع را نشان می‌دهند. در هر دو، سطرهایی که با حروف مورب آمده‌اند جای سطرهای ۶ و ۷ شکل ۳.۵ را می‌گیرند.

۳.۱.۵ آدرس‌دهی غیرمستقیم پیشرفته‌تر

جای تعجب نیست که آدرس‌دهی غیرمستقیم[۱۳] بیشتر همراه با آرایه‌ها به کار می‌رود. کلی‌ترین شکل یک ارجاع غیرمستقیم به حافظه چنین است:

[۱۲] صفر کردن AH به طور ضمنی فرض می‌کند که AL یک عدد بی‌علامت است. اگر AL علامت‌دار باشد، کار درست آن است که یک دستورالعمل CBW میان سطرهای ۶ و ۷ قرار داده شود.

[13] indirect addressing

```
1       mov     ebx, array1             ; ebx = address of array1
2       mov     dx, 0                   ; dx will hold sum
3       mov     ecx, 5
4   lp:
5       add     dl, [ebx]               ; dl += *ebx
6       adc     dh, 0                   ; dh += carry flag + 0
7       inc     ebx                     ; bx++
8       loop    lp
```

شکل ۵.۵: جمع عناصر یک آرایه (نسخهٔ ۳)

[*base reg* + *factor***index reg* + *constant*]

که در آن:

base reg ثبات پایه[۱۴] است و می‌تواند یکی از ثبات‌های EAX، EBX، ECX، EDX، EBP، ESP، ESI یا EDI باشد.

factor ضریب است و مقدار آن ۱، ۲، ۴ یا ۸ است. (اگر ضریب ۱ باشد، آن را نمی‌نویسند.)

index reg ثبات اندیس[۱۵] است و می‌تواند یکی از ثبات‌های EAX، EBX، ECX، EDX، EBP، ESI یا EDI باشد. (دقت کنید که ESP در این فهرست نیست.)

constant یک ثابت ۳۲ بیتی است. این ثابت می‌تواند یک برچسب (یا عبارتی شامل برچسب) باشد.

۴.۱.۵ مثال

در اینجا مثالی می‌آوریم که از یک آرایه استفاده می‌کند و آن را به یک تابع می‌فرستد. این برنامه به جای برنامهٔ `driver.c`، از برنامهٔ `array1c.c` (که در ادامه آمده است) به عنوان برنامهٔ راه‌انداز[۱۶] استفاده می‌کند.

array1.asm

```
1   %define ARRAY_SIZE 100
2   %define NEW_LINE 10
```

[14] base register
[15] index register
[16] driver

```
3
4  segment .data
5  FirstMsg        db   "First 10 elements of array", 0
6  Prompt          db   "Enter index of element to display: ", 0
7  SecondMsg       db   "Element %d is %d", NEW_LINE, 0
8  ThirdMsg        db   "Elements 20 through 29 of array", 0
9  InputFormat     db   "%d", 0
10
11 segment .bss
12 array           resd ARRAY_SIZE
13
14 segment .text
15         extern  _puts, _printf, _scanf, _dump_line
16         global  _asm_main
17 _asm_main:
18         enter   4,0               ; local dword variable at EBP - 4
19         push    ebx
20         push    esi
21
22 ; initialize array to 100, 99, 98, 97, ...
23
24         mov     ecx, ARRAY_SIZE
25         mov     ebx, array
26 init_loop:
27         mov     [ebx], ecx
28         add     ebx, 4
29         loop    init_loop
30
31         push    dword FirstMsg         ; print out FirstMsg
32         call    _puts
33         pop     ecx
34
35         push    dword 10
36         push    dword array
37         call    _print_array           ; print first 10 elements of array
38         add     esp, 8
39
40 ; prompt user for element index
41 Prompt_loop:
42         push    dword Prompt
43         call    _printf
44         pop     ecx
```

```
45
46          lea     eax, [ebp-4]      ; eax = address of local dword
47          push    eax
48          push    dword InputFormat
49          call    _scanf
50          add     esp, 8
51          cmp     eax, 1               ; eax = return value of scanf
52          je      InputOK
53
54          call    _dump_line  ; dump rest of line and start over
55          jmp     Prompt_loop          ; if input invalid
56
57  InputOK:
58          mov     esi, [ebp-4]
59          push    dword [array + 4*esi]
60          push    esi
61          push    dword SecondMsg      ; print out value of element
62          call    _printf
63          add     esp, 12
64
65          push    dword ThirdMsg       ; print out elements 20-29
66          call    _puts
67          pop     ecx
68
69          push    dword 10
70          push    dword array + 20*4     ; address of array[20]
71          call    _print_array
72          add     esp, 8
73
74          pop     esi
75          pop     ebx
76          mov     eax, 0            ; return back to C
77          leave
78          ret
79
80  ;
81  ; routine _print_array
82  ; C-callable routine that prints out elements of a double word array as
83  ; signed integers.
84  ; C prototype:
85  ; void print_array( const int * a, int n);
86  ; Parameters:
```

```
87  ;   a - pointer to array to print out (at ebp+8 on stack)
88  ;   n - number of integers to print out (at ebp+12 on stack)
89
90  segment .data
91  OutputFormat    db   "%-5d %5d", NEW_LINE, 0
92
93  segment .text
94          global  _print_array
95  _print_array:
96          enter   0,0
97          push    esi
98          push    ebx
99
100         xor     esi, esi                ; esi = 0
101         mov     ecx, [ebp+12]           ; ecx = n
102         mov     ebx, [ebp+8]            ; ebx = address of array
103 print_loop:
104         push    ecx                     ; printf might change ecx!
105
106         push    dword [ebx + 4*esi]     ; push array[esi]
107         push    esi
108         push    dword OutputFormat
109         call    _printf
110         add     esp, 12                 ; remove parameters (leave ecx!)
111
112         inc     esi
113         pop     ecx
114         loop    print_loop
115
116         pop     ebx
117         pop     esi
118         leave
119         ret
```

array1.asm

array1c.c

```
1  #include <stdio.h>
2
3  int asm_main( void );
4  void dump_line( void );
5
6  int main()
```

```
7  {
8    int ret_status;
9    ret_status = asm_main();
10   return ret_status;
11 }
12
13 /*
14  * function dump_line
15  * dumps all chars left in current line from input buffer
16  */
17 void dump_line()
18 {
19   int ch;
20
21   while( (ch = getchar()) != EOF && ch != '\n')
22     /* null body*/ ;
23 }
```

array1c.c

نگاهی دوباره به دستورالعمل LEA

دستورالعمل LEA فقط برای محاسبهٔ آدرس‌ها نیست و می‌توان آن را برای کارهای دیگری هم به کار برد. یکی از کاربردهای نسبتاً رایج آن، انجام محاسبات سریع است. دستورالعمل زیر را در نظر بگیرید:

```
lea    ebx, [4*eax + eax]
```

این دستورالعمل در عمل مقدار $5 \times$ EAX را در EBX ذخیره می‌کند. انجام این کار با LEA هم ساده‌تر و هم سریع‌تر از به کار بردن MUL است. با این حال باید توجه داشت که عبارت درون کروشه‌ها **حتماً** باید یک آدرس غیرمستقیم معتبر باشد. از این رو، برای نمونه، نمی‌توان از این دستورالعمل برای ضرب سریع در ۶ استفاده کرد.

۵.۱.۵ آرایه‌های چندبعدی

آرایه‌های چندبعدی[۱۷] در واقع تفاوت چندانی با آرایه‌های ساده و یک‌بعدی‌ای که تا اینجا بررسی کردیم ندارند. در حقیقت، این آرایه‌ها در حافظه دقیقاً به همان شکل، یعنی به صورت یک آرایهٔ یک‌بعدی ساده، نمایش داده می‌شوند.

[17] multidimensional array

```
1    mov    eax, [ebp - 44]          ; ebp - 44 is i's location
2    sal    eax, 1                   ; multiple i by 2
3    add    eax, [ebp - 48]          ; add j
4    mov    eax, [ebp + 4*eax - 40]  ; ebp - 40 is the address of a[0][0]
5    mov    [ebp - 52], eax          ; store result into x (at ebp - 52)
```

شکل ۶.۵: کد اسمبلی عبارت `x = a[i][j]`

آرایه‌های دوبعدی

طبیعی است که ساده‌ترین آرایهٔ چندبعدی، آرایهٔ دوبعدی باشد. آرایهٔ دوبعدی را اغلب به صورت شبکه‌ای از عناصر نشان می‌دهند که هر عنصر آن با یک جفت اندیس مشخص می‌شود. طبق قرارداد، اندیس نخست شمارهٔ سطر عنصر و اندیس دوم شمارهٔ ستون آن را نشان می‌دهد.

آرایه‌ای با سه سطر و دو ستون را در نظر بگیرید که به این صورت تعریف شده است:

```
int a [3][2];
```

کامپایلر C برای این تعریف، فضای یک آرایهٔ ۶ ($= 2 \times 3$) عنصری از اعداد صحیح را رزرو می‌کند و عناصر را به این ترتیب در آن جای می‌دهد:

اندیس	0	1	2	3	4	5
عنصر	a[0][0]	a[0][1]	a[1][0]	a[1][1]	a[2][0]	a[2][1]

منظور این جدول آن است که عنصری که با `a[0][0]` به آن ارجاع می‌شود، در ابتدای آرایهٔ یک‌بعدی ۶ عنصری ذخیره می‌شود. عنصر `a[0][1]` در خانهٔ بعدی (اندیس ۱) قرار می‌گیرد و به همین ترتیب تا آخر. به بیان دیگر، هر سطر آرایهٔ دوبعدی به صورت پیوسته در حافظه ذخیره می‌شود و پس از آخرین عنصر هر سطر، نخستین عنصر سطر بعدی می‌آید. این شیوه را نمایش **سطری**[۱۸] آرایه می‌نامند و کامپایلرهای C/C++ آرایه را به همین شکل نمایش می‌دهند.

اما کامپایلر چگونه تشخیص می‌دهد که `a[i][j]` در کجای نمایش سطری قرار دارد؟ یک فرمول ساده اندیس را از روی `i` و `j` محاسبه می‌کند که در این مورد $2i + j$ است. به‌دست آوردن این فرمول چندان دشوار نیست. هر سطر دو عنصر دارد، پس نخستین عنصر سطر i در جایگاه $2i$ قرار می‌گیرد و سپس با افزودن j به $2i$ جایگاه

[18] rowwise representation

ستون j به دست می‌آید. همین تحلیل نشان می‌دهد که فرمول برای آرایه‌ای با N ستون چگونه تعمیم می‌یابد: $N \times i + j$. دقت کنید که این فرمول به تعداد سطرها **وابسته نیست**.

برای نمونه، ببینیم gcc کد زیر را (با همان آرایهٔ a که در بالا تعریف شد) چگونه کامپایل می‌کند:

```
x = a[i][j];
```

کد اسمبلی حاصل از این ترجمه در شکل ۶.۵ آمده است. بنابراین کامپایلر در اصل این کد را به صورت زیر درمی‌آورد:

```
x = *(&a[0][0] + 2*i + j);
```

و در واقع برنامه‌نویس نیز می‌تواند کد را به همین شکل بنویسد و همان نتیجه را بگیرد.

انتخاب نمایش سطری برای آرایه هیچ راز خاصی ندارد و نمایش ستونی نیز به همان خوبی کار می‌کند:

اندیس	0	1	2	3	4	5
عنصر	a[0][0]	a[1][0]	a[2][0]	a[0][1]	a[1][1]	a[2][1]

در نمایش ستونی[۱۹]، هر ستون به صورت پیوسته ذخیره می‌شود و عنصر `[j][i]` در جایگاه $i + 3j$ قرار می‌گیرد. برخی زبان‌های دیگر (برای نمونه FORTRAN) از نمایش ستونی استفاده می‌کنند. این نکته هنگامی اهمیت پیدا می‌کند که بخواهیم کدهایی را که به چند زبان مختلف نوشته شده‌اند با هم پیوند دهیم.

آرایه‌هایی با بیش از دو بعد

برای آرایه‌هایی که بیش از دو بعد دارند نیز همین ایدهٔ اصلی به کار می‌رود. یک آرایهٔ سه‌بعدی را در نظر بگیرید:

```
int b [4][3][2];
```

این آرایه در حافظه طوری ذخیره می‌شود که گویی چهار آرایهٔ دوبعدی، هر یک به اندازهٔ [2] [3]، پشت سر هم قرار گرفته‌اند. جدول زیر آغاز این چیدمان را نشان می‌دهد:

[19] columnwise representation

اندیس	0	1	2	3	4	5
عنصر	b[0][0][0]	b[0][0][1]	b[0][1][0]	b[0][1][1]	b[0][2][0]	b[0][2][1]
اندیس	6	7	8	9	10	11
عنصر	b[1][0][0]	b[1][0][1]	b[1][1][0]	b[1][1][1]	b[1][2][0]	b[1][2][1]

فرمول محاسبهٔ جایگاه `b[i][j][k]` برابر با $6i + 2j + k$ است که در آن عدد ۶ از اندازهٔ آرایه‌های `[2][3]` به دست می‌آید. در حالت کلی، برای آرایه‌ای با ابعاد `a[L][M][N]`، جایگاه عنصر `a[i][j][k]` برابر با $M \times N \times i + N \times j + k$ خواهد بود. باز هم دقت کنید که بعد نخست (`L`) در این فرمول ظاهر نمی‌شود.

برای ابعاد بالاتر نیز همین روند تعمیم می‌یابد. در یک آرایهٔ n بعدی با ابعاد D_1 تا D_n، جایگاه عنصری که با اندیس‌های i_1 تا i_n مشخص می‌شود از این فرمول به دست می‌آید:

$$D_2 \times D_3 \cdots \times D_n \times i_1 + D_3 \times D_4 \cdots \times D_n \times i_2 + \cdots + D_n \times i_{n-1} + i_n$$

و برای علاقه‌مندان دوآتشهٔ ریاضی، می‌توان آن را فشرده‌تر نیز چنین نوشت:

$$\sum_{j=1}^{n} \left(\prod_{k=j+1}^{n} D_k \right) i_j$$

در اینجا نیز بعد نخست، یعنی D_1، در فرمول ظاهر نمی‌شود.

فرمول کلی برای نمایش ستونی چنین است:

$$i_1 + D_1 \times i_2 + \cdots + D_1 \times D_2 \times \cdots \times D_{n-2} \times i_{n-1} + D_1 \times D_2 \times \cdots \times D_{n-1} \times i_n$$

از همین‌جا می‌توان فهمید که نویسنده دانش‌آموختهٔ فیزیک است. (یا شاید اشاره به FORTRAN پیش‌تر این راز را فاش کرده بود؟)

یا با نمادگذاری دلخواه علاقه‌مندان دوآتشهٔ ریاضی:

$$\sum_{j=1}^{n} \left(\prod_{k=1}^{j-1} D_k \right) i_j$$

در این حالت، این بعد آخر یعنی D_n است که در فرمول ظاهر نمی‌شود.

ارسال آرایه‌های چندبعدی به عنوان پارامتر در C

نمایش سطری آرایه‌های چندبعدی اثر مستقیمی بر برنامه‌نویسی به زبان C دارد. در آرایه‌های یک‌بعدی، برای پیدا کردن محل هر عنصر در حافظه نیازی به دانستن اندازهٔ آرایه نیست، اما در آرایه‌های چندبعدی وضع چنین نیست و کامپایلر برای دسترسی به

عناصر این آرایه‌ها باید همهٔ ابعاد به جز بعد نخست را بداند. این موضوع وقتی آشکار می‌شود که پیش‌نمایهٔ[۲۰] تابعی را در نظر بگیریم که یک آرایهٔ چندبعدی را به عنوان پارامتر[۲۱] می‌گیرد. کد زیر کامپایل نمی‌شود:

```
void f( int a[ ][ ] );  /* no dimension information */
```

اما کد زیر بدون مشکل کامپایل می‌شود:

```
void f( int a[ ][2] );
```

هر آرایهٔ دوبعدی با دو ستون را می‌توان به این تابع فرستاد، زیرا بعد نخست لازم نیست.[۲۲]

تابعی با پیش‌نمایهٔ زیر نباید شما را به اشتباه بیندازد:

```
void f( int * a[ ] );
```

این پیش‌نمایه یک آرایهٔ یک‌بعدی از اشاره‌گرهایی[۲۳] به اعداد صحیح را تعریف می‌کند. (اتفاقاً با چنین آرایه‌ای می‌توان آرایه‌ای از آرایه‌ها ساخت که رفتاری بسیار شبیه به یک آرایهٔ دوبعدی دارد.)

در آرایه‌هایی با ابعاد بالاتر نیز هنگام تعریف پارامتر باید همهٔ ابعاد آرایه به جز بعد نخست مشخص شوند. برای نمونه، یک پارامتر آرایهٔ چهاربعدی را می‌توان به این صورت فرستاد:

```
void f( int a[ ][4][3][2] );
```

۲.۵ دستورالعمل‌های آرایه و رشته

خانوادهٔ پردازنده‌های 80x86 چند دستورالعمل[۲۴] در اختیار برنامه‌نویس می‌گذارد که ویژهٔ کار با آرایه‌ها[۲۵] طراحی شده‌اند و به آن‌ها **دستورالعمل‌های رشته‌ای**[۲۶] گفته می‌شود. این دستورالعمل‌ها برای انجام عمل خود از ثبات‌های[۲۷] اندیس (ESI و EDI) استفاده می‌کنند و پس از آن، یکی از این دو ثبات یا هر دوی آن‌ها را به‌طور خودکار افزایش یا کاهش می‌دهند. اینکه ثبات‌های اندیس افزایش یابند یا کاهش، به **پرچم**

[20] prototype
[21] parameter
[۲۲] می‌توان اندازه‌ای را در اینجا مشخص کرد، اما کامپایلر آن را نادیده می‌گیرد.
[23] pointer
[24] instruction
[25] array
[26] string instructions
[27] register

LODSB AL = [DS:ESI] ESI = ESI ± 1	STOSB [ES:EDI] = AL EDI = EDI ± 1
LODSW AX = [DS:ESI] ESI = ESI ± 2	STOSW [ES:EDI] = AX EDI = EDI ± 2
LODSD EAX = [DS:ESI] ESI = ESI ± 4	STOSD [ES:EDI] = EAX EDI = EDI ± 4

شکل ۷.۵: دستورالعمل‌های رشته‌ای خواندن و نوشتن

جهت[۲۸] (DF) در ثبات FLAGS بستگی دارد. دو دستورالعمل برای تغییر پرچم جهت وجود دارد:

CLD پرچم جهت را پاک می‌کند. در این حالت، ثبات‌های اندیس افزایش می‌یابند.

STD پرچم جهت را یک می‌کند. در این حالت، ثبات‌های اندیس کاهش می‌یابند.

یکی از اشتباهات **بسیار** رایج در برنامه‌نویسی 80x86 این است که برنامه‌نویس فراموش می‌کند پرچم جهت را به‌صراحت در وضعیت درست قرار دهد. نتیجهٔ این کار اغلب کدی است که بیشتر وقت‌ها درست کار می‌کند (یعنی هر بار که پرچم جهت به‌طور اتفاقی در وضعیت دلخواه باشد)، اما **همیشه** درست کار نمی‌کند.

۱.۲.۵ خواندن از حافظه و نوشتن در آن

ساده‌ترین دستورالعمل‌های رشته‌ای یا از حافظه[۲۹] می‌خوانند، یا در آن می‌نویسند، یا هر دو کار را انجام می‌دهند و در هر بار می‌توانند یک بایت[۳۰]، یک کلمه[۳۱] یا یک کلمهٔ مضاعف[۳۲] را بخوانند یا بنویسند. شکل ۷.۵ این دستورالعمل‌ها را همراه با توصیفی کوتاه به شکل شبه‌کد از کاری که انجام می‌دهند نشان می‌دهد. در اینجا چند نکته درخور توجه است. نخست اینکه ESI برای خواندن و EDI برای نوشتن به کار می‌رود. به خاطر سپردن این نکته آسان است، به شرط آنکه بدانیم SI کوتاه‌شدهٔ *Source Index* (اندیس مبدأ) و DI کوتاه‌شدهٔ *Destination Index* (اندیس مقصد) است. دوم اینکه ثباتی که داده را نگه می‌دارد ثابت است (یکی از AL، AX یا EAX). سرانجام اینکه دستورالعمل‌های ذخیره‌سازی برای تعیین قطعه‌ای[۳۳] که باید در آن بنویسند از

[28] direction flag
[29] memory
[30] byte
[31] word
[32] double word
[33] segment

```
1  segment .data
2  array1  dd  1, 2, 3, 4, 5, 6, 7, 8, 9, 10
3
4  segment .bss
5  array2  resd 10
6
7  segment .text
8        cld                    ; don't forget this!
9        mov    esi, array1
10       mov    edi, array2
11       mov    ecx, 10
12 lp:
13       lodsd
14       stosd
15       loop  lp
```

شکل ۸.۵: نمونه‌ای از بارکردن و ذخیره‌کردن

ES استفاده می‌کنند، نه از DS. در برنامه‌نویسی حالت حفاظت‌شده[۳۴] این موضوع معمولاً مشکلی ایجاد نمی‌کند، زیرا تنها یک قطعهٔ داده وجود دارد و ES باید به‌طور خودکار طوری مقداردهی شود که به همان قطعه اشاره کند (درست همان‌طور که DS به آن اشاره می‌کند). با این حال، در برنامه‌نویسی حالت حقیقی[۳۵]، **بسیار** مهم است که برنامه‌نویس ES را با مقدار درست انتخاب‌گر[۳۶] قطعه مقداردهی کند[۳۷]. شکل ۸.۵ نمونه‌ای از کاربرد این دستورالعمل‌ها را نشان می‌دهد که یک آرایه را در آرایه‌ای دیگر کپی می‌کند.

ترکیب یک دستورالعمل LODSx با یک دستورالعمل STOSx (مانند سطرهای ۱۳ و ۱۴ در شکل ۸.۵) بسیار رایج است. در واقع، همین ترکیب را می‌توان تنها با یک دستورالعمل رشته‌ای MOVSx انجام داد. شکل ۹.۵ عملیاتی را که این دستورالعمل‌ها انجام می‌دهند توصیف می‌کند. بنابراین می‌توان سطرهای ۱۳ و ۱۴ شکل ۸.۵ را با یک دستورالعمل MOVSD جایگزین کرد و همان نتیجه را گرفت. تنها تفاوت این است

[34] protected mode

[35] real mode

[36] selector

[۳۷] دشواری دیگر این است که نمی‌توان مقدار ثبات DS را با یک دستورالعمل MOV به‌طور مستقیم در ثبات ES کپی کرد. به جای آن، باید مقدار DS را نخست در یک ثبات همه‌منظوره (مانند AX) کپی کرد و سپس آن را با دو دستورالعمل MOV از آن ثبات به ES منتقل کرد.

```
MOVSB  byte [ES:EDI] = byte [DS:ESI]
       ESI = ESI ± 1
       EDI = EDI ± 1
MOVSW  word [ES:EDI] = word [DS:ESI]
       ESI = ESI ± 2
       EDI = EDI ± 2
MOVSD  dword [ES:EDI] = dword [DS:ESI]
       ESI = ESI ± 4
       EDI = EDI ± 4
```

شکل ۹.۵: دستورالعمل‌های رشته‌ای انتقال حافظه

```
1  segment .bss
2  array  resd 10
3
4  segment .text
5        cld                    ; don't forget this!
6        mov    edi, array
7        mov    ecx, 10
8        xor    eax, eax
9        rep stosd
```

شکل ۱۰.۵: نمونهٔ صفرکردن آرایه

که در این حالت، ثبات EAX اصلاً در حلقه[۳۸] به کار نمی‌رود.

۲.۲.۵ پیشوند دستورالعمل REP

خانوادهٔ 80x86 پیشوند دستورالعمل ویژه‌ای[۳۹] به نام REP فراهم می‌کند که می‌توان آن را همراه با دستورالعمل‌های رشته‌ای بالا به کار برد. این پیشوند به پردازنده می‌گوید که دستورالعمل رشته‌ای بعدی را به تعداد دفعات مشخصی تکرار کند. برای شمارش دفعات تکرار نیز (درست مانند دستورالعمل LOOP) از ثبات ECX استفاده می‌شود. به کمک پیشوند REP، می‌توان حلقهٔ شکل ۸.۵ (سطرهای ۱۲ تا ۱۵) را با تنها یک سطر

[38] loop

[۳۹] پیشوند دستورالعمل خودش دستورالعمل نیست، بلکه بایت ویژه‌ای است که پیش از یک دستورالعمل رشته‌ای قرار می‌گیرد و رفتار آن را تغییر می‌دهد. پیشوندهای دیگری نیز وجود دارند که برای کنار گذاشتن قطعه‌های پیش‌فرض در دسترسی به حافظه به کار می‌روند.

CMPSB	مقایسهٔ byte [DS:ESI] با byte [ES:EDI] ESI = ESI ± 1 EDI = EDI ± 1
CMPSW	مقایسهٔ word [DS:ESI] با word [ES:EDI] ESI = ESI ± 2 EDI = EDI ± 2
CMPSD	مقایسهٔ dword [DS:ESI] با dword [ES:EDI] ESI = ESI ± 4 EDI = EDI ± 4
SCASB	مقایسهٔ AL با [ES:EDI] EDI ± 1
SCASW	مقایسهٔ AX با [ES:EDI] EDI ± 2
SCASD	مقایسهٔ EAX با [ES:EDI] EDI ± 4

شکل ۱۱.۵: دستورالعمل‌های رشته‌ای مقایسه

جایگزین کرد:

```
rep movsd
```

شکل ۱۰.۵ نمونهٔ دیگری را نشان می‌دهد که محتوای یک آرایه را صفر می‌کند.

۳.۲.۵ دستورالعمل‌های رشته‌ای مقایسه

شکل ۱۱.۵ چند دستورالعمل رشته‌ای تازه را نشان می‌دهد که با آن‌ها می‌توان حافظه را با بخش دیگری از حافظه یا با یک ثبات مقایسه کرد. این دستورالعمل‌ها برای مقایسهٔ آرایه‌ها یا جست‌وجو در آن‌ها سودمند هستند و ثبات FLAGS را درست مانند دستورالعمل CMP تنظیم می‌کنند. دستورالعمل‌های CMPSx خانه‌های متناظر حافظه را با هم مقایسه می‌کنند، در حالی که دستورالعمل‌های SCASx خانه‌های حافظه را برای یافتن مقداری مشخص می‌پیمایند.

شکل ۱۲.۵ قطعه‌کد کوتاهی را نشان می‌دهد که در یک آرایه از کلمه‌های مضاعف به دنبال عدد ۱۲ می‌گردد. دستورالعمل SCASD در سطر ۱۰ همیشه ۴ واحد به EDI می‌افزاید، حتی اگر مقدار مورد جست‌وجو پیدا شده باشد. از این رو، اگر بخواهیم آدرس[۴۰] عدد ۱۲ پیداشده در آرایه را به دست آوریم، باید ۴ را از EDI کم کنیم

[40] address

```
1  segment .bss
2  array        resd 100
3
4  segment .text
5        cld
6        mov    edi, array    ; pointer to start of array
7        mov    ecx, 100      ; number of elements
8        mov    eax, 12       ; number to scan for
9  lp:
10       scasd
11       je     found
12       loop   lp
13   ; code to perform if not found
14       jmp    onward
15 found:
16       sub    edi, 4         ; edi now points to 12 in array
17   ; code to perform if found
18 onward:
```

شکل ۱۲.۵: نمونهٔ جست‌وجو

(همان کاری که سطر ۱۶ انجام می‌دهد).

۴.۲.۵ پیشوندهای دستورالعمل REPx

چند پیشوند دستورالعمل دیگر، شبیه به REP، نیز وجود دارد که می‌توان آن‌ها را همراه با دستورالعمل‌های رشته‌ای مقایسه به کار برد. شکل ۱۳.۵ دو پیشوند تازه را نشان می‌دهد و کار آن‌ها را توصیف می‌کند. REPE و REPZ در واقع دو نام مترادف برای یک پیشوند هستند (REPNE و REPNZ نیز همین‌طور). اگر اجرای تکراری یک دستورالعمل رشته‌ای مقایسه به سبب نتیجهٔ مقایسه متوقف شود، ثبات یا ثبات‌های اندیس همچنان افزایش می‌یابند و از ECX نیز یک واحد کم می‌شود، اما ثبات FLAGS هنوز همان وضعیتی را نگه می‌دارد که به تکرار پایان داده است. بنابراین می‌توان با کمک پرچم Z تشخیص داد که مقایسه‌های تکراری به سبب نتیجهٔ یک مقایسه متوقف شده‌اند یا به این دلیل که ECX به صفر رسیده است.

چرا نمی‌توان پس از مقایسهٔ تکراری، تنها بررسی کرد که آیا ECX صفر شده است یا نه؟

شکل ۱۴.۵ قطعه‌کدی نمونه را نشان می‌دهد که بررسی می‌کند آیا دو بلوک حافظه با هم برابر هستند یا نه. دستورالعمل JE در سطر ۷ این نمونه، نتیجهٔ دستورالعمل پیشین را بررسی می‌کند. اگر مقایسهٔ تکراری به این دلیل متوقف شده باشد که دو

REPZ ،REPE	دستورالعمل را تا وقتی پرچم Z یک است، حداکثر ECX بار تکرار می‌کند
REPNZ ،REPNE	دستورالعمل را تا وقتی پرچم Z پاک است، حداکثر ECX بار تکرار می‌کند

شکل ۱۳.۵: پیشوندهای دستورالعمل REPx

```
segment .text
      cld
      mov    esi, block1        ; address of first block
      mov    edi, block2        ; address of second block
      mov    ecx, size          ; size of blocks in bytes
      repe   cmpsb              ; repeat while Z flag is set
      je     equal              ; if Z set, blocks equal
   ; code to perform if blocks are not equal
      jmp    onward
equal:
   ; code to perform if equal
onward:
```

شکل ۱۴.۵: مقایسهٔ بلوک‌های حافظه

بایت نابرابر یافته است، پرچم Z همچنان پاک است و هیچ انشعابی[۴۱] رخ نمی‌دهد. اما اگر مقایسه‌ها به این دلیل متوقف شده باشند که ECX صفر شده است، پرچم Z همچنان یک است و اجرای کد به برچسب[۴۲] equal منشعب می‌شود.

۵.۲.۵ مثال

در این بخش یک فایل منبع[۴۳] اسمبلی می‌آید که چند تابع[۴۴] را در بر دارد. این توابع عملیاتی را روی آرایه‌ها با کمک دستورالعمل‌های رشته‌ای پیاده‌سازی می‌کنند و بسیاری از آن‌ها همان کار توابع آشنای کتابخانهٔ C را انجام می‌دهند.

[41] branch
[42] label
[43] source file
[44] function

memory.asm

```
global _asm_copy, _asm_find, _asm_strlen, _asm_strcpy

segment .text
; function _asm_copy
; copies blocks of memory
; C prototype
; void asm_copy( void * dest, const void * src, unsigned sz);
; parameters:
;   dest - pointer to buffer to copy to
;   src  - pointer to buffer to copy from
;   sz   - number of bytes to copy

; next, some helpful symbols are defined

%define dest [ebp+8]
%define src  [ebp+12]
%define sz   [ebp+16]
_asm_copy:
        enter   0, 0
        push    esi
        push    edi

        mov     esi, src        ; esi = address of buffer to copy from
        mov     edi, dest       ; edi = address of buffer to copy to
        mov     ecx, sz         ; ecx = number of bytes to copy

        cld                     ; clear direction flag
        rep     movsb           ; execute movsb ECX times

        pop     edi
        pop     esi
        leave
        ret


; function _asm_find
; searches memory for a given byte
; void * asm_find( const void * src, char target, unsigned sz);
; parameters:
;   src    - pointer to buffer to search
;   target - byte value to search for
```

```
42  ;   sz     - number of bytes in buffer
43  ; return value:
44  ;   if target is found, pointer to first occurrence of target in buffer
45  ;   is returned
46  ;   else
47  ;     NULL is returned
48  ; NOTE: target is a byte value, but is pushed on stack as a dword value.
49  ;       The byte value is stored in the lower 8-bits.
50  ;
51  %define src    [ebp+8]
52  %define target [ebp+12]
53  %define sz     [ebp+16]
54
55  _asm_find:
56          enter   0,0
57          push    edi
58
59          mov     eax, target     ; al has value to search for
60          mov     edi, src
61          mov     ecx, sz
62          cld
63
64          repne   scasb           ; scan until ECX == 0 or [ES:EDI] == AL
65
66          je      found_it        ; if zero flag set, then found value
67          mov     eax, 0          ; if not found, return NULL pointer
68          jmp     short quit
69  found_it:
70          mov     eax, edi
71          dec     eax              ; if found return (DI - 1)
72  quit:
73          pop     edi
74          leave
75          ret
76
77
78  ; function _asm_strlen
79  ; returns the size of a string
80  ; unsigned asm_strlen( const char * );
81  ; parameter:
82  ;   src - pointer to string
83  ; return value:
```

```
;   number of chars in string (not counting, ending 0) (in EAX)

%define src [ebp + 8]
_asm_strlen:
        enter   0,0
        push    edi

        mov     edi, src        ; edi = pointer to string
        mov     ecx, 0FFFFFFFFh ; use largest possible ECX
        xor     al,al           ; al = 0
        cld

        repnz   scasb           ; scan for terminating 0

;
; repnz will go one step too far, so length is FFFFFFFE - ECX,
; not FFFFFFFF - ECX
;
        mov     eax,0FFFFFFFEh
        sub     eax, ecx        ; length = 0FFFFFFFEh - ecx

        pop     edi
        leave
        ret

; function _asm_strcpy
; copies a string
; void asm_strcpy( char * dest, const char * src);
; parameters:
;   dest - pointer to string to copy to
;   src  - pointer to string to copy from
;
%define dest [ebp + 8]
%define src  [ebp + 12]
_asm_strcpy:
        enter   0,0
        push    esi
        push    edi

        mov     edi, dest
        mov     esi, src
        cld
```

```
126 cpy_loop:
127         lodsb                   ; load AL & inc si
128         stosb                   ; store AL & inc di
129         or      al, al          ; set condition flags
130         jnz     cpy_loop        ; if not past terminating 0, continue
131
132         pop     edi
133         pop     esi
134         leave
135         ret
```

memory.asm

memex.c

```
1  #include <stdio.h>
2
3  #define STR_SIZE 30
4  /* prototypes */
5
6  void asm_copy( void *, const void *, unsigned ) __attribute__((cdecl));
7  void * asm_find( const void *,
8                   char target , unsigned ) __attribute__((cdecl));
9  unsigned asm_strlen( const char * ) __attribute__((cdecl));
10 void asm_strcpy( char *, const char * ) __attribute__((cdecl));
11
12 int main()
13 {
14   char st1[STR_SIZE] = "test string";
15   char st2[STR_SIZE];
16   char * st;
17   char   ch;
18
19   asm_copy(st2, st1, STR_SIZE); /* copy all 30 chars of string */
20    printf ("%s\n", st2);
21
22    printf ("Enter a char: "); /* look for byte in string */
23   scanf("%c%*[^\n]", &ch);
24   st = asm_find(st2, ch, STR_SIZE);
25   if ( st )
26     printf ("Found it: %s\n", st);
27   else
28     printf ("Not found\n");
29
```

```
30    st1[0] = 0;
31     printf("Enter string:");
32    scanf("%s", st1);
33     printf("len = %u\n", asm_strlen(st1));
34
35    asm_strcpy( st2, st1 );       /* copy meaningful data in string */
36     printf("%s\n", st2 );
37
38    return 0;
39  }
```

memex.c

فصل ۶

ممیز شناور

۱.۶ نمایش ممیز شناور

۱.۱.۶ اعداد دودویی غیرصحیح

در فصل اول، آنجا که دستگاه‌های عددنویسی را بررسی کردیم، تنها به مقادیر صحیح پرداختیم. اما روشن است که اعداد غیرصحیح را نیز باید بتوان در مبناهایی غیر از دهدهی نمایش داد. در دستگاه دهدهی، هر رقمی که در سمت راست ممیز اعشار قرار می‌گیرد با یکی از توان‌های منفی ده متناظر است:

$$0.123 = 1 \times 10^{-1} + 2 \times 10^{-2} + 3 \times 10^{-3}$$

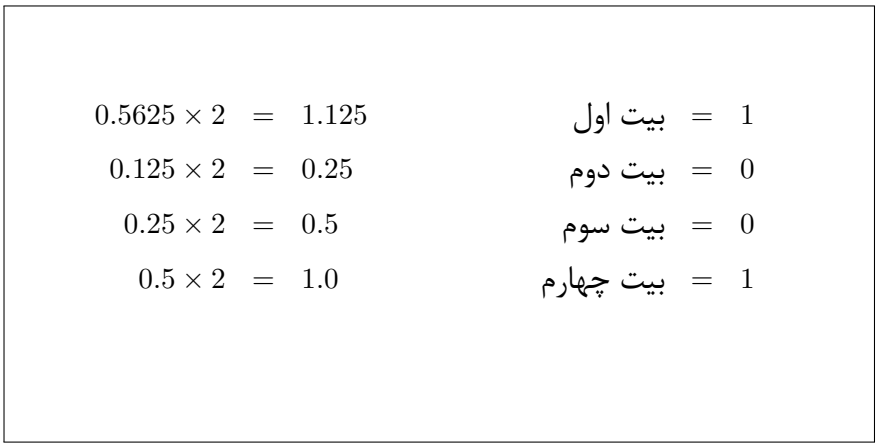
جای شگفتی نیست که اعداد دودویی نیز به همین شیوه عمل می‌کنند:

$$0.101_2 = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 0.625$$

اگر این ایده را با روش‌هایی که در فصل ۱ برای اعداد صحیح دیدیم ترکیب کنیم، می‌توانیم هر عدد دلخواهی را تبدیل کنیم:

$$110.011_2 = 4 + 2 + 0.25 + 0.125 = 6.375$$

تبدیل از دهدهی به دودویی هم چندان دشوار نیست. به طور کلی، عدد دهدهی را به دو بخش صحیح و کسری تقسیم می‌کنیم. آنگاه بخش صحیح را با روش‌های فصل ۱ به دودویی تبدیل می‌کنیم و برای بخش کسری از روشی بهره می‌گیریم که در ادامه شرح داده می‌شود.



شکل ۱.۶: تبدیل 0.5625 به دودویی

کسری دودویی را در نظر بگیرید که بیت‌های آن را به ترتیب با $a, b, c, \ldots$ نام‌گذاری کرده‌ایم. در این صورت، عدد در دستگاه دودویی چنین نوشته می‌شود:

$$0.abcdef\ldots$$

اگر این عدد را در دو ضرب کنیم، نمایش دودویی عدد جدید به این صورت در می‌آید:

$$a.bcdef\ldots$$

توجه کنید که بیت اول اکنون در جایگاه یکان نشسته است. حال اگر به جای a مقدار 0 را قرار دهیم، به عدد زیر می‌رسیم:

$$0.bcdef\ldots$$

و با ضرب دوبارهٔ آن در دو، این عدد به دست می‌آید:

$$b.cdef\ldots$$

اکنون بیت دوم (b) در جایگاه یکان قرار گرفته است. این روند را می‌توان آن قدر تکرار کرد تا به تعداد بیت‌های مورد نیاز دست یابیم. شکل ۱.۶ نمونه‌ای واقعی را نشان می‌دهد که در آن عدد 0.5625 به دودویی تبدیل می‌شود. این روش هنگامی متوقف می‌شود که بخش کسری به صفر برسد.

برای نمونه‌ای دیگر، تبدیل 23.85 به دودویی را در نظر بگیرید. تبدیل بخش صحیح آن ($23 = 10111_2$) ساده است، اما با بخش کسری (0.85) چه باید کرد؟ شکل ۲.۶ آغاز این محاسبه را نشان می‌دهد. اگر با دقت به اعداد نگاه کنیم، به یک حلقهٔ بی‌پایان

$$
\begin{aligned}
0.85 \times 2 &= 1.7 \\
0.7 \times 2 &= 1.4 \\
0.4 \times 2 &= 0.8 \\
0.8 \times 2 &= 1.6 \\
0.6 \times 2 &= 1.2 \\
0.2 \times 2 &= 0.4 \\
0.4 \times 2 &= 0.8 \\
0.8 \times 2 &= 1.6
\end{aligned}
$$

شکل ۲.۶: تبدیل 0.85 به دودویی

برمی‌خوریم! این بدان معناست که 0.85 در دستگاه دودویی عددی متناوب است (همان‌گونه که در مبنای ده از اعشار متناوب سخن می‌گوییم)[۱]. اعداد این محاسبه از الگویی پیروی می‌کنند و با نگاه به این الگو درمی‌یابیم که $0.85 = 0.11\overline{0110}_2$ است. بنابراین $23.85 = 10111.11\overline{0110}_2$ خواهد بود.

یکی از پیامدهای مهم محاسبهٔ بالا این است که 23.85 را نمی‌توان با تعداد محدودی بیت به طور **دقیق** در دستگاه دودویی نمایش داد، درست همان‌طور که $\frac{1}{3}$ را نمی‌توان با تعداد محدودی رقم در دستگاه دهدهی نوشت. چنان‌که در این فصل خواهیم دید، متغیرهای `float` و `double` در زبان C به صورت دودویی ذخیره می‌شوند. از این رو، مقادیری مانند 23.85 را نمی‌توان به طور دقیق در این متغیرها ذخیره کرد و تنها تقریبی از آن‌ها ذخیره می‌شود.

برای ساده‌تر شدن سخت‌افزار، اعداد ممیز شناور[۲] در قالبی یکسان ذخیره می‌شوند. این قالب از نمادگذاری علمی بهره می‌گیرد، با این تفاوت که به جای توان‌های ده، از توان‌های دو در دستگاه دودویی استفاده می‌کند. برای نمونه، 23.85 یا همان

[۱] اینکه عددی در یک مبنا متناوب باشد و در مبنایی دیگر نباشد، نباید چندان شگفت‌آور باشد. عدد $\frac{1}{3}$ را در نظر بگیرید که در دستگاه دهدهی متناوب است، اما در دستگاه سه‌تایی (مبنای ۳) به صورت 0.1_3 نوشته می‌شود.

[2] floating point

$10111.11011001100110\ldots_2$ به این صورت ذخیره می‌شود:

$$1.0111110110011001 10\ldots \times 2^{100}$$

که در آن نما (100) نیز دودویی است. یک عدد ممیز شناور **بهنجار**[۳] شکل کلی زیر را دارد:

$$1.sssssssssssssss \times 2^{eeeeeee}$$

که در آن $1.sssssssssssssss$ **مانتیس**[۴] و $eeeeeeee$ **نما**[۵] نامیده می‌شود.

۲.۱.۶ نمایش ممیز شناور IEEE

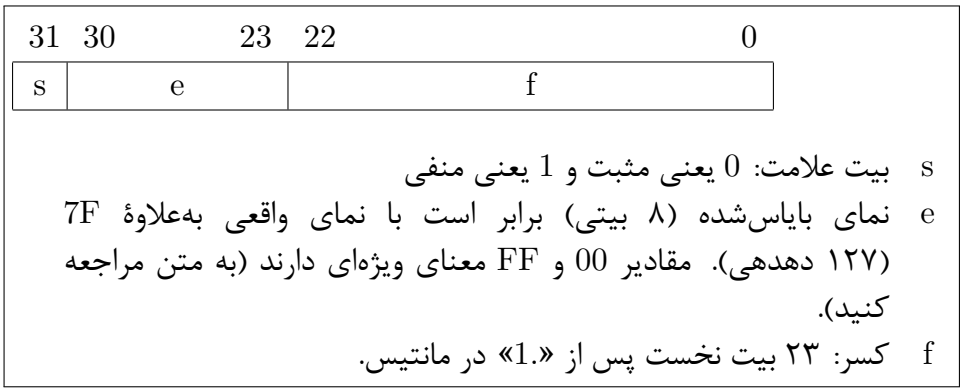
IEEE (مؤسسهٔ مهندسان برق و الکترونیک[۶]) سازمانی بین‌المللی است که برای ذخیره‌سازی اعداد ممیز شناور قالب‌های دودویی مشخصی را طراحی کرده است. بیشتر رایانه‌هایی که امروزه ساخته می‌شوند (البته نه همهٔ آن‌ها!) از این قالب استفاده می‌کنند و در بسیاری از موارد خود سخت‌افزار رایانه از آن پشتیبانی می‌کند. برای نمونه، کمک‌پردازنده‌های ریاضی[۷] اینتل، که از پنتیوم به بعد در همهٔ پردازنده‌های این شرکت تعبیه شده‌اند، همین قالب را به کار می‌برند. IEEE دو قالب با دقت‌های[۸] متفاوت تعریف کرده است: دقت ساده[۹] و دقت مضاعف[۱۰]. در زبان C، متغیرهای `float` از دقت ساده و متغیرهای `double` از دقت مضاعف استفاده می‌کنند.

کمک‌پردازندهٔ ریاضی اینتل دقت سومی را نیز به کار می‌گیرد که از هر دو بالاتر است و **دقت گسترش‌یافته**[۱۱] نام دارد. در واقع، همهٔ داده‌های درون خود کمک‌پردازنده با همین دقت نگهداری می‌شوند و هنگامی که از کمک‌پردازنده در حافظه ذخیره می‌شوند، به طور خودکار به دقت ساده یا مضاعف تبدیل می‌شوند.[۱۲] قالب کلی دقت گسترش‌یافته اندکی با قالب‌های float و double استاندارد IEEE تفاوت دارد و به همین دلیل در اینجا به آن نمی‌پردازیم.

---

[3] normalized
[4] significand
[5] exponent
[6] Institute of Electrical and Electronic Engineers
[7] numeric coprocessor
[8] precision
[9] single precision
[10] double precision
[11] extended precision

[۱۲] نوع `long double` در برخی کامپایلرها (مانند بورلند) از همین دقت گسترش‌یافته استفاده می‌کند. با این حال، کامپایلرهای دیگر برای هر دو نوع `double` و `long double` دقت مضاعف را به کار می‌برند. استاندارد ANSI C این کار را مجاز می‌داند.



s بیت علامت: 0 یعنی مثبت و 1 یعنی منفی
e نمای بایاس‌شده (۸ بیتی) برابر است با نمای واقعی به‌علاوهٔ 7F (۱۲۷ دهدهی). مقادیر 00 و FF معنای ویژه‌ای دارند (به متن مراجعه کنید).
f کسر: ۲۳ بیت نخست پس از «.1» در مانتیس.

شکل ۳.۶: دقت سادهٔ IEEE

دقت سادهٔ IEEE

در قالب ممیز شناور با دقت ساده، هر عدد با ۳۲ بیت کدگذاری می‌شود و معمولاً دقتی در حدود ۷ رقم بامعنای دهدهی دارد. قالب ذخیره‌سازی اعداد ممیز شناور بسیار پیچیده‌تر از قالب اعداد صحیح است. شکل ۳.۶ قالب پایهٔ یک عدد با دقت سادهٔ IEEE را نشان می‌دهد. این قالب چند ویژگی غیرمعمول دارد. نخست آنکه اعداد ممیز شناور برای نمایش اعداد منفی از نمایش مکمل دو[۱۳] استفاده نمی‌کنند، بلکه نمایش علامت و اندازه[۱۴] را به کار می‌برند. همان‌طور که در شکل دیده می‌شود، بیت ۳۱ علامت عدد را تعیین می‌کند.

نمای دودویی نیز مستقیماً ذخیره نمی‌شود. در عوض، حاصل جمع نما و 7F در بیت‌های ۲۳ تا ۳۰ ذخیره می‌شود. این **نمای بایاس‌شده**[۱۵] همواره نامنفی است.

در بخش کسری فرض بر این است که مانتیس بهنجار است (یعنی به شکل $1.ssssssss$ است). از آنجا که بیت نخست همواره یک است، این یکِ ابتدایی **ذخیره نمی‌شود!** این کار امکان ذخیرهٔ یک بیت اضافی را در انتها فراهم می‌کند و بدین ترتیب دقت را اندکی افزایش می‌دهد. این ایده با نام **نمایش یکِ پنهان**[۱۶] شناخته می‌شود.

23.85 چگونه ذخیره می‌شود؟ نخست، این عدد مثبت است، پس بیت علامت آن 0 است. سپس، نمای واقعی برابر ۴ است و در نتیجه نمای بایاس‌شده برابر $7F+4 = 83_{16}$ خواهد بود. سرانجام، کسر برابر 01111101100110011001100 است (به یاد داشته باشید که یکِ ابتدایی پنهان است). اگر همهٔ این‌ها را کنار هم بگذاریم، به عدد زیر

همواره باید به خاطر داشت که بایت‌های 41 BE CC CD بسته به اینکه برنامه با آن‌ها چه کند، به شیوه‌های گوناگونی تفسیر می‌شوند! این بایت‌ها اگر عددی ممیز شناور با دقت ساده در نظر گرفته شوند، 23.850000381 را نشان می‌دهند، اما اگر عدد صحیحی از نوع کلمهٔ مضاعف به شمار آیند، نمایانگر ۱۱۰۳۰۲۳۳۰۹ هستند! پردازنده نمی‌داند کدام تفسیر درست است!

[13] two's complement
[14] signed magnitude
[15] biased exponent
[16] hidden one representation

$e = 0$ و $f = 0$	نشان‌دهندهٔ عدد صفر است (که نمی‌توان آن را بهنجار کرد). توجه کنید که هم صفر مثبت و هم صفر منفی وجود دارد.
$e = 0$ و $f \neq 0$	نشان‌دهندهٔ یک **عدد نابهنجار** است. این اعداد در بخش بعد بررسی می‌شوند.
$e = \text{FF}$ و $f = 0$	نشان‌دهندهٔ بی‌نهایت (∞) است. بی‌نهایت هم مثبت و هم منفی دارد.
$e = \text{FF}$ و $f \neq 0$	نشان‌دهندهٔ نتیجه‌ای تعریف‌نشده است که NaN (مخفف Not a Number به معنای «عدد نیست») نام دارد.

جدول ۱.۶: مقادیر ویژهٔ f و e

می‌رسیم. برای روشن‌تر شدن بخش‌های گوناگون قالب ممیز شناور، زیر بیت علامت و کسر خط کشیده‌ایم و بیت‌ها را در گروه‌های چهاربیتی (نیم‌بایت‌ها[۱۷]) دسته‌بندی کرده‌ایم:

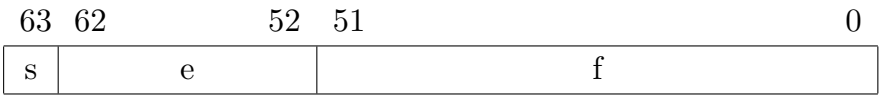
$$\underline{0}\,100\ 0001\ 1\underline{011\ 1110\ 1100\ 1100\ 1100\ 1100}_2 = 41\text{BECCCC}_{16}$$

این عدد دقیقاً 23.85 نیست، زیرا 23.85 در دودویی متناوب است. اگر عدد بالا را دوباره به دهدهی تبدیل کنیم، تقریباً 23.849998474 به دست می‌آید که بسیار به 23.85 نزدیک است، اما با آن برابر نیست. در واقع، زبان C عدد 23.85 را دقیقاً به صورت بالا نمایش نمی‌دهد. چون چپ‌ترین بیتی که از نمایش دقیق حذف شده 1 است، بیت آخر به سمت بالا گرد می‌شود و 1 می‌گردد. بنابراین 23.85 با دقت ساده به صورت شانزده‌شانزدهی 41 BE CC CD نمایش داده می‌شود. تبدیل این مقدار به دهدهی عدد 23.850000381 را به دست می‌دهد که تقریب اندکی بهتری از 23.85 است.

-23.85 چگونه نمایش داده می‌شود؟ کافی است بیت علامت را تغییر دهیم تا به C1 BE CC CD برسیم. مکمل دو را **نگیرید!**

ترکیب‌های خاصی از e و f در اعداد ممیز شناور IEEE معنای ویژه‌ای دارند که جدول ۱.۶ آن‌ها را شرح می‌دهد. بی‌نهایت در اثر سرریز[۱۸] یا تقسیم بر صفر پدید می‌آید. نتیجهٔ تعریف‌نشده نیز از یک عمل نامعتبر حاصل می‌شود، مانند تلاش برای

[17] nibbles
[18] overflow



شکل ۴.۶: دقت مضاعف IEEE

یافتن ریشهٔ دوم یک عدد منفی، جمع دو بی‌نهایت و مانند آن.

اندازهٔ اعداد بهنجار با دقت ساده می‌تواند از 1.0×2^{-126} ($\approx 1.1755 \times 10^{-38}$) تا $1.11111\ldots \times 2^{127}$ ($\approx 3.4028 \times 10^{38}$) باشد.

اعداد نابهنجار

از اعداد نابهنجار[۱۹] می‌توان برای نمایش اعدادی استفاده کرد که اندازهٔ آن‌ها کوچک‌تر از آن است که بتوان بهنجارشان کرد (**یعنی** کمتر از 1.0×2^{-126} هستند). برای نمونه، عدد $1.001_2 \times 2^{-129}$ ($\approx 1.6530 \times 10^{-39}$) را در نظر بگیرید. نمای این عدد در شکل بهنجار آن بیش از حد کوچک است، اما می‌توان آن را به شکل نابهنجار $0.01001_2 \times 2^{-127}$ نمایش داد. برای ذخیرهٔ این عدد، نمای بایاس‌شده برابر 0 قرار داده می‌شود (جدول ۱.۶ را ببینید) و کسر برابر کل مانتیس عدد در حالتی است که به صورت حاصل‌ضربی در 2^{-127} نوشته شده باشد؛ **به بیان دیگر** همهٔ بیت‌ها، از جمله بیتی که در سمت چپ ممیز قرار دارد، ذخیره می‌شوند. بنابراین نمایش 1.001×2^{-129} چنین است:

0 000 0000 0 001 0010 0000 0000 0000 0000

دقت مضاعف IEEE

دقت مضاعف IEEE برای نمایش اعداد از ۶۴ بیت استفاده می‌کند و معمولاً دقتی در حدود ۱۵ رقم بامعنای دهدهی دارد. همان‌طور که شکل ۴.۶ نشان می‌دهد، قالب پایهٔ آن بسیار به دقت ساده شبیه است، با این تفاوت که برای نمای بایاس‌شده (۱۱ بیت) و کسر (۵۲ بیت) بیت‌های بیشتری نسبت به دقت ساده در نظر گرفته شده است.

گستردگی بیشتر بازهٔ نمای بایاس‌شده دو پیامد دارد. نخست آنکه این نما از جمع نمای واقعی با 3FF (۱۰۲۳) به دست می‌آید، نه با 7F که در دقت ساده به کار می‌رفت. دوم آنکه بازهٔ بزرگ‌تری از نماهای واقعی، و در نتیجه بازهٔ بزرگ‌تری از اندازه‌ها، مجاز است. اندازهٔ اعداد با دقت مضاعف می‌تواند تقریباً از 10^{-308} تا 10^{308} باشد.

افزایش تعداد رقم‌های بامعنا در مقادیر double نیز حاصل بزرگ‌تر بودن میدان کسر است.

[19] denormalized

برای نمونه، بار دیگر 23.85 را در نظر بگیرید. نمای بایاس‌شدهٔ آن در مبنای شانزده برابر $403 = 3\text{FF} + 4$ خواهد بود. بنابراین نمایش آن با دقت مضاعف چنین است:

$$0\,100\ 0000\ 0011\ \underline{0111\ 1101\ 1001\ 1001\ 1001\ 1001\ 1001\ 1001\ 1001\ 1001\ 1001\ 1001\ 1001\ 1010}$$

که در مبنای شانزده به صورت 40 37 D9 99 99 99 99 9A نوشته می‌شود. اگر این مقدار را دوباره به دهدهی تبدیل کنیم، به 23.850000000000014 می‌رسیم (که ۱۲ صفر دارد!) و تقریب بسیار بهتری از 23.85 است.

دقت مضاعف همان مقادیر ویژهٔ دقت ساده را دارد[۲۰]. اعداد نابهنجار نیز بسیار شبیه هستند و تنها تفاوت اصلی آن‌ها این است که اعداد نابهنجار با دقت مضاعف به جای 2^{-127} از 2^{-1023} استفاده می‌کنند.

۲.۶ حساب ممیز شناور

حساب ممیز شناور در رایانه با حساب در ریاضیات پیوسته تفاوت دارد. در ریاضیات می‌توان همهٔ اعداد را دقیق در نظر گرفت، اما همان‌طور که در بخش پیش دیدیم، بسیاری از اعداد را در رایانه نمی‌توان با تعداد محدودی بیت به طور دقیق نمایش داد و همهٔ محاسبات با دقتی محدود انجام می‌شوند. برای سادگی، در مثال‌های این بخش از اعدادی با مانتیس ۸ بیتی استفاده می‌کنیم.

۱.۲.۶ جمع

برای جمع دو عدد ممیز شناور، نماهای آن‌ها باید برابر باشند. اگر از پیش برابر نباشند، باید با جابه‌جا کردن[۲۱] مانتیس عددی که نمای کوچک‌تری دارد، آن‌ها را برابر کرد. برای نمونه، $16.71875 = 10.375 + 6.34375$ را در نظر بگیرید که در دودویی به این صورت است:

$$\begin{array}{rl} & 1.0100110 \times 2^3 \\ + & 1.1001011 \times 2^2 \\ \hline \end{array}$$

[۲۰] تنها تفاوت این است که برای بی‌نهایت و مقادیر تعریف‌نشده، نمای بایاس‌شده به جای FF برابر 7FF است.

[21] shifting

نمای این دو عدد یکسان نیست. پس مانتیس را جابه‌جا می‌کنیم تا نماها برابر شوند و سپس جمع را انجام می‌دهیم:

$$\begin{array}{rr} & 1.0100110 \times 2^3 \\ + & 0.1100110 \times 2^3 \\ \hline & 10.0001100 \times 2^3 \end{array}$$

توجه کنید که با جابه‌جا کردن 1.1001011×2^2، یک انتهایی آن حذف می‌شود و پس از گرد کردن به 0.1100110×2^3 می‌رسیم. حاصلِ جمع، یعنی 10.0001100×2^3 (یا 1.00001100×2^4)، برابر با 10000.110_2 یا 16.75 است. این مقدار با پاسخ دقیق (16.71875) برابر **نیست!** بلکه تنها تقریبی از آن است که به سبب خطاهای گرد کردن[۲۲] در فرایند جمع پدید آمده است.

باید دانست که حساب ممیز شناور در رایانه (یا ماشین‌حساب) همواره تقریبی است و قوانین ریاضی همیشه دربارهٔ اعداد ممیز شناور در رایانه برقرار نیستند. ریاضیات دقتی بی‌نهایت را فرض می‌کند که هیچ رایانه‌ای توان رسیدن به آن را ندارد. برای نمونه، ریاضیات به ما می‌آموزد که $(a+b)-b=a$ است، اما ممکن است این رابطه در رایانه به طور دقیق برقرار نباشد!

۲.۲.۶ تفریق

تفریق نیز بسیار شبیه جمع انجام می‌شود و همان مشکلات جمع را دارد. برای نمونه، $16.75 - 15.9375 = 0.8125$ را در نظر بگیرید:

$$\begin{array}{rr} & 1.0000110 \times 2^4 \\ - & 1.1111111 \times 2^3 \\ \hline \end{array}$$

با جابه‌جا کردن 1.1111111×2^3 (و گرد کردن به سمت بالا) به 1.0000000×2^4 می‌رسیم:

$$\begin{array}{rr} & 1.0000110 \times 2^4 \\ - & 1.0000000 \times 2^4 \\ \hline & 0.0000110 \times 2^4 \end{array}$$

حاصل برابر $0.0000110 \times 2^4 = 0.11_2 = 0.75$ است که کاملاً درست نیست.

[22] round off errors

۳.۲.۶ ضرب و تقسیم

در ضرب، مانتیس‌ها در هم ضرب و نماها با هم جمع می‌شوند. برای نمونه، $10.375 \times 2.5 = 25.9375$ را در نظر بگیرید:

$$\begin{array}{rr} & 1.0100110 \times 2^3 \\ \times & 1.0100000 \times 2^1 \\ \hline & 10100110 \\ + & 10100110 \\ \hline & 1.10011111000000 \times 2^4 \end{array}$$

البته نتیجهٔ واقعی به ۸ بیت گرد می‌شود و حاصل چنین خواهد بود:

$$1.1010000 \times 2^4 = 11010.000_2 = 26$$

تقسیم پیچیده‌تر است، اما مشکلات مشابهی از نظر خطاهای گرد کردن دارد.

۴.۲.۶ پیامدها برای برنامه‌نویسی

نکتهٔ اصلی این بخش آن است که محاسبات ممیز شناور دقیق نیستند و برنامه‌نویس باید از این موضوع آگاه باشد. یکی از اشتباهات رایج برنامه‌نویسان در کار با اعداد ممیز شناور این است که آن‌ها را با این فرض مقایسه می‌کنند که محاسبه دقیق بوده است. برای نمونه، فرض کنید تابعی به نام `f(x)` محاسبه‌ای پیچیده انجام می‌دهد و برنامه‌ای می‌کوشد ریشه‌های[۲۳] این تابع را بیابد[۲۴]. ممکن است وسوسه شویم که برای بررسی ریشه بودن `x` از دستور زیر استفاده کنیم:

```
if ( f(x) == 0.0 )
```

اما اگر `f(x)` مقدار 1×10^{-30} را برگرداند چه؟ به احتمال بسیار زیاد این بدان معناست که `x` تقریب **بسیار** خوبی از یک ریشهٔ واقعی است، با این حال نتیجهٔ مقایسهٔ برابری نادرست خواهد بود. حتی ممکن است به سبب خطاهای گرد کردن در `f(x)`، هیچ مقدار ممیز شناور IEEE برای `x` وجود نداشته باشد که دقیقاً صفر را برگرداند.

روش بسیار بهتر، استفاده از دستور زیر است:

```
if ( fabs(f(x)) < EPS )
```

[23] roots

[۲۴] ریشهٔ یک تابع مقداری مانند x است که به ازای آن $f(x) = 0$ باشد.

که در آن EPS ماکرویی[۲۵] است که با یک مقدار مثبت بسیار کوچک (مانند 1×10^{-10}) تعریف شده است. این شرط هر گاه f(x) بسیار به صفر نزدیک باشد، درست است. به طور کلی، برای مقایسهٔ یک مقدار ممیز شناور (مثلاً x) با مقداری دیگر (y) از دستور زیر استفاده کنید:

if (fabs(x − y)/fabs(y) < EPS)

۳.۶ کمک‌پردازندهٔ ریاضی

۱.۳.۶ سخت‌افزار

نخستین پردازنده‌های اینتل هیچ پشتیبانی سخت‌افزاری از عملیات ممیز شناور[۲۶] نداشتند. البته این بدان معنا نیست که آن‌ها از انجام عملیات ممیز شناور ناتوان بودند، بلکه این عملیات باید با رویه‌هایی[۲۷] انجام می‌شد که از شمار زیادی دستورالعمل غیرممیز شناور ساخته شده بودند. اینتل برای این سیستم‌های اولیه تراشهٔ جداگانه‌ای به نام **کمک‌پردازندهٔ ریاضی**[۲۸] عرضه کرد. کمک‌پردازندهٔ ریاضی دستورالعمل‌هایی[۲۹] در سطح ماشین دارد که بسیاری از عملیات ممیز شناور را بسیار سریع‌تر از یک رویهٔ نرم‌افزاری انجام می‌دهند (در پردازنده‌های اولیه دست‌کم ۱۰ برابر سریع‌تر!). کمک‌پردازندهٔ 8086/8088 تراشه‌ای به نام 8087 بود و به همین ترتیب، برای 80286 تراشهٔ 80287 و برای 80386 تراشهٔ 80387 ساخته شد. سرانجام در پردازندهٔ 80486DX کمک‌پردازندهٔ ریاضی درون خود 80486 جای گرفت.[۳۰] از پنتیوم به بعد، همهٔ نسل‌های پردازنده‌های 80x86 کمک‌پردازندهٔ ریاضی داخلی دارند، با این حال این کمک‌پردازنده را هنوز هم به گونه‌ای برنامه‌نویسی می‌کنند که گویی واحدی جداگانه است. حتی روی سیستم‌های قدیمی‌تری که کمک‌پردازنده ندارند نیز می‌توان نرم‌افزاری نصب کرد که کار کمک‌پردازندهٔ ریاضی را شبیه‌سازی می‌کند. این بسته‌های شبیه‌ساز[۳۱] هر گاه برنامه‌ای یکی از دستورالعمل‌های کمک‌پردازنده را اجرا کند، به‌طور خودکار فعال می‌شوند و رویه‌ای نرم‌افزاری را به اجرا درمی‌آورند که همان نتیجهٔ کمک‌پردازنده را تولید می‌کند (هرچند که طبیعتاً بسیار کندتر است).

---

[25] macro

[26] floating point

[27] procedure

[28] math coprocessor

[29] instruction

[۳۰] با این حال، پردازندهٔ 80486SX کمک‌پردازندهٔ یکپارچه **نداشت** و برای این ماشین‌ها تراشهٔ جداگانه‌ای به نام 80487SX عرضه می‌شد.

[31] emulator

کمک‌پردازندهٔ ریاضی هشت ثبات[۳۲] ممیز شناور دارد و هر یک از این ثبات‌ها ۸۰ بیت داده نگه می‌دارد. اعداد ممیز شناور در این ثبات‌ها **همیشه** به صورت اعداد ۸۰ بیتی با دقت گسترش‌یافته[۳۳] ذخیره می‌شوند. نام این ثبات‌ها ST0، ST1، ST2، ... و ST7 است. شیوهٔ به‌کارگیری ثبات‌های ممیز شناور با ثبات‌های صحیح پردازندهٔ اصلی فرق دارد، زیرا این ثبات‌ها به شکل یک **پشته**[۳۴] سازمان‌دهی شده‌اند. به یاد دارید که پشته فهرستی از نوع **آخرین ورودی، نخستین خروجی**[۳۵] است. ST0 همواره به مقدار بالای پشته اشاره دارد. هر عدد تازه به بالای پشته افزوده می‌شود و اعداد موجود یک خانه به پایین پشته رانده می‌شوند تا برای عدد جدید جا باز شود.

کمک‌پردازندهٔ ریاضی یک ثبات وضعیت[۳۶] نیز دارد که چند پرچم[۳۷] را در خود جای داده است. در اینجا تنها چهار پرچمی را بررسی می‌کنیم که در مقایسه‌ها به کار می‌روند، یعنی C_0، C_1، C_2 و C_3، و شیوهٔ استفاده از آن‌ها را بعداً شرح خواهیم داد.

۲.۳.۶ دستورالعمل‌ها

برای اینکه دستورالعمل‌های کمک‌پردازنده به‌آسانی از دستورالعمل‌های معمولی پردازنده بازشناخته شوند، یادیار[۳۸] همهٔ دستورالعمل‌های کمک‌پردازنده با حرف F آغاز می‌شود.

بارگذاری و ذخیره‌سازی

چند دستورالعمل وجود دارد که داده‌ها را در بالای پشتهٔ ثبات‌های کمک‌پردازنده بار می‌کنند:

---

[32] register
[33] extended precision
[34] stack
[35] Last-In First-Out (LIFO)
[36] status register
[37] flag
[38] mnemonic

`FLD` *source* یک عدد ممیز شناور را از حافظه در بالای پشته بار می‌کند. عملگر *source* می‌تواند عددی با دقت ساده، مضاعف یا گسترش‌یافته یا یکی از ثبات‌های کمک‌پردازنده باشد.

`FILD` *source* یک عدد **صحیح** را از حافظه می‌خواند، آن را به ممیز شناور تبدیل می‌کند و نتیجه را در بالای پشته قرار می‌دهد. عملگر *source* می‌تواند یک کلمه، کلمهٔ مضاعف یا کلمهٔ چهارگانه باشد.

`FLD1` عدد یک را در بالای پشته قرار می‌دهد.

`FLDZ` عدد صفر را در بالای پشته قرار می‌دهد.

چند دستورالعمل هم هست که داده‌ها را از پشته در حافظه ذخیره می‌کنند. برخی از این دستورالعمل‌ها عدد را هنگام ذخیره کردن، از پشته **برمی‌دارند**[۳۹] (یعنی آن را از پشته حذف می‌کنند).

[39] pop

FST *dest* بالای پشته (ST0) را در حافظه ذخیره می‌کند. **مقصد** می‌تواند عددی با دقت ساده یا مضاعف یا یکی از ثبات‌های کمک‌پردازنده باشد.

FSTP *dest* مانند FST بالای پشته را در حافظه ذخیره می‌کند، با این تفاوت که پس از ذخیرهٔ عدد، مقدار آن را از پشته برمی‌دارد. **مقصد** می‌تواند عددی با دقت ساده، مضاعف یا گسترش‌یافته یا یکی از ثبات‌های کمک‌پردازنده باشد.

FIST *dest* مقدار بالای پشته را به عدد صحیح تبدیل و در حافظه ذخیره می‌کند. **مقصد** می‌تواند یک کلمه یا کلمهٔ مضاعف باشد و خود پشته تغییری نمی‌کند. چگونگی تبدیل عدد ممیز شناور به عدد صحیح به چند بیت از **کلمهٔ کنترل** (control word) کمک‌پردازنده بستگی دارد. کلمهٔ کنترل ثباتی ویژه (و غیرممیز شناور) به اندازهٔ یک کلمه است که نحوهٔ کار کمک‌پردازنده را تعیین می‌کند. مقدار اولیهٔ پیش‌فرض کلمهٔ کنترل چنان تنظیم شده است که کمک‌پردازنده هنگام تبدیل به عدد صحیح، مقدار را به نزدیک‌ترین عدد صحیح گرد کند. با این حال، می‌توان این رفتار را با دستورالعمل‌های FSTCW (ذخیرهٔ کلمهٔ کنترل (Store Control Word)) و FLDCW (بارگذاری کلمهٔ کنترل (Load Control Word)) تغییر داد.

FISTP *dest* همانند FIST است، جز در دو مورد: بالای پشته برداشته می‌شود و **مقصد** می‌تواند یک کلمهٔ چهارگانه نیز باشد.

دو دستورالعمل دیگر نیز هست که داده‌ها را درون خود پشته جابه‌جا یا حذف می‌کنند.

FXCH STn مقادیر ST0 و STn را در پشته با هم عوض می‌کند (n شمارهٔ ثبات و عددی از ۱ تا ۷ است).

FFREE STn یکی از ثبات‌های پشته را با علامت‌گذاری به عنوان «استفاده‌نشده» یا «خالی» آزاد می‌کند.

```
segment .bss
array        resq SIZE
sum          resq 1

segment .text
      mov    ecx, SIZE
      mov    esi, array
      fldz                  ; ST0 = 0
lp:
      fadd   qword [esi]    ; ST0 += *(esi)
      add    esi, 8         ; move to next double
      loop   lp
      fstp   qword sum      ; store result into sum
```

شکل ۵.۶: مثال جمع عناصر یک آرایه

جمع و تفریق

هر یک از دستورالعمل‌های جمع، مجموع ST0 و یک عملگر[40] دیگر را محاسبه می‌کند و نتیجه همیشه در یکی از ثبات‌های کمک‌پردازنده ذخیره می‌شود.

`FADD` *src* — ST0 += *src*. عملگر *src* می‌تواند هر یک از ثبات‌های کمک‌پردازنده یا عددی با دقت ساده یا مضاعف در حافظه باشد.

`FADD` *dest*, ST0 — *dest* += ST0. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.

`FADDP` *dest* یا `FADDP` *dest*, ST0 — عمل *dest* += ST0 را انجام می‌دهد و سپس یک مقدار از پشته برمی‌دارد. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.

`FIADD` *src* — ST0 += (float) *src*. یک عدد صحیح را با ST0 جمع می‌کند. عملگر *src* باید یک کلمه یا کلمهٔ مضاعف در حافظه باشد.

شمار دستورالعمل‌های تفریق دو برابر دستورالعمل‌های جمع است، زیرا در تفریق ترتیب عملگرها اهمیت دارد (یعنی $a + b = b + a$، اما $a - b \neq b - a$!). از این رو، برای هر دستورالعمل تفریق، دستورالعمل دیگری هم وجود دارد که تفریق را به ترتیب

[40] operand

معکوس انجام می‌دهد و نام همهٔ این دستورالعمل‌های معکوس به R یا RP ختم می‌شود. شکل ۵.۶ قطعه‌کد کوتاهی را نشان می‌دهد که عناصر آرایه‌ای از اعداد double را با هم جمع می‌کند. در سطرهای ۱۰ و ۱۳ باید اندازهٔ عملگر حافظه را صریحاً مشخص کرد، زیرا در غیر این صورت اسمبلر نمی‌داند که عملگر حافظه از نوع float (dword) است یا double (qword).

FSUB *src*	ST0 -= *src*. عملگر *src* می‌تواند هر یک از ثبات‌های کمک‌پردازنده یا عددی با دقت ساده یا مضاعف در حافظه باشد.
FSUBR *src*	ST0 = *src* - ST0. عملگر *src* می‌تواند هر یک از ثبات‌های کمک‌پردازنده یا عددی با دقت ساده یا مضاعف در حافظه باشد.
FSUB *dest*, ST0	*dest* -= ST0. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FSUBR *dest*, ST0	*dest* = ST0 - *dest*. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FSUBP *dest* یا FSUBP *dest*, ST0	عمل *dest* -= ST0 را انجام می‌دهد و سپس یک مقدار از پشته برمی‌دارد. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FSUBRP *dest* یا FSUBRP *dest*, ST0	عمل *dest* = ST0 - *dest* را انجام می‌دهد و سپس یک مقدار از پشته برمی‌دارد. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FISUB *src*	ST0 -= (float) *src*. یک عدد صحیح را از ST0 کم می‌کند. عملگر *src* باید یک کلمه یا کلمهٔ مضاعف در حافظه باشد.
FISUBR *src*	ST0 = (float) *src* - ST0. مقدار ST0 را از یک عدد صحیح کم می‌کند. عملگر *src* باید یک کلمه یا کلمهٔ مضاعف در حافظه باشد.

ضرب و تقسیم

دستورالعمل‌های ضرب کاملاً همانند دستورالعمل‌های جمع هستند.

FMUL *src*	ST0 *= src*. عملگر *src* می‌تواند هر یک از ثبات‌های کمک‌پردازنده یا عددی با دقت ساده یا مضاعف در حافظه باشد.
FMUL *dest*, ST0	*dest* *= ST0. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FMULP *dest* یا FMULP *dest*, ST0	عمل *dest* *= ST0 را انجام می‌دهد و سپس یک مقدار از پشته برمی‌دارد. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FIMUL *src*	ST0 *= (float) *src*. یک عدد صحیح را در ST0 ضرب می‌کند. عملگر *src* باید یک کلمه یا کلمهٔ مضاعف در حافظه باشد.

جای شگفتی نیست که دستورالعمل‌های تقسیم نیز همانند دستورالعمل‌های تفریق باشند. تقسیم بر صفر به بی‌نهایت می‌انجامد.

FDIV *src*	ST0 /= *src*. عملگر *src* می‌تواند هر یک از ثبات‌های کمک‌پردازنده یا عددی با دقت ساده یا مضاعف در حافظه باشد.
FDIVR *src*	ST0 = *src* / ST0. عملگر *src* می‌تواند هر یک از ثبات‌های کمک‌پردازنده یا عددی با دقت ساده یا مضاعف در حافظه باشد.
FDIV *dest*, ST0	*dest* /= ST0. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FDIVR *dest*, ST0	*dest* = ST0 / *dest*. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FDIVP *dest* یا FDIVP *dest*, ST0	عمل *dest* /= ST0 را انجام می‌دهد و سپس یک مقدار از پشته برمی‌دارد. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FDIVRP *dest* یا FDIVRP *dest*, ST0	عمل *dest* = ST0 / *dest* را انجام می‌دهد و سپس یک مقدار از پشته برمی‌دارد. عملگر *dest* می‌تواند هر یک از ثبات‌های کمک‌پردازنده باشد.
FIDIV *src*	ST0 /= (float) *src*. مقدار ST0 را بر یک عدد صحیح تقسیم می‌کند. عملگر *src* باید یک کلمه یا کلمهٔ مضاعف در حافظه باشد.
FIDIVR *src*	ST0 = (float) *src* / ST0. یک عدد صحیح را بر ST0 تقسیم می‌کند. عملگر *src* باید یک کلمه یا کلمهٔ مضاعف در حافظه باشد.

مقایسه

کمک‌پردازنده مقایسهٔ اعداد ممیز شناور را نیز انجام می‌دهد و این کار بر عهدهٔ خانوادهٔ دستورالعمل‌های FCOM است.

```
;     if ( x > y )
;
      fld    qword [x]       ; ST0 = x
      fcomp  qword [y]       ; compare ST0 and y
      fstsw  ax              ; move C bits into FLAGS
      sahf
      jna    else_part       ; if x not above y, goto else_part
then_part:
      ; code for then part
      jmp    end_if
else_part:
      ; code for else part
end_if:
```

شکل ۶.۶: مثال مقایسه

`FCOM` *src* ST0 و *src* را با هم مقایسه می‌کند. عملگر *src* می‌تواند یکی از ثبات‌های کمک‌پردازنده یا یک عدد float یا double در حافظه باشد.

`FCOMP` *src* ST0 و *src* را با هم مقایسه می‌کند و سپس یک مقدار از پشته برمی‌دارد. عملگر *src* می‌تواند یکی از ثبات‌های کمک‌پردازنده یا یک عدد float یا double در حافظه باشد.

`FCOMPP` ST0 و ST1 را با هم مقایسه می‌کند و سپس دو مقدار از پشته برمی‌دارد.

`FICOM` *src* ST0 و *src* (float) را با هم مقایسه می‌کند. عملگر *src* می‌تواند یک عدد صحیح به اندازهٔ کلمه یا کلمهٔ مضاعف در حافظه باشد.

`FICOMP` *src* ST0 و *src*(float) را با هم مقایسه می‌کند و سپس یک مقدار از پشته برمی‌دارد. عملگر *src* می‌تواند یک عدد صحیح به اندازهٔ کلمه یا کلمهٔ مضاعف در حافظه باشد.

`FTST` ST0 را با ۰ مقایسه می‌کند.

این دستورالعمل‌ها بیت‌های C_0، C_1، C_2 و C_3 ثبات وضعیت کمک‌پردازنده را تغییر می‌دهند. متأسفانه پردازنده نمی‌تواند مستقیماً به این بیت‌ها دسترسی داشته باشد، چون دستورالعمل‌های انشعاب[۴۱] شرطی از ثبات FLAGS استفاده می‌کنند، نه

[41] branch

از ثبات وضعیت کمک‌پردازنده. با این حال، به کمک چند دستورالعمل جدید می‌توان بیت‌های کلمهٔ وضعیت را نسبتاً به‌سادگی به بیت‌های متناظر در ثبات FLAGS منتقل کرد:

`FSTSW` *dest* کلمهٔ وضعیت کمک‌پردازنده را در یک کلمه از حافظه یا در ثبات AX ذخیره می‌کند.

`SAHF` محتوای ثبات AH را در ثبات FLAGS ذخیره می‌کند.

`LAHF` بیت‌های ثبات FLAGS را در ثبات AH بار می‌کند.

شکل ۶.۶ قطعه‌کد کوتاهی را به عنوان نمونه نشان می‌دهد. سطرهای ۵ و ۶ بیت‌های C_0، C_1، C_2 و C_3 کلمهٔ وضعیت کمک‌پردازنده را به ثبات FLAGS منتقل می‌کنند. این بیت‌ها چنان منتقل می‌شوند که با نتیجهٔ مقایسهٔ دو عدد صحیح **بی‌علامت** همانند باشند و به همین دلیل است که در سطر ۷ از دستورالعمل `JNA` استفاده شده است.

پنتیوم پرو (و پردازنده‌های پس از آن، یعنی پنتیوم II و III) از دو اپراتور مقایسهٔ جدید پشتیبانی می‌کنند که ثبات FLAGS پردازنده را مستقیماً تغییر می‌دهند.

`FCOMI` *src* `ST0` و *src* را با هم مقایسه می‌کند. عملگر *src* باید یکی از ثبات‌های کمک‌پردازنده باشد.

`FCOMIP` *src* `ST0` و *src* را با هم مقایسه می‌کند و سپس یک مقدار از پشته برمی‌دارد. عملگر *src* باید یکی از ثبات‌های کمک‌پردازنده باشد.

شکل ۷.۶ زیرروالی[۴۲] نمونه را نشان می‌دهد که با دستورالعمل `FCOMIP` بزرگ‌ترین عدد را از میان دو عدد double پیدا می‌کند. این دستورالعمل‌ها را با توابع مقایسهٔ اعداد صحیح (`FICOM` و `FICOMP`) اشتباه نگیرید.

دستورالعمل‌های متفرقه

در این بخش به چند دستورالعمل متفرقهٔ دیگر می‌پردازیم که کمک‌پردازنده در اختیار برنامه‌نویس می‌گذارد.

`FCHS` `ST0 = - ST0`. علامت `ST0` را عوض می‌کند.

`FABS` $\texttt{ST0} = |\texttt{ST0}|$. قدر مطلق `ST0` را محاسبه می‌کند.

`FSQRT` $\texttt{ST0} = \sqrt{\texttt{ST0}}$. جذر `ST0` را محاسبه می‌کند.

`FSCALE` $\texttt{ST0} = \texttt{ST0} \times 2^{\lfloor \texttt{ST1} \rfloor}$. `ST0` را به‌سرعت در توانی از ۲ ضرب می‌کند، بی‌آنکه `ST1` از پشتهٔ کمک‌پردازنده حذف شود. شکل ۸.۶ نمونه‌ای از کاربرد این دستورالعمل را نشان می‌دهد.

[42] subroutine

```
1  global _dmax
2
3  segment .text
4  ; function _dmax
5  ; returns the larger of its two double arguments
6  ; C prototype
7  ; double dmax( double d1, double d2 )
8  ; Parameters:
9  ;   d1   - first double
10 ;   d2   - second double
11 ; Return value:
12 ;   larger of d1 and d2 (in ST0)
13 %define d1   ebp+8
14 %define d2   ebp+16
15 _dmax:
16         enter   0, 0
17
18         fld     qword [d2]
19         fld     qword [d1]          ; ST0 = d1, ST1 = d2
20         fcomip  st1                 ; ST0 = d2
21         jna     short d2_bigger
22         fcomp   st0                 ; pop d2 from stack
23         fld     qword [d1]          ; ST0 = d1
24         jmp     short exit
25 d2_bigger:                          ; if d2 is max, nothing to do
26 exit:
27         leave
28         ret
```

شکل ۷.۶: مثال FCOMIP

```
1 segment .data
2 x             dq  2.75          ; converted to double format
3 five          dw  5
4
5 segment .text
6       fild    dword [five]      ; ST0 = 5
7       fld     qword [x]         ; ST0 = 2.75, ST1 = 5
8       fscale                    ; ST0 = 2.75 * 32, ST1 = 5
```

شکل ۸.۶: مثال FSCALE

۳.۳.۶ مثال‌ها

۴.۳.۶ فرمول معادلهٔ درجهٔ دوم

نخستین مثال نشان می‌دهد که فرمول معادلهٔ درجهٔ دوم[۴۳] را چگونه می‌توان به زبان اسمبلی پیاده‌سازی کرد. به یاد دارید که این فرمول جواب‌های معادلهٔ درجهٔ دوم زیر را به دست می‌دهد:

$$ax^2 + bx + c = 0$$

خود فرمول دو جواب x_1 و x_2 را برای x به دست می‌دهد:

$$x_1, x_2 = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

عبارت زیر رادیکال ($b^2 - 4ac$) **مبین**[۴۴] نام دارد و مقدار آن به ما کمک می‌کند تا دریابیم کدام‌یک از سه حالت زیر برای جواب‌ها برقرار است.

۱. اگر $b^2 - 4ac = 0$ باشد، تنها یک جواب حقیقی تکراری[۴۵] وجود دارد.

۲. اگر $b^2 - 4ac > 0$ باشد، دو جواب حقیقی وجود دارد.

۳. اگر $b^2 - 4ac < 0$ باشد، دو جواب مختلط وجود دارد.

در ادامه برنامهٔ کوچکی به زبان C آمده است که از این زیرروال اسمبلی استفاده می‌کند:

[43] quadratic formula
[44] discriminant
[45] degenerate solution

quadt.c

```
#include <stdio.h>

int quadratic( double, double, double, double *, double *);

int main()
{
  double a,b,c, root1, root2;

   printf ("Enter a, b, c: ");
  scanf("%lf %lf %lf", &a, &b, &c);
  if (quadratic( a, b, c, &root1, &root2) )
     printf ("roots: %.10g %.10g\n", root1, root2);
  else
     printf ("No real roots\n");
  return 0;
}
```

quadt.c

روال اسمبلی نیز به این صورت است:

quad.asm

```
; function quadratic
; finds solutions to the quadratic equation:
;       a*x^2 + b*x + c = 0
; C prototype:
;   int quadratic( double a, double b, double c,
;                  double * root1, double *root2 )
; Parameters:
;   a, b, c - coefficients of powers of quadratic equation (see above)
;   root1   - pointer to double to store first root in
;   root2   - pointer to double to store second root in
; Return value:
;   returns 1 if real roots found, else 0

%define a               qword [ebp+8]
%define b               qword [ebp+16]
%define c               qword [ebp+24]
%define root1           dword [ebp+32]
%define root2           dword [ebp+36]
```

```
19 %define disc            qword [ebp-8]
20 %define one_over_2a     qword [ebp-16]
21
22 segment .data
23 MinusFour       dw      -4
24
25 segment .text
26         global  _quadratic
27 _quadratic:
28         push    ebp
29         mov     ebp, esp
30         sub     esp, 16         ; allocate 2 doubles (disc & one_over_2a)
31         push    ebx             ; must save original ebx
32
33         fild    word [MinusFour]; stack -4
34         fld     a               ; stack: a, -4
35         fld     c               ; stack: c, a, -4
36         fmulp   st1             ; stack: a*c, -4
37         fmulp   st1             ; stack: -4*a*c
38         fld     b
39         fld     b               ; stack: b, b, -4*a*c
40         fmulp   st1             ; stack: b*b, -4*a*c
41         faddp   st1             ; stack: b*b - 4*a*c
42         ftst                    ; test with 0
43         fstsw   ax
44         sahf
45         jb      no_real_solutions ; if disc < 0, no real solutions
46         fsqrt                   ; stack: sqrt(b*b - 4*a*c)
47         fstp    disc            ; store and pop stack
48         fld1                    ; stack: 1.0
49         fld     a               ; stack: a, 1.0
50         fscale                  ; stack: a * 2^(1.0) = 2*a, 1
51         fdivp   st1             ; stack: 1/(2*a)
52         fst     one_over_2a     ; stack: 1/(2*a)
53         fld     b               ; stack: b, 1/(2*a)
54         fld     disc            ; stack: disc, b, 1/(2*a)
55         fsubrp  st1             ; stack: disc - b, 1/(2*a)
56         fmulp   st1             ; stack: (-b + disc)/(2*a)
57         mov     ebx, root1
58         fstp    qword [ebx]     ; store in *root1
59         fld     b               ; stack: b
60         fld     disc            ; stack: disc, b
```

```
61          fchs                    ; stack: -disc, b
62          fsubrp  st1             ; stack: -disc - b
63          fmul    one_over_2a     ; stack: (-b - disc)/(2*a)
64          mov     ebx, root2
65          fstp    qword [ebx]     ; store in *root2
66          mov     eax, 1          ; return value is 1
67          jmp     short quit
68
69  no_real_solutions:
70          mov     eax, 0          ; return value is 0
71
72  quit:
73          pop     ebx
74          mov     esp, ebp
75          pop     ebp
76          ret
```

quad.asm

۵.۳.۶ خواندن آرایه از فایل

در این مثال، یک روال اسمبلی اعداد double را از یک فایل می‌خواند. برنامهٔ آزمون کوتاهی به زبان C برای آن در ادامه آمده است:

readt.c

```
1  /*
2   * This program tests the 32-bit read_doubles() assembly procedure.
3   * It reads the doubles from stdin. (Use redirection to read from file.)
4   */
5  #include <stdio.h>
6  extern int read_doubles( FILE *, double *, int );
7  #define MAX 100
8
9  int main()
10 {
11   int i,n;
12   double a[MAX];
13
14   n = read_doubles(stdin, a, MAX);
15
16   for( i=0; i < n; i++ )
17     printf ("%3d %g\n", i, a[i]);
18   return 0;
```

```
19 }
```

readt.c

روال اسمبلی نیز به این صورت است:

read.asm

```
1  segment .data
2  format  db      "%lf", 0        ; format for fscanf()
3
4  segment .text
5          global  _read_doubles
6          extern  _fscanf
7
8  %define SIZEOF_DOUBLE   8
9  %define FP              dword [ebp + 8]
10 %define ARRAYP          dword [ebp + 12]
11 %define ARRAY_SIZE      dword [ebp + 16]
12 %define TEMP_DOUBLE     [ebp - 8]
13
14 ;
15 ; function _read_doubles
16 ; C prototype:
17 ;   int read_doubles( FILE * fp, double * arrayp, int array_size );
18 ; This function reads doubles from a text file into an array, until
19 ; EOF or array is full.
20 ; Parameters:
21 ;   fp         - FILE pointer to read from (must be open for input)
22 ;   arrayp     - pointer to double array to read into
23 ;   array_size - number of elements in array
24 ; Return value:
25 ;   number of doubles stored into array (in EAX)
26
27 _read_doubles:
28         push    ebp
29         mov     ebp,esp
30         sub     esp, SIZEOF_DOUBLE      ; define one double on stack
31
32         push    esi                     ; save esi
33         mov     esi, ARRAYP             ; esi = ARRAYP
34         xor     edx, edx                ; edx = array index (initially 0)
35
```

```
36 while_loop:
37         cmp     edx, ARRAY_SIZE         ; is edx < ARRAY_SIZE?
38         jnl     short quit              ; if not, quit loop
39 ;
40 ; call fscanf() to read a double into TEMP_DOUBLE
41 ; fscanf() might change edx so save it
42 ;
43         push    edx                     ; save edx
44         lea     eax, TEMP_DOUBLE
45         push    eax                     ; push &TEMP_DOUBLE
46         push    dword format            ; push &format
47         push    FP                      ; push file pointer
48         call    _fscanf
49         add     esp, 12
50         pop     edx                     ; restore edx
51         cmp     eax, 1                  ; did fscanf return 1?
52         jne     short quit              ; if not, quit loop
53
54 ;
55 ; copy TEMP_DOUBLE into ARRAYP[edx]
56 ; (The 8-bytes of the double are copied by two 4-byte copies)
57 ;
58         mov     eax, [ebp - 8]
59         mov     [esi + 8*edx], eax      ; first copy lowest 4 bytes
60         mov     eax, [ebp - 4]
61         mov     [esi + 8*edx + 4], eax  ; next copy highest 4 bytes
62
63         inc     edx
64         jmp     while_loop
65
66 quit:
67         pop     esi                     ; restore esi
68
69         mov     eax, edx                ; store return value into eax
70
71         mov     esp, ebp
72         pop     ebp
73         ret
```

read.asm

۶.۳.۶ یافتن اعداد اول

این مثال پایانی بار دیگر به یافتن اعداد اول[۴۶] می‌پردازد. پیاده‌سازی تازه از پیاده‌سازی پیشین کارآمدتر است، زیرا اعداد اولی را که پیدا می‌کند در یک آرایه نگه می‌دارد و برای یافتن اعداد اول جدید، به‌جای تقسیم بر همهٔ اعداد فرد، تنها بر اعداد اولی که پیش‌تر یافته است تقسیم می‌کند.

تفاوت دیگر این است که برنامه جذر عددی را که به عنوان حدس بعدی برای عدد اول برگزیده است محاسبه می‌کند تا دریابد جست‌وجوی عامل‌ها را در کجا می‌تواند متوقف کند. برای این کار، کلمهٔ کنترل کمک‌پردازنده را چنان تغییر می‌دهد که هنگام ذخیرهٔ جذر به صورت عدد صحیح، به‌جای گرد کردن، بخش اعشاری را کنار بگذارد. این رفتار را بیت‌های ۱۰ و ۱۱ کلمهٔ کنترل تعیین می‌کنند که بیت‌های RC (کنترل گرد کردن[۴۷]) نام دارند. اگر هر دو بیت ۰ باشند (حالت پیش‌فرض)، کمک‌پردازنده هنگام تبدیل به عدد صحیح عدد را گرد می‌کند و اگر هر دو ۱ باشند، در تبدیل به عدد صحیح بخش اعشاری را قطع[۴۸] می‌کند. توجه کنید که این روال با دقت، کلمهٔ کنترل اصلی را ذخیره می‌کند و پیش از بازگشت، آن را به حالت نخست برمی‌گرداند.

برنامهٔ راه‌انداز[۴۹] نوشته‌شده به زبان C در ادامه آمده است:

fprime.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  /*
4   * function find_primes
5   * finds the indicated number of primes
6   * Parameters:
7   *    a — array to hold primes
8   *    n — how many primes to find
9   */
10 extern void find_primes( int * a, unsigned n );
11
12 int main()
13 {
14   int status;
15   unsigned i;
```

[46] prime number
[47] Rounding Control
[48] truncate
[49] driver

```
16    unsigned max;
17    int * a;
18
19     printf ("How many primes do you wish to find? ");
20    scanf("%u", &max);
21
22    a = calloc( sizeof(int), max);
23
24    if ( a ) {
25
26      find_primes(a,max);
27
28      /* print out the last 20 primes found */
29      for(i= ( max > 20 ) ? max - 20 : 0; i < max; i++ )
30         printf ("%3d %d\n", i+1, a[i]);
31
32      free(a);
33      status = 0;
34    }
35    else {
36       fprintf ( stderr , "Can not create array of %u ints\n", max);
37      status = 1;
38    }
39
40    return status;
41  }
```

fprime.c

روال اسمبلی نیز به این صورت است:

prime2.asm

```
1  segment .text
2          global  _find_primes
3  ;
4  ; function find_primes
5  ; finds the indicated number of primes
6  ; Parameters:
7  ;   array  - array to hold primes
8  ;   n_find - how many primes to find
9  ; C Prototype:
10 ;extern void find_primes( int * array, unsigned n_find )
```

```
11  ;
12  %define array         ebp + 8
13  %define n_find        ebp + 12
14  %define n             ebp - 4          ; number of primes found so far
15  %define isqrt         ebp - 8          ; floor of sqrt of guess
16  %define orig_cntl_wd  ebp - 10         ; original control word
17  %define new_cntl_wd   ebp - 12         ; new control word
18
19  _find_primes:
20          enter   12,0                   ; make room for local variables
21
22          push    ebx                    ; save possible register variables
23          push    esi
24
25          fstcw   word [orig_cntl_wd]    ; get current control word
26          mov     ax, [orig_cntl_wd]
27          or      ax, 0C00h              ; set rounding bits to 11 (truncate)
28          mov     [new_cntl_wd], ax
29          fldcw   word [new_cntl_wd]
30
31          mov     esi, [array]           ; esi points to array
32          mov     dword [esi], 2         ; array[0] = 2
33          mov     dword [esi + 4], 3     ; array[1] = 3
34          mov     ebx, 5                 ; ebx = guess = 5
35          mov     dword [n], 2           ; n = 2
36  ;
37  ; This outer loop finds a new prime each iteration, which it adds to the
38  ; end of the array. Unlike the earlier prime finding program, this function
39  ; does not determine primeness by dividing by all odd numbers. It only
40  ; divides by the prime numbers that it has already found. (That's why they
41  ; are stored in the array.)
42  ;
43  while_limit:
44          mov     eax, [n]
45          cmp     eax, [n_find]          ; while ( n < n_find )
46          jnb     short quit_limit
47
48          mov     ecx, 1                 ; ecx is used as array index
49          push    ebx                    ; store guess on stack
50          fild    dword [esp]            ; load guess onto coprocessor stack
51          pop     ebx                    ; get guess off stack
52          fsqrt                          ; find sqrt(guess)
```

```
53          fistp   dword [isqrt]           ; isqrt = floor(sqrt(quess))
54  ;
55  ; This inner loop divides guess (ebx) by earlier computed prime numbers
56  ; until it finds a prime factor of guess (which means guess is not prime)
57  ; or until the prime number to divide is greater than floor(sqrt(guess))
58  ;
59  while_factor:
60          mov     eax, dword [esi + 4*ecx]        ; eax = array[ecx]
61          cmp     eax, [isqrt]                    ; while ( isqrt < array[ecx]
62          jnbe    short quit_factor_prime
63          mov     eax, ebx
64          xor     edx, edx
65          div     dword [esi + 4*ecx]
66          or      edx, edx                        ; && guess % array[ecx] != 0 )
67          jz      short quit_factor_not_prime
68          inc     ecx                             ; try next prime
69          jmp     short while_factor
70
71  ;
72  ; found a new prime !
73  ;
74  quit_factor_prime:
75          mov     eax, [n]
76          mov     dword [esi + 4*eax], ebx        ; add guess to end of array
77          inc     eax
78          mov     [n], eax                        ; inc n
79
80  quit_factor_not_prime:
81          add     ebx, 2                          ; try next odd number
82          jmp     short while_limit
83
84  quit_limit:
85
86          fldcw   word [orig_cntl_wd]             ; restore control word
87          pop     esi                             ; restore register variables
88          pop     ebx
89
90          leave
91          ret
```

prime2.asm

فصل ۷

ساختارها و C++

۱.۷ ساختارها

۱.۱.۷ مقدمه

در زبان C از ساختارها[1] برای گردآوردن داده‌های مرتبط در قالب یک متغیر مرکب استفاده می‌شود. این شیوه چند مزیت دارد:

۱. کد را روشن‌تر می‌کند، زیرا نشان می‌دهد که داده‌های تعریف‌شده در ساختار پیوندی نزدیک با یکدیگر دارند.

۲. ارسال داده‌ها به توابع را ساده‌تر می‌کند؛ به جای آنکه چندین متغیر را جداگانه بفرستیم، می‌توانیم همهٔ آن‌ها را یکجا و در قالب یک واحد ارسال کنیم.

۳. **محلیت**[2] کد را افزایش می‌دهد.[3]

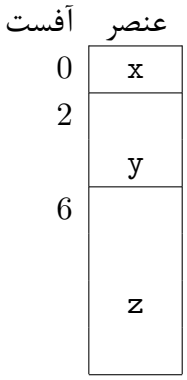
از دیدگاه اسمبلی، ساختار را می‌توان آرایه‌ای در نظر گرفت که عناصر آن اندازه‌های **متفاوتی** دارند. در آرایه‌های واقعی، عناصر همواره هم‌اندازه و هم‌نوع هستند و درست همین ویژگی است که به ما امکان می‌دهد با دانستن آدرس شروع آرایه، اندازهٔ عناصر و اندیس عنصر مورد نظر، آدرس هر عنصری را محاسبه کنیم.

---

[1] structure

[2] locality

[3] برای آشنایی با این اصطلاح، به بخش مدیریت حافظهٔ مجازی در هر کتاب درسی سیستم‌عامل مراجعه کنید.



شکل ۱.۷: ساختار S

اما عناصر یک ساختار لزومی ندارد هم‌اندازه باشند (و معمولاً هم نیستند). به همین دلیل، هر عنصر ساختار باید به‌صراحت مشخص شود و به جای اندیس عددی، یک **برچسب**[۴] (یا نام) به آن داده می‌شود.

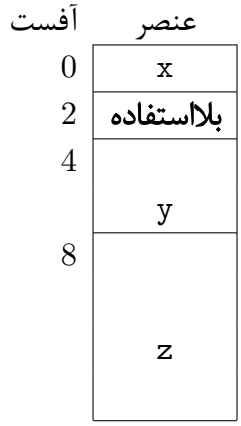
در اسمبلی، دسترسی به عنصر یک ساختار شبیه دسترسی به عنصر یک آرایه است؛ یعنی برای دسترسی به هر عنصر باید آدرس شروع ساختار و **آفست نسبی**[۵] آن عنصر از ابتدای ساختار را بدانیم. تفاوت در این است که در آرایه این آفست از روی اندیس عنصر محاسبه می‌شود، در حالی که آفست هر عنصر ساختار را کامپایلر تعیین می‌کند.

برای نمونه، ساختار زیر را در نظر بگیرید:

```
struct S {
  short int x;    /* 2-byte integer */
  int       y;    /* 4-byte integer */
  double    z;    /* 8-byte float   */
};
```

شکل ۱.۷ نشان می‌دهد که متغیری از نوع S ممکن است در حافظهٔ رایانه چگونه قرار گیرد. بنا بر استاندارد ANSI C، عناصر ساختار به همان ترتیبی در حافظه چیده می‌شوند که در تعریف `struct` آمده‌اند. این استاندارد همچنین تصریح می‌کند که نخستین عنصر درست در ابتدای ساختار (**یعنی** در آفست صفر) قرار دارد. افزون بر این، ماکروی سودمند دیگری به نام `()offsetof` را در فایل سرآیند[۶] `stddef.h` تعریف می‌کند که آفست هر عنصر دلخواه از یک ساختار را محاسبه می‌کند و برمی‌گرداند. این

[4] tag
[5] relative offset
[6] header file



شکل ۲.۷: ساختار S

ماکرو دو پارامتر می‌گیرد: اولی نام **نوع** ساختار است و دومی نام عنصری که می‌خواهیم آفست آن را بیابیم. بنابراین با توجه به شکل ۱.۷، حاصل `offsetof(S, y)` برابر ۲ خواهد بود.

۲.۱.۷ هم‌ترازی حافظه

به یاد دارید که آدرسی روی مرز کلمهٔ مضاعف قرار دارد که بر ۴ بخش‌پذیر باشد.

اگر با کامپایلر gcc و به کمک ماکروی `offsetof` آفست y را بیابید، خواهید دید که مقدار ۴ را برمی‌گرداند، نه ۲! چرا؟ زیرا gcc (و بسیاری از کامپایلرهای دیگر) به طور پیش‌فرض متغیرها را روی مرز کلمهٔ مضاعف[7] هم‌تراز[8] می‌کند؛ چون در حالت حفاظت‌شدهٔ ۳۲ بیتی، اگر داده از مرز کلمهٔ مضاعف شروع شود، پردازنده آن را سریع‌تر از حافظه می‌خواند. شکل ۲.۷ نشان می‌دهد که ساختار S با gcc در واقع چه شکلی دارد: کامپایلر دو بایت بلااستفاده در ساختار می‌گنجاند تا y (و z) روی مرز کلمهٔ مضاعف هم‌تراز شوند. همین نکته روشن می‌کند که چرا هنگام کار با ساختارهای تعریف‌شده در C، بهتر است آفست‌ها را به جای محاسبهٔ دستی، با `offsetof` به دست آوریم.

البته اگر ساختار فقط در اسمبلی به کار رود، برنامه‌نویس می‌تواند آفست‌ها را خود تعیین کند. اما وقتی قرار است C و اسمبلی با هم کار کنند، بسیار مهم است که کد اسمبلی و کد C بر سر آفست عناصر ساختار توافق داشته باشند! یکی از دشواری‌های کار این است که کامپایلرهای مختلف C ممکن است به عناصر آفست‌های متفاوتی بدهند. برای نمونه، همان‌طور که دیدیم، کامپایلر gcc ساختار S را مانند شکل ۲.۷

[7] double word boundary
[8] align

```
struct S {
  short int x;     /* 2-byte integer */
  int       y;     /* 4-byte integer */
  double    z;     /* 8-byte float   */
} __attribute__((packed));
```

شکل ۳.۷: ساختار فشرده با gcc

می‌سازد، در حالی که کامپایلر بورلند آن را مانند شکل ۱.۷ ایجاد می‌کند. کامپایلرهای C راه‌هایی برای تعیین هم‌ترازی داده‌ها فراهم کرده‌اند؛ اما استاندارد ANSI C شیوهٔ این کار را مشخص نکرده است و از این رو هر کامپایلر آن را به روش خود انجام می‌دهد.

کامپایلر gcc روشی انعطاف‌پذیر و البته پیچیده برای تعیین هم‌ترازی دارد و اجازه می‌دهد با نحو ویژه‌ای، هم‌ترازی هر نوع دلخواهی را مشخص کنیم. برای نمونه، خط زیر:

```
typedef short int unaligned_int __attribute__((aligned(1)));
```

نوع جدیدی به نام `unaligned_int` تعریف می‌کند که روی مرز بایت هم‌تراز می‌شود. (بله، همهٔ این پرانتزها پس از `__attribute__` لازم هستند!) عدد ۱ در پارامتر `aligned` را می‌توان با توان‌های دیگری از ۲ جایگزین کرد تا هم‌ترازی‌های دیگری به دست آید (۲ برای هم‌ترازی روی مرز کلمه، ۴ برای هم‌ترازی روی مرز کلمهٔ مضاعف و **الی آخر**). اگر نوع عنصر `y` در ساختار به `unaligned_int` تغییر کند، gcc آن را در آفست ۲ قرار می‌دهد؛ با این حال `z` همچنان در آفست ۸ می‌ماند، زیرا نوع `double` نیز به طور پیش‌فرض روی مرز کلمهٔ مضاعف هم‌تراز می‌شود. برای آنکه `z` در آفست ۶ قرار گیرد، باید تعریف نوع آن را هم تغییر داد.

کامپایلر gcc همچنین امکان **فشرده‌سازی**[9] ساختار را فراهم می‌کند؛ یعنی می‌توان به کامپایلر گفت که برای ساختار کمترین فضای ممکن را به کار ببرد. شکل ۳.۷ نشان می‌دهد که S را چگونه می‌توان به این صورت بازنویسی کرد. این شکل از S کمترین تعداد بایت ممکن، یعنی ۱۴ بایت، را اشغال می‌کند.

کامپایلرهای مایکروسافت و بورلند هر دو از روش یکسانی برای تعیین هم‌ترازی پشتیبانی می‌کنند که بر رهنمود `#pragma` استوار است.

```
#pragma pack(1)
```

رهنمود بالا به کامپایلر می‌گوید که عناصر ساختارها را روی مرز بایت فشرده کند

[9] pack

```
#pragma pack(push) /* save alignment state */
#pragma pack(1)    /* set byte alignment   */

struct S {
  short int x;     /* 2-byte integer */
  int       y;     /* 4-byte integer */
  double    z;     /* 8-byte float   */
};

#pragma pack(pop)  /* restore original alignment */
```

شکل ۴.۷: ساختار فشرده با کامپایلر مایکروسافت یا بورلند

(**یعنی** بدون هیچ لایه‌گذاری[10] اضافی). به جای عدد یک می‌توان دو، چهار، هشت یا شانزده گذاشت تا هم‌ترازی به ترتیب روی مرزهای کلمه، کلمهٔ مضاعف، کلمهٔ چهارگانه و پاراگراف انجام شود. این رهنمود تا زمانی که رهنمود دیگری آن را لغو نکند، برقرار می‌ماند و همین می‌تواند دردسرساز شود، زیرا چنین رهنمودهایی اغلب در فایل‌های سرآیند به کار می‌روند. اگر چنین فایل سرآیندی پیش از فایل‌های سرآیند دیگری که ساختار دارند گنجانده شود، ممکن است آن ساختارها به شکلی متفاوت با حالت پیش‌فرض چیده شوند. نتیجه می‌تواند خطایی باشد که یافتن آن بسیار دشوار است: ماژول‌های مختلف یک برنامه ممکن است عناصر ساختارها را در جاهای **متفاوتی** قرار دهند!

خوشبختانه برای پرهیز از این مشکل راهی وجود دارد. مایکروسافت و بورلند امکانی فراهم کرده‌اند که با آن می‌توان وضعیت فعلی هم‌ترازی را ذخیره کرد و بعداً آن را بازگرداند. شکل ۴.۷ نشان می‌دهد که این کار چگونه انجام می‌شود.

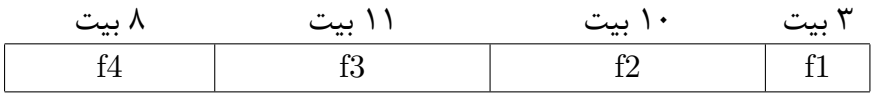
۳.۱.۷ میدان‌های بیتی

میدان‌های بیتی[11] این امکان را فراهم می‌کنند که اعضایی از یک `struct` تعریف کنیم که تنها تعداد مشخصی بیت را به کار می‌گیرند و این تعداد لازم نیست مضربی از هشت باشد. عضو میدان بیتی مانند یک عضو `unsigned int` یا `int` تعریف می‌شود، با این تفاوت که یک دونقطه و اندازهٔ میدان بر حسب بیت به دنبال آن می‌آید. شکل ۵.۷ نمونه‌ای از این تعریف را نشان می‌دهد که یک متغیر ۳۲ بیتی می‌سازد و آن را به بخش‌های زیر تقسیم می‌کند:

[10] padding
[11] bit field

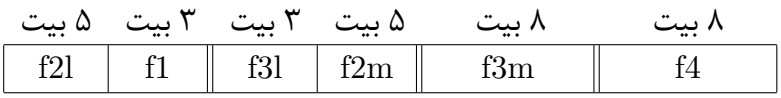
```
struct S {
  unsigned f1 : 3;   /* 3-bit field  */
  unsigned f2 : 10;  /* 10-bit field */
  unsigned f3 : 11;  /* 11-bit field */
  unsigned f4 : 8;   /* 8-bit field  */
};
```

شکل ۵.۷: نمونه‌ای از میدان بیتی



نخستین میدان بیتی به کم‌ارزش‌ترین بیت‌های کلمهٔ مضاعف خود اختصاص می‌یابد.[۱۲]

اما اگر ببینیم بیت‌ها در عمل چگونه در حافظه ذخیره می‌شوند، درمی‌یابیم که قالب به این سادگی نیست. دشواری زمانی پیش می‌آید که میدان‌های بیتی از مرز بایت‌ها عبور کنند، زیرا در پردازنده‌ای با ترتیب کوچک‌انتها[۱۳] بایت‌ها در حافظه وارونه قرار می‌گیرند. برای نمونه، میدان‌های بیتی ساختار S در حافظه به این صورت خواهند بود:



برچسب *f2l* به پنج بیت آخر (**یعنی** پنج بیت کم‌ارزش‌تر) میدان بیتی *f2* اشاره دارد و برچسب *f2m* به پنج بیت پرارزش‌تر آن. خطوط عمودی دوتایی مرز بایت‌ها را نشان می‌دهند. اگر همهٔ بایت‌ها را وارونه کنیم، تکه‌های میدان‌های *f2* و *f3* دوباره در جای درست خود به هم می‌پیوندند.

چیدمان فیزیکی داده‌ها در حافظه معمولاً اهمیتی ندارد، مگر آنکه داده‌ها به برنامه وارد یا از آن خارج شوند (که اتفاقاً در کار با میدان‌های بیتی بسیار رایج است). رابط‌های دستگاه‌های سخت‌افزاری اغلب از تعداد بیت‌های نامتعارفی استفاده می‌کنند و میدان‌های بیتی برای نمایش چنین داده‌هایی می‌توانند بسیار سودمند باشند.

یکی از این نمونه‌ها SCSI[۱۴] است. برای خواندن مستقیم از یک دستگاه SCSI،

---

[۱۲] در واقع استاندارد ANSI/ISO C در نحوهٔ دقیق چیدمان بیت‌ها به کامپایلر قدری آزادی عمل می‌دهد. با این حال، کامپایلرهای رایج C (gcc، مایکروسافت و بورلند) میدان‌ها را به همین شکل می‌چینند.

[13] little endian

[۱۴] مخفف Small Computer Systems Interface، استانداردی صنعتی برای دیسک‌های سخت و **دستگاه‌هایی از این دست**.

بیت \ بایت	7	6	5	4	3	2	1	0
0	کد عملیاتی (08h)							
1	شمارهٔ واحد منطقی			بیت‌های پرارزش LBA				
2	بخش میانی آدرس بلوک منطقی							
3	بیت‌های کم‌ارزش آدرس بلوک منطقی							
4	طول انتقال							
5	کنترل							

شکل ۶.۷: قالب فرمان خواندن SCSI

پیامی شش‌بایتی با قالبی که در شکل ۶.۷ آمده است به دستگاه فرستاده می‌شود. دشواری نمایش این فرمان با میدان‌های بیتی به **آدرس بلوک منطقی**[15] برمی‌گردد که در سه بایت مختلف فرمان پخش شده است. از شکل ۶.۷ پیداست که داده‌ها با ترتیب بزرگ‌انتها[16] ذخیره شده‌اند. شکل ۷.۷ تعریفی را نشان می‌دهد که می‌کوشد با همهٔ کامپایلرها کار کند. دو خط نخست ماکرویی تعریف می‌کنند که اگر کد با کامپایلر مایکروسافت یا بورلند کامپایل شود، مقدار درست دارد. بخش‌هایی که ممکن است گیج‌کننده به نظر برسند، خطوط ۱۱ تا ۱۴ هستند. نخست ممکن است بپرسید چرا میدان‌های `lba_mid` و `lba_lsb` جداگانه تعریف شده‌اند و نه به صورت یک میدان ۱۶ بیتی واحد. دلیل این است که داده‌ها ترتیب بزرگ‌انتها دارند، در حالی که کامپایلر یک میدان ۱۶ بیتی را با ترتیب کوچک‌انتها ذخیره می‌کند. دوم اینکه به نظر می‌رسد جای میدان‌های `lba_msb` و `logical_unit` برعکس شده است؛ اما چنین نیست و این دو میدان باید به همین ترتیب قرار گیرند. شکل ۸.۷ نشان می‌دهد که این میدان‌ها چگونه به صورت یک موجودیت ۴۸ بیتی نگاشته می‌شوند (مرز بایت‌ها باز هم با خطوط دوتایی مشخص شده است). وقتی این موجودیت با ترتیب کوچک‌انتها در حافظه ذخیره شود، بیت‌ها درست با قالب مطلوب (شکل ۶.۷) چیده می‌شوند.

ماجرا از این هم پیچیده‌تر است، زیرا تعریف `SCSI_read_cmd` با کامپایلر C مایکروسافت کاملاً درست کار نمی‌کند. اگر عبارت `sizeof(SCSI_read_cmd)` ارزیابی شود، C مایکروسافت مقدار ۸ را برمی‌گرداند، نه ۶! علت این است که کامپایلر مایکروسافت برای تعیین نحوهٔ نگاشت بیت‌ها، نوع میدان بیتی را هم در نظر می‌گیرد. از آنجا که همهٔ میدان‌های بیتی از نوع `unsigned` تعریف شده‌اند، کامپایلر دو بایت در انتهای ساختار می‌افزاید تا اندازهٔ آن مضرب صحیحی از کلمهٔ مضاعف شود. برای رفع

[15] logical block address
[16] big endian

```
#define MS_OR_BORLAND (defined(__BORLANDC__) \
                       || defined(_MSC_VER))

#if MS_OR_BORLAND
# pragma pack(push)
# pragma pack(1)
#endif

struct SCSI_read_cmd {
  unsigned opcode : 8;
  unsigned lba_msb : 5;
  unsigned logical_unit : 3;
  unsigned lba_mid : 8;     /* middle bits */
  unsigned lba_lsb : 8;
  unsigned transfer_length : 8;
  unsigned control : 8;
}
#if defined(__GNUC__)
   __attribute__((packed))
#endif
;

#if MS_OR_BORLAND
# pragma pack(pop)
#endif
```

شکل ۷.۷: ساختار قالب فرمان خواندن SCSI

این مشکل می‌توان همهٔ میدان‌ها را به جای آن از نوع `unsigned short` تعریف کرد. در این صورت کامپایلر مایکروسافت نیازی به افزودن بایت لایه‌گذاری ندارد، زیرا شش بایت مضرب صحیحی از کلمه‌های دوبایتی است.[۱۷] کامپایلرهای دیگر نیز با این تغییر درست کار می‌کنند. شکل ۹.۷ تعریف دیگری را نشان می‌دهد که با هر سه کامپایلر کار می‌کند و با به کار بردن `unsigned char`، جز دو میدان، از همهٔ میدان‌های بیتی پرهیز می‌کند.

اگر بحث پیشین را گیج‌کننده یافتید، دلسرد نشوید؛ چون این بحث به‌راستی گیج‌کننده است! نویسنده خود اغلب ترجیح می‌دهد به‌کلی از میدان‌های بیتی چشم

[۱۷] ترکیب انواع مختلف میدان‌های بیتی به رفتاری بسیار گیج‌کننده می‌انجامد! از خواننده دعوت می‌شود که خود این موضوع را بیازماید.

۸ بیت	۸ بیت	۸ بیت	۸ بیت	۳ بیت	۵ بیت	۸ بیت
control	transfer_length	lba_lsb	lba_mid	logical_unit	lba_msb	opcode

شکل ۸.۷: نگاشت میدان‌های SCSI_read_cmd

```
1  struct SCSI_read_cmd {
2    unsigned char opcode;
3    unsigned char lba_msb : 5;
4    unsigned char logical_unit  :  3;
5    unsigned char lba_mid;    /* middle bits */
6    unsigned char lba_lsb;
7    unsigned char transfer_length;
8    unsigned char control;
9  }
10 #if defined(__GNUC__)
11     __attribute__((packed))
12 #endif
13 ;
```

شکل ۹.۷: ساختار جایگزین برای قالب فرمان خواندن SCSI

بپوشد و با عملیات بیتی، بیت‌ها را به صورت دستی بررسی و تغییر دهد، چون این راه را کمتر گیج‌کننده می‌یابد.

۴.۱.۷ استفاده از ساختارها در اسمبلی

همان‌طور که پیش‌تر گفتیم، دسترسی به یک ساختار در اسمبلی بسیار شبیه دسترسی به یک آرایه است. برای نمونه‌ای ساده، فرض کنید می‌خواهیم روالی اسمبلی بنویسیم که عنصر y از یک ساختار S را صفر کند. اگر پیش‌نمونهٔ[۱۸] این روال چنین باشد:

```
void zero_y( S * s_p );
```

روال اسمبلی به این صورت خواهد بود:

```
1  %define      y_offset  4
2  _zero_y:
3        enter  0,0
4        mov    eax, [ebp + 8]      ; get s_p (struct pointer) from stack
```

[۱۸] prototype

```
5       mov    dword [eax + y_offset], 0
6       leave
7       ret
```

زبان C اجازه می‌دهد ساختار را به صورت مقداری[19] به تابع ارسال کنیم؛ اما این کار تقریباً همیشه ایدهٔ بدی است، زیرا در ارسال مقداری، کل داده‌های ساختار باید در پشته کپی شوند و سپس روال آن‌ها را از پشته بازیابی کند. بسیار کارآمدتر است که به جای این کار، اشاره‌گری به ساختار ارسال شود.

زبان C همچنین اجازه می‌دهد یک نوع ساختار به عنوان مقدار بازگشتی تابع به کار رود. پیداست که ساختار را نمی‌توان در ثبات EAX برگرداند و کامپایلرهای مختلف این وضعیت را به روش‌های متفاوتی مدیریت می‌کنند. یک راه‌حل رایج این است که کامپایلر تابع را در درون خود به تابعی بازنویسی کند که اشاره‌گری به ساختار را به عنوان پارامتر می‌گیرد. سپس از این اشاره‌گر برای قرار دادن مقدار بازگشتی در ساختاری استفاده می‌شود که بیرون از روال فراخوانی‌شده تعریف شده است.

بیشتر اسمبلرها (از جمله NASM) پشتیبانی داخلی برای تعریف ساختارها در کد اسمبلی دارند. برای آگاهی از جزئیات، به مستندات اسمبلر خود مراجعه کنید.

۲.۷ اسمبلی و C++

زبان برنامه‌نویسی C++ گسترشی از زبان C است و از این رو بسیاری از قاعده‌های پایه‌ای که برای ارتباط میان C و زبان اسمبلی[20] گفتیم، در C++ نیز به کار می‌آیند. با این حال، برخی از این قاعده‌ها باید تغییر کنند. از سوی دیگر، درک پاره‌ای از امکاناتی که C++ به C افزوده است، با آشنایی با زبان اسمبلی بسیار آسان‌تر می‌شود. در این بخش فرض بر این است که خواننده با مبانی C++ آشنا است.

۱.۲.۷ سربارگذاری و درهم‌سازی نام

زبان C++ اجازه می‌دهد چند تابع (و نیز چند تابع عضو[21] یک کلاس) با نام یکسان تعریف شوند. هنگامی که چند تابع نام مشترکی داشته باشند، می‌گوییم این توابع **سربارگذاری**[22] شده‌اند. اگر در C دو تابع هم‌نام تعریف کنیم، پیونددهنده[23] خطا

[19] pass by value
[20] assembly language
[21] member function
[22] overloading
[23] linker

```
1  #include <stdio.h>
2
3  void f( int x )
4  {
5     printf ("%d\n", x);
6  }
7
8  void f( double x )
9  {
10    printf ("%g\n", x);
11 }
```

شکل ۱۰.۷: دو تابع `f()`

می‌دهد، زیرا در فایل‌های شیء[۲۴] که در حال پیونددادن آن‌ها است، دو تعریف برای یک نماد[۲۵] پیدا می‌کند. برای نمونه، کد شکل ۱۰.۷ را در نظر بگیرید. کد اسمبلی معادل آن دو برچسب[۲۶] با نام `f_` تعریف می‌کند که آشکارا خطا است.

$C++$ از همان فرایند پیونددهی C استفاده می‌کند، اما با انجام **درهم‌سازی نام**[۲۷]، یعنی تغییر نمادی که برچسب تابع را می‌سازد، از این خطا جلوگیری می‌کند. در واقع C نیز به گونه‌ای درهم‌سازی نام انجام می‌دهد، زیرا هنگام ساختن برچسب هر تابع، یک خط زیرین به ابتدای نام آن می‌افزاید. با این حال، C نام هر دو تابع شکل ۱۰.۷ را به یک شکل درهم می‌سازد و در نتیجه باز هم خطا رخ می‌دهد. در مقابل، $C++$ از فرایند درهم‌سازی پیچیده‌تری بهره می‌گیرد که برای این دو تابع دو برچسب متفاوت پدید می‌آورد. برای نمونه، DJGPP به تابع نخست شکل ۱۰.۷ برچسب `_f__Fi` و به تابع دوم برچسب `_f__Fd` را نسبت می‌دهد و به این ترتیب هیچ خطایی در پیونددهی رخ نمی‌دهد.

متأسفانه هیچ استانداردی برای شیوهٔ مدیریت نام‌ها در $C++$ وجود ندارد و کامپایلرهای گوناگون نام‌ها را به روش‌های متفاوتی درهم می‌سازند. برای نمونه، Bor-land $C++$ برای دو تابع شکل ۱۰.۷ برچسب‌های `@f$qi` و `@f$qd` را به کار می‌برد. با این همه، این قاعده‌ها کاملاً دلبخواهی نیستند، زیرا نام درهم‌سازی‌شده **امضای**[۲۸]

[24] object file
[25] symbol
[26] label
[27] name mangling
[28] signature

تابع را در خود رمزگذاری می‌کند. امضای یک تابع را ترتیب و نوع پارامترهای[۲۹] آن تعیین می‌کند. دقت کنید که تابعی که تنها یک آرگومان[۳۰] از نوع `int` می‌گیرد، در انتهای نام درهم‌سازی‌شدهٔ خود یک i دارد (هم در DJGPP و هم در بورلند) و تابعی که آرگومانی از نوع `double` می‌گیرد، در انتهای نام درهم‌سازی‌شدهٔ خود یک d دارد. اگر تابعی با نام `f` و با پیش‌نمونهٔ[۳۱] زیر وجود داشت:

```
void f( int x, int y, double z);
```

DJGPP نام آن را به `_f__Fiid` و بورلند آن را به `@f$qiid` درهم می‌ساخت.

نوع بازگشتی تابع جزء امضای آن **نیست** و در نام درهم‌سازی‌شدهٔ آن نیز رمزگذاری نمی‌شود. همین واقعیت یکی از قاعده‌های سربارگذاری در $C++$ را توضیح می‌دهد: تنها توابعی را می‌توان سربارگذاری کرد که امضای آن‌ها یکتا باشد. همان‌طور که می‌بینید، اگر در $C++$ دو تابع با نام و امضای یکسان تعریف شوند، نام درهم‌سازی‌شدهٔ یکسانی پدید می‌آورند و در نتیجه خطای پیونددهی رخ می‌دهد. به طور پیش‌فرض، نام همهٔ توابع $C++$ درهم‌سازی می‌شود، حتی توابعی که سربارگذاری نشده‌اند. دلیل این است که کامپایلر[۳۲] هنگام کامپایل یک فایل هیچ راهی ندارد که بداند آیا تابعی خاص سربارگذاری شده است یا نه، پس همهٔ نام‌ها را درهم می‌سازد. در واقع کامپایلر نام متغیرهای سراسری[۳۳] را نیز درهم می‌سازد و این کار را با رمزگذاری نوع متغیر، به شیوه‌ای مشابه امضای توابع، انجام می‌دهد. بنابراین اگر کسی متغیری سراسری را در یک فایل با نوعی معین تعریف کند و سپس بکوشد آن را در فایل دیگری با نوعی نادرست به کار ببرد، خطای پیونددهی رخ می‌دهد. این ویژگی $C++$ را **پیونددهی با ایمنی نوع**[۳۴] می‌نامند. این ویژگی نوع دیگری از خطا را نیز آشکار می‌کند که ناسازگاری پیش‌نمونه‌ها است. این خطا زمانی رخ می‌دهد که تعریف یک تابع در یک پیمانه[۳۵] با پیش‌نمونه‌ای که پیمانهٔ دیگری به کار می‌برد، هم‌خوانی نداشته باشد. در C اشکال‌زدایی[۳۶] چنین مشکلی می‌تواند بسیار دشوار باشد، زیرا C این خطا را تشخیص نمی‌دهد. برنامه بی‌هیچ مشکلی کامپایل و پیونددهی می‌شود، اما رفتار آن تعریف‌نشده خواهد بود، چون کد فراخواننده انواعی را در پشته[۳۷] قرار می‌دهد که با انواع مورد انتظار

[29] parameter
[30] argument
[31] prototype
[32] compiler
[33] global variable
[34] typesafe linking
[35] module
[36] debugging
[37] stack

تابع متفاوت هستند. در C++ همین وضعیت به یک خطای پیونددهی می‌انجامد.

هنگامی که کامپایلر C++ یک فراخوانی تابع را تجزیه می‌کند، با بررسی نوع آرگومان‌هایی که به تابع فرستاده می‌شوند، به دنبال تابعی می‌گردد که با آن‌ها جور باشد[۳۸]. اگر چنین تابعی پیدا شود، کامپایلر با استفاده از قاعده‌های درهم‌سازی نام خود، یک دستورالعمل CALL به تابع درست تولید می‌کند.

از آنجا که کامپایلرهای گوناگون قاعده‌های درهم‌سازی متفاوتی دارند، ممکن است نتوان کدهای C++ را که با کامپایلرهای مختلف کامپایل شده‌اند با هم پیوند داد. این نکته هنگام استفاده از یک کتابخانهٔ ازپیش‌کامپایل‌شدهٔ C++ اهمیت زیادی دارد! اگر کسی بخواهد تابعی را به زبان اسمبلی بنویسد که قرار است همراه با کد C++ به کار رود، باید قاعده‌های درهم‌سازی نام کامپایلر C++ مورد استفاده را بداند (یا از روشی که در ادامه توضیح داده می‌شود بهره بگیرد).

دانشجوی تیزبین ممکن است بپرسد که آیا کد شکل ۱۰.۷ همان‌گونه که انتظار می‌رود کار می‌کند یا نه. از آنجا که C++ نام همهٔ توابع را درهم می‌سازد، نام تابع printf نیز درهم‌سازی می‌شود و کامپایلر دستور CALL به برچسب _printf را تولید نخواهد کرد. این نگرانی کاملاً بجا است! اگر پیش‌نمونهٔ printf صرفاً در بالای فایل قرار داده می‌شد، دقیقاً همین اتفاق می‌افتاد. این پیش‌نمونه چنین است:

```
int  printf ( const char *,  ...);
```

DJGPP این نام را به _printf__FPCce درهم می‌ساخت. (در اینجا F نشانهٔ **تابع**[۳۹]، P نشانهٔ **اشاره‌گر**[۴۰]، C نشانهٔ *const*، c نشانهٔ *char* و e نشانهٔ سه‌نقطه[۴۱] است.) در این صورت تابع printf معمولی کتابخانهٔ C فراخوانی نمی‌شد! بدیهی است که باید راهی وجود داشته باشد تا کد C++ بتواند کد C را فراخوانی کند. این موضوع اهمیت بسیاری دارد، زیرا کدهای قدیمی و سودمند C **بسیار زیادی** در دسترس هستند. C++ افزون بر اینکه اجازه می‌دهد کدهای قدیمی C را فراخوانی کنیم، امکان فراخوانی کد اسمبلی را نیز با قراردادهای معمول درهم‌سازی C فراهم می‌کند.

C++ کلیدواژهٔ extern را گسترش داده است تا بتوان با آن مشخص کرد که تابع یا متغیر سراسری مورد نظر از قراردادهای معمول C پیروی می‌کند. در اصطلاح C++ گفته می‌شود که این تابع یا متغیر سراسری **پیوند** *C*[۴۲] دارد. برای نمونه، برای

[۳۸] لازم نیست این تطابق دقیق باشد، زیرا کامپایلر تطابق‌هایی را هم که با تبدیل نوع آرگومان‌ها به دست می‌آیند در نظر می‌گیرد. قاعده‌های این فرایند از حوصلهٔ این کتاب بیرون است و برای جزئیات آن باید به کتاب‌های C++ مراجعه کرد.

[39] function
[40] pointer
[41] ellipsis
[42] C linkage

اعلان printf با پیوند C از پیش‌نمونهٔ زیر استفاده می‌کنیم:

```
extern "C" int  printf ( const char *,  ...  );
```

این اعلان به کامپایلر می‌گوید که قاعده‌های درهم‌سازی نام C++ را روی این تابع به کار نبرد و به جای آن از قاعده‌های C استفاده کند. البته با این کار دیگر نمی‌توان تابع printf را سربارگذاری کرد. همین روش ساده‌ترین راه برای ارتباط میان C++ و اسمبلی است، یعنی تابع را با پیوند C تعریف کنیم و سپس از قرارداد فراخوانی[۴۳] C بهره بگیریم.

برای راحتی کار، C++ اجازه می‌دهد پیوند مجموعه‌ای از توابع و متغیرهای سراسری را نیز یک‌جا تعیین کنیم. این مجموعه، مانند همیشه، با آکولاد مشخص می‌شود:

```
extern "C" {
  /* C linkage global  variables  and function  prototypes */
}
```

اگر فایل‌های سرآیند[۴۴] ANSI C را که امروزه همراه با کامپایلرهای C/C++ عرضه می‌شوند بررسی کنید، نزدیک به ابتدای هر یک از آن‌ها قطعه‌کد زیر را خواهید یافت:

```
#ifdef __cplusplus
extern "C" {
#endif
```

و نزدیک به انتهای فایل نیز ساختار مشابهی وجود دارد که آکولاد بسته را در بر می‌گیرد. کامپایلرهای C++ ماکروی[۴۵] __cplusplus را (با **دو** خط زیرین در ابتدای آن) تعریف می‌کنند. بنابراین قطعه‌کد بالا، اگر فایل سرآیند به عنوان C++ کامپایل شود، کل آن را درون یک بلوک extern "C" قرار می‌دهد، اما اگر به عنوان C کامپایل شود، هیچ کاری انجام نمی‌دهد (زیرا کامپایلر C برای extern "C" خطای نحوی می‌دهد). هر برنامه‌نویسی می‌تواند از همین روش برای ساختن فایل سرآیندی برای روال‌های اسمبلی خود استفاده کند که هم در C و هم در C++ قابل استفاده باشد.

[43] calling convention
[44] header file
[45] macro

```
1  void f( int & x )      // the & denotes a reference parameter
2  { x++; }
3
4  int main()
5  {
6    int y = 5;
7    f(y);                // reference to y is passed, note no & here!
8    printf ("%d\n", y);  // prints out 6!
9    return 0;
10 }
```

شکل ۱۱.۷: نمونه‌ای از ارجاع

۲.۲.۷ ارجاع‌ها

ارجاع‌ها[۴۶] یکی دیگر از امکانات تازهٔ C++ هستند و به ما اجازه می‌دهند بدون استفادهٔ صریح از اشاره‌گرها، پارامترها را به توابع بفرستیم. برای نمونه، کد شکل ۱۱.۷ را در نظر بگیرید. در واقع پارامترهای ارجاعی بسیار ساده هستند، زیرا چیزی جز همان اشاره‌گرها نیستند و کامپایلر تنها این موضوع را از دید برنامه‌نویس پنهان می‌کند (درست همان‌طور که کامپایلرهای پاسکال پارامترهای var را به صورت اشاره‌گر پیاده‌سازی می‌کنند). هنگامی که کامپایلر برای فراخوانی تابع در سطر ۷ کد اسمبلی تولید می‌کند، **آدرس** y را به تابع می‌فرستد. اگر کسی بخواهد تابع f را به زبان اسمبلی بنویسد، باید چنان رفتار کند که گویی پیش‌نمونهٔ تابع چنین است[۴۷]:

```
void f( int * xp);
```

ارجاع‌ها تنها برای راحتی کار هستند، اما به‌ویژه در سربارگذاری اپراتورها[۴۸] بسیار سودمند واقع می‌شوند. سربارگذاری اپراتور یکی دیگر از امکانات C++ است که به ما اجازه می‌دهد برای اپراتورهای رایج روی انواع ساختار[۴۹] یا کلاس[۵۰] معنای تازه‌ای تعریف کنیم. برای نمونه، یکی از کاربردهای رایج این است که اپراتور جمع (+) را برای الحاق اشیای رشته‌ای تعریف کنیم. به این ترتیب، اگر a و b رشته باشند، a + b حاصل الحاق دو رشتهٔ a و b را برمی‌گرداند. C++ در عمل برای انجام این کار

[46] references

[۴۷] البته ممکن است بخواهد، همان‌طور که در بخش ۱.۲.۷ گفته شد، تابع را با پیوند C اعلان کند تا از درهم‌سازی نام پرهیز شود.

[48] operator overloading

[49] structure

[50] class

یک تابع را فراخوانی می‌کند (در واقع می‌توان این عبارت را با نمادگذاری تابعی به صورت operator +(a,b) بازنویسی کرد). برای کارایی بیشتر، بهتر است به جای فرستادن اشیای رشته‌ای با مقدار، آدرس آن‌ها را بفرستیم. بدون ارجاع‌ها این کار به صورت operator +(&a,&b) ممکن است، اما در آن صورت باید آن را با نحو اپراتوری به شکل &a + &b نوشت که بسیار ناخوشایند و گیج‌کننده است. در مقابل، با استفاده از ارجاع‌ها می‌توان آن را به صورت a + b نوشت که کاملاً طبیعی به نظر می‌رسد.

۳.۲.۷ توابع درون‌خطی

توابع درون‌خطی[۵۱] نیز یکی دیگر از امکانات C++ هستند[۵۲]. هدف از توابع درون‌خطی جایگزین کردن ماکروهای پارامتردار مبتنی بر پیش‌پردازنده[۵۳] است که مستعد خطا هستند. از زبان C به یاد دارید که ماکرویی که عددی را به توان دو می‌رساند، ممکن است چنین نوشته شود:

```
#define SQR(x) ((x)*(x))
```

از آنجا که پیش‌پردازنده زبان C را نمی‌فهمد و تنها جایگزینی‌های ساده انجام می‌دهد، این پرانتزها لازم هستند تا در بیشتر موارد پاسخ درست به دست آید. با این حال، حتی همین نسخه نیز برای SQR(x++) پاسخ درست نمی‌دهد.

ماکروها به این دلیل به کار می‌روند که سربار فراخوانی تابع را برای توابع ساده از میان برمی‌دارند. همان‌طور که در فصل زیربرنامه‌ها دیدیم، فراخوانی یک تابع چند مرحله دارد و برای تابعی بسیار ساده، زمانی که صرف فراخوانی آن می‌شود ممکن است از زمان اجرای خود عملیات درون تابع بیشتر باشد! توابع درون‌خطی روش بسیار دلپذیرتری برای نوشتن کدی هستند که ظاهری مانند یک تابع معمولی دارد، اما یک بلوک کد مشترک را با CALL فراخوانی **نمی‌کند**. در عوض، هر فراخوانی تابع درون‌خطی با کدی جایگزین می‌شود که کار آن تابع را انجام می‌دهد. در C++ برای درون‌خطی کردن یک تابع کافی است کلیدواژهٔ inline را پیش از تعریف آن قرار دهیم. برای نمونه، توابعی را که در شکل ۱۲.۷ اعلان شده‌اند در نظر بگیرید. فراخوانی تابع f در سطر ۱۰ یک فراخوانی معمولی تابع است (به زبان اسمبلی، با این فرض که x در آدرس ebp-8 و y در ebp-4 قرار دارد):

```
1       push    dword [ebp-8]
```

[51] inline functions

[۵۲] کامپایلرهای C اغلب این امکان را به عنوان گسترشی بر ANSI C پشتیبانی می‌کنند.

[53] preprocessor

```
1  inline int inline_f( int x )
2  { return x*x; }
3
4  int f( int x )
5  { return x*x; }
6
7  int main()
8  {
9    int y, x = 5;
10   y = f(x);
11   y = inline_f(x);
12   return 0;
13 }
```

شکل ۱۲.۷: نمونه‌ای از درون‌خطی‌سازی

```
2       call    _f
3       pop     ecx
4       mov     [ebp-4], eax
```

اما فراخوانی تابع `inline_f` در سطر ۱۱ به این صورت خواهد بود:

```
1       mov     eax, [ebp-8]
2       imul    eax, eax
3       mov     [ebp-4], eax
```

در این مثال، درون‌خطی‌سازی دو مزیت دارد. نخست اینکه تابع درون‌خطی سریع‌تر است، چون هیچ پارامتری در پشته قرار داده نمی‌شود، هیچ قاب پشته‌ای[۵۴] ساخته و سپس نابود نمی‌شود و هیچ انشعابی[۵۵] هم رخ نمی‌دهد. دوم اینکه فراخوانی تابع درون‌خطی کد کمتری مصرف می‌کند! البته این نکتهٔ دوم هرچند در این مثال درست است، در همهٔ موارد صدق نمی‌کند.

عیب اصلی درون‌خطی‌سازی این است که کد درون‌خطی پیونددهی نمی‌شود و از این رو کد یک تابع درون‌خطی باید در دسترس **همهٔ** فایل‌هایی باشد که از آن استفاده می‌کنند. کد اسمبلی مثال پیشین این موضوع را به‌خوبی نشان می‌دهد. برای فراخوانی تابع غیردرون‌خطی کافی است پارامترها، نوع مقدار بازگشتی، قرارداد فراخوانی و نام

[54] stack frame
[55] branch

```
1  class Simple {
2  public:
3    Simple();                  // default constructor
4    ~Simple();                 // destructor
5    int get_data() const;      // member functions
6    void set_data( int );
7  private:
8    int data;                  // member data
9  };
10
11 Simple::Simple()
12 { data = 0; }
13
14 Simple::~Simple()
15 { /* null body */ }
16
17 int Simple::get_data() const
18 { return data; }
19
20 void Simple::set_data( int x )
21 { data = x; }
```

شکل ۱۳.۷: یک کلاس ساده در C++

برچسب تابع را بدانیم و همهٔ این اطلاعات از پیش‌نمونهٔ تابع به دست می‌آید. اما استفاده از تابع درون‌خطی مستلزم آگاهی از تمام کد آن است. معنای این سخن آن است که اگر **هر** بخشی از یک تابع درون‌خطی تغییر کند، **همهٔ** فایل‌های منبعی[۵۶] که از آن استفاده می‌کنند باید دوباره کامپایل شوند. به یاد دارید که در توابع غیردرون‌خطی، اگر پیش‌نمونه تغییر نکند، اغلب نیازی به کامپایل دوبارهٔ فایل‌هایی که از تابع استفاده می‌کنند نیست. به همهٔ این دلایل، کد توابع درون‌خطی معمولاً در فایل‌های سرآیند قرار داده می‌شود. این شیوه با قاعدهٔ سفت‌وسخت و همیشگی C در تضاد است که می‌گوید دستورات کد اجرایی **هرگز** در فایل‌های سرآیند قرار نمی‌گیرند.

[56] source file

```
void set_data( Simple * object, int x )
{
  object->data = x;
}
```

شکل ۱۴.۷: نسخهٔ C تابع Simple::set_data()

```
1  _set_data__6Simplei:            ; mangled name
2        push   ebp
3        mov    ebp, esp
4
5        mov    eax, [ebp + 8]    ; eax = pointer to object (this)
6        mov    edx, [ebp + 12]   ; edx = integer parameter
7        mov    [eax], edx        ; data is at offset 0
8
9        leave
10       ret
```

شکل ۱۵.۷: خروجی کامپایلر برای Simple::set_data(int)

۴.۲.۷ کلاس‌ها

هر کلاس[۵۷] در C++ نوعی از **شیء**[۵۸] را توصیف می‌کند. شیء هم عضو داده‌ای دارد و هم عضو تابعی[۵۹]؛ به بیان دیگر، شیء همان `struct` است که داده‌ها و توابعی به آن وابسته شده‌اند. کلاس ساده‌ای را در نظر بگیرید که در شکل ۱۳.۷ تعریف شده است. متغیری از نوع `Simple` درست مانند یک `struct` معمولی در C با تنها یک عضو `int` به نظر می‌رسد. توابع در حافظه‌ای که به ساختار اختصاص یافته است ذخیره **نمی‌شوند**. با این حال، توابع عضو با توابع دیگر فرق دارند، زیرا یک پارامتر **پنهان**[۶۰] نیز به آن‌ها فرستاده می‌شود. این پارامتر اشاره‌گری به شیئی است که تابع عضو روی آن عمل می‌کند.

در واقع C++ برای دسترسی به اشاره‌گرِ شیء مورد عمل از درون تابع عضو، کلیدواژهٔ `this` را به کار می‌برد.

برای نمونه، متد[۶۱] `set_data` از کلاس `Simple` در شکل ۱۳.۷ را در نظر بگیرید.

[57] class
[58] object
[۵۹] این اعضا در C++ معمولاً **توابع عضو** (member functions) و به‌طور کلی‌تر **متد** (methods) نامیده می‌شوند.
[60] hidden parameter
[61] method

اگر این متد به زبان C نوشته می‌شد، به تابعی شبیه بود که اشاره‌گر شیء مورد عمل به‌صراحت به آن فرستاده می‌شود؛ کد شکل ۱۴.۷ همین را نشان می‌دهد. سوئیچ S- در کامپایلر $DJGPP$ (و همچنین کامپایلرهای gcc و بورلند) از کامپایلر می‌خواهد فایلی اسمبلی بسازد که معادل زبان اسمبلیِ کد تولیدشده را در بر دارد. در $DJGPP$ و gcc پسوند این فایل `s` . است و متأسفانه از نحو اسمبلی AT&T پیروی می‌کند که با نحو NASM و MASM تفاوت زیادی دارد[۶۲]. (کامپایلرهای بورلند و مایکروسافت فایلی با پسوند `asm` . و با نحو MASM تولید می‌کنند.) شکل ۱۵.۷ خروجی $DJGPP$ را پس از تبدیل به نحو NASM نشان می‌دهد و برای روشن‌شدن کار هر دستور، توضیحاتی نیز به آن افزوده شده است. در همان سطر نخست دقت کنید که به متد `set_data` برچسبی درهم‌سازی‌شده[۶۳] داده شده است که نام متد، نام کلاس و پارامترها را در خود رمزگذاری می‌کند. نام کلاس از آن رو رمزگذاری می‌شود که ممکن است کلاس‌های دیگری هم متدی به نام `set_data` داشته باشند و این دو متد **باید** برچسب‌های متفاوتی بگیرند. پارامترها نیز رمزگذاری می‌شوند تا کلاس بتواند، درست مانند توابع معمولی $C++$، متد `set_data` را برای پذیرفتن پارامترهای دیگر سربارگذاری[۶۴] کند. با این همه، باز هم کامپایلرهای مختلف این اطلاعات را به شیوه‌های گوناگونی در برچسب درهم‌سازی‌شده رمزگذاری می‌کنند.

پس از آن، در سطرهای ۲ و ۳ پیش‌درآمد[۶۵] آشنای تابع دیده می‌شود. در سطر ۵ نخستین پارامتر روی پشته در `EAX` ذخیره می‌شود. این پارامتر، پارامتر `x` **نیست**! بلکه همان پارامتر پنهانی[۶۶] است که به شیء مورد عمل اشاره می‌کند. سطر ۶ پارامتر `x` را در `EDX` قرار می‌دهد و سطر ۷ مقدار `EDX` را در کلمهٔ مضاعفی ذخیره می‌کند که `EAX` به آن اشاره دارد. این کلمهٔ مضاعف همان عضو `data` از شیء `Simple` مورد عمل است که چون تنها دادهٔ این کلاس است، در آفست ۰ ساختار `Simple` جای می‌گیرد.

---

[۶۲] سیستم کامپایلر gcc اسمبلر ویژهٔ خود را دارد که gas نام دارد. اسمبلر gas از نحو AT&T استفاده می‌کند و به همین دلیل کامپایلر کد را در قالب gas بیرون می‌دهد. صفحات متعددی در وب تفاوت‌های دو قالب اینتل و AT&T را بررسی کرده‌اند. برنامهٔ رایگانی هم به نام `a2i` (`http://www.multimania.com/placr/a2i.html`) وجود دارد که قالب AT&T را به قالب NASM تبدیل می‌کند.

[63] mangled label

[64] overload

[65] prologue

[۶۶] طبق معمول، در کد اسمبلی **هیچ چیز** پنهان نیست!

```
1  class Big_int {
2  public:
3     /*
4     * Parameters:
5     *    size            - size of integer expressed as number of
6     *                      normal unsigned int's
7     *    initial_value   - initial value of Big_int as a normal unsigned int
8     */
9     explicit Big_int( size_t     size,
10                      unsigned initial_value = 0);
11    /*
12     * Parameters:
13     *    size            - size of integer expressed as number of
14     *                      normal unsigned int's
15     *    initial_value   - initial value of Big_int as a string holding
16     *                      hexadecimal representation of value.
17     */
18    Big_int( size_t          size,
19             const char *  initial_value );
20
21    Big_int( const Big_int & big_int_to_copy);
22    ~Big_int();
23
24    // returns size of Big_int (in terms of unsigned int's)
25    size_t  size() const;
26
27    const Big_int & operator = ( const Big_int & big_int_to_copy);
28    friend Big_int operator + ( const Big_int & op1,
29                                const Big_int & op2 );
30    friend Big_int operator - ( const Big_int & op1,
31                                const Big_int & op2);
32    friend bool operator == ( const Big_int & op1,
33                              const Big_int & op2 );
34    friend bool operator < ( const Big_int & op1,
35                             const Big_int & op2);
36    friend ostream & operator << ( ostream &      os,
37                                   const Big_int & op );
38 private:
39    size_t       size_;     // size of unsigned array
40    unsigned * number_; // pointer to unsigned array holding value
41 };
```

شکل ۱۶.۷: تعریف کلاس Big_int

```
1  // prototypes for assembly routines
2  extern "C" {
3    int add_big_ints( Big_int &       res,
4                      const Big_int & op1,
5                      const Big_int & op2);
6    int sub_big_ints( Big_int &       res,
7                      const Big_int & op1,
8                      const Big_int & op2);
9  }
10
11 inline Big_int operator + ( const Big_int & op1, const Big_int & op2)
12 {
13   Big_int result(op1.size());
14   int res = add_big_ints(result, op1, op2);
15   if (res == 1)
16     throw Big_int::Overflow();
17   if (res == 2)
18     throw Big_int::Size_mismatch();
19   return result;
20 }
21
22 inline Big_int operator - ( const Big_int & op1, const Big_int & op2)
23 {
24   Big_int result(op1.size());
25   int res = sub_big_ints(result, op1, op2);
26   if (res == 1)
27     throw Big_int::Overflow();
28   if (res == 2)
29     throw Big_int::Size_mismatch();
30   return result;
31 }
```

شکل ۱۷.۷: کد حسابی کلاس Big_int

مثال

در این بخش، با به کار بستن ایده‌های این فصل، کلاسی در C++ می‌سازیم که یک عدد صحیح بی‌علامت با اندازهٔ دلخواه را نمایش می‌دهد. چون این عدد صحیح می‌تواند هر اندازه‌ای داشته باشد، در آرایه‌ای از اعداد صحیح بی‌علامت (کلمه‌های مضاعف) ذخیره می‌شود و به کمک تخصیص پویا[۶۷] می‌توان آن را به هر اندازه‌ای درآورد. کلمه‌های مضاعف به ترتیب معکوس ذخیره می‌شوند[۶۸]، یعنی کم‌ارزش‌ترین کلمهٔ مضاعف در اندیس ۰ قرار دارد. تعریف کلاس `Big_int` در شکل ۱۶.۷ آمده است[۶۹]. اندازهٔ هر `Big_int` با اندازهٔ آرایهٔ `unsigned` سنجیده می‌شود که داده‌های آن را در خود نگه می‌دارد. به عضو دادهٔ `_size` در این کلاس آفست صفر و به عضو `_number` آفست ۴ اختصاص داده شده است.

برای آنکه این مثال ساده بماند، فقط نمونه‌هایی[۷۰] از شیء را می‌توان با هم جمع یا از هم کم کرد که آرایه‌هایشان هم‌اندازه باشد.

این کلاس سه سازنده[۷۱] دارد: سازندهٔ اول (سطر ۹) نمونهٔ کلاس را با یک عدد صحیح بی‌علامت معمولی مقداردهی اولیه می‌کند، سازندهٔ دوم (سطر ۱۸) نمونه را با رشته‌ای مقداردهی می‌کند که یک مقدار شانزده‌شانزدهی را در بر دارد و سازندهٔ سوم (سطر ۲۱) **سازندهٔ کپی**[۷۲] است.

تمرکز این بحث بر نحوهٔ کار اپراتورهای[۷۳] جمع و تفریق است، زیرا زبان اسمبلی درست در همین جا به کار رفته است. شکل ۱۷.۷ بخش‌های مربوط به این اپراتورها را از فایل سرآیند[۷۴] نشان می‌دهد و روشن می‌کند که اپراتورها چگونه برای فراخوانی روال‌های اسمبلی آماده می‌شوند. از آنجا که کامپایلرهای مختلف برای توابع اپراتور از قواعد درهم‌سازی بسیار متفاوتی پیروی می‌کنند، توابع اپراتور درون‌خطی[۷۵] به کار رفته‌اند تا فراخوانی روال‌های اسمبلی با پیوند[۷۶] به سبک C را آماده کنند. به این ترتیب، انتقال کد به کامپایلرهای دیگر نسبتاً آسان می‌شود و سرعت آن نیز درست به

---

[67] dynamic allocation

[۶۸] چرا؟ چون در این صورت عملیات جمع همیشه از ابتدای آرایه آغاز می‌شود و رو به جلو پیش می‌رود.

[۶۹] کد کامل این مثال را در کدهای نمونهٔ کتاب ببینید؛ متن فقط به بخشی از آن کد اشاره می‌کند.

[70] instance

[71] constructor

[72] copy constructor

[73] operator

[74] header file

[75] inline

[76] linkage

اندازهٔ فراخوانی مستقیم است. افزون بر این، با این روش دیگر لازم نیست استثنا[۷۷] از درون کد اسمبلی پرتاب شود!

اصلاً چرا در اینجا از اسمبلی استفاده شده است؟ به یاد بیاورید که در حساب با دقت چندگانه[۷۸]، رقم نقلی[۷۹] باید از یک کلمهٔ مضاعف گرفته و با کلمهٔ مضاعف پرارزش‌تر بعدی جمع شود. C++ (و C) به برنامه‌نویس اجازه نمی‌دهند به پرچم نقلی[۸۰] پردازنده دسترسی داشته باشد. بنابراین، تنها راه انجام این جمع آن بود که C++ پرچم نقلی را جداگانه دوباره محاسبه کند و آن را به‌طور شرطی با کلمهٔ مضاعف بعدی جمع کند. نوشتن این کد به زبان اسمبلی، که در آن می‌توان به پرچم نقلی دسترسی داشت، بسیار کارآمدتر است و در آن استفاده از دستورالعمل ADC، که پرچم نقلی را خودکار در جمع وارد می‌کند، کاملاً منطقی است.

برای کوتاه‌شدن بحث، در اینجا فقط به بررسی روال اسمبلی add_big_ints می‌پردازیم. کد این روال (از فایل big_math.asm) در ادامه آمده است:

big_math.asm

```
1  segment .text
2          global  add_big_ints, sub_big_ints
3  %define size_offset 0
4  %define number_offset 4
5
6  %define EXIT_OK 0
7  %define EXIT_OVERFLOW 1
8  %define EXIT_SIZE_MISMATCH 2
9
10 ; Parameters for both add and sub routines
11 %define res ebp+8
12 %define op1 ebp+12
13 %define op2 ebp+16
14
15 add_big_ints:
16         push    ebp
17         mov     ebp, esp
18         push    ebx
19         push    esi
20         push    edi
21         ;
```

[77] exception
[78] multiple precision arithmetic
[79] carry
[80] carry flag

```
22          ; first set up esi to point to op1
23          ;              edi to point to op2
24          ;              ebx to point to res
25          mov     esi, [op1]
26          mov     edi, [op2]
27          mov     ebx, [res]
28          ;
29          ; make sure that all 3 Big_int's have the same size
30          ;
31          mov     eax, [esi + size_offset]
32          cmp     eax, [edi + size_offset]
33          jne     sizes_not_equal                 ; op1.size_ != op2.size_
34          cmp     eax, [ebx + size_offset]
35          jne     sizes_not_equal                 ; op1.size_ != res.size_
36
37          mov     ecx, eax                        ; ecx = size of Big_int's
38          ;
39          ; now, set registers to point to their respective arrays
40          ;       esi = op1.number_
41          ;       edi = op2.number_
42          ;       ebx = res.number_
43          ;
44          mov     ebx, [ebx + number_offset]
45          mov     esi, [esi + number_offset]
46          mov     edi, [edi + number_offset]
47
48          clc                                     ; clear carry flag
49          xor     edx, edx                        ; edx = 0
50          ;
51          ; addition loop
52  add_loop:
53          mov     eax, [edi+4*edx]
54          adc     eax, [esi+4*edx]
55          mov     [ebx + 4*edx], eax
56          inc     edx                             ; does not alter carry flag
57          loop    add_loop
58
59          jc      overflow
60  ok_done:
61          xor     eax, eax                        ; return value = EXIT_OK
62          jmp     done
63  overflow:
```

```
64          mov     eax, EXIT_OVERFLOW
65          jmp     done
66  sizes_not_equal:
67          mov     eax, EXIT_SIZE_MISMATCH
68  done:
69          pop     edi
70          pop     esi
71          pop     ebx
72          leave
73          ret
```
big_math.asm

امیدواریم تا اینجا بیشتر این کد برای خواننده روشن باشد. سطرهای ۲۵ تا ۲۷ اشاره‌گرهای اشیای Big_int را که به تابع فرستاده شده‌اند در ثبات‌ها قرار می‌دهند. به یاد داشته باشید که ارجاع‌ها[۸۱] در واقع چیزی جز اشاره‌گر نیستند. سطرهای ۳۱ تا ۳۵ بررسی می‌کنند که اندازهٔ آرایه‌های هر سه شیء یکسان باشد. (دقت کنید که برای دسترسی به عضو داده، آفست _size با اشاره‌گر جمع می‌شود.) سطرهای ۴۴ تا ۴۶ ثبات‌ها را طوری تنظیم می‌کنند که به جای خود اشیا، به آرایه‌ای اشاره کنند که هر شیء به کار می‌برد. (در اینجا نیز آفست عضو _number با اشاره‌گر شیء جمع می‌شود.)

حلقهٔ سطرهای ۵۲ تا ۵۷ اعداد صحیح ذخیره‌شده در آرایه‌ها را با هم جمع می‌کند؛ این حلقه نخست کم‌ارزش‌ترین کلمه‌های مضاعف را جمع می‌کند، سپس کلمه‌های مضاعف کم‌ارزش بعدی را، **و الی آخر**. در حساب با دقت گسترش‌یافته جمع باید با همین ترتیب انجام شود (بخش ۵.۱.۲ را ببینید). سطر ۵۹ سرریز را بررسی می‌کند، زیرا اگر سرریز رخ دهد، آخرین جمع، یعنی جمع پرارزش‌ترین کلمهٔ مضاعف، پرچم نقلی را یک می‌کند. از آنجا که کلمه‌های مضاعف آرایه به ترتیب کوچک‌انتها[۸۲] ذخیره شده‌اند، حلقه از ابتدای آرایه آغاز می‌شود و رو به جلو به سوی انتهای آن پیش می‌رود.

شکل ۱۸.۷ مثال کوتاهی از کاربرد کلاس Big_int را نشان می‌دهد. دقت کنید که ثابت‌های Big_int باید مانند سطر ۱۶ به‌صراحت اعلان شوند. این کار به دو دلیل لازم است. نخست آنکه هیچ سازندهٔ تبدیلی[۸۳] وجود ندارد که یک unsigned int را به Big_int تبدیل کند. دوم آنکه فقط اشیای Big_int هم‌اندازه را می‌توان با هم جمع کرد و همین موضوع تبدیل را دشوار می‌کند، چون به سختی می‌توان دانست که مقدار باید به چه اندازه‌ای تبدیل شود. در پیاده‌سازی پیشرفته‌تری از این کلاس می‌شد

[81] reference
[82] little endian
[83] conversion constructor

```
#include "big_int.hpp"
#include <iostream>
using namespace std;

int main()
{
  try {
    Big_int b(5,"8000000000000a00b");
    Big_int a(5,"80000000000010230");
    Big_int c = a + b;
    cout << a << " + " << b << " = " << c << endl;
    for( int i=0; i < 2; i++ ) {
      c = c + a;
      cout << "c = " << c << endl;
    }
    cout << "c-1 = " << c - Big_int(5,1) << endl;
    Big_int d(5, "12345678");
    cout << "d = " << d << endl;
    cout << "c == d " << (c == d) << endl;
    cout << "c > d " << (c > d) << endl;
  }
  catch( const char * str ) {
    cerr << "Caught: " << str << endl;
  }
  catch( Big_int::Overflow ) {
    cerr << "Overflow" << endl;
  }
  catch( Big_int::Size_mismatch ) {
    cerr << "Size mismatch" << endl;
  }
  return 0;
}
```

شکل ۱۸.۷: کاربردی ساده از `Big_int`

هر اندازه‌ای را با هر اندازهٔ دیگری جمع کرد، اما نویسنده نخواست با پیاده‌سازی این قابلیت در اینجا مثال را بیش از حد پیچیده کند. (با این حال، خواننده تشویق می‌شود که این کار را خود انجام دهد.)

۵.۲.۷ وراثت و چندریختی

وراثت[۸۴] به یک کلاس امکان می‌دهد داده‌ها و متدهای کلاس دیگری را به ارث ببرد. برای نمونه، کد شکل ۱۹.۷ را در نظر بگیرید. در این کد دو کلاس A و B تعریف شده‌اند که کلاس B از A ارث می‌برد. خروجی این برنامه چنین است:

```
Size of a: 4 Offset of ad: 0
Size of b: 8 Offset of ad: 0 Offset of bd: 4
A::m()
A::m()
```

دقت کنید که عضو دادهٔ ad در هر دو کلاس (که B آن را از A به ارث برده است) در آفست یکسانی قرار دارد. این نکته اهمیت دارد، زیرا ممکن است به تابع f اشاره‌گری به یک شیء A یا به هر شیئی از نوعی مشتق‌شده[۸۵] از A (یعنی نوعی که از A ارث برده است) فرستاده شود. شکل ۲۰.۷ کد اسمبلی (ویرایش‌شدهٔ) این تابع را نشان می‌دهد که *gcc* تولید کرده است.

در خروجی دقت کنید که متد m کلاس A هم برای شیء a و هم برای شیء b فراخوانی شده است. کد اسمبلی نشان می‌دهد که فراخوانی A::m() به‌طور ثابت در خود تابع نوشته شده است. اما در برنامه‌نویسی شیءگرای واقعی، متدی که فراخوانی می‌شود باید به نوع شیئی بستگی داشته باشد که به تابع فرستاده شده است. این ویژگی **چندریختی**[۸۶] نام دارد. $C++$ این ویژگی را به‌طور پیش‌فرض خاموش نگه می‌دارد و برای روشن‌کردن آن باید کلیدواژهٔ *virtual* (مجازی) را به کار برد. شکل ۲۱.۷ نشان می‌دهد که دو کلاس چگونه باید تغییر کنند و هیچ بخش دیگری از کد به تغییر نیاز ندارد. چندریختی را می‌توان به روش‌های بسیاری پیاده‌سازی کرد. متأسفانه پیاده‌سازی *gcc* در زمان نگارش این متن در حال گذار است و به‌مراتب از پیاده‌سازی اولیهٔ خود پیچیده‌تر می‌شود. از این رو، برای ساده‌ماندن بحث، نویسنده فقط به پیاده‌سازی چندریختی در کامپایلرهای ویندوزی مایکروسافت و بورلند می‌پردازد. این

[84] inheritance
[85] derived
[86] polymorphism

```
1  #include <cstddef>
2  #include <iostream>
3  using namespace std;
4
5  class A {
6  public:
7    void __cdecl m() { cout << "A::m()" << endl; }
8    int ad;
9  };
10
11 class B : public A {
12 public:
13   void __cdecl m() { cout << "B::m()" << endl; }
14   int bd;
15 };
16
17 void f( A * p )
18 {
19   p->ad = 5;
20   p->m();
21 }
22
23 int main()
24 {
25   A a;
26   B b;
27   cout << "Size of a: " << sizeof(a)
28        << " Offset of ad: " << offsetof(A,ad) << endl;
29   cout << "Size of b: " << sizeof(b)
30        << " Offset of ad: " << offsetof(B,ad)
31        << " Offset of bd: " << offsetof(B,bd) << endl;
32   f(&a);
33   f(&b);
34   return 0;
35 }
```

شکل ۱۹.۷: وراثت ساده

```
_f__FP1A:                       ; mangled function name
      push   ebp
      mov    ebp, esp
      mov    eax, [ebp+8]       ; eax points to object
      mov    dword [eax], 5     ; using offset 0 for ad
      mov    eax, [ebp+8]       ; passing address of object to A::m()
      push   eax
      call   _m__1A             ; mangled method name for A::m()
      add    esp, 4
      leave
      ret
```

شکل ۲۰.۷: کد اسمبلی وراثت ساده

```
class A {
public:
  virtual void __cdecl m() { cout << "A::m()" << endl; }
  int ad;
};

class B : public A {
public:
  virtual void __cdecl m() { cout << "B::m()" << endl; }
  int bd;
};
```

شکل ۲۱.۷: وراثت چندریختی

```
1  ?f@@YAXPAVA@@@Z:
2        push   ebp
3        mov    ebp, esp
4
5        mov    eax, [ebp+8]
6        mov    dword [eax+4], 5  ; p->ad = 5;
7
8        mov    ecx, [ebp + 8]    ; ecx = p
9        mov    edx, [ecx]        ; edx = pointer to vtable
10       mov    eax, [ebp + 8]    ; eax = p
11       push   eax               ; push "this" pointer
12       call   dword [edx]       ; call first function in vtable
13       add    esp, 4            ; clean up stack
14
15       pop    ebp
16       ret
```

شکل ۲۲.۷: کد اسمبلی تابع f()

پیاده‌سازی سال‌هاست که تغییری نکرده است و احتمالاً در آیندهٔ قابل پیش‌بینی نیز تغییر نخواهد کرد.

با این تغییرات، خروجی برنامه چنین می‌شود:

```
Size of a: 8 Offset of ad: 4
Size of b: 12 Offset of ad: 4 Offset of bd: 8
A::m()
B::m()
```

اکنون فراخوانی دوم f، متد B::m() را فراخوانی می‌کند، زیرا یک شیء B به آن فرستاده شده است. اما این تنها تغییر نیست. اندازهٔ A اکنون ۸ است (و اندازهٔ B برابر ۱۲). همچنین آفست ad به جای ۰، اکنون ۴ است. پس در آفست ۰ چه چیزی قرار دارد؟ پاسخ این پرسش‌ها به نحوهٔ پیاده‌سازی چندریختی بازمی‌گردد.

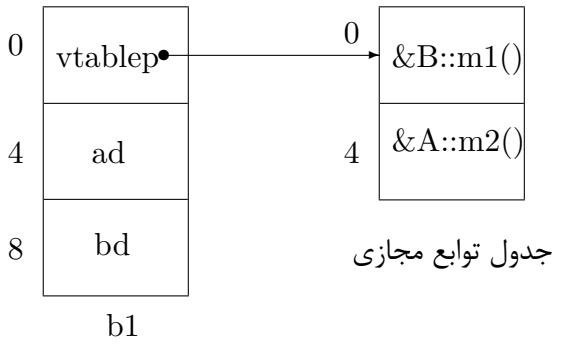
هر کلاس C++ که متد مجازی[87] داشته باشد، یک فیلد پنهان اضافی می‌گیرد که اشاره‌گری به آرایه‌ای از اشاره‌گرهای متد است[88]. این جدول را اغلب **جدول توابع**

[87] virtual method

[88] کامپایلرهای C++ کلاس‌های بدون متد مجازی را همیشه با یک struct معمولی در C که همان اعضای داده را دارد سازگار نگه می‌دارند.

```
1  class A {
2  public:
3    virtual void __cdecl m1() { cout << "A::m1()" << endl; }
4    virtual void __cdecl m2() { cout << "A::m2()" << endl; }
5    int ad;
6  };
7
8  class B : public A {     // B inherits A's m2()
9  public:
10   virtual void __cdecl m1() { cout << "B::m1()" << endl; }
11   int bd;
12 };
13 /* prints the vtable of given object */
14 void print_vtable( A * pa )
15 {
16   // p sees pa as an array of dwords
17   unsigned * p = reinterpret_cast<unsigned *>(pa);
18   // vt sees vtable as an array of pointers
19   void ** vt = reinterpret_cast<void **>(p[0]);
20   cout << hex << "vtable address = " << vt << endl;
21   for( int i=0; i < 2; i++ )
22     cout << "dword " << i << ": " << vt[i] << endl;
23
24   // call virtual functions in EXTREMELY non-portable way!
25   void (*m1func_pointer)(A *);   // function pointer variable
26   m1func_pointer = reinterpret_cast<void (*)(A*)>(vt[0]);
27   m1func_pointer(pa);            // call method m1 via function pointer
28
29   void (*m2func_pointer)(A *);   // function pointer variable
30   m2func_pointer = reinterpret_cast<void (*)(A*)>(vt[1]);
31   m2func_pointer(pa);            // call method m2 via function pointer
32 }
33
34 int main()
35 {
36   A a;    B b1;  B b2;
37   cout << "a: " << endl;   print_vtable(&a);
38   cout << "b1: " << endl;  print_vtable(&b1);
39   cout << "b2: " << endl;  print_vtable(&b2);
40   return 0;
41 }
```

شکل ۲۳.۷: مثالی پیچیده‌تر



شکل ۲۴.۷: نمایش درونی b1

مجازی (*vtable*) می‌نامند. در کلاس‌های A و B این اشاره‌گر در آفست ۰ ذخیره می‌شود. کامپایلرهای ویندوز همیشه این اشاره‌گر را در ابتدای کلاسی قرار می‌دهند که در بالای درخت وراثت[۸۹] جای دارد. اگر به کد اسمبلی (شکل ۲۲.۷) نگاه کنید که برای تابع f (از شکل ۱۹.۷) در نسخهٔ متد مجازی برنامه تولید شده است، می‌بینید که فراخوانی متد m به یک برچسب نیست. سطر ۹ آدرس جدول توابع مجازی را از روی شیء پیدا می‌کند. در سطر ۱۱ آدرس شیء در پشته قرار داده می‌شود و سطر ۱۲ با انشعاب به نخستین آدرس جدول توابع مجازی، متد مجازی را فراخوانی می‌کند[۹۰]. این فراخوانی از برچسب استفاده نمی‌کند، بلکه به آدرس کدی انشعاب می‌کند که EDX به آن اشاره دارد. چنین فراخوانی‌ای نمونه‌ای از **انقیاد دیرهنگام**[۹۱] است. در انقیاد دیرهنگام، تصمیم‌گیری دربارهٔ اینکه کدام متد فراخوانی شود تا زمان اجرای کد به تأخیر می‌افتد و همین امر به کد امکان می‌دهد متد مناسبِ هر شیء را فراخوانی کند. در حالت معمولی (شکل ۲۰.۷) فراخوانی یک متد مشخص به‌طور ثابت در کد نوشته می‌شود که به آن **انقیاد زودهنگام**[۹۲] می‌گویند (چون در اینجا متد زود، یعنی در زمان کامپایل، مقید می‌شود).

خوانندهٔ دقیق حتماً از خود می‌پرسد که چرا متدهای کلاس در شکل ۲۱.۷ با کلیدواژهٔ __cdecl به‌صراحت طوری اعلان شده‌اند که از قرارداد فراخوانی[۹۳] C پیروی کنند. مایکروسافت به‌طور پیش‌فرض برای متدهای کلاس در C++ قرارداد فراخوانی

[89] inheritance tree

[۹۰] البته این مقدار از پیش در ثبات ECX قرار دارد، چون در سطر ۸ در آنجا گذاشته شده است. بنابراین می‌شد سطر ۱۰ را حذف کرد و سطر بعد را طوری تغییر داد که ECX را در پشته قرار دهد. این کد چندان کارآمد نیست، زیرا بدون روشن‌کردن بهینه‌سازی‌های کامپایلر تولید شده است.

[91] late binding

[92] early binding

[93] calling convention

```
a:
vtable address = 004120E8
dword 0: 00401320
dword 1: 00401350
A::m1()
A::m2()
b1:
vtable address = 004120F0
dword 0: 004013A0
dword 1: 00401350
B::m1()
A::m2()
b2:
vtable address = 004120F0
dword 0: 004013A0
dword 1: 00401350
B::m1()
A::m2()
```

شکل ۲۵.۷: خروجی برنامهٔ شکل ۲۳.۷

دیگری جز قرارداد استاندارد C به کار می‌برد. در این قرارداد، اشاره‌گر شیئی که متد روی آن عمل می‌کند به جای پشته در ثبات ECX فرستاده می‌شود، هرچند پشته همچنان برای دیگر پارامترهای صریح متد به کار می‌رود. اصلاح‌گر[۹۴] __cdecl به کامپایلر می‌گوید از قرارداد فراخوانی استاندارد C استفاده کند. بورلند C++ به‌طور پیش‌فرض از قرارداد فراخوانی C استفاده می‌کند.

اکنون مثال کمی پیچیده‌تری را بررسی کنیم (شکل ۲۳.۷). در این مثال، هر یک از کلاس‌های A و B دو متد به نام‌های m1 و m2 دارد. به یاد داشته باشید که چون کلاس B متد m2 ویژهٔ خود را تعریف نمی‌کند، این متد را از کلاس A به ارث می‌برد. شکل ۲۴.۷ نشان می‌دهد که شیء b در حافظه چگونه به نظر می‌رسد و شکل ۲۵.۷ خروجی برنامه را نشان می‌دهد. نخست به آدرس جدول توابع مجازی هر شیء نگاه کنید. آدرس‌های دو شیء B یکسان است و بنابراین این دو شیء در یک جدول توابع مجازی شریک

[94] modifier

هستند. جدول توابع مجازی ویژگیِ کلاس است، نه ویژگی شیء (درست مانند یک عضو دادهٔ `static`). سپس به آدرس‌های درون جدول‌های توابع مجازی نگاه کنید. با بررسی خروجی اسمبلی می‌توان دریافت که اشاره‌گر متد `m1` در آفست ۰ (یا کلمهٔ مضاعف ۰) و اشاره‌گر `m2` در آفست ۴ (کلمهٔ مضاعف ۱) قرار دارد. اشاره‌گرهای متد `m2` در جدول توابع مجازی کلاس `A` و کلاس `B` یکسان هستند، زیرا کلاس `B` متد `m2` را از کلاس `A` به ارث می‌برد.

سطرهای ۲۵ تا ۳۲ نشان می‌دهند که چگونه می‌توان یک تابع مجازی را با خواندن آدرس آن از جدول توابع مجازی شیء فراخوانی کرد[۹۵]. آدرس متد در یک اشاره‌گر به تابعِ سبک C ذخیره می‌شود و اشاره‌گر *this* به‌صراحت به آن فرستاده می‌شود. خروجی شکل ۲۵.۷ نشان می‌دهد که این روش واقعاً کار می‌کند. با این حال، لطفاً **هرگز** چنین کدی ننویسید! این کد فقط برای آن آمده است که نشان دهد متدهای مجازی چگونه از جدول توابع مجازی استفاده می‌کنند.

از این بحث چند درس عملی می‌توان گرفت. یک نکتهٔ مهم آن است که هنگام خواندن و نوشتن متغیرهای کلاس در یک فایل دودویی باید بسیار محتاط بود. نمی‌توان به‌سادگی کل شیء را به صورت دودویی خواند یا نوشت، زیرا در این صورت اشاره‌گر جدول توابع مجازی نیز از فایل خوانده یا در آن نوشته می‌شود! این اشاره‌گر به جایی از حافظهٔ برنامه اشاره می‌کند که جدول توابع مجازی در آن قرار دارد و از برنامه‌ای به برنامهٔ دیگر تغییر می‌کند. همین مشکل در C نیز ممکن است با ساختارها پیش بیاید، اما در C ساختارها فقط وقتی اشاره‌گر دارند که برنامه‌نویس به‌صراحت اشاره‌گری در آن‌ها قرار دهد، در حالی که در کلاس‌های `A` و `B` هیچ اشاره‌گر آشکاری تعریف نشده است.

باز هم باید توجه داشت که کامپایلرهای مختلف متدهای مجازی را به شیوه‌های متفاوتی پیاده‌سازی می‌کنند. در ویندوز، اشیای کلاس‌های COM (مدل شیء مؤلفه‌ای یا Component Object Model) برای پیاده‌سازی واسط‌های[۹۶] COM از جدول توابع مجازی استفاده می‌کنند[۹۷]. تنها کامپایلرهایی می‌توانند کلاس‌های COM بسازند که جدول توابع مجازی را به همان شیوهٔ مایکروسافت پیاده‌سازی کنند. به همین دلیل است که بورلند همان پیاده‌سازی مایکروسافت را به کار می‌برد و این یکی از دلایلی است که نمی‌توان با *gcc* کلاس‌های COM ساخت.

کد خود متد مجازی دقیقاً مانند کد یک متد غیرمجازی است و تنها کدی که آن

[۹۵] به یاد داشته باشید که این کد فقط با کامپایلرهای مایکروسافت و بورلند کار می‌کند، نه با *gcc*.

[96] interface

[۹۷] کلاس‌های COM همچنین به جای قرارداد استاندارد C از قرارداد فراخوانی `__stdcall` پیروی می‌کنند.

را فراخوانی می‌کند تفاوت دارد. اگر کامپایلر کاملاً مطمئن باشد که کدام متد مجازی فراخوانی خواهد شد، می‌تواند جدول توابع مجازی را نادیده بگیرد و متد را مستقیماً فراخوانی کند (یعنی از انقیاد زودهنگام استفاده کند).

۶.۲.۷ دیگر ویژگی‌های C++

سازوکار دیگر ویژگی‌های C++ (مانند اطلاعات نوع در زمان اجرا[۹۸]، مدیریت استثنا[۹۹] و وراثت چندگانه[۱۰۰]) از حوزهٔ این کتاب بیرون است. خواننده‌ای که بخواهد در این زمینه بیشتر پیش برود، می‌تواند از دو کتاب آغاز کند که نقطهٔ شروع خوبی هستند: کتاب *The Annotated C++ Reference Manual* نوشتهٔ الیس و استراستروپ، و کتاب *The Design and Evolution of C++* نوشتهٔ استراستروپ.

[98] RunTime Type Information
[99] exception handling
[100] multiple inheritance

پیوست الف

دستورالعمل‌های 80x86

الف.۱ دستورالعمل‌های غیر ممیز شناور

در این بخش، دستورالعمل‌های[1] غیر ممیز شناور[2] خانوادهٔ پردازنده‌های 80x86 اینتل فهرست شده و عملکرد و قالب هر یک از آن‌ها شرح داده شده است.

در نوشتن قالب‌ها، اختصارهای زیر به کار رفته است:

R	ثبات عمومی
R8	ثبات ۸ بیتی
R16	ثبات ۱۶ بیتی
R32	ثبات ۳۲ بیتی
SR	ثبات قطعه
M	حافظه
M8	بایت
M16	کلمه
M32	کلمهٔ مضاعف
I	مقدار بی‌واسطه

برای دستورالعمل‌هایی که چند عملگر[3] دارند، این اختصارها با هم ترکیب می‌شوند؛ برای نمونه، قالب $R,\ R$ یعنی دستورالعمل دو عملگر ثباتی[4] می‌گیرد. از آنجا که

[1] instruction
[2] floating point
[3] operand
[4] register

بسیاری از دستورالعمل‌های دوعملگری عملگرهای یکسانی را می‌پذیرند، این مجموعهٔ مشترک را با اختصار *O2* نشان می‌دهیم که به معنای قالب‌های *R,R R,M R,I M,R M,I* است. همچنین اگر بتوان برای یک عملگر از ثبات یا خانهٔ حافظهٔ ۸ بیتی استفاده کرد، اختصار *R/M8* به کار می‌رود.

جدول زیر همچنین نشان می‌دهد که هر دستورالعمل چه تأثیری بر بیت‌های گوناگون ثبات FLAGS می‌گذارد. خالی بودن ستون یک بیت به این معناست که آن بیت اصلاً تغییر نمی‌کند. اگر بیت همواره مقدار مشخصی بگیرد، همان مقدار، یعنی 1 یا 0، در ستون نوشته شده است و اگر مقدار تازهٔ بیت به عملگرهای دستورالعمل بستگی داشته باشد، حرف *C* در ستون آمده است. سرانجام، اگر بیت به شکلی نامعین تغییر کند، در ستون علامت *?* دیده می‌شود. پرچم جهت[5] در ستون‌های پرچم‌ها نیامده است، زیرا تنها دستورالعمل‌هایی که آن را تغییر می‌دهند CLD و STD هستند.

نام	توضیح	قالب‌ها	پرچم‌ها					
			O	**S**	**Z**	**A**	**P**	**C**
ADC	جمع با رقم نقلی (carry)	O2	C	C	C	C	C	C
ADD	جمع اعداد صحیح	O2	C	C	C	C	C	C
AND	AND بیتی	O2	0	C	C	?	C	0
BSWAP	وارونه کردن ترتیب بایت‌ها	R32						
CALL	فراخوانی روال	R M I						
CBW	تبدیل بایت به کلمه							
CDQ	تبدیل کلمهٔ مضاعف به کلمهٔ چهارگانه							
CLC	صفر کردن پرچم نقلی							0
CLD	صفر کردن پرچم جهت							
CMC	وارونه کردن پرچم نقلی							C
CMP	مقایسهٔ اعداد صحیح	O2	C	C	C	C	C	C
CMPSB	مقایسهٔ بایت‌ها		C	C	C	C	C	C
CMPSW	مقایسهٔ کلمه‌ها		C	C	C	C	C	C
CMPSD	مقایسهٔ کلمه‌های مضاعف		C	C	C	C	C	C
CWD	تبدیل کلمه به کلمهٔ مضاعف در DX:AX							
CWDE	تبدیل کلمه به کلمهٔ مضاعف در EAX							

[5] direction flag

نام	توضیح	قالب‌ها	پرچم‌ها					
			O	S	Z	A	P	C
DEC	کاهش یک‌واحدی عدد صحیح	R M	C	C	C	C	C	
DIV	تقسیم بی‌علامت	R M	?	?	?	?	?	?
ENTER	ساختن قاب پشته (stack frame)	I,0						
IDIV	تقسیم علامت‌دار	R M	?	?	?	?	?	?
IMUL	ضرب علامت‌دار	R M	C	?	?	?	?	C
		R16,R/M16						
		R32,R/M32						
		R16,I R32,I						
		R16,R/M16,I						
		R32,R/M32,I						
INC	افزایش یک‌واحدی عدد صحیح	R M	C	C	C	C	C	
INT	ایجاد وقفه (interrupt)	I						
JA	پرش اگر بالاتر باشد	I						
JAE	پرش اگر بالاتر یا مساوی باشد	I						
JB	پرش اگر پایین‌تر باشد	I						
JBE	پرش اگر پایین‌تر یا مساوی باشد	I						
JC	پرش اگر پرچم نقلی یک باشد	I						
JCXZ	پرش اگر $CX = 0$ باشد	I						
JE	پرش اگر مساوی باشد	I						
JG	پرش اگر بزرگ‌تر باشد	I						
JGE	پرش اگر بزرگ‌تر یا مساوی باشد	I						
JL	پرش اگر کوچک‌تر باشد	I						
JLE	پرش اگر کوچک‌تر یا مساوی باشد	I						
JMP	پرش غیرشرطی	R M I						
JNA	پرش اگر بالاتر نباشد	I						
JNAE	پرش اگر بالاتر یا مساوی نباشد	I						

نام	توضیح	قالب‌ها	پرچم‌ها					
			O	S	Z	A	P	C
JNB	پرش اگر پایین‌تر نباشد	I						
JNBE	پرش اگر پایین‌تر یا مساوی نباشد	I						
JNC	پرش اگر پرچم نقلی صفر باشد	I						
JNE	پرش اگر مساوی نباشد	I						
JNG	پرش اگر بزرگ‌تر نباشد	I						
JNGE	پرش اگر بزرگ‌تر یا مساوی نباشد	I						
JNL	پرش اگر کوچک‌تر نباشد	I						
JNLE	پرش اگر کوچک‌تر یا مساوی نباشد	I						
JNO	پرش اگر سرریز رخ ندهد	I						
JNS	پرش اگر پرچم علامت صفر باشد	I						
JNZ	پرش اگر صفر نباشد	I						
JO	پرش اگر سرریز رخ دهد	I						
JPE	پرش اگر توازن (parity) زوج باشد	I						
JPO	پرش اگر توازن فرد باشد	I						
JS	پرش اگر پرچم علامت یک باشد	I						
JZ	پرش اگر صفر باشد	I						
LAHF	بار کردن FLAGS در AH							
LEA	بار کردن آدرس مؤثر (effective address)	R32,M						
LEAVE	برچیدن قاب پشته							
LODSB	بار کردن بایت							
LODSW	بار کردن کلمه							
LODSD	بار کردن کلمهٔ مضاعف							
LOOP	حلقه	I						
LOOPE/LOOPZ	حلقه اگر مساوی باشد	I						
LOOPNE/LOOPNZ	حلقه اگر مساوی نباشد	I						

نام	توضیح	قالب‌ها	پرچم‌ها					
			O	S	Z	A	P	C
MOV	انتقال داده	O2						
		SR,R/M16						
		R/M16,SR						
MOVSB	انتقال بایت							
MOVSW	انتقال کلمه							
MOVSD	انتقال کلمهٔ مضاعف							
MOVSX	انتقال علامت‌دار	R16,R/M8						
		R32,R/M8						
		R32,R/M16						
MOVZX	انتقال بی‌علامت	R16,R/M8						
		R32,R/M8						
		R32,R/M16						
MUL	ضرب بی‌علامت	R M	C	?	?	?	?	C
NEG	قرینه کردن	R M	C	C	C	C	C	C
NOP	بدون عملیات							
NOT	مکمل یک	R M						
OR	OR بیتی	O2	0	C	C	?	C	0
POP	برداشتن از پشته	R/M16						
		R/M32						
POPA	برداشتن همهٔ ثبات‌ها از پشته							
POPF	برداشتن FLAGS از پشته		C	C	C	C	C	C
PUSH	قراردادن در پشته	R/M16						
		R/M32 I						
PUSHA	قراردادن همهٔ ثبات‌ها در پشته							
PUSHF	قراردادن FLAGS در پشته							
RCL	چرخش (rotate) به چپ	R/M,I	C					C
	همراه با رقم نقلی	R/M,CL						
RCR	چرخش به راست همراه با	R/M,I	C					C
	رقم نقلی	R/M,CL						
REP	تکرار							
REPE/REPZ	تکرار اگر مساوی باشد							
REPNE/REPNZ	تکرار اگر مساوی نباشد							
RET	بازگشت							

نام	توضیح	قالب‌ها	پرچم‌ها					
			O	S	Z	A	P	C
ROL	چرخش به چپ	R/M,I R/M,CL	C					C
ROR	چرخش به راست	R/M,I R/M,CL	C					C
SAHF	رونوشت AH در FLAGS			C	C	C	C	C
SAL	جابه‌جایی (shift) به چپ	R/M,I R/M, CL						C
SBB	تفریق با رقم قرضی (borrow)	O2	C	C	C	C	C	C
SCASB	جست‌وجوی بایت		C	C	C	C	C	C
SCASW	جست‌وجوی کلمه		C	C	C	C	C	C
SCASD	جست‌وجوی کلمهٔ مضاعف		C	C	C	C	C	C
SETA	تنظیم اگر بالاتر باشد	R/M8						
SETAE	تنظیم اگر بالاتر یا مساوی باشد	R/M8						
SETB	تنظیم اگر پایین‌تر باشد	R/M8						
SETBE	تنظیم اگر پایین‌تر یا مساوی باشد	R/M8						
SETC	تنظیم اگر پرچم نقلی یک باشد	R/M8						
SETE	تنظیم اگر مساوی باشد	R/M8						
SETG	تنظیم اگر بزرگ‌تر باشد	R/M8						
SETGE	تنظیم اگر بزرگ‌تر یا مساوی باشد	R/M8						
SETL	تنظیم اگر کوچک‌تر باشد	R/M8						
SETLE	تنظیم اگر کوچک‌تر یا مساوی باشد	R/M8						
SETNA	تنظیم اگر بالاتر نباشد	R/M8						
SETNAE	تنظیم اگر بالاتر یا مساوی نباشد	R/M8						
SETNB	تنظیم اگر پایین‌تر نباشد	R/M8						
SETNBE	تنظیم اگر پایین‌تر یا مساوی نباشد	R/M8						
SETNC	تنظیم اگر پرچم نقلی صفر باشد	R/M8						

نام	توضیح	قالب‌ها	پرچم‌ها					
			O	S	Z	A	P	C
SETNE	تنظیم اگر مساوی نباشد	R/M8						
SETNG	تنظیم اگر بزرگ‌تر نباشد	R/M8						
SETNGE	تنظیم اگر بزرگ‌تر یا مساوی نباشد	R/M8						
SETNL	تنظیم اگر کوچک‌تر نباشد	R/M8						
SETNLE	تنظیم اگر کوچک‌تر یا مساوی نباشد	R/M8						
SETNO	تنظیم اگر سرریز رخ ندهد	R/M8						
SETNS	تنظیم اگر پرچم علامت صفر باشد	R/M8						
SETNZ	تنظیم اگر صفر نباشد	R/M8						
SETO	تنظیم اگر سرریز رخ دهد	R/M8						
SETPE	تنظیم اگر توازن زوج باشد	R/M8						
SETPO	تنظیم اگر توازن فرد باشد	R/M8						
SETS	تنظیم اگر پرچم علامت یک باشد	R/M8						
SETZ	تنظیم اگر صفر باشد	R/M8						
SAR	جابه‌جایی حسابی به راست	R/M,I R/M, CL						C
SHR	جابه‌جایی منطقی به راست	R/M,I R/M, CL						C
SHL	جابه‌جایی منطقی به چپ	R/M,I R/M, CL						C
STC	یک کردن پرچم نقلی							1
STD	یک کردن پرچم جهت							
STOSB	ذخیرهٔ بایت							
STOSW	ذخیرهٔ کلمه							
STOSD	ذخیرهٔ کلمهٔ مضاعف							
SUB	تفریق	O2	C	C	C	C	C	C
TEST	مقایسهٔ منطقی	R/M,R R/M,I	0	C	C	?	C	0
XCHG	مبادلهٔ مقادیر	R/M,R R,R/M						
XOR	XOR بیتی	O2	0	C	C	?	C	0

الف.۲ دستورالعمل‌های ممیز شناور

این بخش بسیاری از دستورالعمل‌های کمک‌پردازندهٔ ریاضی[†] 80x86 را معرفی می‌کند. در ستون توضیح، کار هر دستورالعمل به‌کوتاهی شرح داده شده است؛ البته برای صرفه‌جویی در فضا، اینکه دستورالعمل مقداری را از پشته برمی‌دارد یا نه، در این توضیح نیامده است.

ستون قالب نشان می‌دهد که با هر دستورالعمل از چه نوع عملگرهایی می‌توان استفاده کرد. در این ستون، اختصارهای زیر به کار رفته است:

STn	یک ثبات کمک‌پردازنده
F	عدد با دقت ساده در حافظه
D	عدد با دقت مضاعف در حافظه
E	عدد با دقت گسترش‌یافته در حافظه
I16	عدد صحیح یک‌کلمه‌ای در حافظه
I32	عدد صحیح به اندازهٔ کلمهٔ مضاعف در حافظه
I64	عدد صحیح به اندازهٔ کلمهٔ چهارگانه در حافظه

دستورالعمل‌هایی که به پردازندهٔ پنتیوم پرو یا پردازنده‌ای جدیدتر نیاز دارند، با ستاره (*) مشخص شده‌اند.

دستورالعمل	توضیح	قالب
`FABS`	`ST0` $= \lvert$`ST0`$\rvert$	
`FADD` *src*	`ST0 +=` *src*	STn F D
`FADD` *dest*`, ST0`	*dest* `+= ST0`	STn
`FADDP` *dest*`[,ST0]`	*dest* `+= ST0`	STn
`FCHS`	`ST0` $= -$`ST0`	
`FCOM` *src*	مقایسهٔ `ST0` با *src*	STn F D
`FCOMP` *src*	مقایسهٔ `ST0` با *src*	STn F D
`FCOMPP` *src*	مقایسهٔ `ST0` با `ST1`	
`FCOMI`* *src*	مقایسه و ثبت نتیجه در FLAGS	STn
`FCOMIP`* *src*	مقایسه و ثبت نتیجه در FLAGS	STn
`FDIV` *src*	`ST0 /=` *src*	STn F D
`FDIV` *dest*`, ST0`	*dest* `/= ST0`	STn
`FDIVP` *dest*`[,ST0]`	*dest* `/= ST0`	STn

[†] math coprocessor

دستورالعمل	توضیح	قالب
FDIVR *src*	ST0 = *src*/ST0	STn F D
FDIVR *dest*, ST0	*dest* = ST0/*dest*	STn
FDIVRP *dest*[,ST0]	*dest* = ST0/*dest*	STn
FFREE *dest*	علامت‌گذاری به‌عنوان خالی	STn
FIADD *src*	ST0 += *src*	I16 I32
FICOM *src*	مقایسهٔ ST0 با *src*	I16 I32
FICOMP *src*	مقایسهٔ ST0 با *src*	I16 I32
FIDIV *src*	ST0 /= *src*	I16 I32
FIDIVR *src*	ST0 = *src*/ST0	I16 I32
FILD *src*	قراردادن *src* در پشته	I16 I32 I64
FIMUL *src*	ST0 *= *src*	I16 I32
FINIT	مقداردهی اولیهٔ کمک‌پردازنده	
FIST *dest*	ذخیرهٔ ST0	I16 I32
FISTP *dest*	ذخیرهٔ ST0	I16 I32 I64
FISUB *src*	ST0 -= *src*	I16 I32
FISUBR *src*	ST0 = *src* - ST0	I16 I32
FLD *src*	قراردادن *src* در پشته	STn F D E
FLD1	قراردادن 1.0 در پشته	
FLDCW *src*	بار کردن ثبات کلمهٔ کنترل (control word)	I16
FLDPI	قراردادن π در پشته	
FLDZ	قراردادن 0.0 در پشته	
FMUL *src*	ST0 *= *src*	STn F D
FMUL *dest*, ST0	*dest* *= ST0	STn
FMULP *dest*[,ST0]	*dest* *= ST0	STn
FRNDINT	گرد کردن ST0	
FSCALE	$\text{ST0} = \text{ST0} \times 2^{\lfloor \text{ST1} \rfloor}$	
FSQRT	$\text{ST0} = \sqrt{\text{ST0}}$	
FST *dest*	ذخیرهٔ ST0	STn F D
FSTP *dest*	ذخیرهٔ ST0	STn F D E
FSTCW *dest*	ذخیرهٔ ثبات کلمهٔ کنترل	I16
FSTSW *dest*	ذخیرهٔ ثبات کلمهٔ وضعیت (status word)	I16 AX
FSUB *src*	ST0 -= *src*	STn F D
FSUB *dest*, ST0	*dest* -= ST0	STn
FSUBP *dest*[,ST0]	*dest* -= ST0	STn
FSUBR *src*	ST0 = *src*-ST0	STn F D

دستورالعمل	توضیح	قالب
FSUBR *dest*, ST0	*dest* = ST0-*dest*	STn
FSUBRP *dest*[,ST0]	*dest* = ST0-*dest*	STn
FTST	مقایسهٔ ST0 با 0.0	
FXCH *dest*	مبادلهٔ ST0 و *dest*	STn